

COMPLETE DEVOPS INTERVIEW HANDBOOK

Read · Understand · Practice · Recall · Answer

Prepared for

AASIM RAZA

DevOps Engineer · Gurgaon, India

AWS · Kubernetes · Docker · Terraform · Ansible · Jenkins
GitHub Actions · Prometheus · Grafana · Linux · Python · Bash

25 PARTS · 600+ INTERVIEW QUESTIONS · 300+ FLASHCARDS

5 Mock Interviews · 30-Day Study Plan · Production Troubleshooting Runbooks

TABLE OF CONTENTS

Part 1. MY DEVOPS INTERVIEW PROFILE	11
1.1 How to Use This Handbook	11
1.2 Your Resume, Extracted and Organised	11
1.3 Your Strongest Technical Areas	13
1.4 Areas That Require Deeper Preparation	13
1.5 How to Explain Your Background Professionally	14
1.6 Your Introductions - 30 Seconds, 60 Seconds, 2 Minutes	15
1.7 The Full "Tell Me About Yourself" Answer	16
1.8 How to Explain Your Project Without Exaggerating	17
Part 2. DEVOPS FUNDAMENTALS	18
2.1 What Is DevOps?	18
2.2 Why DevOps? The Business Case	19
2.3 The DevOps Lifecycle	19
2.4 CI vs CD - The Question You Will Definitely Be Asked	20
2.5 Infrastructure as Code (IaC)	21
2.6 Configuration Management	22
2.7 Containerization	22
2.8 Orchestration	22
2.9 Monitoring	23
2.10 Observability	23
2.11 Logging	23
2.12 Alerting	24
2.13 Incident Management	24
2.14 Root Cause Analysis (RCA)	25
2.15 Rollback	25
2.16 Deployment Strategies	26
2.17 Immutable Infrastructure	27
2.18 Automation	27
2.19 DevSecOps Basics	27
2.20 SRE Basics	28
2.21 High Availability, Scalability, Reliability	28
Part 3. LINUX COMPLETE INTERVIEW GUIDE	30
3.1 Linux Architecture	30
3.2 The Filesystem Hierarchy	30
3.3 Users, Groups, Ownership	31
3.4 Permissions and Ownership	31
3.5 Core File and Text Commands	33

3.6 Processes	35
3.7 Services and systemd	36
3.8 Disk and Filesystems	37
3.9 Memory	38
3.10 Logs	39
3.11 Linux Networking Commands	39
3.12 Package Management	41
3.13 Bash Scripting for Linux Work	41
3.14 Hands-On Exercises	43
3.15 Linux Interview Questions and Answers	43
Part 4. NETWORKING FOR DEVOPS	48
4.1 The OSI Model	48
4.2 TCP/IP Model and How It Maps	48
4.3 IP Addressing, Subnetting and CIDR	48
4.4 Ports, TCP and UDP	49
4.5 DNS	50
4.6 HTTP, HTTPS and TLS	51
4.7 Proxies, Reverse Proxies and Load Balancers	53
4.8 NAT, Firewalls, Routing and VPN	54
4.9 "What Happens When You Type https://example.com?"	54
4.10 Troubleshooting Method: "The Application Is Not Reachable"	55
4.11 Scenario Catalogue	56
4.12 Networking Interview Questions	58
Part 5. GIT AND GITHUB	62
5.1 Git Architecture - The Four Areas	62
5.2 The Everyday Commands	62
5.3 Branching	63
5.4 Merge vs Rebase - The Classic Question	63
5.5 Merge Conflicts	64
5.6 Undoing Things - The Recovery Toolkit	65
5.7 Git Internals Worth Knowing	66
5.8 Pull Requests, Protection and Hooks	66
5.9 Git Interview Questions	67
Part 6. JENKINS COMPLETE GUIDE	72
6.1 Jenkins Architecture	72
6.2 Job Types	73
6.3 Declarative vs Scripted Pipelines	73
6.4 A Complete Jenkinsfile, Explained Line by Line	74

6.5 The Complete CI/CD Flow	77
6.6 Credentials and Security	77
6.7 Shared Libraries	78
6.8 Multibranch Pipelines, Webhooks and Triggers	79
6.9 Agents, Including Docker and Kubernetes Agents	80
6.10 Jenkins Troubleshooting	81
6.11 Jenkins Interview Questions	82
Part 7. DOCKER COMPLETE GUIDE	87
7.1 Docker Architecture	87
7.2 Images and Layers	87
7.3 Dockerfile Instructions	88
7.4 Docker Commands	90
7.5 Docker Networking	91
7.6 Volumes and Persistence	92
7.7 Docker Compose	93
7.8 Container Lifecycle and Restart Policies	94
7.9 Docker Troubleshooting Scenarios	95
7.10 Docker Interview Questions	97
Part 8. AWS COMPLETE DEVOPS GUIDE	101
8.1 EC2 - Elastic Compute Cloud	101
8.2 IAM - Identity and Access Management	102
8.3 VPC - Virtual Private Cloud	104
8.4 S3 - Simple Storage Service	105
8.5 CloudWatch	106
8.6 Route 53	106
8.7 Load Balancing on AWS	107
8.8 A Complete Architecture, and Why Each Layer Exists	107
8.9 AWS Troubleshooting Scenarios	108
8.10 AWS Interview Questions	109
Part 9. TERRAFORM COMPLETE GUIDE	115
9.1 Terraform Architecture	115
9.2 HCL - The Building Blocks	115
9.3 The Command Workflow	118
9.4 Terraform State - Explained Deeply	119
9.5 Modules	120
9.6 Workspaces and Environment Separation	122
9.7 Terraform in CI/CD	122
9.8 What Actually Happens During `terraform apply`	123

9.9 Terraform Interview Questions	123
Part 10. ANSIBLE COMPLETE GUIDE	128
10.1 Ansible Architecture	128
10.2 Inventory	128
10.3 Ad-Hoc Commands	130
10.4 Playbooks	130
10.5 Idempotency - The Core Concept	132
10.6 Variables, Facts, Templates	132
10.7 Roles	134
10.8 Ansible Vault	134
10.9 Running Playbooks	135
10.10 Practical Playbook Examples	135
10.11 Terraform vs Ansible - Say This Clearly	137
10.12 Ansible Interview Questions	138
Part 11. KUBERNETES COMPLETE GUIDE	142
11.1 Architecture	142
11.2 Pods	143
11.3 Deployments and ReplicaSets	145
11.4 Services and Networking	146
11.5 ConfigMaps and Secrets	147
11.6 The Other Objects	148
11.7 kubectl - The Commands You Must Know Cold	149
11.8 The 15 Troubleshooting Scenarios	150
11.9 Kubernetes Interview Questions	154
Part 12. PROMETHEUS AND GRAFANA	161
12.1 Metrics and Time Series	161
12.2 Prometheus Architecture	161
12.3 Configuration	162
12.4 PromQL	163
12.5 Alerting	164
12.6 Grafana	166
12.7 What to Monitor	166
12.8 Monitoring Interview Questions	167
Part 13. AWS CLOUDWATCH	170
13.1 The Components	170
13.2 Key Metrics by Service	170
13.3 The CloudWatch Agent	171

13.4 Alarms	172
13.5 Logs Insights	173
13.6 CloudWatch Alongside Prometheus	174
13.7 CloudWatch Interview Scenarios	174
Part 14. PYTHON FOR DEVOPS	176
14.1 The Language Subset You Actually Need	176
14.2 Script 1 - Disk Space Check	178
14.3 Script 2 - Memory Check	179
14.4 Script 3 - Process Check	180
14.5 Script 4 - TCP Port Check	180
14.6 Script 5 - HTTP API Call with Retry	181
14.7 Script 6 - Log Parser	182
14.8 Script 7 - File Cleanup	183
14.9 Script 8 - Service Check	184
14.10 Script 9 - Composite Health Check	185
14.11 Python Interview Questions	186
Part 15. BASH SCRIPTING	190
15.1 The Safety Header	190
15.2 Variables, Arguments and Defaults	190
15.3 Conditions	191
15.4 Loops, Functions and Traps	191
15.5 Script 1 - Health Check with Retry	192
15.6 Script 2 - Disk Space Alert	193
15.7 Script 3 - Log Rotation and Archive	193
15.8 Script 4 - Deployment with Rollback	194
15.9 Script 5 - Service Watchdog	195
15.10 Script 6 - Backup with Verification	196
15.11 Script 7 - Parallel Remote Command	196
15.12 Script 8 - Docker Cleanup	197
15.13 Script 9 - SSL Certificate Expiry Check	197
15.14 Script 10 - Container Entrypoint	198
15.15 Bash Interview Questions	199
Part 16. PRODUCTION TROUBLESHOOTING HANDBOOK	201
16.1 The Universal Method	201
16.2 Incident 1 - Server Down / Unreachable	201
16.3 Incident 2 - CPU at 100%	202
16.4 Incident 3 - Memory Exhausted / OOM	202
16.5 Incident 4 - Disk Full	203

16.6 Incident 5 - Application Down	204
16.7 Incident 6 - Database Unavailable	204
16.8 Incident 7 - Jenkins Failure	204
16.9 Incident 8 - Docker Failure	205
16.10 Incident 9 - Kubernetes Failure	205
16.11 Incident 10 - Network Timeout	206
16.12 Incident 11 - DNS Issue	206
16.13 Incident 12 - SSL/TLS Issue	206
16.14 Incident 13 - Deployment Failure	207
16.15 Incident 14 - Rollback Required	207
16.16 Incident 15 - Monitoring Alert Storm	208
16.17 Incident 16 - Application Latency	208
16.18 Incident 17 - HTTP 500	208
16.19 Incident 18 - HTTP 502	209
16.20 Incident 19 - HTTP 503	209
16.21 Incident 20 - HTTP 504	210
16.22 The Incident Communication Template	210
16.23 The RCA Template	210
Part 17. SCENARIO-BASED INTERVIEW QUESTIONS	212
17.1 Scenarios 1-10: Availability	212
17.2 Scenarios 11-20: CI/CD and Deployment	213
17.3 Scenarios 21-32: Kubernetes and Containers	215
17.4 Scenarios 33-42: Linux, AWS and Networking	216
17.5 Scenarios 43-50: Judgement, Process and People	218
Part 18. PROJECT EXPLANATION	220
18.1 The Architecture You Are Describing	220
18.2 The 30-Second Version	221
18.3 The One-Minute Version	221
18.4 The Three-Minute Version	221
18.5 The Five-Minute Technical Version	222
18.6 Fifty Questions an Interviewer May Ask About Your Project	224
Part 19. RESUME CROSS-QUESTIONING	229
19.1 Summary Line: "3 years of experience automating CI/CD pipelines, cloud infrastructure and production deployments"	229
19.2 "Hands-on with AWS, Kubernetes, Docker, Terraform, Ansible, Jenkins, GitHub Actions, Prometheus, Grafana, Python and Bash"	229
19.3 "Skilled in Infrastructure as Code, containerized deployments, monitoring and production incident resolution"	229
19.4 Skills Line: "AWS (EC2, S3, IAM, VPC, CloudWatch, Route 53, Security Groups)"	230
19.5 "Docker, Kubernetes (Pods, Deployments, Services, ConfigMaps, Secrets)"	230

19.6 "Terraform, Terraform Modules, Infrastructure Provisioning"	230
19.7 "Ansible, YAML" and Configuration Management	231
19.8 "Jenkins, Jenkins Pipelines (pipeline-as-code), GitHub Actions"	231
19.9 "Prometheus, Grafana, AWS CloudWatch, Application Monitoring, Log Analysis, Alerting"	232
19.10 "Python, Bash, PowerShell, Automation Scripts"	232
19.11 "Git, GitHub, Branching, Pull Requests"	233
19.12 "Linux, Ubuntu, Windows Server, TCP/IP, HTTP/HTTPS, DNS, RTSP, Ports, Network Troubleshooting"	233
19.13 "Production Support, Incident Troubleshooting, RCA, Release Management"	233
19.14 Current Role Bullet 1: "Manage DevOps activities across Linux and Windows"	233
19.15 Current Role Bullet 2: "Built and maintain Jenkins CI/CD pipelines... reducing manual deployment effort and improving release consistency"	234
19.16 Current Role Bullet 3: "Provisioned AWS infrastructure with Terraform... automated configuration with Ansible"	234
19.17 Current Role Bullet 4: "Containerized application components with Docker and support Kubernetes deployments"	234
19.18 Current Role Bullet 5: "Implemented monitoring with Prometheus, Grafana and AWS CloudWatch; maintain GitHub Actions workflows"	234
19.19 Current Role Bullet 6: "Developed Python and Bash scripts for deployment support, system checks, log analysis and network troubleshooting"	235
19.20 Current Role Bullet 7: "Investigate production incidents... perform RCA, coordinate fixes with development and QA, validate application upgrades"	235
19.21 Previous Role Bullets	235
19.22 Education and Certifications	236
19.23 The Self-Audit Checklist	236
Part 20. HR AND MANAGERIAL ROUND	238
20.1 Tell Me About Yourself	238
20.2 Why DevOps?	238
20.3 Why Are You Switching / Why Are You Leaving?	238
20.4 Why Should We Hire You?	239
20.5 Why This Company?	239
20.6 Strengths and Weaknesses	239
20.7 Tell Me About a Difficult Incident	240
20.8 Tell Me About a Production Issue You Caused	240
20.9 Tell Me About a Disagreement	240
20.10 How Do You Handle Pressure?	241
20.11 How Do You Prioritise Incidents?	241
20.12 Are You Comfortable With On-Call?	241
20.13 Where Do You See Yourself?	241
20.14 Expected Salary	242
20.15 Notice Period	242
20.16 Other Questions You Will Be Asked	242
20.17 The Consistency Check	243

Part 21. RAPID REVISION	244
21.1 Linux Command Sheet	244
21.2 Git Command Sheet	245
21.3 Docker Command Sheet	245
21.4 Kubernetes Command Sheet	246
21.5 Terraform and Ansible Sheets	246
21.6 PromQL Sheet	247
21.7 AWS Quick Facts	247
21.8 The Difference Tables	248
21.9 HTTP Status Decision Tree	250
21.10 Troubleshooting Flowcharts	250
21.11 Definitions You Must Be Able to State in One Sentence	250
21.12 The Twelve Sentences That Win Interviews	251
Part 22. FLASHCARDS	252
22.1 DevOps Fundamentals (1-30)	252
22.2 Linux (31-70)	253
22.3 Networking (71-105)	254
22.4 Git (106-130)	255
22.5 Jenkins and CI/CD (131-160)	256
22.6 Docker (161-190)	258
22.7 Kubernetes (191-235)	259
22.8 Terraform and Ansible (236-265)	260
22.9 AWS and Monitoring (266-310)	261
Part 23. FIVE COMPLETE MOCK INTERVIEWS	264
23.1 Mock Interview 1 - Junior / Intermediate DevOps (45 minutes)	264
23.2 Mock Interview 2 - Two to Three Year DevOps (60 minutes)	265
23.3 Mock Interview 3 - AWS and Cloud Infrastructure (60 minutes)	266
23.4 Mock Interview 4 - Kubernetes and CI/CD (60 minutes)	267
23.5 Mock Interview 5 - Production Troubleshooting (45 minutes)	268
23.6 Self-Scoring Sheet	269
Part 24. 30-DAY STUDY PLAN	270
24.1 Week 1 - Foundations and Linux	270
24.2 Week 2 - Networking, Git, Jenkins	270
24.3 Week 3 - Containers, CI/CD, Cloud	271
24.4 Week 4 - Kubernetes, IaC, Monitoring, Practice	271
24.5 Daily Routine	272
24.6 If You Only Have One Week	272
24.7 If You Only Have One Day	273

24.8 Progress Tracker	273
Part 25. FINAL INTERVIEW CHECKLIST	274
25.1 The Week Before	274
25.2 The Day Before	274
25.3 Thirty Minutes Before	274
25.4 During the Introduction	274
25.5 During the Technical Round	275
25.6 Questions to Ask Them	275
25.7 Things Not to Say	276
25.8 Handling Questions You Cannot Answer	276
25.9 During the Managerial and HR Round	277
25.10 After the Interview	277
25.11 The Final Reminders	277

PART 1

MY DEVOPS INTERVIEW PROFILE

Your resume, decoded into interview language

This part converts your resume into a structured, defensible interview narrative. Everything in the grey "From Your Resume" boxes is taken directly from your CV. Anything marked "Interview Knowledge / Preparation" is study material to strengthen your fundamentals - it is not a claim about your experience, and you must never present it as one.

1.1 How to Use This Handbook

This book is built for five passes. Do not read it once and stop.

Pass	What you do	Where you work
Read	Read the concept sections end to end, slowly. Do not skip the "simple explanation" - that is the wording you will use with a non-technical manager.	Parts 2-15
Understand	Redraw every architecture diagram on paper from memory. If you cannot draw it, you do not know it.	Diagrams in every part
Practice	Run the commands on a laptop VM / EC2 free tier / Minikube. Break things deliberately, then fix them.	Hands-on exercises
Recall	Flashcards and the Rapid Revision part, 20 minutes a day.	Parts 21-22
Answer	Mock interviews out loud, timed, recorded on your phone.	Part 23

THE SINGLE RULE OF THIS HANDBOOK

Never claim experience you do not have. When you hit a topic you only studied, the correct interview sentence is: "I have not run that in production, but I understand how it works - here is my understanding, and here is what I would verify first." That answer scores higher than a confident bluff that collapses on the follow-up question.

1.2 Your Resume, Extracted and Organised

1.2.1 Identity and headline

HEADER FACTS

- Name: Aasim Raza
- Title: DevOps Engineer
- Location: Gurgaon, India
- Contact: +91 9766613006, aasimdevops@gmail.com
- Total experience stated: 3 years

1.2.2 Professional summary (as written on your CV)

PROFESSIONAL SUMMARY

DevOps Engineer with 3 years of experience automating CI/CD pipelines, cloud infrastructure and production deployments. Hands-on with AWS, Kubernetes, Docker, Terraform, Ansible, Jenkins, GitHub Actions, Prometheus, Grafana, Python and Bash. Skilled in Infrastructure as Code, containerized deployments, monitoring and production incident resolution, working closely with development and QA teams to improve release reliability and operational efficiency.

Every single word of that paragraph is a question the interviewer is allowed to ask. Part 19 cross-questions each one.

1.2.3 Current role

DEVOPS ENGINEER - I2V SYSTEMS, GURGAON (MAY 2025 - PRESENT)

- Manage DevOps activities for application environments: deployment automation, infrastructure provisioning, configuration management, monitoring, and production troubleshooting across **Linux and Windows**.
- Built and maintain **Jenkins CI/CD pipelines** automating build, test, packaging and deployment workflows, reducing manual deployment effort and improving release consistency.
- Provisioned **AWS infrastructure with Terraform**, and automated server and application configuration with **Ansible** for repeatable, controlled environments.
- Containerized application components with **Docker**; support **Kubernetes** deployments - troubleshooting Pods, Deployments, Services, ConfigMaps and application connectivity.
- Implemented monitoring with **Prometheus, Grafana and AWS CloudWatch**; maintain **GitHub Actions** workflows for CI/CD and deployment orchestration.
- Developed **Python and Bash** scripts for deployment support, system checks, log analysis and network troubleshooting; work in Git/GitHub branching and pull-request workflows.
- Investigate production incidents using application logs, Windows services, Linux commands, database connectivity checks and network diagnostics (**HTTP/HTTPS, TCP/IP, DNS, RTSP**); perform **RCA**, coordinate fixes with development and QA, validate application upgrades and releases.

1.2.4 Previous role**DEVOPS ENGINEER - NARESH I TECHNOLOGY, HYDERABAD (JAN 2024 - APR 2025)**

- Created and maintained **Jenkins pipelines** for automated build, deployment and release workflows integrated with Git; packaged applications with **Docker** to standardize environments.
- Automated infrastructure provisioning and configuration with **Terraform and Ansible**; operated AWS services (**EC2, S3, IAM, VPC, CloudWatch, Route 53**) for hosting, access management and monitoring.
- Wrote **Python and Bash** scripts for operational tasks, environment checks and service management; monitored health with **Prometheus, Grafana, CloudWatch** and investigated alerts.
- Performed **Linux and Windows troubleshooting** (service failures, disk utilization, network connectivity, application logs); collaborated with QA and development during releases, documenting recurring issues and procedures.

1.2.5 Technology inventory from your CV

Category	What your CV lists	Handbook part
Cloud	AWS - EC2, S3, IAM, VPC, CloudWatch, Route 53, Security Groups	Part 8, 13
Containers & Orchestration	Docker; Kubernetes - Pods, Deployments, Services, ConfigMaps, Secrets	Parts 7, 11
Infrastructure as Code	Terraform, Terraform Modules, infrastructure provisioning	Part 9
Configuration Management	Ansible, YAML	Part 10
CI/CD	Jenkins, Jenkins pipeline-as-code, GitHub Actions, release & deployment automation	Part 6
Monitoring & Observability	Prometheus, Grafana, CloudWatch, application monitoring, log analysis, alerting	Parts 12, 13
Scripting	Python, Bash, PowerShell, automation scripts	Parts 14, 15
Version control	Git, GitHub, branching, pull requests	Part 5
OS & Networking	Linux, Ubuntu, Windows Server, TCP/IP, HTTP/HTTPS, DNS, RTSP, ports, network troubleshooting	Parts 3, 4
Practices	Production support, incident troubleshooting, RCA, release management	Parts 16, 17

1.2.6 Key projects on your CV

Project	Resume wording	Cross-questioned in
CI/CD Deployment Automation	Jenkins pipeline-as-code workflows integrating Git, automated build, test and deployment stages to standardize release workflows	Part 6, Part 18
Infrastructure as Code with Terraform	Reusable Terraform modules provisioning AWS infrastructure, enabling repeatable environment creation and reducing manual configuration	Part 9, Part 18
Containerized Deployment & Observability	Dockerized components on Kubernetes (Pods, Deployments, Services, ConfigMaps, Secrets) with Prometheus, Grafana and CloudWatch dashboards and alerting	Parts 7, 11, 12, 18

1.2.7 Certifications and education

CERTIFICATIONS & EDUCATION

- Certifications / continuous learning: AWS, DevOps, Terraform, Ansible. Currently strengthening Kubernetes administration, AWS architecture, GitHub Actions and cloud-native observability.
- Education: B.Tech, Mechanical Engineering - Government College of Engineering, Jalgaon (2018-2022).

ON THE CERTIFICATION LINE

Your CV lists these as "Certifications & Continuous Learning" without certificate IDs or issuing bodies. If an interviewer asks "Which AWS certification?", answer precisely and without inflation: name the training you completed and say whether the exam is scheduled or not. Saying "I completed AWS and Terraform training as part of my DevOps program; I am preparing for the AWS Solutions Architect Associate exam" is safe. Claiming a certification number you cannot produce is the fastest way to lose an offer at the background-check stage.

1.3 Your Strongest Technical Areas

Ranked by how much your CV supports them - lead with the top of this list.

R a n k	Area	Why it is strong for you	How to lead with it
1	Jenkins CI/CD and pipeline-as-code	Appears in both roles and as a named project. Two years of continuous exposure.	Make the pipeline your default story: Git -> webhook -> build -> test -> package -> image -> deploy -> health check.
2	Production troubleshooting and RCA	Your current role bullet is unusually specific: logs, Windows services, Linux commands, DB connectivity, HTTP/DNS/RTSP diagnostics.	This is your differentiator. Most 3-year candidates cannot troubleshoot; you can describe a method.
3	Terraform + Ansible automation	Both roles, plus a named module project.	Pair them correctly: Terraform provisions, Ansible configures.
4	Docker containerization	Both roles. Image building and standardizing environments.	Dockerfile, layers, CMD vs ENTRYPOINT, networking, volumes.
5	AWS core services	EC2, S3, IAM, VPC, CloudWatch, Route 53, Security Groups.	Stay inside those services. Do not volunteer EKS, ECS, Lambda depth you do not have.
6	Monitoring - Prometheus, Grafana, CloudWatch	Both roles, plus project three.	Metrics -> dashboards -> alerts -> action.
7	Linux and Windows administration	Explicitly on the CV; rarer than you think in a candidate pool.	Windows Server + RTSP diagnostics is genuinely unusual - use it.
8	Python and Bash automation	Both roles. System checks, log analysis, network troubleshooting.	Have three scripts you can write on a whiteboard. Part 14/15 gives them to you.

1.4 Areas That Require Deeper Preparation

Be honest with yourself here - this is where interviews are lost.

Gap	Why it is a risk	Fix before the interview
Kubernetes depth	Your CV says you "support Kubernetes deployments" and troubleshoot objects. It does not say you built or administer a cluster. A K8s-heavy interviewer will go to scheduling, networking and etcd.	Part 11 in full, plus Minikube/kind hands-on. Be ready to say you troubleshoot workloads, not that you administer control planes.
VPC / networking design	Your CV lists VPC as a used service, not as something you architected.	Part 4 and Part 8.3. Be able to draw public/private subnet + NAT gateway from memory.
Terraform state at scale	Modules are on your CV. Remote state, locking, drift and imports are the standard follow-ups.	Part 9.6 to 9.9.
Kubernetes on AWS (EKS)	Not on your CV at all.	Know the concept, never claim the experience. Part 11.15.
Security / DevSecOps	Not on your CV. Commonly asked at 3-year level.	Part 2.24 plus IAM least privilege in Part 8.2.
Metrics you cannot quote	Your CV wisely avoids invented numbers ("reduced deploy time by 80%"). Some interviewers push for numbers.	Answer with direction and mechanism, not invented figures. See the box below.

WHEN THE INTERVIEWER ASKS "BY HOW MUCH DID YOU IMPROVE IT?"

Wrong: inventing "we cut deployment time by 70% and saved 40 engineer-hours a month." If you are pushed on methodology you will contradict yourself.

Right: "I did not own the measurement, so I will not quote a number I cannot defend. What I can describe is the mechanism: the manual deployment was a sequence of copy, stop service, replace build, start service, verify - it took a person and was inconsistent between environments. The pipeline made it one triggered job with the same steps every time, which removed the manual variance and the need for a person to be available at release time."

1.5 How to Explain Your Background Professionally

1.5.1 The Mechanical Engineering question

You hold a B.Tech in Mechanical Engineering and work in DevOps. You will be asked about this in most interviews. Treat it as a strength, delivered in three sentences - never apologetically.

"YOU STUDIED MECHANICAL ENGINEERING. WHY DEVOPS?"

"My degree is in Mechanical Engineering, and the part of it I enjoyed most was systems - how a process behaves end to end, where it fails, and how you make it repeatable. That is the same instinct DevOps rewards. I moved into DevOps deliberately: I took formal DevOps training covering AWS, Terraform, Ansible and CI/CD, then started working as a DevOps engineer in January 2024. Three years in, my day is pipelines, infrastructure automation and production troubleshooting, and the engineering discipline from my degree - measure, find the root cause, don't guess - is what I use most."

NEVER SAY THESE ABOUT YOUR DEGREE

"I couldn't find a job in mechanical." / "Mechanical has no scope." / "I just wanted a higher salary." All three tell the interviewer your motivation is external and your first better offer will move you again. Frame it as a deliberate, trained transition - which is what it was.

1.5.2 How to introduce your current role

Say what you own, on which platforms, with which tools, and how you know it works. Four parts, in that order.

INTRODUCING I2V SYSTEMS

"At I2V Systems I handle DevOps for our application environments - that means deployment automation, provisioning, configuration management, monitoring and production troubleshooting, across both Linux and Windows servers. Concretely, I build and maintain the Jenkins pipelines that take code from Git through build, test, packaging and deployment; I provision AWS infrastructure using Terraform and configure it using Ansible; I containerize application components with Docker and support our Kubernetes deployments; and I run monitoring through Prometheus, Grafana and CloudWatch so we see problems before users report them. When a production issue comes in, I am usually the one working through logs, services, database connectivity and network diagnostics to find root cause."

1.5.3 The two-company question

"WHY DID YOU CHANGE COMPANIES IN APRIL 2025?"

"At Naresh I Technology I was building pipelines and automating infrastructure, and I learned the fundamentals there - Jenkins, Docker, Terraform, Ansible and the AWS core services. The move to I2V Systems in May 2025 gave me broader ownership: I now handle the full environment rather than just the pipeline, including Kubernetes deployments, monitoring implementation and production incident work across Linux and Windows. It was a step up in responsibility, not a sideways move."

1.6 Your Introductions - 30 Seconds, 60 Seconds, 2 Minutes

Memorise the structure, not the words. Delivering a memorised paragraph sounds memorised; delivering a memorised structure sounds fluent.

WHO	WHAT I DO NOW	WITH WHAT	PROOF	WHY HERE	
+-----+	+-----+	+-----+	+-----+	+-----+	
Name, --> Current role, -> Tool stack -> One concrete -> Fit to the	years	ownership	(top 5 only)	example	JD / role
+-----+	+-----+	+-----+	+-----+	+-----+	
3 sec	8 sec	6 sec	25 sec	8 sec	

The introduction skeleton - works at any length

1.6.1 The 30-second introduction

30 SECONDS - USE FOR SCREENING CALLS AND "QUICK INTRO PLEASE"

"I'm Aasim Raza, a DevOps Engineer with three years of experience, currently at I2V Systems in Gurgaon. My work is CI/CD automation with Jenkins and GitHub Actions, infrastructure automation on AWS using Terraform and Ansible, containerized deployments with Docker and Kubernetes, and monitoring with Prometheus, Grafana and CloudWatch. A large part of my role is production support - investigating incidents across Linux and Windows and doing root cause analysis. I'm looking for a role where I can go deeper on cloud infrastructure and Kubernetes."

1.6.2 The 60-second introduction

60 SECONDS - THE STANDARD OPENING FOR A TECHNICAL ROUND

"I'm Aasim Raza, a DevOps Engineer with three years of experience. I started in DevOps at Naresh I Technology in January 2024, where I built and maintained Jenkins pipelines for build and deployment, packaged applications with Docker, and automated AWS infrastructure with Terraform and Ansible - working with EC2, S3, IAM, VPC, CloudWatch and Route 53.

Since May 2025 I've been at I2V Systems in Gurgaon, where I own DevOps for our application environments. I build the Jenkins CI/CD pipelines, provision infrastructure with Terraform, configure it with Ansible, containerize components with Docker and support Kubernetes deployments - troubleshooting Pods, Deployments, Services and ConfigMaps. I implemented monitoring using Prometheus, Grafana and CloudWatch, and I maintain GitHub Actions workflows alongside Jenkins.

The part of the job I'm strongest at is production troubleshooting - working through application logs, Linux and Windows services, database connectivity and network diagnostics across HTTP, DNS and RTSP, finding root cause and coordinating the fix with development and QA. My background is a B.Tech in Mechanical Engineering, and I moved into DevOps through formal training in AWS, Terraform, Ansible and CI/CD."

1.6.3 The 2-minute introduction

2 MINUTES - WHEN THE INTERVIEWER SAYS "WALK ME THROUGH YOUR BACKGROUND"

"I'm Aasim Raza, a DevOps Engineer based in Gurgaon with three years of experience across CI/CD automation, cloud infrastructure and production operations.

My background is a B.Tech in Mechanical Engineering from Government College of Engineering, Jalgaon, in 2022. What drew me to DevOps was the systems side of engineering - repeatability, failure analysis, automation. I took formal DevOps training covering AWS, Terraform, Ansible and CI/CD, and moved into the field in January 2024.

My first DevOps role was at Naresh I Technology in Hyderabad. I created and maintained Jenkins pipelines that automated build, deployment and release workflows integrated with Git, and packaged applications with Docker so environments stopped drifting between dev, test and production. I automated provisioning with Terraform and configuration with Ansible, and operated AWS - EC2 for compute, S3 for storage, IAM for access, VPC for networking, CloudWatch and Route 53. I also wrote Python and Bash for operational tasks and environment checks, monitored health with Prometheus and Grafana, and did Linux and Windows troubleshooting - service failures, disk utilization, network connectivity and application logs.

Since May 2025 I've been at I2V Systems, where the scope is wider. I manage DevOps across our application environments end to end: deployment automation, infrastructure provisioning, configuration management, monitoring and production troubleshooting, on both Linux and Windows. I build and maintain the Jenkins CI/CD pipelines for build, test, packaging and deployment; provision AWS infrastructure with Terraform as code; automate server and application configuration with Ansible; containerize components with Docker; and support Kubernetes deployments, where most of my work is troubleshooting Pods, Deployments, Services, ConfigMaps and connectivity. I implemented monitoring with Prometheus, Grafana and CloudWatch, and I maintain GitHub Actions workflows for CI/CD and deployment orchestration.

Production support is a core part of my role. When an incident comes in, I work through application logs, Windows services, Linux commands, database connectivity checks and network diagnostics - HTTP/HTTPS, TCP/IP, DNS and RTSP, since we work with video streaming components. I do root cause analysis, coordinate the fix with development and QA, and validate application upgrades and releases.

Right now I'm strengthening Kubernetes administration, AWS architecture and cloud-native observability, and I'm looking for a role where infrastructure and Kubernetes are a bigger part of the day-to-day."

1.7 The Full "Tell Me About Yourself" Answer

THE STRUCTURE BEHIND THE ANSWER

PRESENT (what I do now, 40%) -> PAST (how I got here, 30%) -> FUTURE (why this role, 20%) -> HOOK (one line inviting the next question, 10%). Always end with a hook so the interviewer's next question lands on ground you have prepared.

FULL-LENGTH "TELL ME ABOUT YOURSELF"

"Sure. I'm a DevOps Engineer with three years of experience, currently at I2V Systems in Gurgaon.

Day to day, I own the DevOps side of our application environments. That breaks into four things. First, CI/CD - I build and maintain Jenkins pipelines that take a commit from Git through build, test, packaging and deployment, and I also maintain GitHub Actions workflows. Second, infrastructure - I provision AWS with Terraform and configure servers and applications with Ansible, so environments are reproducible rather than hand-built. Third, containers and monitoring - I containerize components with Docker, support our Kubernetes deployments, and I implemented monitoring with Prometheus, Grafana and CloudWatch. Fourth, production support - I investigate incidents using logs, Linux commands, Windows services, database connectivity checks and network diagnostics, do RCA and coordinate fixes with development and QA.

Before this, I was at Naresh I Technology in Hyderabad from January 2024, doing the same core work at a smaller scope - Jenkins pipelines, Docker packaging, Terraform and Ansible automation, and AWS operations across EC2, S3, IAM, VPC, CloudWatch and Route 53.

My degree is a B.Tech in Mechanical Engineering. I moved into DevOps deliberately through structured training in AWS, Terraform, Ansible and CI/CD, because the systems-thinking part of engineering - repeatability and root-cause analysis - is what I enjoy, and DevOps is where that pays off daily.

What I'm looking for now is depth: more Kubernetes, more cloud architecture, and more ownership of reliability. That's why this role interested me. I'm happy to go deeper on any of it - the CI/CD pipeline design or a production incident are probably the most useful places to start."

1.8 How to Explain Your Project Without Exaggerating

Three rules that keep you safe under cross-examination:

1. **Describe mechanism, not magnitude.** "The pipeline builds the image, tags it with the build number and pushes it to the registry" is verifiable. "We handle 10,000 deployments a month" is not, and it is not on your CV.
2. **Use the right verb.** Your CV uses "built and maintain" for Jenkins, "provisioned" for Terraform, "support" for Kubernetes. Keep those verbs. "Support Kubernetes deployments" is honest; "architected our Kubernetes platform" is not supported by your CV.
3. **Separate what you did from what the system does.** "The system runs on AWS behind a load balancer" describes the system. "I wrote the Terraform modules that create the VPC, subnets and EC2 instances" describes you. Interviewers are buying the second one.

THREE SENTENCES THAT WILL GET YOU CAUGHT

- "I managed a 200-node Kubernetes cluster." Your CV says you support deployments and troubleshoot objects. A follow-up on etcd backups or node draining ends the round.
- "I designed our entire AWS architecture." Your CV says you provisioned infrastructure with Terraform and operated core services.
- "We had zero downtime for a year." You cannot prove it, and the next question is "what was your SLO and how did you measure it?"

PART 1 RECALL POINTS

- Lead with Jenkins CI/CD and production troubleshooting - both roles support them.
- Kubernetes: say "support and troubleshoot", not "administer".
- Mechanical Engineering is a deliberate, trained transition - three sentences, no apology.
- Refuse to invent metrics; offer mechanism instead.
- Introductions: WHO -> WHAT NOW -> STACK -> PROOF -> WHY HERE.
- Always end "tell me about yourself" with a hook that steers the next question.

PART 2

DEVOPS FUNDAMENTALS

The 28 concepts every DevOps interview opens with

Interviewers use fundamentals as a filter. They are not testing whether you can recite a definition - they are testing whether you understand the problem the practice solves. For each topic below you get a plain-language explanation, the technical explanation, a real-world analogy, how it shows up in a DevOps stack, interview questions with model answers, and the mistakes that cost marks.

2.1 What Is DevOps?

Simple explanation

DevOps is a way of working where the people who write software and the people who run it share one goal, one set of tools and one definition of "done". Instead of developers throwing a build over a wall and operations catching it, both sides own the path from commit to running system.

Technical explanation

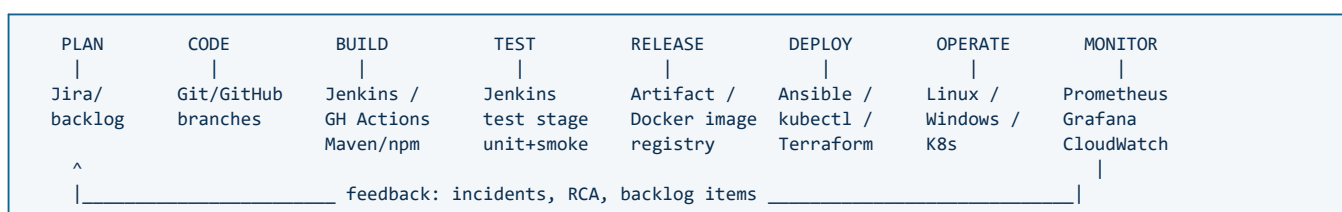
DevOps is a set of cultural practices and engineering automation that shortens the time between a change being committed and that change running safely in production, while keeping the system reliable. It rests on four pillars:

Pillar	What it means	Typical implementation
Culture	Shared ownership, blameless postmortems, developers on call or at least accountable for their service	Blameless RCA, shared on-call, common Slack/Teams channels
Automation	No manual step that a machine can repeat	Jenkins, GitHub Actions, Terraform, Ansible
Measurement	You cannot improve what you do not see	Prometheus, Grafana, CloudWatch, logs, DORA metrics
Sharing	Runbooks, code review, documentation, internal tooling	Git, pull requests, shared libraries, wikis

Real-world analogy

A restaurant where the chefs never speak to the waiters: food goes out cold, complaints come back to the kitchen days later, and nobody knows whose fault it is. DevOps is the open kitchen - one team, one pass, immediate feedback.

In a DevOps stack



The DevOps loop, mapped to the tools on your CV

THE ANSWER THAT SCORES

Do not open with "DevOps is a culture, not a tool." Interviewers hear that twenty times a day. Open with the problem: "DevOps exists because the handoff between development and operations was where releases broke. It replaces that handoff with automation and shared ownership - pipelines instead of tickets, infrastructure as code instead of manual builds, and monitoring so both sides see the same truth." Then name your tools.

Q. What is DevOps in your own words?

DevOps is the practice of removing the manual handoff between writing code and running it. Practically, it means every change goes through an automated pipeline - build, test, package, deploy - and the infrastructure it lands on is defined as code rather than built by hand, so environments are reproducible. It also means the team monitors what it ships, so

problems are found by dashboards and alerts rather than by users. In my current role that is Jenkins and GitHub Actions for the pipeline, Terraform and Ansible for the infrastructure, Docker and Kubernetes for packaging and running, and Prometheus, Grafana and CloudWatch for visibility.

Q. Is DevOps a role, a tool or a culture?

It is a way of working that is supported by tools. "DevOps Engineer" has become a role title because someone has to build and own the automation, but the practice only works if development, QA and operations share responsibility for the release. The tools are interchangeable - Jenkins or GitHub Actions, Terraform or CloudFormation - the practice is not.

Q. What problem was DevOps invented to solve?

The gap between development throughput and operations stability. Development is measured on change; operations is measured on not breaking. Left alone, those incentives conflict and releases become rare, large and risky. DevOps makes releases small, frequent and automated so that each one carries less risk.

COMMON MISTAKES ON THIS QUESTION

- Listing tools with no problem statement: "DevOps is Jenkins, Docker, Kubernetes." That is a toolchain, not a definition.
- Claiming DevOps means "no operations team".
- Saying "DevOps means continuous deployment to production" - most organisations, quite reasonably, stop at continuous delivery.

2.2 Why DevOps? The Business Case

Without DevOps	With DevOps	The mechanism that changes it
Releases monthly or quarterly, large and risky	Releases small and frequent	Automated build/test/deploy removes the cost of releasing
Environments differ; "works on my machine"	Environments reproducible	IaC and container images
Failures found by customers	Failures found by alerts	Monitoring and alerting
Recovery takes hours of manual work	Recovery is a rollback or redeploy	Versioned artifacts and automated deploy
Knowledge lives in one person's head	Knowledge lives in code and runbooks	Git, pipelines as code, documentation

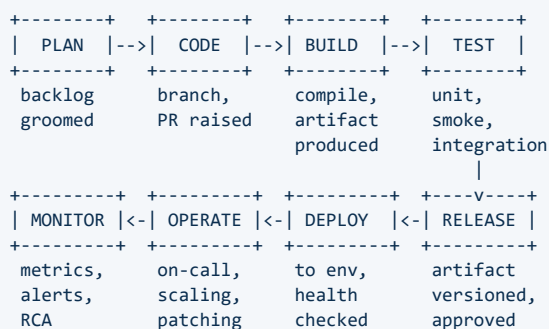
The four DORA metrics - know these by name

Metric	Question it answers	Direction you want
Deployment Frequency	How often do you ship to production?	Higher
Lead Time for Changes	Commit to running in production - how long?	Lower
Change Failure Rate	What percentage of deployments cause an incident?	Lower
Mean Time to Restore (MTTR)	How long to recover from a failure?	Lower

USING DORA SAFELY

Naming the four metrics shows maturity. Do not attach numbers to your own organisation unless you actually measured them. Safe phrasing: "We didn't formally track DORA metrics, but the pipeline work directly targets deployment frequency and MTTR - a deploy became a triggered job rather than a manual sequence, and a rollback became redeploying the previous artifact."

2.3 The DevOps Lifecycle



The eight phases and what "done" means at each one

Q. Walk me through the DevOps lifecycle.

Plan - the change is defined and prioritised. Code - the developer works on a branch and raises a pull request; review happens here. Build - CI compiles and packages the change into a versioned artifact, in our case usually a Docker image. Test - automated tests run against that artifact; nothing untested moves forward. Release - the artifact is tagged and stored in a registry and approved for an environment. Deploy - it is rolled out, in my experience through Jenkins calling Ansible or kubectl, followed by a health check. Operate - the system runs; we handle scaling, patching and configuration. Monitor - Prometheus, Grafana and CloudWatch show whether it is healthy, and anything wrong feeds back into the plan phase. The critical property is that the same artifact moves through all environments; you never rebuild per environment.

BUILD ONCE, DEPLOY MANY

The artifact tested in staging must be the exact artifact deployed to production - same image digest, same build number. If you rebuild for production, you have tested something that never shipped. This is a top-five interview answer and it is almost always a discriminator between a junior and a mid-level candidate.

2.4 CI vs CD - The Question You Will Definitely Be Asked

	Continuous Integration	Continuous Delivery	Continuous Deployment
Goal	Every commit is merged and verified continuously	Every verified build is always releasable	Every verified build is released automatically
Ends at	A tested artifact	An artifact approved and staged for release	Running in production
Human gate	No - automated on commit	Yes - a person clicks approve	No - fully automated
Fails when	Build or tests break	Release packaging or environment promotion breaks	Production deploy breaks
Typical tool usage	Jenkins/GH Actions on push and PR	Jenkins input step, environment approvals	Automated pipeline to prod with automated rollback

THE ONE-LINE ANSWER

"CI verifies every change works together. Continuous Delivery means every verified change is ready to release at any time, with a human deciding when. Continuous Deployment removes that human - it ships automatically. Most teams, including mine, run CI plus Continuous Delivery, because production releases need a business decision."

2.4.1 Continuous Integration in depth

CI means developers merge into the mainline frequently - at least daily - and every merge triggers an automated build and test run. Its purpose is to keep the integration cost near zero. Without CI, integration is a painful event at the end of a project; with CI it is a non-event that happens dozens of times a day.

CI requirement	Why it matters	What breaks without it
Single shared mainline	One source of truth	Long-lived branches drift and merge conflicts explode
Automated build on every push	Fast feedback	Broken code sits undiscovered for days
Automated tests in the pipeline	Build success means something	Green builds that ship bugs

CI requirement	Why it matters	What breaks without it
Fast pipeline (under about 10 minutes)	Developers wait for it	Developers stop waiting, batch changes, and CI value collapses
Broken build is the top priority	Keeps mainline releasable	The build stays red and everyone ignores it

2.4.2 Continuous Delivery in depth

Continuous Delivery extends CI: after a build passes, it is automatically packaged, versioned, published to an artifact store or registry, and deployed to a test or staging environment. Production release remains a decision, not an accident. The property you are claiming is "we could release any green build right now, in minutes, with a known rollback path."

2.4.3 Continuous Deployment in depth

Every change that passes the pipeline goes to production with no human approval. This requires very high test confidence, feature flags so unfinished work can ship dark, progressive rollout (canary), and automated rollback on error-rate regression. It is a legitimate goal but do not claim it unless you actually ran it.

Q. Does your organisation do continuous deployment?

No - we do continuous integration and continuous delivery. The pipeline builds, tests, packages and deploys automatically through the lower environments, and the production release is a controlled step that is approved and scheduled. That fits the product: releases are validated with QA before they go out. Continuous deployment would need broader automated test coverage and automated rollback on metric regression before I would recommend it.

2.5 Infrastructure as Code (IaC)

Simple explanation

Instead of clicking around a cloud console to create servers, you write a file that describes the servers you want, and a tool makes reality match the file. The file is in Git, so infrastructure has history, review and rollback like application code.

Technical explanation

IaC is the practice of defining infrastructure in machine-readable declarative files, applied by a tool that reconciles desired state with actual state. Key properties:

Property	Meaning	Why the interviewer cares
Declarative	You describe the end state, not the steps	Enables plan/diff and idempotency
Idempotent	Applying twice produces the same result	Safe to re-run; the core of automation
Versioned	Lives in Git	Review, audit trail, rollback
Reproducible	Same code produces the same environment	Kills environment drift
Reviewable	Changes go through PRs	Infrastructure changes get four eyes

YOUR IAC CLAIM

Your CV says: "Provisioned AWS infrastructure with Terraform (Infrastructure as Code)" and "Built reusable Terraform modules to provision AWS infrastructure, enabling repeatable environment creation and reducing manual configuration." Part 9 cross-questions this in full - especially modules, state and plan/apply.

Q. What is Infrastructure as Code and why does it matter?

It is defining infrastructure in code that a tool applies, rather than building it by hand. It matters for three reasons. First, reproducibility - I can create an identical environment from the same code instead of trying to remember what I clicked. Second, review and audit - changes go through Git and a pull request, and I can see who changed a security group and when. Third, safety - with Terraform I can run a plan and see exactly what will be created, changed or destroyed before anything happens. In my work that meant writing reusable Terraform modules so a new environment was a set of variable values rather than a manual build.

Q. What is the difference between declarative and imperative infrastructure?

Imperative says how: a script that runs create-instance, then attach-volume, then open-port, in that order. Declarative says what: this is the instance, the volume and the security group I want, and the tool works out the actions. Terraform and Kubernetes manifests are declarative; a Bash script using the AWS CLI is imperative. Declarative gives you idempotency and a diff before you act; imperative gives you fine control but you own the ordering and the failure handling.

2.6 Configuration Management

Configuration management is the layer that gets a provisioned machine into the state the application needs: packages installed, config files rendered, users created, services enabled and started.

	Provisioning (Terraform)	Configuration Management (Ansible)
Creates	Infrastructure - VPCs, EC2, security groups, S3	Nothing infrastructural; configures what exists
Model	Declarative with state file	Procedural tasks that are individually idempotent
Tracks state	Yes, in terraform.tfstate	No state file; checks reality per task each run
Agent needed	No - talks to cloud APIs	No - SSH (Linux) or WinRM (Windows)
Answer to remember	Terraform builds the house	Ansible furnishes it

THE SENTENCE THAT ENDS THIS QUESTION CLEANLY

"Terraform provisions the infrastructure; Ansible configures what runs on it. I use Terraform to create the VPC, security groups and EC2 instances, and Ansible to install packages, push configuration files and manage services on those instances. They are complementary, not competing."

2.7 Containerization

Simple explanation

A container packages an application together with everything it needs to run - libraries, runtime, config - so it behaves the same on a laptop, a test server and production.

Technical explanation

A container is a process (or process group) on the host kernel, isolated using Linux kernel features: namespaces for isolation (PID, network, mount, UTS, IPC, user) and cgroups for resource limits (CPU, memory, I/O). It is not a virtual machine: there is no guest kernel and no hardware emulation.

	Virtual Machine	Container
Isolation boundary	Hypervisor, full guest OS	Kernel namespaces and cgroups
Kernel	Its own	Shares the host kernel
Size	Gigabytes	Megabytes
Start time	Tens of seconds to minutes	Under a second typically
Density per host	Tens	Hundreds
Isolation strength	Stronger	Weaker - kernel is shared
Best for	Mixed OS, strong isolation, legacy	Microservices, CI, elastic scaling

DOCKER VS VM - THE EXAM ANSWER

"A VM virtualises hardware and runs its own kernel; a container virtualises the operating system and shares the host kernel. That is why containers start in under a second and a VM takes minutes, and it is also why containers give weaker isolation - a kernel vulnerability is shared."

2.8 Orchestration

Running one container is easy. Running hundreds across many machines, keeping the right number alive, replacing failures, rolling out new versions without downtime and giving them stable network names is orchestration - and that is Kubernetes' job.

Problem when you scale containers	What the orchestrator provides
Which machine should this container run on?	Scheduling based on requests, taints, affinity
A container died	Self-healing - the controller recreates it

Problem when you scale containers	What the orchestrator provides
I need 10 copies, not 1	Replicas managed by a Deployment/ReplicaSet
Container IPs keep changing	Service - a stable virtual IP and DNS name
Deploy a new version without downtime	Rolling update with readiness gates
Configuration and passwords	ConfigMaps and Secrets
Traffic from outside the cluster	Ingress and LoadBalancer Services

2.9 Monitoring

Simple explanation

Monitoring is watching known signals and being told when they cross a line you care about.

Technical explanation

Monitoring collects time-series metrics from systems and applications, stores them, evaluates rules against them and raises alerts. It answers predefined questions: Is CPU high? Is the error rate above 1%? Are pods restarting?

The four golden signals (SRE)	What it measures	Example metric
Latency	How long requests take - separate success from failure	p95 request duration
Traffic	Demand on the system	requests per second
Errors	Rate of failing requests	rate of HTTP 5xx
Saturation	How full the system is	CPU%, memory%, queue depth, disk%

The USE method (for resources)	Question
Utilisation	What percentage of the resource is busy?
Saturation	How much queued work is waiting?
Errors	Is the resource reporting errors?

2.10 Observability

Monitoring answers questions you already knew to ask. Observability is the property of a system that lets you ask new questions about it without shipping new code - essential when the failure is one you have never seen before.

Pillar	What it is	Tooling in your stack
Metrics	Numeric time series, cheap, aggregatable, low cardinality	Prometheus, CloudWatch metrics
Logs	Discrete timestamped events with detail	Application logs, journald, CloudWatch Logs
Traces	The path of one request across services with timing per hop	Jaeger, X-Ray (study topic, not on your CV)

MONITORING VS OBSERVABILITY, SAID WELL

"Monitoring tells me the error rate is up. Observability is what lets me find out why - which endpoint, which customer, which dependency, which deploy. Metrics tell me something is wrong, logs tell me what happened, and traces tell me where in the request path it happened."

2.11 Logging

Practice	Why	Anti-pattern
Structured logs (JSON or key=value)	Machine-parseable, filterable	Free-text logs that need regex archaeology
Correlation/request ID on every line	Follow one request across services	No way to join logs across components

Practice	Why	Anti-pattern
Consistent levels: ERROR, WARN, INFO, DEBUG	Signal-to-noise control	Everything logged at ERROR
Log rotation	Logs are the number one cause of disk-full incidents	Unrotated logs filling the root partition
Never log secrets, tokens, card data or full PII	Compliance and breach risk	Passwords in application logs
Centralise	You cannot grep 50 machines	SSH to each server to read logs

LOGS ARE THE NUMBER ONE DISK-FULL CAUSE

When a filesystem fills, check log directories first: `du -sh /var/log/* | sort -h | tail`. If a file was deleted but space did not return, a process still holds the handle - find it with `lsof | grep deleted` and restart that process. Fix the cause with logrotate, do not just delete the file.

2.12 Alerting

Rule	Reason
Alert on symptoms, not causes	"Error rate above 2%" is actionable; "CPU 80%" often is not
Every alert must be actionable	If nobody does anything, it is a dashboard, not an alert
Every alert must link a runbook	3am is not the time to invent a procedure
Use for durations to avoid flapping	A 30-second CPU spike is not an incident
Severity tiers: page vs ticket	Only page for things that need a human now
Review alert noise regularly	Alert fatigue is how real incidents get missed

Prometheus alert rule - symptom-based, with a wait window and a runbook link

```
groups:
- name: application-availability
  rules:
  - alert: HighHTTP5xxRate
    expr: |
      sum(rate(http_requests_total{status=~"5.."}[5m]))
      / sum(rate(http_requests_total[5m])) > 0.02
    for: 5m
    labels:
      severity: critical
    annotations:
      summary: "5xx error rate above 2% for 5 minutes"
      runbook: "https://wiki.internal/runbooks/http-5xx"
```

2.13 Incident Management

DETECT	-->	TRIAGE	-->	COMMUNICATE	-->	MITIGATE	-->	RESOLVE	-->	RCA	-->	PREVENT
alert,		severity,		status update,		restore		confirm		5 whys,		action items
user		is it		stakeholders		service		healthy		timeline		with owners
report		real?		informed		FIRST						and dates

Incident lifecycle

MITIGATE BEFORE YOU DIAGNOSE

In a live incident, restoring service comes before understanding the cause. Roll back, restart, fail over, scale out - whatever restores users fastest - and preserve the evidence (logs, a copy of the failing container, a heap dump) so RCA is still possible afterwards. Candidates who insist on root-causing a live outage before mitigating are marked down.

Severity	Typical definition	Response
SEV1	Complete outage or data loss; all users affected	Immediate page, incident channel, exec comms
SEV2	Major feature broken or severe degradation	Page during business hours, urgent fix

Severity	Typical definition	Response
SEV3	Minor degradation, workaround available	Ticket, next business day
SEV4	Cosmetic or low impact	Backlog

2.14 Root Cause Analysis (RCA)

YOUR RCA CLAIM

Your CV: "perform RCA, coordinate fixes with development and QA, and validate application upgrades and releases." Expect the interviewer to ask you to walk through one. Part 16 and Part 17 give you the structures; Part 20 gives you the story form.

The RCA document structure

Section	Content
Summary	Two sentences: what broke, for how long, who was affected
Impact	Duration, users/services affected, business effect - factual only
Timeline	Timestamped: first symptom, detection, actions taken, resolution
Root cause	The actual technical cause, not "human error"
Detection	How we found out - and how long that took
Mitigation	What restored service
Action items	Preventive changes with an owner and a date

The 5 Whys, applied properly

Level	Question	Example answer
Why 1	Why did the application return 502?	The application container was not responding.
Why 2	Why was it not responding?	It had been OOM-killed by the kernel.
Why 3	Why was it OOM-killed?	Its memory limit was lower than its actual working set under load.
Why 4	Why was the limit too low?	The limit was copied from a different service and never load-tested.
Why 5	Why did nobody notice?	There was no alert on container restarts or on memory approaching the limit.

Note the two action items that fall out: set limits from measured usage, and alert on restarts and memory saturation. A root cause with no action item is not a root cause.

RCA ANSWERS THAT LOSE MARKS

- "Root cause: human error." Humans err; the system that allowed the error to reach production is the root cause.
- Naming a person. RCAs are blameless - name systems and processes.
- Stopping at the first why. "The server ran out of disk" is a symptom; why did nothing alert at 80%, and why did logs grow unbounded?

2.15 Rollback

Rollback method	How it works	Speed	Risk
Redeploy previous artifact	Deploy the previous image tag or build number	Fast	Needs immutable, versioned artifacts
<code>kubectl rollout undo</code>	Kubernetes reverts to the previous ReplicaSet	Seconds	Config/secret changes are not reverted
Blue-Green switch back	Point the load balancer back to blue	Near instant	Needs double capacity

Rollback method	How it works	Speed	Risk
Canary abort	Stop the rollout, shift traffic back to stable	Fast	Needs traffic-splitting support
Database rollback	Restore or run a down-migration	Slow and risky	Often impossible without data loss

THE DATABASE IS WHY ROLLBACK IS HARD

Application rollback is easy; schema rollback usually is not. The standard practice is expand-and-contract: deploy a backward-compatible schema change first (add the column, keep the old one), ship application code that works with both, and only drop the old column in a later release once no running version needs it. Say this sentence in an interview and you will stand out.

2.16 Deployment Strategies

2.16.1 Recreate (big bang)

Stop all old instances, start new ones. Simple, causes downtime, and is still the right choice for some stateful or single-instance components.

2.16.2 Rolling deployment

```

Start:  [v1][v1][v1][v1]
Step 1: [v2][v1][v1][v1]    <- one pod replaced, must pass readiness before continuing
Step 2: [v2][v2][v1][v1]
Step 3: [v2][v2][v2][v1]
Done:   [v2][v2][v2][v2]
Controlled in Kubernetes by maxSurge (extra pods allowed) and maxUnavailable (pods allowed down)

```

Rolling update - replace instances in waves

Pros: no extra infrastructure, no downtime, default in Kubernetes. Cons: two versions run simultaneously, so the application and its database schema must be backward compatible; rollback takes as long as the rollout did.

2.16.3 Blue-Green deployment

```

      +-----+
Users --> | Load Balancer |
      +-----+
          | 100% traffic
          +-----V-----+
          | BLUE (v1) |    | GREEN (v2) |
          | LIVE   |    | idle, tested |
          +-----+      +-----+
Cutover = repoint the load balancer to GREEN. Rollback = repoint to BLUE.

```

Blue-Green - two full environments, one live

Pros: instant switch, instant rollback, test green with production-like traffic before cutover. Cons: double the infrastructure cost during the window; shared databases still need compatible schemas; long-lived sessions can break at cutover.

2.16.4 Canary deployment

```

Users
|
+--V-----+
| Load Balancer |
+--V-----+
| 95%          | 5%
+--V-----+   +--V-----+   Watch: error rate, p95 latency, restarts
| v1 |         | v2 |       Healthy -> 25% -> 50% -> 100%
| stable |     | canary |    Unhealthy -> route 100% back to v1
+-----+     +-----+

```

Canary - a small slice of real traffic on the new version

Pros: the smallest possible blast radius; real traffic validation. Cons: needs traffic splitting (ingress controller, service mesh or weighted target groups) and good metrics - a canary without metrics is just a slow deployment.

Strategy	Downtime	Extra cost	Rollback speed	Blast radius	Complexity
Recreate	Yes	None	Redeploy old	All users	Lowest
Rolling	No	Small (surge)	Slow (reverse rollout)	Growing during rollout	Low
Blue-Green	No	2x during window	Instant	All users at once after cutover	Medium
Canary	No	Small	Fast	Smallest	Highest

CHOOSING A STRATEGY OUT LOUD

"It depends on the failure cost and what the platform supports. Kubernetes gives me rolling updates by default with readiness probes, and that covers most services. If a release is high risk and I need instant rollback, blue-green is better because the switch is a load balancer change. Canary is the safest but it needs traffic splitting and good metrics to decide whether the canary is healthy."

2.17 Immutable Infrastructure

Servers are never modified in place. To change anything - a package, a config file, the application version - you build a new image and replace the instance. The opposite, mutable infrastructure, produces snowflake servers: machines that have accumulated years of undocumented manual changes and cannot be rebuilt.

Mutable (pets)	Immutable (cattle)
SSH in and patch the running server	Build a new AMI or container image and replace
Configuration drift accumulates silently	Drift is impossible - instances are replaced, not edited
"Do not touch that server, only Ravi knows how it was set up"	Any instance can be destroyed and rebuilt identically
Rollback means undoing manual changes	Rollback means deploying the previous image

Containers are inherently immutable - you do not patch a running container, you redeploy a new image. That is a good example to use when explaining the concept.

2.18 Automation

Automate when	Do not automate when
The task is repeated	It is genuinely one-off and low risk
The task is error-prone by hand	The process is still changing weekly
It must run out of hours	Automating costs more than the task ever will
It must be auditable	A human judgement call is required
More than one person must be able to run it	The blast radius of a bug is unbounded and untested

AUTOMATION PHILOSOPHY ANSWER

"My rule is that anything I do twice manually gets scripted, and anything that touches production gets a dry-run mode and logging before it gets a schedule. Automation without visibility just makes mistakes faster."

2.19 DevSecOps Basics

Shift security left - make it part of the pipeline rather than a gate at the end.

Stage	Security control	Example tooling
Code	Secret scanning, pre-commit hooks	git-secrets, gitleaks
Build	SAST - static analysis of source	SonarQube, Bandit for Python
Dependencies	SCA - known vulnerable libraries	Trivy, Dependabot, npm audit
Image	Container image scanning	Trivy, Clair, ECR scan on push
IaC	Terraform policy scanning	tfsec, Checkov

Stage	Security control	Example tooling
Runtime	Least privilege, network policy, secrets management	IAM roles, K8s NetworkPolicy, Vault, AWS Secrets Manager

SECRETS - THE THREE RULES

1) Never commit a secret to Git - once pushed, treat it as compromised and rotate it, because it stays in history. 2) Inject secrets at runtime from a secret store (Jenkins credentials, AWS Secrets Manager, Kubernetes Secrets), never bake them into an image. 3) Use IAM roles instead of long-lived access keys wherever the platform supports it.

2.20 SRE Basics

Term	Definition	Example
SLI - Service Level Indicator	The measurement	Proportion of HTTP requests returning under 300ms with status under 500
SLO - Service Level Objective	The internal target	99.9% of requests meet the SLI over 30 days
SLA - Service Level Agreement	The contract with a customer, with penalties	99.5% uptime or service credits apply
Error budget	100% minus the SLO - the allowed failure	99.9% = about 43 minutes of failure per 30 days
Toil	Manual, repetitive, automatable operational work	Manually restarting a service every morning

THE ERROR BUDGET ANSWER

"The error budget turns reliability into a shared decision. If the service is inside its budget, the team can keep shipping features; if the budget is exhausted, the next work item is reliability, not features. It stops reliability from being an argument and makes it a number."

2.21 High Availability, Scalability, Reliability

Concept	Definition	How you achieve it
High Availability	The system keeps serving despite component failure	Redundancy across AZs, load balancing, health checks, automated failover, no single point of failure
Scalability	The system handles more load by adding resources	Horizontal scaling behind a load balancer, autoscaling groups, HPA in Kubernetes, stateless application design
Reliability	The system does the right thing consistently over time	Testing, gradual rollouts, monitoring, rollback, capacity planning, RCA that closes loops
Fault tolerance	The system continues correctly even while a component is failing	Retries with backoff, timeouts, circuit breakers, graceful degradation
Disaster recovery	Recovering from catastrophic loss	Backups tested by restore, cross-region copies, RTO/RPO targets

Horizontal scaling	Vertical scaling
Add more instances	Make the instance bigger
Near-linear, effectively unbounded	Bounded by the largest instance type
Usually no downtime	Usually requires a restart or resize
Requires stateless application design	Works for stateful software such as databases
Example: ASG from 2 to 10 EC2 instances; HPA 3 to 20 pods	Example: t3.medium to m5.2xlarge

Availability target	Downtime per month	Downtime per year
99%	about 7h 18m	about 3d 15h
99.9% ("three nines")	about 43m	about 8h 45m
99.99% ("four nines")	about 4m 23s	about 52m

Availability target	Downtime per month	Downtime per year
99.999% ("five nines")	about 26s	about 5m 15s

THE HA TRAP QUESTION

"We have two web servers, so we are highly available." Not necessarily. If both sit in one Availability Zone, behind one load balancer node, talking to a single database instance, then the AZ and the database are still single points of failure. HA is a property of the whole path, not of one tier. Always trace the request end to end and name every component that exists only once.

PART 2 RECALL POINTS

- DevOps = removing the handoff; four pillars are culture, automation, measurement, sharing.
- DORA: deployment frequency, lead time, change failure rate, MTTR.
- Build once, deploy many - the same artifact moves through every environment.
- CI verifies; Continuous Delivery makes it releasable; Continuous Deployment ships automatically.
- Terraform provisions, Ansible configures.
- Container shares the host kernel; a VM brings its own.
- Golden signals: latency, traffic, errors, saturation.
- Mitigate first, RCA after - and always preserve evidence.
- Rollback is easy for code, hard for schemas - expand and contract.
- Rolling = default, Blue-Green = instant switch, Canary = smallest blast radius.
- SLI measures, SLO targets, SLA contracts, error budget is what is left.

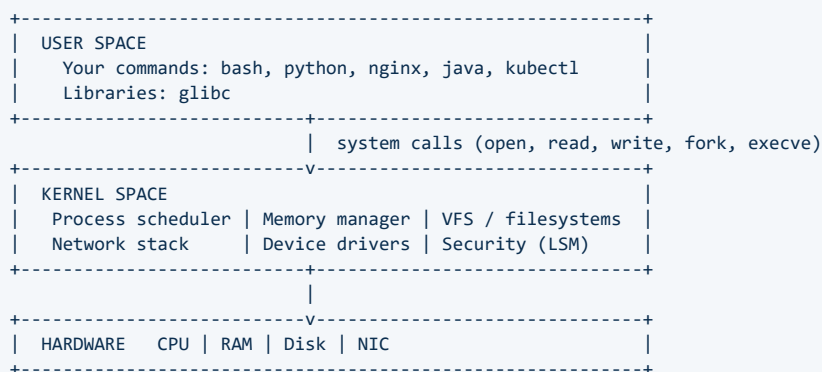
PART 3

LINUX COMPLETE INTERVIEW GUIDE

Architecture, commands, permissions, processes, disk, memory, logs, networking, scripting

Linux is the highest-yield subject in a DevOps interview because it is where every other topic bottoms out. Your CV claims Linux and Windows troubleshooting - service failures, disk utilization, network connectivity and application logs. This part makes that claim defensible under pressure. Every command here is one you should run at least once before your interview.

3.1 Linux Architecture



From your command to the hardware

Layer	Responsibility	Interview one-liner
Kernel	Owens hardware, schedules processes, manages memory, implements filesystems and the network stack	"The kernel is the only thing that talks to hardware; everything else asks it through system calls."
Shell	Interprets commands, expands variables and globs, manages jobs and redirection	"The shell is a program that turns what I type into process execution."
User space	Every program you run, including systemd	"Nothing in user space touches hardware directly."
System calls	The kernel's API	"strace shows me exactly which system calls a process makes - it is how I debug 'permission denied' with no useful log."

3.1.1 The boot sequence - a frequent question

Step	What happens	Where it breaks
BIOS/UEFI	Power-on self-test, finds the boot device	Wrong boot order, failed disk
Bootloader (GRUB)	Loads the kernel and initramfs, passes kernel parameters	Corrupt GRUB config after a kernel update
Kernel	Initialises hardware, mounts the root filesystem	Missing driver in initramfs, wrong root UUID
init - systemd (PID 1)	Starts units to reach the default target	A failed unit, or a bad /etc/fstab entry blocking boot
Targets	multi-user.target (CLI) or graphical.target	Wrong default target
Login	getty or SSH	PAM or SSH config errors

3.2 The Filesystem Hierarchy

Path	Purpose	DevOps relevance
/	Root of everything	Filling this partition takes the system down

Path	Purpose	DevOps relevance
/etc	System-wide configuration	nginx, ssh, fstab, systemd unit overrides
/var	Variable data - logs, spool, caches	/var/log is the usual disk-full culprit
/var/log	Log files	First place you look in an incident
/home	User home directories	SSH keys in ~/.ssh
/opt	Optional/third-party application software	Where vendor applications often land
/usr	User binaries and libraries	/usr/bin, /usr/local/bin
/bin, /sbin	Essential and system binaries (symlinks to /usr on modern distros)	
/tmp	Temporary files, world-writable, cleared on reboot	Often a separate small filesystem; builds fill it
/proc	Virtual filesystem - kernel and process information	/proc/PID/, /proc/meminfo, /proc/cpuinfo
/sys	Virtual filesystem - devices and kernel objects	Tuning and device info
/dev	Device files	/dev/null, /dev/sda1, /dev/urandom
/mnt, /media	Mount points	Attached EBS volumes, NFS

EVERYTHING IS A FILE

Files, directories, devices, sockets and pipes are all accessed through the same file API. That is why `>/dev/null` discards output, why you can `cat /proc/meminfo`, and why `lsof` (list open files) is the tool for "which process is holding this deleted log file".

3.3 Users, Groups, Ownership

File	Contents	Note
/etc/passwd	username:x:UID:GID:comment:home:shell	World-readable; x means the password is in shadow
/etc/shadow	Hashed passwords and ageing policy	Root-only
/etc/group	Group name, GID, member list	
/etc/sudoers	Who may run what as root	Edit only with <code>visudo</code> - a syntax error can lock out sudo

User and group management

```
id aasim           # UID, GID and all group memberships
whoami            # current effective user
groups aasim      # groups this user belongs to
sudo useradd -m -s /bin/bash deploy    # create user with home dir and bash shell
sudo passwd deploy      # set password
sudo usermod -aG docker deploy    # ADD to the docker group (-a is critical)
sudo userdel -r olduser    # delete user and home directory
sudo groupadd appops
last -n 10              # recent logins
sudo chage -l deploy    # password ageing information
```

USERMOD WITHOUT -A

`sudo usermod -G docker deploy` REPLACES all of the user's secondary groups with just `docker`, silently removing them from `sudo`, `wheel` and everything else. Always use `-aG` to append. This is a classic interview trap - know it.

3.4 Permissions and Ownership

```
-rwxr-x--- 1 deploy appops 8432 Sep 23 10:14 deploy.sh
```

	permissions	owner	group	size	date	time	filename
	rwxr-x---	deploy	appops	8432	Sep 23	10:14	deploy.sh
							+-- filename
							+----- last modified
							+----- size in bytes
							+----- group owner
							+----- user owner
							+----- hard link count
	+++-----						OTHER = --- (0)
	+++-----						GROUP = r-x (5)
	+++-----						USER = rwx (7)
	first character:	- file	d directory	l symlink	b/c device	s socket	

Reading ls -l output

Permission	On a file	On a directory	Numeric
r (read)	Read contents	List names in it	4
w (write)	Modify contents	Create, delete, rename entries inside it	2
x (execute)	Run it as a program	Enter it / access items through it	1

THE DIRECTORY PERMISSION TRAP

Deleting a file depends on write permission on the DIRECTORY, not on the file. You can delete a read-only file from a directory you can write to. And without x on a directory you cannot access anything inside it even if you can list the names with r. Interviewers love this one.

Numeric	Symbolic	Typical use
755	<code>rxwxr-xr-x</code>	Executables, directories
644	<code>rw-r--r--</code>	Normal config and data files
600	<code>rw-----</code>	Private keys, credential files
700	<code>rxw-----</code>	<code>~/ .ssh</code> directory
640	<code>rw-r-----</code>	Config readable by an application group
777	<code>rxwxrwxrwx</code>	Almost always wrong - a red flag in an interview

chmod, chown, chgrp and the umask

```

chmod 755 deploy.sh                # numeric
chmod u+x,g-w deploy.sh           # symbolic
chmod -R 750 /opt/app              # recursive - be careful on large trees
chmod o-rwx /etc/app/secrets.conf # remove all access for others

chown deploy:appops /opt/app/config.yml # change user and group
chown -R deploy /opt/app                # recursive owner change
chgrp appops /opt/app/shared            # change only the group

umask                # shows the default-permission mask, typically 0022
# umask 022 -> new files 666-022 = 644, new directories 777-022 = 755

```

3.4.1 Special bits - SUID, SGID, sticky

Bit	Nu mer ic	On what	Effect	Example
SUID	400	Executable	Runs with the FILE OWNER's privileges, not the caller's	<code>/usr/bin/passwd</code> runs as root so a user can edit <code>/etc/shadow</code>
SGID	200	Executable	Runs with the file's group privileges	<code>/usr/bin/wall</code>
SGID	200	Directory	New files inside inherit the directory's group	Shared team directories
Sticky	100	Directory	Only the file owner (or root) can delete files inside	<code>/tmp</code> - mode 1777

Setting and finding special bits

```

chmod 4755 /usr/local/bin/tool      # SUID -> shows as rwsr-xr-x
chmod 2775 /srv/shared              # SGID -> shows as rwxrwsr-x
chmod 1777 /srv/upload              # sticky-> shows as rwxrwxrwt

# Security audit: find unexpected SUID binaries (a standard hardening check)
sudo find / -perm -4000 -type f -exec ls -l {} \; 2>/dev/null

```

A LOWERCASE S MEANS THE EXECUTE BIT IS ALSO SET; AN UPPERCASE S MEANS SUID/SGID IS SET BUT EXECUTE IS NOT - USUALLY A MISTAKE. SAME FOR T VERSUS T ON THE STICKY BIT.

3.5 Core File and Text Commands

3.5.1 Navigation and listing

pwd, ls, cd

```

pwd                # print working directory
ls -l              # long listing
ls -lah            # long, all (incl. hidden), human-readable sizes
ls -lt            # newest first - what changed most recently
ls -ltr            # oldest first, newest at the bottom - best for log directories
ls -ld /opt/app    # show the DIRECTORY itself, not its contents
ls -li            # show inode numbers
cd /var/log        # absolute path
cd ../../         # up two levels
cd -               # back to the previous directory
cd                # home directory

```

3.5.2 Creating, copying, moving, deleting

mkdir, touch, cp, mv, rm

```

mkdir -p /opt/app/config/env      # -p creates parents and never errors if it exists
touch app.log                     # create empty file, or update its timestamp
cp app.conf app.conf.bak         # copy
cp -r /opt/app /opt/app-backup   # recursive copy of a directory
cp -p file dest                  # preserve mode, ownership and timestamps
cp -a /src /dst                  # archive: recursive + preserve everything (best for backups)
mv old.log archive/              # move
mv app.conf app.conf.old         # rename (same command)
rm file.txt                      # recursive
rm -r directory/                 # recursive + force: no prompt, no error if missing
rm -rf directory/                # interactive - prompt per file
rm -i *.log

```

RM -RF DISCIPLINE

Never type `rm -rf` with a variable you have not printed first. `rm -rf "$DIR"/` with an empty `$DIR` targets `.`. Rules: always quote variables, always echo the command before running it in a script, add `set -u` to your scripts so unset variables abort, and prefer moving to a `.trash` directory in automation rather than deleting.

3.5.3 Viewing file content

cat, less, head, tail

```

cat app.conf          # whole file to stdout
cat -n app.conf        # with line numbers
cat f1 f2 > combined.txt # concatenate
less /var/log/syslog   # pager: /search n next G end g start q quit F follow
head -n 20 access.log  # first 20 lines
tail -n 50 access.log  # last 50 lines
tail -f /var/log/app/app.log # FOLLOW - live tail, the incident-response default
tail -F /var/log/app/app.log # follow by name - survives log rotation (use this one)
tail -n 200 app.log | less # last 200 lines in a pager

```

TAIL -F VERSUS TAIL -F: LOWERCASE -F FOLLOWS THE FILE DESCRIPTOR, SO WHEN LOGROTATE MOVES THE FILE YOU ARE SILENTLY WATCHING A FILE NOBODY WRITES TO ANY MORE. UPPERCASE -F RE-OPENS BY NAME. IN PRODUCTION ALWAYS USE -F.

3.5.4 grep - the most important command in an incident

grep essentials

```
grep "ERROR" app.log           # basic match
grep -i "error" app.log        # case-insensitive
grep -r "database" /etc/app/    # recursive through a directory
grep -n "timeout" app.log       # show line numbers
grep -v "DEBUG" app.log         # INVERT - everything except DEBUG
grep -c "ERROR" app.log         # count matching lines
grep -w "test" file             # whole word only (not "latest")
grep -A 5 "Exception" app.log    # 5 lines After the match - the stack trace
grep -B 3 "Exception" app.log    # 3 lines Before - what led to it
grep -C 4 "Exception" app.log    # 4 lines of Context either side
grep -E "ERROR|FATAL|CRITICAL" app.log # extended regex alternation
grep -l "apikey" -r /opt/app     # list only the FILENAMES that match
grep -o "[0-9]\{1,3\}\.[0-9]\{1,3\}\.[0-9]\{1,3\}" access.log # print only the match
zgrep "ERROR" app.log.2.gz       # grep inside gzipped rotated logs
```

The incident one-liners you should have memorised

```
# Top 10 IPs hitting the service
awk '{print $1}' access.log | sort | uniq -c | sort -rn | head -10

# Count of each HTTP status code
awk '{print $9}' access.log | sort | uniq -c | sort -rn

# All 5xx responses in the last 1000 requests
tail -1000 access.log | awk '$9 ~ /^5/ {print $0}'

# Errors in the last hour only (log timestamps like 2026-09-23T14:)
grep "$(date -u +%Y-%m-%dT%H)" app.log | grep -c ERROR

# Which error messages are most frequent
grep ERROR app.log | awk -F'ERROR' '{print $2}' | sort | uniq -c | sort -rn | head
```

3.5.5 awk - field processing

awk in practice

```
awk '{print $1, $7}' access.log # print fields 1 and 7 (space-separated by default)
awk -F: '{print $1}' /etc/passwd # custom field separator - all usernames
awk '$3 > 1000 {print $1}' /etc/passwd # conditional: only rows where field 3 exceeds 1000
awk -F: '$3 >= 1000 {print $1, $6}' /etc/passwd # real users and their home directories
awk '{sum += $10} END {print sum}' access.log # sum bytes column
awk '{sum+=$10; n++;} END {print sum/n}' access.log # average
awk 'NR==5,NR==10' app.log # print lines 5 to 10
awk 'length > 200' app.log # lines longer than 200 characters
df -h | awk '$5+0 > 80 {print $6, $5}' # filesystems over 80% used - a monitoring one-liner
```

3.5.6 sed - stream editing

sed in practice

```
sed 's/old/new/' file          # replace FIRST occurrence on each line
sed 's/old/new/g' file         # replace ALL occurrences
sed -i 's/8080/9090/g' app.conf # edit the file IN PLACE - no output, file changed
sed -i.bak 's/8080/9090/g' app.conf # in place, keeping app.conf.bak - always do this in production
sed -n '10,20p' app.log        # print only lines 10-20 (-n suppresses default printing)
sed '/^#/d' app.conf            # delete comment lines
sed '/^$/d' app.conf            # delete blank lines
sed -n '/ERROR/,/END/p' app.log # print from an ERROR line to the next END line
sed 's/password=.*password=REDACTED/' app.log # redact before sharing a log
```

SED -I ON A PRODUCTION CONFIG WITH NO BACKUP AND NO DIFF. DO IT AS: COPY THE FILE, EDIT THE COPY, DIFF THE TWO, THEN MOVE IT INTO PLACE - OR AT MINIMUM USE SED -I.BAK. BEING ABLE TO UNDO IS THE DIFFERENCE BETWEEN AN EDIT AND AN INCIDENT.

3.5.7 sort, uniq, cut, wc, tr

Text pipeline building blocks

```
sort names.txt                # alphabetical
```

```

sort -n numbers.txt          # numeric
sort -rn numbers.txt         # numeric, descending - the usual "top N" sort
sort -k2 -n data.txt         # sort by the 2nd field, numerically
sort -u list.txt             # sort and remove duplicates
sort -h                     # human-numeric: understands 1K, 200M, 3G (pairs with du -h)

uniq file.txt                # remove ADJACENT duplicates - always sort first
sort file.txt | uniq -c       # count occurrences
sort file.txt | uniq -d       # only duplicated lines
sort file.txt | uniq -u       # only unique lines

cut -d: -f1 /etc/passwd      # field 1, colon-delimited
cut -d, -f2,5 data.csv       # fields 2 and 5 of a CSV
cut -c1-10 file.txt          # characters 1 to 10

wc -l app.log                # line count
wc -c file                   # bytes
grep -c ERROR app.log        # faster than grep ERROR | wc -l

tr 'a-z' 'A-Z' < file        # translate to uppercase
tr -d '\r' < dos.txt > unix.txt # strip Windows carriage returns - a real DevOps fix
tr -s ' '                    # squeeze repeated spaces into one

```

3.5.8 find - locate and act

find, the swiss army knife

```

find /var/log -name "*.log"          # by name
find /var/log -iname "*.LOG"         # case-insensitive
find /opt -type f -size +100M        # files larger than 100 MB
find /opt -type d -name "cache"      # directories only
find /var/log -mtime +30             # modified more than 30 days ago
find /var/log -mmin -60              # modified in the last 60 minutes
find / -perm -4000 -type f 2>/dev/null # SUID binaries
find /opt/app -user deploy           # owned by a user
find /tmp -type f -mtime +7 -delete  # delete files older than 7 days
find /var/log -name "*.log" -mtime +30 -exec gzip {} \; # act on each match
find /var/log -name "*.gz" -mtime +90 -print0 | xargs -0 rm -f # safe with spaces in names
find . -type f -newer reference.txt  # changed since a reference file - great for "what did that
install touch?"

locate nginx.conf                # instant, but from a database - run `sudo updatedb` if results look stale
which kubect1                    # path of the executable that would run
type -a python                   # shows aliases, functions and all matching binaries

```

3.5.9 xargs, tee, echo, printf

Gluing commands together

```

# xargs converts stdin into ARGUMENTS for another command
cat servers.txt | xargs -I {} ssh {} "uptime"
find . -name "*.tmp" | xargs rm -f
find . -name "*.log" -print0 | xargs -0 grep -l "ERROR" # null-safe with odd filenames
cat urls.txt | xargs -P 5 -I {} curl -s -o /dev/null -w "%{http_code} {}\n" {} # 5 in parallel

# tee writes to a file AND passes output on
./deploy.sh 2>&1 | tee -a /var/log/deploy.log
echo "net.core.somaxconn=1024" | sudo tee -a /etc/sysctl.conf # the way to append as root

echo "Deploy started at $(date)"
echo -n "no trailing newline"
printf "%-20s %s\n" "Service:" "running" # printf gives you formatting; echo does not
printf "Disk used: %d%\n" 87

```

SUDO ECHO "X" >> /ETC/FILE DOES NOT WORK - THE REDIRECTION IS PERFORMED BY YOUR SHELL, WHICH IS NOT ROOT. USE `ECHO "X"``

3.6 Processes

Term	Meaning
PID	Process ID - unique identifier
PPID	Parent process ID
UID	User the process runs as
Zombie	Finished but its parent has not read its exit status; shows as Z, cannot be killed - fix the parent
Orphan	Parent died; re-parented to PID 1 (systemd)
Daemon	Long-running background service, usually managed by systemd
Process state	R running, S interruptible sleep, D uninterruptible sleep (usually I/O - a red flag), Z zombie, T stopped

Inspecting processes

```
ps aux                # every process, BSD style (the one to memorise)
ps -ef               # every process, System V style
ps aux | grep [n]ginx # the bracket trick excludes the grep itself
ps -ef --forest      # process tree with parent/child relationships
ps -eo pid,ppid,user,%cpu,%mem,etime,cmd --sort=-%cpu | head # top CPU consumers
ps -eo pid,rss,cmd --sort=-rss | head # top memory consumers (RSS in KB)
pgrep -a java        # PIDs matching a name, with the command line
pstree -p            # visual process tree
top                  # live: P sort by CPU, M sort by memory, 1 per-core, k kill, q quit
htop                 # friendlier top, if installed - F6 sort, F9 kill
```

Signals and killing

```
kill 1234            # SIGTERM (15) - polite: asks the process to clean up and exit
kill -9 1234         # SIGKILL - the kernel destroys it; no cleanup, possible data loss
kill -HUP 1234       # SIGHUP (1) - many daemons reload config without restarting
kill -l              # list all signals
killall nginx        # kill by process NAME
pkill -f "python app.py" # kill by matching the full command line
```

ALWAYS SIGTERM BEFORE SIGKILL

Explain it this way: "I send SIGTERM first so the process can flush buffers, close connections and exit cleanly. I only use kill -9 if it does not exit within a reasonable time, because SIGKILL cannot be caught - the process gets no chance to clean up, which can corrupt state. And if a process is stuck in uninterruptible sleep, state D, even kill -9 will not remove it until the I/O completes."

Priorities and job control

```
nice -n 10 ./backup.sh # start with lower priority (-20 highest .. 19 lowest)
sudo renice -n -5 -p 1234 # change priority of a running process (negative needs root)

./long-job.sh &        # run in background
jobs                   # list this shell's jobs
fg %1                  # bring job 1 to the foreground
bg %1                  # resume a stopped job in the background
Ctrl+Z                 # suspend the foreground job
Ctrl+C                 # send SIGINT to the foreground job

nohup ./deploy.sh > deploy.out 2>&1 & # survives logout; output to deploy.out
setsid ./server.sh      # fully detach into a new session
# Better for long work over SSH: tmux new -s deploy (detach Ctrl+B then D, reattach: tmux attach -t deploy)
```

3.7 Services and systemd

systemctl - the commands you will be asked for

```
systemctl status nginx # state, PID, memory, and the last log lines
systemctl start nginx
systemctl stop nginx
systemctl restart nginx # stop then start
systemctl reload nginx # re-read config without dropping connections
systemctl reload-or-restart nginx
systemctl enable nginx # start automatically at boot
systemctl disable nginx
```

```

systemctl is-active nginx          # "active" / "inactive" - scriptable
systemctl is-enabled nginx
systemctl list-units --type=service --state=running
systemctl list-units --failed      # FAILED UNITS - first command after an unexpected reboot
systemctl daemon-reload           # MUST run after editing any unit file
systemctl cat nginx                # show the effective unit file
systemctl edit nginx               # create a drop-in override (safer than editing the vendor unit)

```

A systemd unit file, line by line

```

[Unit]
Description=Application API service
After=network-online.target        # order: start after the network is really up
Wants=network-online.target
Requires=postgresql.service       # hard dependency: if this fails, we fail

[Service]
Type=simple                       # simple: the ExecStart process IS the service
User=appuser                      # never run as root without a reason
WorkingDirectory=/opt/app
EnvironmentFile=/etc/app/app.env  # load environment variables from a file
ExecStart=/usr/bin/python3 /opt/app/main.py
ExecReload=/bin/kill -HUP $MAINPID
Restart=on-failure                # restart if it exits non-zero
RestartSec=5                      # wait 5s between restarts
StandardOutput=journal            # logs go to journald -> journalctl -u app
LimitNOFILE=65536                 # raise the open-file limit

[Install]
WantedBy=multi-user.target        # what `systemctl enable` hooks it into

```

A SERVICE WILL NOT START - THE 60-SECOND ROUTINE

1. `systemctl status appname -l` - read the exit code and the last lines.
2. `journalctl -u appname -n 100 --no-pager` - the actual error.
3. `journalctl -u appname --since "10 minutes ago"` - narrow to the incident window.
4. Config valid? `nginx -t`, `httpd -t`, or run the binary manually as the service user.
5. Port already taken? `sudo ss -tulpn | grep :8080`.
6. Permissions? Run as the service user: `sudo -u appuser ls -l /opt/app`.
7. Resources? `df -h` (disk full stops many services) and `free -h`.
8. SELinux/AppArmor blocking it? `sudo ausearch -m avc -ts recent` OR `dmesg | tail`.
9. Did you edit the unit file? `systemctl daemon-reload` and try again.

3.8 Disk and Filesystems

Disk inspection

```

df -h                             # filesystem usage, human-readable - the first disk command
df -i                             # INODE usage - "No space left on device" with df -h showing free space
df -hT                            # include filesystem type
du -sh /var/log                   # total size of a directory
du -sh /var/log/* | sort -h       # each child, smallest to largest
du -ah /var/log | sort -h | tail -20 # the 20 largest files
du -xh --max-depth=1 / | sort -h  # top-level offenders, staying on one filesystem
lsblk                             # block devices and mount points as a tree
lsblk -f                          # with filesystem type and UUID
blkid                             # UUIDs for /etc/fstab
sudo fdisk -l                     # partition tables
mount | column -t                  # what is mounted and how
findmnt /var                       # cleaner view of one mount

```

Mounting a new disk (the EBS volume workflow)

```

lsblk                             # identify the new device, e.g. /dev/xvdf
sudo file -s /dev/xvdf             # "data" means no filesystem yet
sudo mkfs -t ext4 /dev/xvdf        # create filesystem (DESTRUCTIVE)
sudo mkdir -p /data
sudo mount /dev/xvdf /data         # mount now

```

```

sudo blkid /dev/xvdf          # get the UUID
echo 'UUID=xxxx-xxxx /data ext4 defaults,nofail 0 2' | sudo tee -a /etc/fstab
sudo mount -a                # TEST fstab before rebooting
sudo umount /data            # unmount
sudo umount -l /data          # lazy unmount when it says "target is busy"
sudo fuser -vm /data          # which processes hold the mount open

```

A BAD /ETC/FSTAB ENTRY CAN STOP A SERVER BOOTING, AND ON A CLOUD VM YOU MAY HAVE NO CONSOLE. ALWAYS RUN `SUDO MOUNT -A` AFTER EDITING FSTAB, AND ALWAYS INCLUDE `NOFAIL` ON NON-ESSENTIAL VOLUMES SO A MISSING DISK DOES NOT BLOCK BOOT.

3.8.1 Inodes

An inode stores a file's metadata - permissions, owner, timestamps, size and the pointers to its data blocks - but not its name. The name lives in the directory entry, which maps a name to an inode number. A filesystem has a fixed number of inodes created at format time, so millions of tiny files can exhaust inodes while gigabytes of space remain free.

Inode troubleshooting

```

df -i                        # look for IUse% at 100%
ls -li file                  # show a file's inode number
find /var -xdev -printf '%h\n' | sort | uniq -c | sort -rn | head    # dirs with the most files
find /var/spool -type f | wc -l    # common culprits: mail spool, session files, PHP sessions

```

DISK FULL - THE FULL DECISION TREE

1. `df -h` - which filesystem, and how full? Also run `df -i` for inodes.
2. `du -xh --max-depth=1 / | sort -h` then descend into the biggest directory.
3. Usual suspects: `/var/log` (unrotated logs), `/var/lib/docker` (images, stopped containers, volumes), `/tmp` (build leftovers), application data directories, core dumps.
4. Space not returning after deleting a large file? `sudo lsof +L1` or `lsof | grep deleted` - a process still holds the handle. Restart that process; truncating with `> /proc/PID/fd/N` also works.
5. Docker hosts: `docker system df` then `docker system prune -a --volumes` (know exactly what that deletes before running it).
6. Immediate relief: `journalctl --vacuum-size=200M`, gzip old logs, clear package caches (`apt clean`, `yum clean all`).
7. Permanent fix: logrotate config, an alert at 80%, or a bigger volume - and for cloud, growing the EBS volume then `growpart` and `resize2fs`.

3.9 Memory

Memory inspection

```

free -h                    # total / used / free / shared / buff-cache / available
free -m                    # in megabytes
cat /proc/meminfo           # everything the kernel knows
vmstat 1 5                 # 5 samples, 1s apart: r, b, swpd, free, si, so, us, sy, id, wa
ps -eo pid,rss,%mem,cmd --sort=-rss | head    # biggest memory consumers
swapon --show               # active swap
sudo swapoff -a && sudo swapon -a              # clear swap (only if there is free RAM)
dmesg -T | grep -i -E "out of memory|oom-killer|killed process" # OOM evidence

```

"FREE" VERSUS "AVAILABLE" - A CLASSIC TRAP

Linux deliberately uses spare RAM as page cache, so `free` looks low on a healthy system. The column that matters is **available** - memory that can be given to a new process, including reclaimable cache. If someone shows you `free -h` with 200MB free and 6GB of buff/cache and asks "is this server out of memory?", the answer is no - look at `available` and at swap activity (`si/so` in `vmstat`).

Symptom	Meaning	Action
available is very low, si/so active in vmstat	Genuine memory pressure, swapping	Find the consumer; add memory; set limits

Symptom	Meaning	Action
Process killed, dmesg shows oom-killer	Kernel killed the largest-scoring process to save the system	Raise the limit or fix the leak; in K8s check the container memory limit
RSS grows steadily and never falls	Memory leak	Heap profiling; schedule restarts as a stopgap; fix the code
High buff/cache, high available	Normal, healthy	No action

3.10 Logs

Where the truth lives

```
ls -ltr /var/log/           # newest at the bottom
tail -F /var/log/syslog    # Debian/Ubuntu general log
tail -F /var/log/messages # RHEL/CentOS general log
tail -F /var/log/auth.log  # Ubuntu authentication; /var/log/secure on RHEL
tail -F /var/log/nginx/error.log
dmesg -T | tail -50        # kernel ring buffer with human timestamps
```

journalctl - systemd's log

```
journalctl -u nginx        # one unit
journalctl -u nginx -f     # live follow
journalctl -u nginx -n 200 --no-pager # last 200 lines, no pager (scripts, CI)
journalctl --since "2026-09-23 14:00" --until "2026-09-23 15:00"
journalctl --since "30 min ago"
journalctl -p err -b       # priority error and worse, this boot
journalctl -b -1           # the PREVIOUS boot - what happened before the crash
journalctl -k              # kernel messages only
journalctl -u app --grep "timeout"
journalctl --disk-usage
sudo journalctl --vacuum-time=7d # trim to 7 days
```

logrotate - preventing the number one disk-full cause

```
# /etc/logrotate.d/myapp
/var/log/myapp/*.log {
    daily           # rotate every day
    rotate 14       # keep 14 old files
    size 100M       # or rotate early if it reaches 100M
    compress        # gzip old logs
    delaycompress   # do not compress the most recent rotation
    missingok       # do not error if the file is absent
    notifempty      # skip rotation if empty
    copytruncate    # copy then truncate - for apps that hold the file open
    create 0640 appuser appgroup
    postrotate
        systemctl reload myapp > /dev/null 2>&1 || true
    endscript
}

sudo logrotate -d /etc/logrotate.d/myapp # DEBUG/dry run - always test first
sudo logrotate -f /etc/logrotate.d/myapp # force a rotation now
```

3.11 Linux Networking Commands

Addresses, routes, sockets

```
ip addr show           # interfaces and IP addresses (replaces ifconfig)
ip -br addr            # brief, readable summary
ip route show          # routing table; "default via ..." is your gateway
ip route get 8.8.8.8   # which interface and source IP would be used
ip link set eth0 up

ss -tulpn              # listening TCP/UDP sockets with the owning process (replaces netstat)
ss -tan state established # established connections
ss -s                 # socket summary counts
ss -tn dst 10.0.2.50   # connections to a specific host
netstat -tulpn         # older equivalent; still asked about in interviews
```

Reachability and DNS

```

ping -c 4 10.0.1.20      # ICMP reachability (may be blocked by firewalls - absence is not proof)
traceroute example.com  # hop-by-hop path
mtr example.com         # continuous traceroute + loss statistics - best for intermittent issues

nslookup example.com    # simple DNS lookup
dig example.com         # full DNS answer with TTL and record type
dig +short example.com
dig example.com A +noall +answer
dig MX example.com
dig @8.8.8.8 example.com # query a SPECIFIC resolver - proves whether it is your resolver
dig -x 93.184.216.34    # reverse lookup
host example.com
cat /etc/resolv.conf     # which resolvers this box uses
getent hosts example.com # resolution the way the system libraries do it (includes /etc/hosts)

```

Port and HTTP testing

```

nc -zv 10.0.2.50 3306    # is TCP port 3306 open? the fastest connectivity test
nc -zv host 20-25        # scan a small range
timeout 3 bash -c "</dev/tcp/10.0.2.50/3306" && echo open || echo closed # no netcat needed
telnet 10.0.2.50 3306    # old but universally available

curl -I https://example.com      # headers only
curl -v https://example.com      # verbose: DNS, TCP, TLS handshake, headers
curl -s -o /dev/null -w "%{http_code} %{time_total}s\n" https://example.com
curl -X POST -H "Content-Type: application/json" -d '{"k":"v"}' http://api/endpoint
curl --resolve example.com:443:10.0.1.5 https://example.com # test a specific backend, bypassing DNS
curl -k https://self-signed.local # skip certificate verification (diagnostics only)
wget -q https://host/file.tar.gz -O /tmp/file.tar.gz

```

SSH, SCP, rsync

```

ssh deploy@10.0.1.20
ssh -i ~/.ssh/prod.pem ec2-user@ec2-host
ssh -p 2222 user@host
ssh -v user@host # verbose - the fastest way to debug key/auth problems
ssh -o StrictHostKeyChecking=no user@host # CI only; understand that you lose MITM protection
ssh user@jump -J # jump host syntax: ssh -J user@bastion user@private-host
ssh user@host "df -h; uptime" # run remote commands and return

ssh-keygen -t ed25519 -C "aasim@laptop" # generate a key pair (ed25519 preferred over RSA)
ssh-copy-id -i ~/.ssh/id_ed25519.pub user@host # install the public key on the server
chmod 700 ~/.ssh && chmod 600 ~/.ssh/authorized_keys ~/.ssh/id_ed25519

scp app.tar.gz user@host:/opt/app/ # copy to remote
scp -r user@host:/var/log/app ./logs/ # copy a directory back
rsync -avz --progress /src/ user@host:/dst/ # archive, compress, show progress
rsync -avz --delete /src/ /backup/ # mirror: delete extras at the destination
rsync -avzn /src/ /dst/ # -n DRY RUN - always run this first

```

SSH file	Purpose	Required permissions
~/.ssh/id_ed25519	Your PRIVATE key - never leaves your machine	600
~/.ssh/id_ed25519.pub	Your public key - safe to distribute	644
~/.ssh/authorized_keys	Public keys allowed to log in AS this user	600
~/.ssh/known_hosts	Host keys you have accepted - protects against MITM	644
~/.ssh/config	Per-host shortcuts: Host, User, IdentityFile, ProxyJump	600
/etc/ssh/sshd_config	Server config: PermitRootLogin, PasswordAuthentication, Port	644

SSH "PERMISSION DENIED (PUBLICKEY)" - THE CHECKLIST

1. Are you using the right key and user? `ssh -i key.pem -v ec2-user@host` and read the verbose output for "Offering public key".
2. Key permissions: the private key must be 600 - SSH refuses keys that are group- or world-readable.
3. Server-side: `~/.ssh` must be 700 and `authorized_keys` 600, both owned by the user.
4. Home directory must not be group-writable.
5. Correct default user for the AMI: `ec2-user` (Amazon Linux/RHEL), `ubuntu` (Ubuntu), `admin` (Debian), `centos` (CentOS).
6. Server config: is `PubkeyAuthentication` yes set, and is the user permitted by `AllowUsers/AllowGroups`?
7. SELinux context problems: `sudo restorecon -R -v ~/.ssh`.
8. Nothing at all happening (timeout, not "denied")? That is a NETWORK problem - security group, NACL, route table or the host being down - not an authentication problem. Distinguish the two immediately.

3.12 Package Management

Task	Debian/Ubuntu (apt)	RHEL/CentOS (yum/dnf)
Refresh metadata	<code>sudo apt update</code>	<code>sudo yum check-update</code>
Install	<code>sudo apt install nginx</code>	<code>sudo yum install nginx</code>
Remove	<code>sudo apt remove nginx</code>	<code>sudo yum remove nginx</code>
Remove with config	<code>sudo apt purge nginx</code>	-
Upgrade all	<code>sudo apt upgrade</code>	<code>sudo yum update</code>
Search	<code>apt search nginx</code>	<code>yum search nginx</code>
Package info	<code>apt show nginx</code>	<code>yum info nginx</code>
List installed	<code>dpkg -l</code>	<code>rpm -qa</code>
Which package owns a file	<code>dpkg -S /usr/sbin/nginx</code>	<code>rpm -qf /usr/sbin/nginx</code>
Files in a package	<code>dpkg -L nginx</code>	<code>rpm -ql nginx</code>
Install a local package	<code>sudo dpkg -i pkg.deb</code>	<code>sudo rpm -ivh pkg.rpm</code>
Clean cache	<code>sudo apt clean</code>	<code>sudo yum clean all</code>

APT UPDATE REFRESHES THE PACKAGE INDEX; APT UPGRADE INSTALLS NEWER VERSIONS. CANDIDATES CONFUSE THESE CONSTANTLY. ALSO KNOW THAT DNF IS THE MODERN SUCCESSOR TO YUM ON RHEL 8+ AND THE SYNTAX IS COMPATIBLE.

3.13 Bash Scripting for Linux Work

Variables, arguments and exit codes

```
#!/usr/bin/env bash
set -euo pipefail          # -e exit on error, -u error on unset var, pipefail catches pipeline failures
IFS=$'\n\t'

NAME="app"                 # no spaces around =
COUNT=5
readonly LOG_DIR="/var/log/${NAME}" # constant
TODAY=$(date +%F)          # command substitution
echo "Backing up ${NAME} on ${TODAY} to ${LOG_DIR}"

# Script arguments
echo "script name : $0"
echo "first arg   : $1"
echo "all args    : @$@"
echo "arg count   : $#"
```

```
echo "exit code of last command: $?"
echo "PID of this script: $$"

# Defaults and required arguments
ENVIRONMENT="${1:-staging}" # default if not supplied
```

```
: "${API_TOKEN:?API_TOKEN must be set}"          # abort with a message if unset
```

Conditions

```
if [ -f /etc/app/app.conf ]; then echo "config exists"; fi
if [ -d /opt/app ]; then echo "directory exists"; fi
if [ -r file ]; then echo readable; fi
if [ -s file ]; then echo "exists and is non-empty"; fi
if [ -z "$VAR" ]; then echo "VAR is empty"; fi
if [ -n "$VAR" ]; then echo "VAR is set"; fi
if [ "$a" = "$b" ]; then echo "strings equal"; fi
if [ "$x" -eq 5 ]; then echo "numeric equal"; fi      # -eq -ne -lt -le -gt -ge
if [[ "$name" == app* ]]; then echo "pattern match"; fi  # [[ ]] is bash-only, safer
if [[ "$line" =~ ^ERROR ]]; then echo "regex match"; fi

# Combining
if [ "$USED" -gt 80 ] && [ "$ENV" = "prod" ]; then echo "page someone"; fi

case "$1" in
  start)  systemctl start app ;;
  stop)   systemctl stop app ;;
  restart) systemctl restart app ;;
  *)      echo "Usage: $0 {start|stop|restart}"; exit 1 ;;
esac
```

Loops and functions

```
for host in web1 web2 web3; do
  ssh "$host" "uptime"
done

for f in /var/log/*.log; do
  gzip "$f"
done

for i in {1..5}; do echo "attempt $i"; done

while read -r line; do
  echo "processing: $line"
done < servers.txt

# Retry loop with backoff - a genuinely useful DevOps pattern
attempt=1
until curl -sf http://localhost:8080/health > /dev/null; do
  if [ "$attempt" -ge 10 ]; then echo "health check failed after 10 attempts"; exit 1; fi
  echo "waiting for service, attempt $attempt"
  sleep $(( attempt * 2 ))
  attempt=$(( attempt + 1 ))
done
echo "service healthy"

log() { echo "[$(date +%F %T)] $*" | tee -a /var/log/deploy.log; }
fail() { log "ERROR: $*"; exit 1; }
check_service() {
  local svc="$1"
  systemctl is-active --quiet "$svc" && return 0 || return 1
}
check_service nginx || fail "nginx is not running"
```

cron and environment

```
crontab -e          # edit the current user's crontab
crontab -l          # list
sudo crontab -u root -l
# Format: minute hour day-of-month month day-of-week command
#      *      *      *      *      *
0 2 * * *      /opt/scripts/backup.sh >> /var/log/backup.log 2>&1  # daily at 02:00
*/5 * * * *    /opt/scripts/healthcheck.sh                      # every 5 minutes
0 */6 * * *    /opt/scripts/cleanup.sh                          # every 6 hours
0 3 * * 0      /opt/scripts/weekly.sh                            # 03:00 every Sunday
@reboot        /opt/scripts/startup.sh

# Environment variables
export APP_ENV=production      # exported: visible to child processes
```

```
printenv | sort                # everything in the environment
echo "$PATH"
env APP_ENV=test ./script.sh   # set for one command only
# Persist: ~/.bashrc (interactive), /etc/environment (system-wide), EnvironmentFile= in systemd
```

WHY CRON JOBS FAIL BUT THE SCRIPT WORKS BY HAND

Cron runs with a minimal environment: PATH is usually just /usr/bin:/bin, and your ~/.bashrc is not sourced. A script that calls aws, kubectl, docker or python3 by bare name will fail with "command not found". Fixes: use absolute paths in cron scripts, set PATH= at the top of the crontab, source the environment explicitly, and always redirect output to a log file with >> /var/log/x.log 2>&1 so the failure is visible instead of being emailed into a void.

3.14 Hands-On Exercises

DO THESE ON A VM OR EC2 FREE-TIER INSTANCE BEFORE YOUR INTERVIEW

1. Create a user deploy, add it to a new group appops without removing its other groups, give it passwordless sudo for systemctl restart nginx only.
2. Create /srv/shared so that every file created inside inherits the appops group and only owners can delete their own files.
3. Fill a disk deliberately with fallocation -l 2G /tmp/big, watch df -h, delete it while tail -f holds it open, and prove with lsof that the space did not return.
4. Write a systemd unit for a Python HTTP server, enable it, break it deliberately (wrong path), and diagnose it using only systemctl status and journalctl.
5. Write a logrotate config for your app and test it with logrotate -d.
6. From access.log, produce the top 10 IPs, the count per status code, and every 5xx in the last hour - using only grep, awk, sort and uniq.
7. Generate an ed25519 key pair, deploy it with ssh-copy-id, disable password authentication in sshd_config, and confirm you can still log in BEFORE closing your session.
8. Write a Bash health-check script that retries with backoff, logs with timestamps, and exits non-zero on failure - then schedule it with cron every 5 minutes and confirm it runs.

3.15 Linux Interview Questions and Answers

Q. 1. What is the difference between a hard link and a soft link?

A hard link is a second directory entry pointing at the same inode, so both names are equally real - deleting one leaves the data intact until the last link is gone, and hard links cannot cross filesystems or point at directories. A soft link (symlink) is a small file containing a path; it can cross filesystems and point at directories, but it breaks if the target is removed. `ln file hard, ln -s file soft; ls -li` shows the shared inode number for hard links.

Q. 2. What does `chmod 755` actually mean?

Owner gets read, write and execute (4+2+1=7); group and others get read and execute (4+1=5). It is the normal mode for executables and directories.

Q. 3. Which permission lets you delete a file?

Write permission on the directory containing it, not on the file. That is why a root-owned read-only file in a directory you own can still be deleted by you.

Q. 4. What is the sticky bit and where is it used?

On a directory, it restricts deletion to the file's owner or root even when everyone can write to the directory. /tmp uses it - mode 1777 - so that users cannot delete each other's temporary files.

Q. 5. What is SUID and why is it a security concern?

A SUID executable runs with the privileges of the file's owner rather than the caller. `passwd` needs it to write /etc/shadow. It is a risk because a bug in a root-owned SUID binary becomes a privilege escalation, so hardening includes auditing them with `find / -perm -4000 -type f`.

Q. 6. Difference between `kill`, `kill -9` and `kill -HUP`?

kill sends SIGTERM (15), which the process can catch and use to shut down cleanly. kill -9 sends SIGKILL, which the process cannot catch - the kernel destroys it immediately, risking unflushed data. kill -HUP sends SIGHUP (1), which most daemons interpret as "reload your configuration" without restarting.

Q. 7. A process shows state D. What does that mean and can you kill it?

D is uninterruptible sleep, almost always waiting on I/O - a slow disk, a hung NFS mount or a failing EBS volume. It cannot be killed, not even with -9, until the I/O completes or fails. The fix is at the storage layer, not the process layer. Several processes in D plus high wa in vmstat points straight at storage.

Q. 8. What is a zombie process and how do you remove it?

A process that has exited but whose parent has not yet read its exit status, so its entry remains in the process table. It uses no resources other than that entry and cannot be killed. You fix it by making the parent reap it, usually by restarting the parent; if the parent dies, init adopts and reaps the child.

Q. 9. df shows space available but writes fail with "No space left on device". Why?

Inode exhaustion. Check df -i. A filesystem has a fixed inode count set at format time, so millions of small files - session files, mail spool, cache - can use them all while blocks remain free. Fix by deleting the small files or reformatting with more inodes.

Q. 10. You deleted a huge log file but disk usage did not drop. What is happening?

A process still holds an open file descriptor to it, so the kernel keeps the blocks allocated until that descriptor closes. Find it with lsof | grep deleted or lsof +L1, then restart that process, or truncate through /proc/PID/fd/N. Prevent it with logrotate using copytruncate or a postrotate reload.

Q. 11. How do you find the largest files on a system?

du -ah /var | sort -h | tail -20 for a directory tree, or find / -xdev -type f -size +500M -exec ls -lh {} \; 2>/dev/null to search by size. -xdev keeps find on one filesystem so it does not wander into /proc or network mounts.

Q. 12. What is the difference between free and available memory?

free is memory not used at all. available is memory that could be given to a new process, which includes page cache the kernel can reclaim. Linux uses spare RAM for cache by design, so low free is normal and healthy; only low available combined with swap activity indicates real pressure.

Q. 13. What is the OOM killer?

When the kernel cannot satisfy an allocation and cannot reclaim enough memory, it selects a process using an oom_score based on memory use and kills it to keep the system alive. Evidence is in dmesg -T | grep -i oom. In containers, hitting a memory limit produces the same effect for that container - in Kubernetes it shows as OOMKilled.

Q. 14. How do you check which process is listening on port 8080?

sudo ss -tulpn | grep :8080, or sudo lsof -i :8080, or the older sudo netstat -tulpn | grep :8080. sudo matters - without it you often see the socket but not the owning process.

Q. 15. Difference between ps aux and top?

ps is a snapshot at one instant; top refreshes continuously and sorts live. Use ps in scripts and for exact filtering, top/htop for watching a problem develop.

Q. 16. How do you run a process so it survives your SSH session ending?

nohup ./script.sh > out.log 2>&1 &, or setsid, or - better for anything interactive or long - run it inside tmux or screen so you can reattach later. In production I prefer systemd for anything that should really be a service.

Q. 17. What does 2>&1 mean?

Redirect file descriptor 2 (stderr) to wherever descriptor 1 (stdout) currently points. Order matters: > file 2>&1 sends both to the file, while 2>&1 > file sends stderr to the terminal and only stdout to the file.

Q. 18. Difference between > and >>?

> truncates the file and writes; >> appends. Using > on a live log file is a classic accidental data loss.

Q. 19. How do you append to a root-owned file without being root in the shell?

echo "line" | sudo tee -a /etc/file. sudo echo "line" >> /etc/file fails because your non-root shell performs the redirection.

Q. 20. What is the difference between soft and hard in ulimit?

The soft limit is the currently enforced value, which a user can raise up to the hard limit; the hard limit is the ceiling and only root can raise it. ulimit -n shows open files - raising it is a standard fix for "too many open files" on busy servers.

Q. 21. How do you troubleshoot high CPU?

top and press 1 to see per-core usage and whether it is user, system, iowait or steal time. Identify the process with `ps -eo pid,%cpu,cmd --sort=-%cpu | head`. For a Java process, get thread-level detail with `top -H -p PID` and map the thread to a stack. Check whether it correlates with a deploy, a traffic spike or a cron job. High wa means it is really disk; high st on a VM means the hypervisor is stealing CPU, which is a capacity problem, not your application.

Q. 22. What is load average and how do you interpret it?

The 1, 5 and 15-minute averages of runnable plus uninterruptible processes. Compare it to core count: a load of 4 on a 4-core box is fully utilised; 8 on 4 cores means work is queueing. The trend matters more than the number - rising is a problem, falling is recovery. And on Linux, load includes processes blocked on I/O, so a high load with low CPU usually means disk.

Q. 23. How do you check disk I/O?

`iostat -xz 1` for per-device utilisation and `await`, `iostat` for per-process I/O, and `vmstat 1` where high wa in the CPU columns and non-zero b indicate processes blocked on I/O.

Q. 24. What is /proc?

A virtual filesystem exposing kernel and process state as files. `/proc/meminfo`, `/proc/cpuinfo`, `/proc/loadavg`, and per-process directories like `/proc/1234/environ`, `/proc/1234/fd` and `/proc/1234/limits`. Nothing there is on disk - it is generated by the kernel on read.

Q. 25. What happens when you run a command in the shell?

The shell parses the line, expands variables and globs, resolves the command through aliases, functions, builtins, then PATH; it forks, the child calls `execve` to replace itself with the program, and the parent waits for the exit status unless you backgrounded it. Redirections are set up in the child before `execve`, which is why `sudo cmd > file` writes the file as you, not as root.

Q. 26. How do you check whether a service starts at boot?

`systemctl is-enabled servicename`, and `systemctl enable/disable` to change it. `enable` only affects boot; `start` affects now. Both are needed for a service to run now and after a reboot.

Q. 27. What is the difference between systemctl reload and restart?

`restart` stops and starts the process - connections drop and there is a brief outage. `reload` signals the running process to re-read its configuration without dropping connections, where the service supports it, such as nginx. Always prefer reload for config-only changes in production.

Q. 28. What do you check first when a server is "slow"?

In order: `uptime` for load, `top` for CPU and who is consuming it, `free -h` for memory pressure and swap, `df -h` for a full disk, `iostat/vmstat` for I/O wait, `ss -s` and interface counters for network, then application logs. It is a resource triage before it is an application investigation - and I check whether something changed recently, because most "suddenly slow" turns out to be a deploy, a cron job or a traffic change.

Q. 29. How do you find files modified in the last hour?

`find /etc -mmin -60 -type f`. This is genuinely useful during an incident to answer "what changed?" - especially `find /etc -mtime -1` after an unexplained behaviour change.

Q. 30. What is a symbolic link used for in deployments?

The classic pattern is releasing to `/opt/app/releases/2026-09-23-1` and pointing `/opt/app/current` at it with a symlink. Deployment becomes an atomic symlink swap and rollback is pointing the symlink back at the previous release directory.

Q. 31. How do you check open files for a process?

`lsof -p PID`, or count them with `ls /proc/PID/fd | wc -l` and compare against `cat /proc/PID/limits`. "Too many open files" means the process hit its `nofile` limit - raise it in the `systemd` unit with `LimitNOFILE=`.

Q. 32. How do you check what a process is doing right now?

`strace -p PID` for system calls, `strace -f -e trace=file ./cmd` to see which files it tries to open - this is how you diagnose an opaque "permission denied" - and `lsof -p PID` for what it currently has open.

Q. 33. Explain the difference between su and sudo.

`su` switches to another user entirely, requiring that user's password, and the whole session runs as them. `sudo` runs a single command as another user, authenticating with YOUR password, controlled by `/etc/sudoers`, and every invocation is logged. `sudo` is preferred because it gives least privilege and an audit trail.

Q. 34. How do you give a user permission to restart only one service?

A sudoers entry restricting the command: `deploy ALL=(root) NOPASSWD: /bin/systemctl restart nginx`, added with visudo or as a file in `/etc/sudoers.d/`. That is least privilege - it avoids handing out full root for one operational task.

Q. 35. What is the difference between `apt update` and `apt upgrade`?

`update` refreshes the list of available packages from the repositories; `upgrade` actually installs newer versions of installed packages. `update` alone changes nothing on the system.

Q. 36. How do you check which package owns a file?

`dpkg -S /usr/sbin/nginx` on Debian/Ubuntu, `rpm -qf /usr/sbin/nginx` on RHEL.

Q. 37. What is a runlevel or target?

`systemd` targets replaced SysV runlevels. `multi-user.target` is the normal multi-user CLI state (old runlevel 3), `graphical.target` adds a GUI (runlevel 5), `rescue.target` is single-user. Check with `systemctl get-default` and change with `systemctl set-default`.

Q. 38. How do you schedule a one-off job in the future?

`at - echo "/opt/scripts/cleanup.sh" | at 02:00` - or a `systemd` timer for anything that should be managed and logged properly. `cron` is for recurring work.

Q. 39. What is the difference between `cron` and `systemd` timers?

Both schedule recurring work. `systemd` timers give you `journald` logging, dependency ordering, `Persistent=true` so missed runs execute after downtime, and randomised delays to avoid thundering herds; `cron` is simpler and universally available. Operationally, the biggest practical difference is that timer output lands in `journalctl` by default, whereas `cron` output goes to mail or nowhere unless you redirect it.

Q. 40. How would you copy 500 GB between servers efficiently?

`rsync -avz --partial --progress` over SSH, because it is resumable, can be re-run to catch changes, and compresses in transit. I would run it inside `tmux` so an SSH drop does not kill it, dry-run with `-n` first, and consider `--bwlimit` so it does not saturate the link during business hours. For a one-time bulk move inside a cloud, an object-store staging hop is often faster than a direct copy.

Q. 41. How do you check the Linux distribution and kernel version?

`cat /etc/os-release` for the distribution, `uname -r` for the kernel, `uname -a` for everything, and `hostnamectl` for a combined summary.

Q. 42. What is swap and should you use it on a production server?

Swap is disk space used as overflow when RAM is exhausted. It prevents immediate OOM kills but performance falls off a cliff when actively swapping. On dedicated database servers and Kubernetes nodes it is usually disabled - `kubelet` historically refused to start with swap enabled - because predictable failure is better than unpredictable slowness. On general-purpose servers a small swap plus a low `vm.swappiness` is a reasonable safety net.

Q. 43. How would you find which directory is consuming the most space, quickly?

`du -xh --max-depth=1 / | sort -h` and then descend into the largest entry repeatedly. `-x` keeps it on one filesystem, which avoids wasting time on `/proc` and network mounts.

Q. 44. How do you test whether a remote port is reachable without netcat?

Bash's pseudo-device: `timeout 3 bash -c "</dev/tcp/10.0.2.50/3306" && echo open || echo closed`. Also `curl -v telnet://host:3306` works on many systems. This matters because minimal container images rarely ship `nc`, `telnet` or `ping`.

PART 3 RECALL POINTS

- Deleting a file needs write on the DIRECTORY.
- 1777 = sticky (/tmp), 4755 = SUID, 2775 = SGID directory.
- `usermod -aG` - forgetting `-a` wipes secondary groups.
- SIGTERM first, SIGKILL last; state D cannot be killed.
- `df -i` when disk space looks free but writes fail.
- `lsof | grep deleted` when deleting a file does not free space.
- available, not free, is the memory number that matters.
- `tail -F` not `-f` in production.
- `echo x | sudo tee -a file` to append as root.
- `systemctl daemon-reload` after any unit file edit.
- `journalctl -u svc -n 100 --no-pager` is your first log command.
- cron has a minimal PATH - use absolute paths and redirect output.

PART 4

NETWORKING FOR DEVOPS

OSI, TCP/IP, subnetting, DNS, TLS, HTTP, load balancers, firewalls

Networking is where most DevOps candidates get exposed. Your CV claims TCP/IP, HTTP/HTTPS, DNS, RTSP, ports and network troubleshooting - so expect an interviewer to go deep. This part gives you the models, the protocols, the diagnostic method, and a scenario catalogue for every failure mode you will be asked about.

4.1 The OSI Model

Layer	Name	Unit	What lives here	DevOps relevance
7	Application	Data	HTTP, HTTPS, DNS, SSH, FTP, SMTP, RTSP	ALB routing by path/host; application errors
6	Presentation	Data	TLS/SSL, encoding, compression	Certificate problems, cipher mismatches
5	Session	Data	Session establishment, RPC	Sticky sessions, keep-alive
4	Transport	Segment	TCP, UDP, ports	NLB, security groups, "connection refused"
3	Network	Packet	IP, ICMP, routing	Route tables, NAT gateway, subnets, ping
2	Data Link	Frame	Ethernet, MAC, ARP, VLAN	Rare in cloud; matters on-prem
1	Physical	Bits	Cables, NICs, radio	Data-centre only

MNEMONIC AND THE INTERVIEW USE

Top-down: **All People Seem To Need Data Processing**. The reason interviewers ask is to see if you can localise a fault by layer. Practise saying: "Layer 3 is reachability - can the packet get there. Layer 4 is whether something is listening on the port. Layer 7 is whether the application answered correctly. My troubleshooting works bottom-up through exactly those three."

4.2 TCP/IP Model and How It Maps

TCP/IP layer	OSI equivalent	Protocols
Application	5, 6, 7	HTTP, HTTPS, DNS, SSH, RTSP
Transport	4	TCP, UDP
Internet	3	IP, ICMP, ARP
Network Access	1, 2	Ethernet, Wi-Fi

```

Application data:           [ HTTP GET /index.html ]
+ TCP header (src port, dst port, seq, flags): [TCP][ HTTP ]
+ IP header (src IP, dst IP, TTL, protocol):  [IP][TCP][ HTTP ]
+ Ethernet frame (src MAC, dst MAC, CRC):     [ETH][IP][TCP][ HTTP ][CRC]
Each layer adds its own header on the way down and strips it on the way up.

```

Encapsulation - what actually goes on the wire

4.3 IP Addressing, Subnetting and CIDR

4.3.1 IPv4 basics

An IPv4 address is 32 bits, written as four octets (0-255). It splits into a network portion and a host portion; the subnet mask says where the split is.

Notation	Mask	Host bits	Usable hosts (cloud)	Typical use
/16	255.255.0.0	16	65531	A whole VPC

Notation	Mask	Host bits	Usable hosts (cloud)	Typical use
/20	255.255.240.0	12	4091	Large subnet
/24	255.255.255.0	8	251	Standard subnet
/26	255.255.255.192	6	59	Small tier subnet
/28	255.255.255.240	4	11	Tiny subnet, e.g. NAT/endpoints
/32	255.255.255.255	0	1	A single host - security group rules

AWS RESERVES 5 ADDRESSES PER SUBNET

In a 10.0.1.0/24 subnet, AWS takes 10.0.1.0 (network), 10.0.1.1 (VPC router), 10.0.1.2 (DNS), 10.0.1.3 (reserved for future use) and 10.0.1.255 (broadcast). So a /24 gives you 251 usable addresses, not 254. This is a favourite AWS interview detail.

4.3.2 Private address ranges (RFC 1918)

Range	CIDR	Size	Common use
10.0.0.0 - 10.255.255.255	10.0.0.0/8	16.7M	VPCs, large corporate networks
172.16.0.0 - 172.31.255.255	172.16.0.0/12	1M	Default Docker bridge lives in 172.17.0.0/16
192.168.0.0 - 192.168.255.255	192.168.0.0/16	65K	Home and small office networks

Also worth knowing: 127.0.0.0/8 is loopback, 169.254.0.0/16 is link-local - and 169.254.169.254 is the cloud instance metadata endpoint, which is how an EC2 instance discovers its own identity and retrieves IAM role credentials.

4.3.3 Subnetting worked example

Splitting 10.0.0.0/16 into a standard three-tier VPC across two AZs

VPC: 10.0.0.0/16 (65536 addresses)

Public subnet AZ-a: 10.0.1.0/24 hosts 10.0.1.4 - 10.0.1.254 (ALB, NAT GW, bastion)

Public subnet AZ-b: 10.0.2.0/24 hosts 10.0.2.4 - 10.0.2.254

Private app AZ-a: 10.0.11.0/24 hosts 10.0.11.4 - 10.0.11.254 (EC2 app / worker nodes)

Private app AZ-b: 10.0.12.0/24

Private data AZ-a: 10.0.21.0/24 (RDS, cache - no route to the internet gateway)

Private data AZ-b: 10.0.22.0/24

Why gaps between 1-2, 11-12, 21-22? Room to grow each tier without renumbering.

Why two AZs? A single AZ is a single point of failure; ALB and RDS Multi-AZ both require two.

THE FAST CIDR MENTAL MATHS

/24 = 256 addresses. Every bit you remove from the prefix doubles the block: /23 = 512, /22 = 1024, /21 = 2048, /20 = 4096. Every bit you add halves it: /25 = 128, /26 = 64, /27 = 32, /28 = 16. If asked "how many hosts in a /26?", the answer is 64 total, 62 usable on-prem, 59 usable in AWS.

4.4 Ports, TCP and UDP

Port	Service	Notes
20, 21	FTP data / control	Largely replaced by SFTP
22	SSH / SCP / SFTP	The one you will use daily
25, 587	SMTP	Outbound mail is blocked by default on most cloud providers
53	DNS	UDP for queries, TCP for large responses and zone transfers
80	HTTP	
443	HTTPS	
554	RTSP	Video streaming control - relevant to your current role
3306	MySQL	
5432	PostgreSQL	

Port	Service	Notes
6379	Redis	
8080	HTTP alternate	Common application port behind a proxy
9090	Prometheus	
9100	node_exporter	
3000	Grafana	
8443	HTTPS alternate / Jenkins TLS	
6443	Kubernetes API server	
2379, 2380	etcd client / peer	
10250	kubelet API	
25565-ish high ports	Ephemeral range 32768-60999 on Linux	Source ports for outbound connections

	TCP	UDP
Connection	Connection-oriented - handshake first	Connectionless - just send
Reliability	Guaranteed delivery, retransmission, ordering	No guarantee, no ordering
Speed	Slower - acknowledgements and congestion control	Faster - no overhead
Header	20 bytes minimum	8 bytes
Use cases	HTTP, SSH, database, most APIs	DNS queries, NTP, syslog, video/voice streaming, metrics push
Failure mode	Retries then timeout	Silent packet loss

4.4.1 The TCP three-way handshake

CLIENT	SERVER
----- SYN (seq=x) ----->	"I want to connect"
<----- SYN-ACK (seq=y, ack=x+1) ----	"OK, and I want to connect too"
----- ACK (ack=y+1) ----->	"Confirmed" - connection ESTABLISHED
===== data transfer =====	
----- FIN ----->	Four-way close:
<----- ACK -----	FIN / ACK / FIN / ACK
<----- FIN -----	
----- ACK ----->	then TIME_WAIT on the closing side

Connection establishment and teardown

What you observe	What it means
Connection refused, immediately	The host is reachable and it sent RST - nothing is listening on that port
Connection timed out	No response at all - a firewall, security group, NACL or routing problem, or the host is down
Many sockets in TIME_WAIT	Normal on a busy client; the closing side holds the socket for 2*MSL to catch stray packets
Many in CLOSE_WAIT	The application is not calling close() - an application bug, and it leaks file descriptors
Many in SYN_RECV	Possible SYN flood, or the application backlog is full

REFUSED VS TIMEOUT IS THE SINGLE MOST USEFUL DISTINCTION IN NETWORK TROUBLESHOOTING

"Connection refused" means you reached the machine - so DNS, routing and the firewall are all fine - and the service is not listening. Go and look at the process. "Connection timed out" means the packet never got an answer - go and look at security groups, NACLs, route tables and whether the host is alive. Saying this sentence unprompted marks you as someone who has actually debugged production.

4.5 DNS

4.5.1 Resolution flow

```

Application
| 1. check /etc/hosts and local cache
v
Stub resolver (from /etc/resolv.conf)
| 2. ask the recursive resolver (VPC .2 address, 8.8.8.8, corporate DNS)
v
Recursive resolver -- 3. ask a ROOT server --> "ask .com"
                  -- 4. ask the .com TLD server --> "ask ns-1.example.com"
                  -- 5. ask the AUTHORITATIVE ns --> "93.184.216.34"
| 6. cache the answer for its TTL, return it
v
Application connects to 93.184.216.34

```

What happens when a name is resolved

Record	Purpose	Example
A	Name to IPv4 address	example.com -> 93.184.216.34
AAAA	Name to IPv6 address	example.com -> 2606:2800:220:1:248:1893:25c8:1946
CNAME	Alias to another NAME (never to an IP; not allowed at the zone apex)	www -> example.com
ALIAS / Route 53 ALIAS	AWS-specific; behaves like a CNAME but works at the apex and resolves to AWS resources free of charge	example.com -> my-alb-123.eu-west-1.elb.amazonaws.com
MX	Mail exchange, with priority	10 mail.example.com
TXT	Arbitrary text - SPF, DKIM, domain verification	"v=spf1 include:_spf.google.com ~all"
NS	Which name servers are authoritative for the zone	ns-1.awsdns.com
SOA	Zone metadata - primary NS, serial, refresh, TTLs	
PTR	Reverse lookup - IP to name	34.216.184.93.in-addr.arpa
SRV	Service location - host and port	Used by Kubernetes DNS for named ports

Route 53 routing policy	Behaviour	Use case
Simple	One record, one answer	Basic
Weighted	Split traffic by assigned weights	Blue-green and canary at the DNS level
Latency-based	Answer with the region with lowest latency for the client	Multi-region applications
Failover	Primary with a health check, secondary on failure	Active-passive DR
Geolocation	Answer based on the user's country	Compliance, localisation
Multivalue answer	Several healthy IPs returned, client picks	Poor man's load balancing

THE TTL TRAP

If you plan a DNS cutover, lower the TTL 24-48 hours BEFORE the change, not at the time of the change. Clients and intermediate resolvers have already cached the old value for the old TTL, and nothing you do afterwards shortens that. Also remember DNS is not a load balancer: browsers and JVMs cache aggressively - the JVM historically caches DNS forever unless `networkaddress.cache.ttl` is set - so removing a record does not immediately stop traffic.

4.6 HTTP, HTTPS and TLS

4.6.1 HTTP request anatomy

A raw HTTP request and response

```

GET /api/v1/status HTTP/1.1
Host: api.example.com
User-Agent: curl/8.4.0
Accept: application/json
Authorization: Bearer eyJhbGciOi...
X-Request-ID: 8f2c1a

HTTP/1.1 200 OK

```

```
Content-Type: application/json
Content-Length: 41
Cache-Control: no-store
X-Request-ID: 8f2c1a

{"status":"ok","version":"1.14.2"}
```

Method	Purpose	Idempotent	Safe
GET	Retrieve	Yes	Yes
POST	Create / submit	No	No
PUT	Replace entirely	Yes	No
PATCH	Partial update	Not necessarily	No
DELETE	Remove	Yes	No
HEAD	Headers only, no body	Yes	Yes
OPTIONS	Supported methods / CORS preflight	Yes	Yes

4.6.2 HTTP status codes - the DevOps subset

Code	Meaning	Who is at fault	First thing you check
200	OK	-	-
201	Created	-	-
204	No content	-	-
301 / 302	Moved permanently / found	-	Redirect loops; 301 is cached by browsers, be careful
304	Not modified	-	Caching working as intended
400	Bad request	Client	Malformed payload or query
401	Unauthorised - not authenticated	Client	Missing or expired token
403	Forbidden - authenticated but not allowed	Client/config	IAM policy, S3 bucket policy, file permissions, WAF rule
404	Not found	Client/config	Wrong path, wrong routing rule, missing file, wrong ingress host
405	Method not allowed	Client	POST to a GET-only route
413	Payload too large	Config	nginx client_max_body_size, ALB limits
429	Too many requests	Client	Rate limiting, throttling
500	Internal server error	Server	The APPLICATION threw - go to application logs and the stack trace
502	Bad gateway	Proxy/backend	The proxy got an invalid or no response - backend crashed or wrong port
503	Service unavailable	Server	No healthy backends, overloaded, or maintenance mode
504	Gateway timeout	Backend slow	The backend did not answer within the proxy timeout

500 VS 502 VS 503 VS 504 - THE ANSWER THAT WINS THE QUESTION

"500 means the application itself raised an error - I go to the application logs and find the exception. 502 means the proxy reached a backend but got garbage or a dropped connection - the backend probably crashed, or the proxy is pointing at the wrong port. 503 means there is no healthy backend at all - in Kubernetes that is usually a Service with no endpoints because the pods are failing readiness; behind an ALB it is an empty or unhealthy target group. 504 means the backend is alive but too slow - the proxy timed out waiting, so I look at backend latency, database queries and the timeout settings."

4.6.3 TLS handshake

```

CLIENT                                SERVER
|-- ClientHello (TLS versions, cipher suites, SNI) ->|
|<- ServerHello (chosen version and cipher)           |
|<- Certificate (server cert + intermediate chain)     | <-- expired / wrong CN / missing chain
|<- ServerHelloDone                                   |
|-- verify chain against local CA store                | <-- untrusted CA, clock skew
|-- ClientKeyExchange (pre-master secret, encrypted)->|
|-- ChangeCipherSpec, Finished ----->|
|<- ChangeCipherSpec, Finished                       |
|===== encrypted application data =====|
TLS 1.3 collapses this into one round trip and drops the older cipher suites.

```

TLS 1.2 handshake (simplified) - what breaks where

TLS symptom	Cause	Fix
certificate has expired	Expired cert	Renew; automate with ACM or cert-manager; alert 30 days out
unable to get local issuer certificate	Missing intermediate chain	Serve the full chain, not just the leaf
hostname mismatch	Cert CN/SAN does not match the requested host	Reissue with the right SAN, or use the correct hostname
self-signed certificate	Internal CA not trusted by the client	Install the CA bundle in the client trust store
handshake failure / no shared cipher	Protocol or cipher mismatch, TLS 1.0/1.1 disabled	Align the TLS version and cipher list

TLS diagnostics

```

openssl s_client -connect example.com:443 -servername example.com < /dev/null
openssl s_client -connect example.com:443 -showcerts                # full chain
echo | openssl s_client -connect example.com:443 2>/dev/null | openssl x509 -noout -dates
openssl x509 -in cert.pem -noout -text | head -30                  # inspect a local cert
openssl x509 -in cert.pem -noout -subject -issuer -dates
curl -vI https://example.com 2>&1 | grep -E "SSL|subject|expire"

```

4.7 Proxies, Reverse Proxies and Load Balancers

	Forward proxy	Reverse proxy
Sits in front of	Clients	Servers
Knows about	The destination on behalf of the client	The backends on behalf of the client
Purpose	Egress control, caching, anonymity, corporate filtering	TLS termination, routing, caching, load balancing, hiding the backend
Example	Corporate web proxy, squid, NAT gateway (conceptually)	nginx, HAProxy, ALB, Ingress controller

	ALB (Application LB, L7)	NLB (Network LB, L4)
Layer	7 - HTTP/HTTPS	4 - TCP/UDP/TLS
Routes by	Host header, URL path, headers, query string, method	IP and port only
TLS termination	Yes, with ACM certificates	Yes (TLS listener) or pass-through
Source IP seen by target	The ALB's IP - the original is in X-Forwarded-For	The client's real IP is preserved
Static IP	No - use a DNS name	Yes, one static/Elastic IP per AZ
Performance	Very high	Extreme, ultra-low latency
Health checks	HTTP/HTTPS path-based	TCP or HTTP
Use it for	Web applications, microservices, path-based routing, WebSockets	Non-HTTP protocols such as RTSP or database, static IP requirements, extreme throughput

GIVEN YOUR ROLE

Your current work involves RTSP, which is not HTTP. That is a perfect concrete answer for "when would you use an NLB over an ALB?": "An ALB only understands HTTP, so it cannot route an RTSP stream - for a non-HTTP TCP protocol you need an NLB, which balances at layer 4 and also preserves the client source IP."

LB algorithm	Behaviour
Round robin	Each backend in turn - the default
Least connections	To the backend with fewest active connections - better for long-lived connections
IP hash / sticky sessions	Same client to the same backend - needed for in-memory session state
Weighted	Proportional to assigned weight - canary deployments

4.8 NAT, Firewalls, Routing and VPN

Concept	What it does	Cloud implementation
NAT	Translates private source addresses to a public one for outbound traffic	NAT Gateway in a public subnet; private subnet route 0.0.0.0/0 -> nat-xxxx
Firewall	Allows or denies traffic by rules	Security Group (instance level), NACL (subnet level), iptables/nftables, Windows Firewall
Routing	Decides which next hop a packet takes	VPC Route Tables; ip route on a host
Default gateway	Where packets go when no specific route matches	Internet Gateway for public subnets
VPN	Encrypted tunnel joining two networks	Site-to-Site VPN, Client VPN, WireGuard/OpenVPN
Peering	Private routing between two VPCs	VPC Peering (non-transitive) or Transit Gateway (hub)

	Security Group	Network ACL
Level	Instance / ENI	Subnet
State	Stateful - return traffic is automatically allowed	Stateless - you must allow the return traffic explicitly, usually the ephemeral range 1024-65535
Rules	Allow only	Allow and Deny
Evaluation	All rules evaluated; if any allows, it is allowed	Rules in numbered order, first match wins
Default	Deny all inbound, allow all outbound	Default NACL allows everything; custom NACL denies everything
Can reference	Other security groups - powerful for tiering	CIDR blocks only

THE STATEFUL/STATELESS ANSWER

"A security group is stateful: if I allow inbound 443, the response goes out automatically. A NACL is stateless: allowing inbound 443 is not enough, I also have to allow outbound on the ephemeral port range, or the response is dropped. That asymmetry is why most 'I opened the port but it still times out' problems in a hardened VPC turn out to be the NACL."

iptables essentials (still asked, even in a cloud-first role)

```

sudo iptables -L -n -v          # list rules with packet counters
sudo iptables -t nat -L -n      # NAT table
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT
sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT
sudo iptables -A INPUT -j DROP  # default deny AFTER the allows
sudo iptables -D INPUT 3        # delete rule number 3
sudo iptables-save > /etc/iptables/rules.v4 # rules are NOT persistent by default

sudo firewall-cmd --list-all    # RHEL firewalld
sudo firewall-cmd --permanent --add-port=8080/tcp && sudo firewall-cmd --reload
sudo ufw status verbose         # Ubuntu ufw
sudo ufw allow 8080/tcp

```

4.9 "What Happens When You Type https://example.com?"

This is the single most common senior-screening question in the industry. Answer it in layers, and stop to say where each step can fail.

1 BROWSER	parse URL, check HSTS list, check its own cache
2 DNS	/etc/hosts -> stub resolver -> recursive resolver -> root -> .com -> authoritative result: 93.184.216.34, cached for the TTL
3 ROUTING	OS consults its route table, picks the interface and gateway; ARP for the next hop MAC
4 TCP	three-way handshake to 93.184.216.34:443 (SYN / SYN-ACK / ACK)
5 TLS	ClientHello with SNI -> ServerHello -> certificate -> chain validated -> keys agreed
6 HTTP	GET / HTTP/1.1 (or HTTP/2 stream) with Host, cookies, Accept headers
7 EDGE	CDN / WAF / load balancer terminates TLS, picks a healthy target, adds X-Forwarded-For
8 SERVER	reverse proxy -> application -> database / cache -> response generated
9 RESPONSE	status line, headers, body travel back; browser parses HTML, fetches sub-resources
10 RENDER	DOM, CSSOM, render tree, paint

The full path of one request

HOW TO DELIVER IT IN AN INTERVIEW

"I'd go layer by layer. First the browser normalises the URL and checks HSTS and its cache. Then name resolution: /etc/hosts, then the stub resolver from resolv.conf, then a recursive resolver that walks root, TLD and authoritative servers, with caching at every step governed by TTL. With an IP, the kernel consults its routing table to choose an interface and gateway and ARPs for the next-hop MAC. Then TCP: a three-way handshake to port 443. Then TLS: ClientHello with SNI, the server's certificate and chain, validation against the trust store, key exchange, and from then on the channel is encrypted. Then the HTTP request itself with the Host header. In a real deployment it lands on a CDN or load balancer that terminates TLS, adds X-Forwarded-For and picks a healthy target; that target is usually a reverse proxy in front of the application, which may call a database or cache. The response comes back and the browser parses and renders it. Each of those steps has its own failure signature - DNS failures look different from a timeout, which looks different from a 502 - and that is how I isolate faults."

4.10 Troubleshooting Method: "The Application Is Not Reachable"

WORK BOTTOM-UP, NEVER GUESS

Step 0 - Scope. Is it everyone or one user? One environment or all? Did it ever work? What changed in the last hour - deploy, config, certificate, DNS, security group?

Step 1 - Name resolution. `dig +short app.example.com` and `dig @8.8.8.8 app.example.com`. Does it resolve, and to the IP you expect? Compare with `getent hosts app.example.com` to catch a stale `/etc/hosts` entry.

Step 2 - Reachability. `ping -c3 IP` (remember ICMP is often blocked, so no reply is not proof of failure), `traceroute IP` or `mtr IP` to see where the path stops.

Step 3 - Port. `nc -zv IP 443` Or `timeout 3 bash -c "</dev/tcp/IP/443"`. Refused means reachable-but-not-listening. Timeout means blocked or dead.

Step 4 - Is the server listening? On the host: `sudo ss -tulpn | grep :443`. Check the bind address: a process bound to 127.0.0.1 is unreachable from outside no matter what the firewall says. That single check solves a surprising share of real incidents.

Step 5 - Firewall path. Security group inbound, NACL inbound AND outbound (stateless!), host firewall (`iptables -L -n, ufw status`), and the route table for the subnet.

Step 6 - TLS. `openssl s_client -connect host:443 -servername host - expiry, chain, hostname`.

Step 7 - Application layer. `curl -v https://host/health`. Read the status code and apply the 500/502/503/504 logic. Then go to logs: application, proxy access and error logs, `journalctl -u service`.

Step 8 - Dependencies. Can the application reach its database, cache and downstream APIs? Test from the application host, as the application user, using the same DNS name and port the application uses.

Step 9 - Capacity. Is the box healthy? `df -h`, `free -h`, `top`, connection counts with `ss -s`, and file descriptor limits.

4.11 Scenario Catalogue

Scenario 1 - Connection refused

Field	Detail	
Symptom	<code>curl: (7) Failed to connect to host port 8080: Connection refused - immediate, no delay</code>	
Meaning	The TCP SYN reached the host and the host replied with RST. Network path is fine.	
Causes	Service crashed or was never started; listening on a different port; bound to 127.0.0.1 only; container not running	
Commands	<code>sudo ss -tulpn \</code>	<code>grep 8080, systemctl status app, docker ps, journalctl -u app -n 100`</code>
Fix	Start the service, correct the port, change the bind address to 0.0.0.0	

Scenario 2 - Connection timed out

Field	Detail
Symptom	Hangs, then Connection timed out after 30-120 seconds
Meaning	No response at all - packets are being dropped silently
Causes	Security group missing the inbound rule; NACL blocking return traffic; wrong route table; host down; wrong IP; host firewall DROP
Commands	<code>traceroute</code> , check SG/NACL/route table in the console or with <code>aws ec2 describe-security-groups, sudo iptables -L -n</code> on the host
Fix	Open the correct path at the layer that is dropping - and verify with a test from inside the same subnet to isolate whether the problem is internal or at the edge

Scenario 3 - DNS failure

Field	Detail
Symptom	Could not resolve host, or it resolves to an old IP
Commands	<code>dig name</code> , <code>dig @8.8.8.8 name</code> , <code>cat /etc/resolv.conf</code> , <code>getent hosts name</code> , <code>systemd-resolve --status</code>
Causes	Record missing or typo'd; TTL caching an old value; split-horizon DNS returning private records externally; resolver unreachable; a stale <code>/etc/hosts</code> entry; in Kubernetes, CoreDNS down or the wrong <code>ndots</code> search path
Fix	Correct the record, wait out or flush the TTL, fix the resolver. Prove where it fails by querying the authoritative server directly.

Scenario 4 - HTTP 404

Field	Detail
Meaning	The server answered - so DNS, TCP, TLS and the proxy all worked - but it has nothing at that path
Causes	Wrong path or base URL; ingress/ALB rule not matching the Host or path; missing file in the image; application context path changed; a rewrite rule stripping or adding a prefix
Commands	<code>curl -v https://host/path</code> , check the ingress/ALB listener rules, <code>kubectl describe ingress</code> , nginx access log to see the path that actually arrived
Fix	Correct the routing rule or the request path. Check the ACCESS log - it tells you the exact path the server received, which is frequently not the path you think you sent.

Scenario 5 - HTTP 500

Field	Detail
Meaning	The application code raised an unhandled error
Causes	Bug in a new release; a dependency such as the database is unreachable; missing environment variable or config; bad credentials; a schema mismatch after a migration
Commands	Application logs first - find the stack trace. <code>kubectl logs -f deploy/app</code> , <code>journalctl -u app</code> , <code>docker logs</code> . Correlate the timestamp with the deploy time.
Fix	If it started at a deploy, roll back first and diagnose afterwards. Otherwise fix the dependency or config.

Scenario 6 - HTTP 502 Bad Gateway

Field	Detail
Meaning	The proxy reached a backend but got an invalid response or the connection was closed
Causes	Backend crashed or is restarting (OOMKilled is a classic); proxy pointing at the wrong port; backend speaking HTTPS while the proxy expects HTTP (or the reverse); response headers too large; backend killed mid-response
Commands	Proxy error log (<code>/var/log/nginx/error.log</code>), backend health from the proxy host (<code>curl -v http://backend:8080/health</code>), <code>kubectl get pods</code> for restarts, ALB target group health
Fix	Restore the backend, correct the upstream port or protocol, raise header buffer sizes if that is the cause

Scenario 7 - HTTP 503 Service Unavailable

Field	Detail
Meaning	No healthy backend is available to serve the request
Causes	All targets failing health checks; Kubernetes Service with zero endpoints because no pod is Ready; deliberate maintenance mode; rate limiting or an overloaded upstream queue
Commands	<code>kubectl get endpoints svc-name</code> (empty is the giveaway), <code>kubectl get pods -o wide</code> , ALB target group health, proxy upstream status
Fix	Get at least one backend passing readiness. Check the readiness probe path, port and timing - an overly aggressive probe can take a healthy fleet out.

Scenario 8 - HTTP 504 Gateway Timeout

Field	Detail
Meaning	The backend was reached but did not respond within the proxy's timeout

Field	Detail
Causes	Slow database query or missing index; downstream API hanging; thread/connection pool exhausted; proxy timeout shorter than the real processing time; deadlock
Commands	Backend latency metrics, slow query log, <code>curl -w "%{time_total}"</code> directly against the backend, thread dump for a JVM
Fix	Fix the slow path first. Raising the timeout hides the problem - only do it when the work genuinely takes that long, and then make the operation asynchronous.

Scenario 9 - Port blocked / firewall

Field	Detail
Symptom	Timeout from outside, works from <code>localhost</code> on the box
Isolation	Test from a host in the same subnet: if that works, the problem is at the subnet/SG boundary, not on the host. <code>sudo ss -tulpn</code> confirms the process is listening on 0.0.0.0.
Layers to check in order	Host firewall -> security group -> NACL -> route table -> corporate/edge firewall

Scenario 10 - Wrong route / asymmetric routing

Field	Detail
Symptom	Some destinations work, others time out; or traffic leaves but nothing returns
Commands	<code>ip route get 10.0.2.50</code> shows exactly which route and source IP would be used; check the VPC route table for the subnet
Cause	Private subnet missing the 0.0.0.0/0 -> NAT gateway route; peering route absent; overlapping CIDRs between peered VPCs

Scenario 11 - Server not listening on the expected interface

Field	Detail	
Symptom	<code>curl localhost:8080</code> works on the host, everything remote times out or is refused	
Command	<code>sudo ss -tulpn \</code>	<code>grep 8080 - look at the ADDRESS column: 127.0.0.1:8080 is local-only, 0.0.0.0:8080 or *:8080 accepts from anywhere</code>
Fix	Change the application's bind address, typically a config value such as <code>host: 0.0.0.0</code> . In Docker this is the number one cause of "the container runs but I cannot reach it".	

4.12 Networking Interview Questions

Q. 1. Explain the OSI model and why it matters operationally.

Seven layers from physical up to application. Operationally I use it as a fault-isolation ladder: layer 3 asks whether the packet can reach the host, layer 4 asks whether something is listening on the port, and layer 7 asks whether the application answered correctly. Localising the fault to a layer is what stops troubleshooting from being guesswork.

Q. 2. Difference between TCP and UDP?

TCP is connection-oriented with a handshake, acknowledgements, retransmission and ordering, so it guarantees delivery at the cost of overhead. UDP just sends datagrams with no handshake or guarantee, which makes it faster and suitable for DNS queries, NTP, syslog and streaming media, where a late packet is worse than a lost one.

Q. 3. Walk me through the TCP three-way handshake.

The client sends SYN with an initial sequence number, the server replies SYN-ACK acknowledging it and sending its own sequence number, and the client sends ACK. The connection is then established. Teardown is four packets - FIN, ACK, FIN, ACK - and the side that closes first holds the socket in TIME_WAIT for twice the maximum segment lifetime to absorb stray packets.

Q. 4. What is the difference between connection refused and connection timed out?

Refused means the host is reachable and actively rejected the connection with an RST - so nothing is listening on that port, and the problem is the service. Timed out means no answer at all - a firewall, security group, NACL or routing problem, or the host is down. The two lead to completely different investigations, so it is the first thing I read from an error.

Q. 5. What is a subnet mask and what does /24 mean?

The mask splits the address into a network part and a host part. /24 means the first 24 bits are the network, leaving 8 host bits - 256 addresses, 254 usable on a normal network, 251 in AWS because it reserves five.

Q. 6. How many usable IPs in a /28 in AWS?

16 total minus AWS's five reserved addresses, so 11 usable.

Q. 7. What are the private IP ranges?

10.0.0.0/8, 172.16.0.0/12 and 192.168.0.0/16. Docker's default bridge sits in 172.17.0.0/16, which is a real source of conflicts when a corporate network also uses 172.16/12.

Q. 8. What is 169.254.169.254?

The link-local instance metadata endpoint in AWS and other clouds. It returns instance identity, user data and, importantly, temporary credentials for the attached IAM role. That is why an application on EC2 should use a role rather than access keys - and why SSRF vulnerabilities are so dangerous, which is what IMDSv2's token requirement mitigates.

Q. 9. How does DNS resolution work?

The resolver checks local caches and `/etc/hosts`, then asks the recursive resolver configured in `resolv.conf`. If it has no cached answer, the recursive resolver queries a root server, which refers it to the TLD server, which refers it to the zone's authoritative name server, which returns the record. Every step caches the answer for the record's TTL.

Q. 10. Difference between A record and CNAME?

An A record maps a name directly to an IPv4 address. A CNAME maps a name to another NAME, which then has to be resolved. A CNAME cannot coexist with other records at the same name and cannot be used at the zone apex - which is precisely why Route 53 has ALIAS records.

Q. 11. Why can you not use a CNAME at the zone apex?

The apex must carry SOA and NS records, and the DNS specification forbids a CNAME coexisting with any other record at the same name. Route 53 solves it with ALIAS, which is resolved server-side and returns the target's A record.

Q. 12. What is TTL in DNS and how does it bite you?

It is how long resolvers may cache a record. If you change a record without lowering the TTL in advance, clients keep using the old value for up to the old TTL. So before a planned cutover you lower the TTL a day or two ahead, make the change, and raise it again afterwards.

Q. 13. What is the difference between a forward and a reverse proxy?

A forward proxy sits in front of clients and fetches the internet on their behalf - egress control and filtering. A reverse proxy sits in front of servers and receives requests on their behalf - TLS termination, routing, caching and load balancing. nginx and ALB are reverse proxies.

Q. 14. ALB versus NLB - when do you use which?

ALB is layer 7, so it can route on host header and URL path and terminate TLS with ACM, which suits web applications and microservices. NLB is layer 4 - it forwards TCP or UDP without understanding the payload, gives you static IPs per AZ, preserves the client source IP, and handles extreme throughput. For a non-HTTP protocol such as RTSP, or when a static IP is required, NLB is the answer.

Q. 15. Security group versus NACL?

Security groups are stateful, instance-level, allow-only, and can reference other security groups. NACLs are stateless, subnet-level, support both allow and deny, and are evaluated in rule-number order. Because NACLs are stateless you must explicitly allow the ephemeral return ports, which is a very common cause of mysterious timeouts.

Q. 16. What is NAT and why do private subnets need it?

NAT translates private source addresses to a public address so instances without public IPs can reach the internet for updates and API calls, while remaining unreachable from the internet. In AWS that is a NAT Gateway placed in a public subnet, with the private subnet's route table sending 0.0.0.0/0 to it.

Q. 17. What is the difference between an Internet Gateway and a NAT Gateway?

An Internet Gateway gives a subnet two-way internet connectivity for resources that have public IPs - that is what makes a subnet public. A NAT Gateway gives one-way outbound connectivity for private resources; nothing on the internet can initiate a connection back through it.

Q. 18. How do you check which process is using a port?

`sudo ss -tulpn | grep :PORT` Or `sudo lsof -i :PORT`. I also check the bind address in the output, because a process bound to 127.0.0.1 is unreachable remotely regardless of firewall rules.

Q. 19. Why might ping fail but the service still work?

ICMP is frequently blocked by firewalls and security groups while TCP on the service port is allowed. Ping failing proves nothing on its own - I test the actual port with `nc -zv` or `curl`.

Q. 20. What is MTU and when does it matter?

Maximum transmission unit - the largest frame that can be sent without fragmentation, normally 1500 bytes. It matters with VPNs, tunnels and some overlay networks where encapsulation overhead pushes packets over the limit. The classic symptom is that small requests work and large ones hang - fixed by lowering MTU or enabling MSS clamping.

Q. 21. What does traceroute actually do?

It sends packets with increasing TTL values; each router that decrements the TTL to zero returns an ICMP time-exceeded message, revealing the hop. Stars mean a hop did not reply, which is often just a router configured not to respond rather than a fault - so I never read a single line of stars as a failure.

Q. 22. What is the ephemeral port range?

The range of source ports the OS assigns to outbound connections, typically 32768-60999 on Linux. It matters for stateless NACLs, which must allow return traffic on that range, and for connection exhaustion under very high concurrency.

Q. 23. How does HTTPS actually protect the request?

TLS provides encryption so the content cannot be read in transit, integrity so it cannot be modified undetected, and authentication of the server through its certificate chain. Note that SNI and the destination IP are still visible - the hostname is not fully hidden on the wire.

Q. 24. What is SNI and why does it matter?

Server Name Indication is the hostname the client puts in the ClientHello, in plaintext, so a server hosting many sites on one IP knows which certificate to present. It matters because without it you cannot host multiple TLS sites on a single IP, and because `openssl s_client` needs `-servername` or you will get the wrong certificate back and misdiagnose the problem.

Q. 25. A certificate is valid in the browser but fails in curl. Why?

Almost always a missing intermediate certificate. Browsers often cache or fetch intermediates, while curl validates strictly against the chain the server sends. The fix is to serve the full chain.

Q. 26. What is X-Forwarded-For?

A header the load balancer or proxy adds containing the original client IP, because the backend otherwise only sees the proxy's IP. Applications that log or rate-limit by IP must read it - and must only trust it when it comes from a known proxy, otherwise clients can forge it.

Q. 27. What is a sticky session and what is the downside?

The load balancer pins a client to one backend, usually with a cookie, so in-memory session state stays valid. The downside is uneven load distribution and the fact that losing that backend loses those sessions. The better answer is to make the application stateless and store sessions in Redis or a database.

Q. 28. How do you test whether a port is open from a machine with no tools installed?

Bash's `/dev/tcp:timeout 3 bash -c "</dev/tcp/10.0.2.50/3306" && echo open`. Minimal container images typically have no ping, netcat or telnet, so this is worth knowing.

Q. 29. What is the difference between latency, bandwidth and throughput?

Latency is the delay for one packet to travel; bandwidth is the theoretical maximum capacity; throughput is what you actually achieve. High bandwidth with high latency still feels slow for chatty protocols, which is why reducing round trips often helps more than adding bandwidth.

Q. 30. What causes intermittent connectivity and how do you catch it?

Packet loss on one path, a single unhealthy backend behind a load balancer, DNS returning several records where one is dead, MTU issues on large payloads, or a flapping health check. I use `mtr` over time to find loss per hop, and I test each backend directly rather than through the load balancer to identify which one is bad.

Q. 31. What is the difference between a VPC peering connection and a Transit Gateway?

Peering is a one-to-one private link between two VPCs and is not transitive - A peered to B and B peered to C does not let A talk to C. Transit Gateway is a hub that many VPCs attach to, giving transitive routing and scaling far better past a handful of VPCs.

Q. 32. Can two VPCs with overlapping CIDRs be peered?

No. Routing would be ambiguous. You have to re-address one side, or use a NAT/PrivateLink approach to expose specific services instead of routing whole networks.

Q. 33. What is a bastion host?

A hardened instance in a public subnet that is the only SSH entry point into the environment; private instances accept SSH only from its security group. It shrinks the attack surface to one auditable box. Modern alternatives are SSM Session Manager or Instance Connect, which remove the need for an open SSH port entirely.

Q. 34. How does a load balancer decide a target is unhealthy?

It sends periodic health checks - a TCP connect or an HTTP request to a configured path - and marks the target unhealthy after a threshold of consecutive failures, then removes it from rotation until it passes a healthy threshold again. Misconfiguring the path or the timeouts is a very common cause of self-inflicted outages.

Q. 35. What is DNS round robin and why is it not a load balancer?

Returning multiple A records so clients spread across them. It is not load balancing because there is no health awareness - a dead IP keeps being handed out until the record is changed - and client caching makes distribution uneven.

Q. 36. What is a reverse DNS lookup used for?

Mapping an IP back to a name using PTR records. Mail servers check it for spam scoring, and some logging and access-control systems rely on it. `dig -x 1.2.3.4`.

Q. 37. What is the difference between HTTP/1.1 and HTTP/2?

HTTP/1.1 uses one request per connection at a time and suffers head-of-line blocking, so browsers open several connections. HTTP/2 multiplexes many streams over one TCP connection, compresses headers and supports server push. Operationally, HTTP/2 means fewer connections per client, which changes connection-count based capacity assumptions.

Q. 38. What is a CDN and what problem does it solve?

A distributed network of edge caches serving content close to users. It reduces latency, absorbs traffic spikes, offloads the origin and provides a DDoS buffer. Operationally the hard part is cache invalidation and making sure you are not caching authenticated responses.

Q. 39. What is RTSP and how is it different from HTTP?

RTSP is a control protocol for streaming media - it handles PLAY, PAUSE and TEARDOWN on port 554 while the media itself usually flows over RTP, often on UDP. It is not HTTP, so a layer-7 HTTP load balancer cannot route it; it needs layer-4 handling, and firewalls have to permit the media ports as well as the control port. In my current role that distinction matters because video components are part of the stack.

Q. 40. Application works from the server itself but not from outside. Walk me through it.

First `ss -tulpn` to check the bind address - 127.0.0.1 versus 0.0.0.0 explains it more often than anything else. If it is bound correctly, I test from another host in the same subnet to separate host-level from network-level problems. Then host firewall, then security group, then NACL including the return path, then route table, then anything at the edge like a load balancer target group being unhealthy. I work outward one layer at a time and confirm each layer before moving on.

PART 4 RECALL POINTS

- Refused = reachable, nothing listening. Timeout = packet dropped.
- AWS reserves 5 IPs per subnet; a /24 gives 251 usable.
- Security groups stateful, NACLs stateless - allow the ephemeral range.
- 500 = app, 502 = bad backend response, 503 = no healthy backend, 504 = backend too slow.
- Lower DNS TTL before a cutover, not during.
- ALB is HTTP-aware; NLB is L4, static IP, preserves source IP - the right choice for RTSP.
- `ss -tulpn` first: check the BIND ADDRESS, not just the port.
- `openssl s_client -connect host:443 -servername host` for every TLS question.

PART 5

GIT AND GITHUB

Architecture, commands, branching, conflicts, and the DevOps workflow

Your CV says you work in "Git/GitHub branching and pull-request workflows". Git questions are almost guaranteed and they are easy marks - provided you can explain the four areas of Git, the difference between merge and rebase, and how to recover from a mistake without panicking.

5.1 Git Architecture - The Four Areas



Where your changes live at each moment

Area	What it holds	Command to inspect
Working tree	The files you can see and edit	<code>git status</code> , <code>git diff</code>
Staging area (index)	The exact snapshot the next commit will contain	<code>git diff --staged</code>
Local repository	Full history, all branches, in <code>.git/</code>	<code>git log</code> , <code>git branch</code>
Remote repository	The shared copy on GitHub	<code>git remote -v</code> , <code>git log origin/main</code>

GIT IS DISTRIBUTED

Every clone contains the complete history, not a working copy of a central server. That is why `git log` and `git diff` work offline, why branching is cheap, and why you can commit on a plane. The "central" repository is central only by convention.

5.2 The Everyday Commands

Setup and starting a repository

```

git config --global user.name "Aasim Raza"
git config --global user.email "aasimdevops@gmail.com"
git config --global init.defaultBranch main
git config --global pull.rebase false           # explicit merge on pull (or true for rebase)
git config --list --show-origin                 # where each setting comes from

git init                                         # new repository in this directory
git clone https://github.com/org/repo.git        # copy an existing one
git clone git@github.com:org/repo.git           # over SSH
git clone --depth 1 https://...                 # shallow clone - fast CI checkouts
git clone -b develop https://...               # clone a specific branch

```

The daily loop

```

git status                                     # what is modified, staged, untracked
git status -s                                 # short form: M modified A added ?? untracked
git add file.txt                               # stage one file
git add .                                       # stage everything in this directory downward
git add -A                                     # stage everything including deletions
git add -p                                     # interactively stage HUNKS - keeps commits clean
git restore file.txt                           # discard working-tree changes (was: git checkout -- file)
git restore --staged file.txt                  # unstage but keep the edit (was: git reset HEAD file)

git commit -m "Add health check stage to pipeline"
git commit -am "Fix timeout"                  # stage tracked files and commit in one step

```

```
git commit --amend          # rewrite the LAST commit - never after pushing to a shared branch

git push origin main
git push -u origin feature/x # set upstream so later `git push` needs no arguments
git push --force-with-lease  # safer force: refuses if someone else pushed in the meantime

git fetch origin            # download refs WITHOUT changing your working tree
git pull origin main        # fetch + merge
git pull --rebase origin main # fetch + rebase - keeps history linear
```

FETCH VS PULL - A GUARANTEED QUESTION

"fetch downloads remote changes into remote-tracking branches but does not touch my working tree, so it is always safe - I can then inspect with `git log HEAD..origin/main` before integrating. pull is fetch plus merge, so it changes my working tree immediately. On anything important I fetch first and look."

5.3 Branching

Branch operations

```
git branch                # local branches; * marks the current one
git branch -a             # include remote-tracking branches
git branch -vv            # show upstream and ahead/behind counts
git branch feature/pipeline-cache # create without switching
git switch feature/pipeline-cache # switch (modern)
git switch -c feature/new   # create and switch
git checkout -b feature/new # the older equivalent
git branch -d feature/done  # delete a MERGED branch
git branch -D feature/abandoned # force delete an unmerged branch
git push origin --delete feature/done # delete the remote branch
git branch --merged        # branches fully merged into the current one - safe to delete
```

Command	Modern equivalent	Why the change
<code>git checkout branch</code>	<code>git switch branch</code>	checkout was overloaded - it switched branches AND restored files
<code>git checkout -- file</code>	<code>git restore file</code>	Separating the two removed a very common accidental data loss
<code>git checkout -b x</code>	<code>git switch -c x</code>	Clearer intent

5.3.1 Branching strategies

Strategy	How it works	Suits
Git Flow	main, develop, feature/*, release/*, hotfix/*	Scheduled releases, versioned products - heavier
GitHub Flow	main is always deployable; short-lived feature branches merged by PR	Continuous delivery - simple, most common
Trunk-based	Everyone commits to main behind feature flags; branches live hours	High-maturity CD teams
Environment branches	dev/staging/prod branches, promoted by merge	Common but causes drift and merge pain - be ready to critique it

"WHAT BRANCHING STRATEGY DO YOU USE?"

"We work close to GitHub Flow: main is the protected, releasable branch, work happens on short-lived feature branches, and everything merges through a pull request with review and a passing pipeline. Release tagging marks what went to production. I prefer short-lived branches because long-lived ones accumulate merge conflicts and delay integration feedback, which is exactly what CI is supposed to prevent."

5.4 Merge vs Rebase - The Classic Question

```

MERGE (preserves the true history, adds a merge commit)
  A---B---C  main
    \       \
    D---E---M  feature merged into main  (M is the merge commit)

REBASE (replays your commits on top of the new base - linear history)
  A---B---C  main
    \       \
    D'---E'   feature rebased (D and E are NEW commits with new hashes)

```

Same work, two histories

	Merge	Rebase
History	Preserves the actual branch topology	Rewritten into a straight line
Extra commit	Yes, a merge commit	No
Commit hashes	Unchanged	New hashes for every replayed commit
Conflict handling	Resolved once, in the merge	Possibly once per replayed commit
Safe on shared branches	Yes	NO - it rewrites history others have
Best for	Integrating a finished feature into main	Updating your own feature branch onto the latest main before a PR

THE GOLDEN RULE OF REBASE

Never rebase a branch that other people have pulled. Rewriting shared history forces everyone else into a painful recovery. Rebase your own unpushed or personal feature branch to keep it current; merge when integrating into a shared branch. Say exactly that in an interview.

Merging and rebasing in practice

```

git switch main && git pull
git merge feature/x           # merge feature into main
git merge --no-ff feature/x   # always create a merge commit - keeps the feature visible
git merge --squash feature/x   # collapse all the feature commits into one staged change

git switch feature/x
git rebase main                # replay my commits on top of the latest main
git rebase --continue          # after resolving a conflict
git rebase --abort              # bail out and return to the previous state
git rebase -i HEAD~4           # interactive: squash, reword, reorder, drop the last 4 commits

```

5.5 Merge Conflicts

A conflict happens when two branches change the same lines of the same file, or one deletes a file the other edited. Git cannot decide, so it stops and asks you.

What a conflict looks like in the file

```

<<<<<<< HEAD
    timeout = 30
=====
    timeout = 60
>>>>>>> feature/increase-timeout

```

HEAD is YOUR current branch. The part after ===== is the incoming change. Resolve by editing the file into what it should actually be, then DELETE all three markers.

Resolving

```

git status           # lists "both modified" files
git diff             # see the conflicting hunks
# ... edit the files, remove the markers ...
git add resolved-file.py  # marking resolved = staging it
git commit           # completes the merge (or: git rebase --continue)

git merge --abort     # give up and go back to before the merge
git checkout --ours file  # take MY version wholesale
git checkout --theirs file # take the INCOMING version wholesale

```

```
git mergetool # open a three-way merge tool if configured
```

CONFLICT MISTAKES THAT SHOW INEXPERIENCE

- Committing a file that still contains <<<<<< markers - it happens more often than you would think, and it breaks the build.
- Blindly taking --ours or --theirs without reading the change, which silently discards someone's work.
- Resolving a conflict without running the tests afterwards. A syntactically merged file can still be semantically wrong.

Prevention: short-lived branches, rebase onto main frequently, and keep commits small.

5.6 Undoing Things - The Recovery Toolkit

Situation	Command	Effect
Discard uncommitted change in a file	<code>git restore file</code>	Working tree reverts; change is gone
Unstage a file but keep the edit	<code>git restore --staged file</code>	Moves it out of the index
Fix the last commit message	<code>git commit --amend -m "new message"</code>	Rewrites the last commit - local only
Add a forgotten file to the last commit	<code>git add f && git commit --amend --no-edit</code>	Same commit, extra file
Undo last commit, KEEP the changes staged	<code>git reset --soft HEAD~1</code>	Commit gone, work preserved and staged
Undo last commit, keep changes unstaged	<code>git reset HEAD~1 (mixed, the default)</code>	Commit gone, work in the working tree
Undo last commit and DESTROY the changes	<code>git reset --hard HEAD~1</code>	Dangerous - work is lost
Undo a commit that is already pushed	<code>git revert <sha></code>	Creates a NEW commit that reverses it - safe on shared branches
Recover a "lost" commit or branch	<code>git reflog then git reset --hard <sha></code>	Reflog records where HEAD has been for ~90 days
Temporarily park work	<code>git stash</code>	Clears the working tree, saves the changes

RESET VS REVERT - NEVER CONFUSE THESE IN AN INTERVIEW

"reset moves the branch pointer and rewrites history, so it is for local commits that nobody has seen. revert creates a new commit that undoes a previous one, leaving history intact, so it is the correct way to undo something already pushed to a shared branch. On a protected branch I always revert."

stash, cherry-pick, tag, and the investigation commands

```
git stash # park working changes
git stash -u # include untracked files
git stash list
git stash pop # reapply the most recent and drop it
git stash apply stash@{2} # reapply a specific one, keeping it in the list
git stash drop stash@{0}

git cherry-pick <sha> # apply ONE commit from another branch onto this one
git cherry-pick <sha1>..<sha2> # a range
git cherry-pick --abort
# Use case: a hotfix committed on main that must also go onto the release branch.

git tag v1.4.2 # lightweight tag
git tag -a v1.4.2 -m "Release 1.4.2" # annotated - has author, date, message; use this
git push origin v1.4.2
git push origin --tags
git tag -l "v1.4.*"
git describe --tags # nearest tag + commits since - great for build versions

git log --oneline --graph --decorate --all # the history picture
git log -p file.py # every change to one file, with diffs
git log --author="Aasim" --since="2 weeks ago"
```

```
git log --grep="timeout"           # search commit messages
git log main..feature/x           # commits on feature not yet on main
git show <sha>                     # full detail of one commit
git diff main..feature/x          # difference between branches
git diff --stat HEAD~5            # summary of files changed
git blame app.py                  # who last changed each line, and in which commit
git bisect start && git bisect bad && git bisect good v1.3.0 # binary search for the breaking commit
```

GIT BISECT IS AN INTERVIEW DIFFERENTIATOR

"If a bug appeared somewhere in the last 200 commits, I use `git bisect`. I mark a known-good tag and the current bad commit, and Git checks out the midpoint repeatedly - I test, say good or bad, and in about eight steps it identifies the exact commit that introduced the problem. It can even be automated with `git bisect run ./test.sh`." Very few candidates mention this.

5.7 Git Internals Worth Knowing

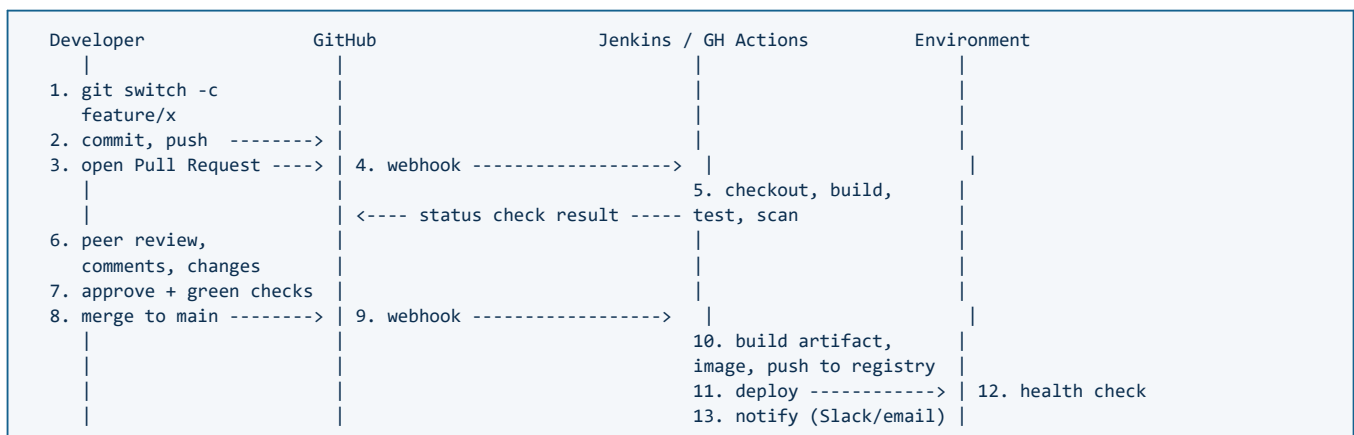
Object	What it is
Blob	The contents of a file - no name, no metadata
Tree	A directory listing: names mapped to blobs and other trees
Commit	A snapshot pointer: one tree, parent commit(s), author, committer, message
Tag (annotated)	A named pointer to a commit with its own metadata
HEAD	A pointer to the current branch (or directly to a commit in "detached HEAD")
Branch	Simply a movable pointer to a commit - which is why branching is instant

DETACHED HEAD, EXPLAINED CALMLY

"It means HEAD points at a commit rather than a branch - usually after checking out a tag or a specific SHA. Commits made there belong to no branch and will eventually be garbage collected. The fix is `git switch -c rescue-branch` to give them a name, or just `git switch main` to leave. It is not an error state, it is just an unnamed position."

5.8 Pull Requests, Protection and Hooks

5.8.1 The pull request workflow



From idea to production - the workflow your CV describes

Branch protection rule	What it prevents
Require a pull request before merging	Direct pushes to main
Require N approvals	Unreviewed code
Require status checks to pass	Merging a red build
Require branches to be up to date	Merging something never tested against current main
Dismiss stale approvals on new commits	Approval of code that has since changed

Branch protection rule	What it prevents
Restrict who can push	Accidental force-push to main
Require signed commits	Unverified authorship
Include administrators	Admins bypassing the rules

5.8.2 Git hooks

Hook	When it fires	DevOps use
pre-commit	Before the commit is created	Lint, format, secret scanning, block large files
commit-msg	After the message is written	Enforce a convention such as JIRA-123: summary
pre-push	Before pushing	Run the fast unit tests
post-receive	On the server after a push	Trigger a deployment or notification
pre-receive	On the server before accepting	Reject pushes violating policy

Hooks live in `.git/hooks/` and are not versioned by default - which is why teams use a framework such as `pre-commit` that stores the configuration in the repository and installs the hooks for everyone.

5.8.3 Authentication

Method	Use	Notes
SSH key	Developer workstations	<code>ssh-keygen -t ed25519</code> , add the public key to GitHub, test with <code>ssh -T git@github.com</code>
Personal access token	HTTPS, scripts, legacy CI	Passwords were removed for Git operations; treat a PAT like a password and scope it minimally
Deploy key	One repository, read-only, on a server	Perfect for a CI agent that only needs to clone one repository
GitHub App / OIDC	Modern CI	Short-lived tokens, no long-lived secrets - the direction of travel

SECRETS IN GIT

If a credential is ever committed and pushed, rotating it is mandatory - removing the file in a later commit does not remove it from history, and anyone with a clone still has it. Cleaning history requires `git filter-repo` or BFG plus a force-push, and coordination with everyone who has a clone. Prevention: `.gitignore`, `pre-commit` secret scanning, GitHub secret scanning, and never putting real values in `.env` files that are tracked.

A practical `.gitignore` for a DevOps repository

```
*.tfstate
*.tfstate.*
.terraform/
.terraform.lock.hcl
*.tfvars          # variable files often contain secrets
.env
*.pem
*.key
id_rsa*
__pycache__/*
*.pyc
.venv/
node_modules/
target/
*.log
.DS_Store
.idea/
.vscode/
```

5.9 Git Interview Questions

Q. 1. What is Git and how is it different from SVN?

Git is a distributed version control system - every clone holds the full history, so committing, branching, diffing and viewing history all work offline. SVN is centralised: the server holds history and most operations need it. Distribution is also why

branching in Git is instant - a branch is just a pointer to a commit.

Q. 2. Explain the three (or four) areas of Git.

Working tree - the files I edit. Staging area or index - the snapshot my next commit will contain. Local repository - the full history in `.git`. And the remote repository, the shared copy. `add` moves work from the tree to the index, `commit` from the index to the local repository, `push` from local to remote.

Q. 3. Difference between `git fetch` and `git pull`?

`Fetch` downloads remote changes into remote-tracking branches without touching my working tree - it is always safe. `Pull` is `fetch` plus `merge`, so it changes my working tree immediately. I usually `fetch` and inspect with `git log HEAD..origin/main` before merging.

Q. 4. Difference between merge and rebase?

`Merge` integrates two branches and creates a merge commit, preserving the real topology. `Rebase` replays my commits on top of a new base, producing new commit hashes and a linear history. I rebase my own feature branch to stay current with main, and I merge to integrate into a shared branch - I never rebase anything others have pulled.

Q. 5. Difference between `git reset` and `git revert`?

`Reset` moves the branch pointer, rewriting history - fine locally, dangerous on a shared branch. `Revert` creates a new commit that undoes a previous one, leaving history intact - the correct choice for anything already pushed.

Q. 6. What are the three modes of `git reset`?

`--soft` moves the branch pointer and leaves the changes staged; `--mixed`, the default, leaves them in the working tree unstaged; `--hard` discards them entirely. Only `--hard` loses work, and even then `git reflog` usually rescues it.

Q. 7. What is `git cherry-pick` and when have you needed it?

It applies a single commit from one branch onto another. The classic case is a hotfix committed on main that also needs to go onto a release branch without bringing the rest of main with it.

Q. 8. How do you resolve a merge conflict?

`git status` lists the conflicted files; I open each one, look at both sides of the conflict markers, edit it into the correct result, remove the markers, `git add` the file to mark it resolved, then `commit` or `git rebase --continue`. Then I run the tests, because a file can merge cleanly at the text level and still be wrong. `git merge --abort` backs out entirely if I want to start again.

Q. 9. What is `git stash` for?

Parking uncommitted work so I can switch context - for example a production hotfix arriving mid-feature. `git stash`, fix the urgent thing, then `git stash pop` to get my work back. `-u` includes untracked files.

Q. 10. How do you recover a commit you deleted with `reset --hard`?

`git reflog` shows everywhere HEAD has pointed, including the commit I "lost". I find its SHA and `git reset --hard <sha>` or `git switch -c recovered <sha>`. `Reflog` entries survive about 90 days by default, which makes almost anything local recoverable.

Q. 11. What is a detached HEAD?

HEAD pointing directly at a commit rather than a branch, usually after checking out a tag or SHA. Any commits made there have no branch and will eventually be garbage collected, so I give them a name with `git switch -c` if I want to keep them.

Q. 12. What does `git rebase -i` let you do?

Rewrite a series of local commits before sharing them - squash noisy WIP commits into one, reword messages, reorder or drop commits, or split one. I use it to turn eight messy commits into one clean, reviewable change before opening a PR.

Q. 13. What is the difference between a lightweight and an annotated tag?

A lightweight tag is just a pointer to a commit. An annotated tag is a real object with a tagger, date, message and optional signature. Releases should use annotated tags because they carry provenance - and `git describe` relies on them.

Q. 14. What is `git bisect`?

A binary search through history to find the commit that introduced a bug. I mark a known-good commit and a bad one, Git checks out midpoints, I test and report good or bad, and it converges in about $\log_2(n)$ steps. It can be fully automated with `git bisect run`.

Q. 15. How do you squash commits?

Either `git rebase -i HEAD~n` and mark commits as `squash/fixup`, or use "Squash and merge" on the pull request so the whole branch lands as one commit on main. Many teams enforce the latter to keep main's history readable.

Q. 16. What is a fast-forward merge?

When the target branch has no new commits since the feature branched, Git can simply move the pointer forward with no merge commit. `--no-ff` forces a merge commit anyway so that the feature remains visible as a unit in history.

Q. 17. What does `git push --force-with-lease` do and why prefer it over `--force`?

Both rewrite the remote branch, but `--force-with-lease` refuses if the remote has commits you have not seen - so it cannot silently destroy a colleague's work pushed since your last fetch. Plain `--force` has no such check.

Q. 18. What is `.gitignore` and what belongs in it?

A list of path patterns Git should not track. For DevOps repositories: state files such as `*.tfstate`, `.terraform/`, `.env`, private keys, build output, `node_modules`, and IDE directories. Note that it only affects UNTRACKED files - if something is already tracked you must `git rm --cached` it first.

Q. 19. How do you remove a file from history after committing a secret?

Rotate the credential first - that is the actual mitigation, because anyone with a clone already has it. Then rewrite history with `git filter-repo` or BFG, force-push, and have everyone re-clone. Prevention is better: pre-commit secret scanning and GitHub secret scanning.

Q. 20. What are Git hooks and give a DevOps example?

Scripts Git runs at lifecycle points. `pre-commit` for linting, formatting and secret scanning; `commit-msg` to enforce a ticket-ID convention; `pre-push` to run fast tests; `post-receive` on the server to trigger a deploy. They live in `.git/hooks` and are not versioned, so teams use the `pre-commit` framework to share them.

Q. 21. What is a fork versus a branch?

A branch is a pointer inside one repository. A fork is a full server-side copy of the repository under a different account, used when contributors do not have write access - typically open source. The contribution then flows back as a pull request across repositories.

Q. 22. What is branch protection and what would you enable on main?

Require a pull request with at least one approval, require status checks to pass, require the branch to be up to date with main, dismiss stale approvals when new commits arrive, forbid force-pushes and deletions, and apply the rules to administrators too. That combination means nothing reaches main without review and a green pipeline.

Q. 23. What is the difference between `git diff`, `git diff --staged` and `git diff HEAD`?

`git diff` shows working tree versus index - what you have not staged. `--staged` (or `--cached`) shows index versus last commit - what you are about to commit. `HEAD` shows working tree versus last commit - everything uncommitted.

Q. 24. How do you see who changed a specific line?

`git blame file` annotates each line with the commit, author and date, then `git show <sha>` for the full context. `git log -S "searchstring"` finds commits that added or removed a specific string, which is often more useful than blame when code has moved.

Q. 25. What is a shallow clone and when do you use one?

`git clone --depth 1` fetches only the latest commit rather than the whole history. In CI it dramatically speeds up checkout and reduces disk usage on agents, which matters when a pipeline runs hundreds of times a day. The caveat is that anything relying on history - `git describe`, `bisect`, `blame` - will not work.

Q. 26. How do you handle a long-running feature branch?

Ideally avoid it - break the work into smaller, mergeable pieces behind a feature flag. If it must exist, rebase or merge main into it frequently so conflicts stay small, and keep it building in CI. The cost of a long-lived branch is paid all at once at merge time, and that is the failure mode CI exists to prevent.

Q. 27. What happens when you `git commit`?

Git creates a tree object representing the staged snapshot, then a commit object containing that tree, the parent commit, author and committer metadata and your message; the branch pointer moves to the new commit. Every object is content-addressed by SHA, which is why history is tamper-evident.

Q. 28. What is the difference between `origin` and `upstream`?

They are just remote names by convention. `origin` is the remote you cloned from - usually your fork or the main repository. `upstream` is conventionally the original project when you are working from a fork, so you can pull in changes from the source.

Q. 29. How does Git store data - as diffs or snapshots?

As snapshots. Each commit references a complete tree of the project; unchanged files simply reuse the same blob object. Git computes diffs on demand for display. Packfiles do apply delta compression for storage, but the model you reason about is snapshots.

Q. 30. How would you set up Git for a CI agent?

A read-only deploy key or a scoped token rather than a personal credential, a shallow clone for speed, `GIT_TERMINAL_PROMPT=0` so it never hangs waiting for input, credentials injected from the CI credential store rather than baked into the job, and the workspace cleaned between builds to avoid stale files affecting results.

Q. 31. Your push is rejected as non-fast-forward. What happened and what do you do?

Someone pushed to that branch after my last fetch, so my history is behind. I `git pull --rebase` to replay my commits on top of theirs, resolve any conflicts, re-run tests and push. I do not force-push to a shared branch to make the error go away - that would delete their commits.

Q. 32. What is the difference between `git rm` and `rm`?

`rm` deletes the file from disk and leaves Git to notice it as an unstaged deletion; `git rm` deletes it and stages the deletion in one step. `git rm --cached` is the important one - it stops tracking a file while leaving it on disk, which is how you fix accidentally committing something that belongs in `.gitignore`.

Q. 33. How do you tag a release from a pipeline?

Create an annotated tag with the build version, push it, and use it as the immutable identifier for the artifact and the container image. `git describe --tags` is a convenient way to derive a version string. The key property is that a tag ties a running artifact back to an exact commit, which is what makes an incident traceable.

Q. 34. Explain the workflow from a developer's commit to production in your environment.

The developer works on a short-lived branch and opens a pull request. That triggers CI - checkout, build, unit tests and scanning - and the result appears as a required status check. A peer reviews it; with an approval and a green build it merges to main. The merge triggers the main pipeline, which builds the artifact and the Docker image, tags them with the build number, pushes the image to the registry, and deploys to the lower environment, followed by a health check. Production is a controlled, approved step using that same artifact, and the deployment notifies the team. If it goes wrong, rollback is deploying the previous image tag.

Q. 35. What is a monorepo and what does it change for CI?

One repository holding many services or components. It simplifies cross-cutting changes and dependency alignment, but CI has to become path-aware - you only build and deploy the components whose paths actually changed, otherwise every commit rebuilds everything. Jenkins multibranch with changeset conditions, or GitHub Actions path filters, handle that.

Q. 36. How do you keep a fork in sync?

Add the original as a remote - `git remote add upstream <url>` - then `git fetch upstream` and `git rebase upstream/main` (or merge) on your branch, and push. The important part is doing it regularly so the divergence never becomes a conflict-heavy rebase.

Q. 37. What does `git clean` do?

Removes untracked files. `git clean -n` previews, `-f` deletes files, `-fd` includes directories, `-fdx` also removes ignored files - useful to guarantee a pristine workspace in CI. It is irreversible, so always run `-n` first.

Q. 38. What is `git worktree` and why might a DevOps engineer use it?

It checks out multiple branches into separate directories from one repository. Practical use: building a hotfix in one directory while a long feature build continues in another, without stashing or re-cloning.

Q. 39. How do you enforce commit message standards?

A `commit-msg` hook, distributed through the `pre-commit` framework, plus a CI check on the pull request that validates the messages against the convention - Conventional Commits, or a ticket ID prefix. Enforcing it in CI rather than only locally is what makes it stick, since local hooks can be bypassed.

Q. 40. What would you check if `git clone` works on your laptop but fails on the Jenkins agent?

Credentials first - is the deploy key or token present in the agent's credential store and does it have access to that repository. Then host key verification: a fresh agent has no entry in `known_hosts`, so SSH clone hangs or fails - which is why agents need the host key pre-seeded. Then network - can the agent reach `github.com` at all, or does it need a proxy. Then disk space and workspace permissions on the agent. `GIT_SSH_COMMAND="ssh -v"` gives you the verbose output needed to tell these apart.

PART 5 RECALL POINTS

- Four areas: working tree, index, local repo, remote.
- fetch is safe, pull changes your working tree.
- Rebase your own branch; merge into shared branches; never rebase shared history.
- reset rewrites (local), revert adds an undo commit (shared).
- `git reflog` recovers almost anything lost locally.
- `--force-with-lease`, never bare `--force`.
- A committed secret must be rotated, not just deleted.
- Branch protection: PR + approval + green checks + no force-push, admins included.

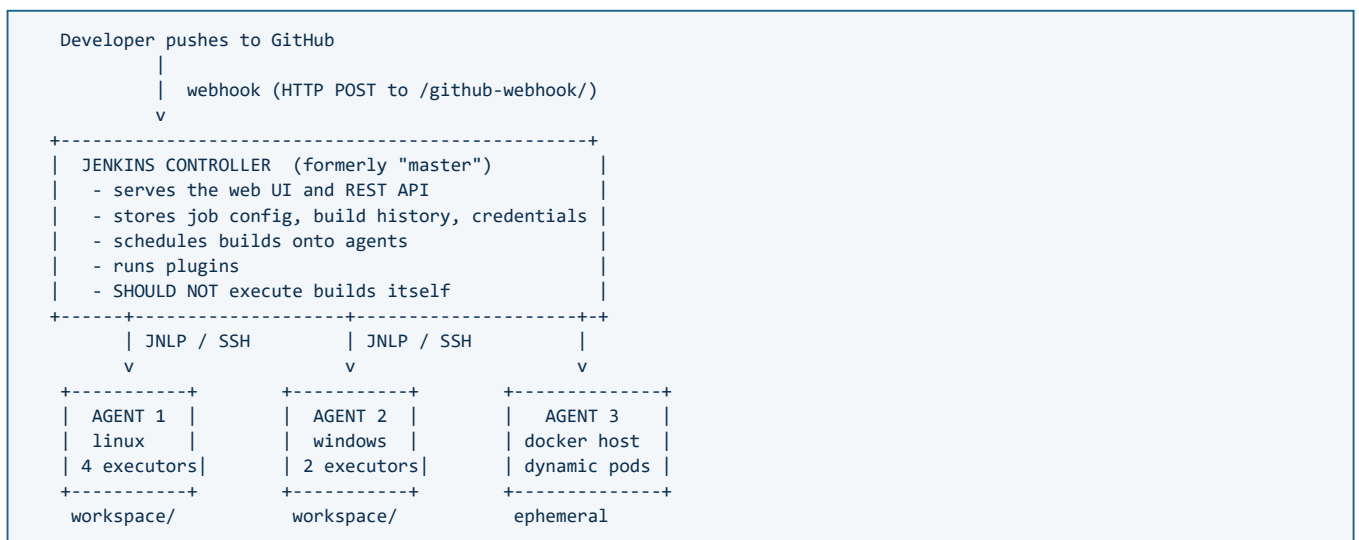
PART 6

JENKINS COMPLETE GUIDE

Architecture, pipelines, Jenkinsfile line by line, troubleshooting, 50 questions

Jenkins appears in both of your roles and as a named project on your CV - "Jenkins pipeline-as-code workflows integrating Git, automated builds, testing and deployment stages". It is therefore the topic where you must be strongest, and the one where an interviewer will feel entitled to go deep. This is the largest part of the handbook for that reason.

6.1 Jenkins Architecture



Controller and agents - who does what

Component	Role	Interview detail
Controller	Orchestration, UI, configuration, scheduling, credentials store	Configuration lives in \$JENKINS_HOME; back that up and you can rebuild the instance
Agent (node)	Executes the build steps	Connected by SSH (controller initiates) or JNLP/inbound (agent initiates - needed when the agent is behind a firewall)
Executor	One concurrent build slot on an agent	Number of executors = how many builds that agent runs at once; set it near the core count
Workspace	Directory on the agent where the job checks out and builds	\$WORKSPACE; not guaranteed clean between builds unless you clean it
Job / Project	A definition of work - freestyle, pipeline, multibranch	Pipeline is the only one worth defending in a modern interview
Build / Run	One execution of a job, with a number and a result	\$BUILD_NUMBER is the natural version tag
Plugin	Extends Jenkins - Git, Docker, Kubernetes, Credentials, Blue Ocean	Plugins are also Jenkins's biggest operational risk: upgrades break pipelines

WHY BUILDS SHOULD NEVER RUN ON THE CONTROLLER

The controller holds credentials, job configuration and build history. Any build script running there has access to the whole Jenkins instance and can take it down by exhausting CPU, memory or disk. Best practice is zero executors on the controller and all work on agents. Saying this unprompted signals real operational experience.

contents	What it is	Backup priority
config.xml	Global configuration	Critical
jobs/	Job definitions and build history	Critical

contents	What it is	Backup priority
credentials.xml, secrets/	Encrypted credentials and the master key	Critical - and useless without secrets/
plugins/	Installed plugins and versions	Important - pin versions
users/	User records	Important
workspace/	Build workspaces	Do not back up - regenerable

6.2 Job Types

Type	What it is	When to use
Freestyle	Click-configured job	Legacy; avoid in new work - config lives only in the UI
Pipeline	A Jenkinsfile defining the whole flow as code	The default choice - versioned, reviewable, diffable
Multibranch Pipeline	Scans a repository and creates a pipeline per branch that has a Jenkinsfile	Feature branch CI and PR validation, automatically
Organization Folder	Scans a whole GitHub org for repositories with Jenkinsfiles	Many repositories, one convention
Matrix / Multi-configuration	Same build across combinations of axes	Testing across OS or versions

"WHY PIPELINE-AS-CODE INSTEAD OF FREESTYLE?"

"Because a freestyle job's configuration only exists inside Jenkins - it is not reviewed, not versioned, and cannot be rolled back. A Jenkinsfile lives in the repository next to the code it builds, so changing the build is a pull request like anything else, every branch can have its own build definition, and rebuilding Jenkins from scratch does not lose the pipeline. It also lets a change to the code and the change to its build ship as one reviewable unit."

6.3 Declarative vs Scripted Pipelines

	Declarative	Scripted
Syntax	Structured pipeline { } blocks	Groovy, starting with node { }
Validation	Validated before running - fails fast on syntax	Errors surface at runtime
Learning curve	Low	Requires Groovy
Error handling	post { always / success / failure } blocks	try/catch/finally
Flexibility	Constrained, but script { } lets you drop into Groovy	Complete
Recommendation	Default for everything	Only for genuinely dynamic pipelines

The minimal declarative skeleton

```
pipeline {
  agent any
  stages {
    stage('Build') {
      steps {
        sh 'echo building'
      }
    }
  }
}
```

The same thing scripted

```
node {
  stage('Build') {
    sh 'echo building'
  }
}
```

6.4 A Complete Jenkinsfile, Explained Line by Line

Production-shaped declarative pipeline

```

pipeline {
    agent any

    options {
        buildDiscarder(logRotator(numToKeepStr: '30'))
        timeout(time: 30, unit: 'MINUTES')
        disableConcurrentBuilds()
        timestamps()
        skipDefaultCheckout(false)
    }

    parameters {
        choice(name: 'ENVIRONMENT', choices: ['dev', 'staging', 'prod'], description: 'Target environment')
        booleanParam(name: 'RUN_TESTS', defaultValue: true, description: 'Run the test stage')
        string(name: 'IMAGE_TAG', defaultValue: '', description: 'Override image tag (blank = build number)')
    }

    environment {
        APP_NAME      = 'payments-api'
        REGISTRY      = 'registry.example.com'
        IMAGE         = "${REGISTRY}/${APP_NAME}"
        TAG           = "${params.IMAGE_TAG ?: env.BUILD_NUMBER}"
        REGISTRY_CRED = credentials('registry-credentials')
    }

    triggers {
        githubPush()
        cron('H 2 * * *')
    }

    stages {
        stage('Checkout') {
            steps {
                checkout scm
                script {
                    env.GIT_SHA = sh(script: 'git rev-parse --short HEAD', returnStdout: true).trim()
                }
                echo "Building ${APP_NAME} at ${env.GIT_SHA} for ${params.ENVIRONMENT}"
            }
        }

        stage('Build') {
            steps {
                sh 'mvn -B -DskipTests clean package'
            }
        }

        stage('Test') {
            when { expression { return params.RUN_TESTS } }
            steps {
                sh 'mvn test'
            }
            post {
                always {
                    junit 'target/surefire-reports/*.xml'
                }
            }
        }

        stage('Static Analysis') {
            steps {
                sh 'mvn sonar:sonar -Dsonar.projectKey=${APP_NAME}'
            }
        }
    }
}

```

```

stage('Docker Build') {
    steps {
        sh """
            docker build \
              --build-arg VERSION=${TAG} \
              -t ${IMAGE}:${TAG} \
              -t ${IMAGE}:latest .
        """
    }
}

stage('Image Scan') {
    steps {
        sh "trivy image --exit-code 1 --severity HIGH,CRITICAL ${IMAGE}:${TAG}"
    }
}

stage('Push Image') {
    steps {
        withCredentials([usernamePassword(credentialsId: 'registry-credentials',
                                         usernameVariable: 'REG_USER',
                                         passwordVariable: 'REG_PASS')]) {
            sh '''
                echo "$REG_PASS" | docker login ${REGISTRY} -u "$REG_USER" --password-stdin
                docker push ${IMAGE}:${TAG}
                docker logout ${REGISTRY}
            '''
        }
    }
}

stage('Approve Production') {
    when { expression { params.ENVIRONMENT == 'prod' } }
    steps {
        timeout(time: 30, unit: 'MINUTES') {
            input message: "Deploy ${IMAGE}:${TAG} to PRODUCTION?", submitter: 'release-managers'
        }
    }
}

stage('Deploy') {
    steps {
        withCredentials([file(credentialsId: 'kubeconfig-' + params.ENVIRONMENT, variable:
'KUBECONFIG')]) {
            sh """
                kubectl set image deployment/${APP_NAME} ${APP_NAME}=${IMAGE}:${TAG} -n
${params.ENVIRONMENT}
                kubectl rollout status deployment/${APP_NAME} -n ${params.ENVIRONMENT} --timeout=300s
            """
        }
    }
}

stage('Health Check') {
    steps {
        script {
            def url = "https://${params.ENVIRONMENT}.example.com/health"
            retry(5) {
                sleep 10
                sh "curl -sf ${url} > /dev/null"
            }
        }
    }
}

}

post {
    success {
        echo "SUCCESS ${APP_NAME}:${TAG}"
    }
    failure {
        echo "FAILED - see ${env.BUILD_URL}"
    }
}

```

```

    }
    unstable {
        echo "UNSTABLE - tests failed"
    }
    always {
        archiveArtifacts artifacts: 'target/*.jar', fingerprint: true, allowEmptyArchive: true
        cleanWs()
    }
}
}

```

6.4.1 Every block explained

Line / block	What it does	Why it matters
pipeline { }	Declares a declarative pipeline	Required outermost block
agent any	Run on any available agent	agent none at top with per-stage agents lets different stages use different machines
options { }	Pipeline-wide behaviour	
buildDiscarder(logRotator(numToKeepStr: '30'))	Keep only the last 30 builds	Prevents \$JENKINS_HOME filling with build history - a real production issue
timeout(time: 30, unit: 'MINUTES')	Abort if the pipeline exceeds 30 minutes	Stops a hung build occupying an executor forever
disableConcurrentBuilds()	One run of this job at a time	Essential when the job deploys - two concurrent deploys corrupt state
timestamps()	Prefix every console line with a time	Makes log analysis and performance debugging possible
parameters { }	Inputs the user supplies at build time	choice, booleanParam, string, password
environment { }	Variables available to all stages	Values are interpolated in double-quoted Groovy strings
credentials('id')	Binds a credential to an env var	For username/password it also creates _USR and _PSW suffixed variables
triggers { }	How the job starts automatically	githubPush(), pollSCM('H/5 * * * *'), cron('H 2 * * *'), upstream(...)
cron('H 2 * * *')	Nightly around 02:00	H spreads load - it hashes the job name to pick a minute, avoiding a thundering herd at exactly 02:00
stages { }	The ordered stages	Each appears as a column in the pipeline view
checkout scm	Clone the repo/branch that this Jenkinsfile came from	Correct for multibranch; you do not hardcode a branch
sh / bat / powershell	Run a shell command	sh on Linux agents, bat or powershell on Windows - relevant to your mixed environment
returnStdout: true	Capture command output into a variable	Always .trim() the result
when { }	Conditional stage execution	branch 'main', expression { }, environment name:, value:, changeset
post { }	Runs after the stage or pipeline	always, success, failure, unstable, changed, aborted
junit	Publish test results	Marks the build UNSTABLE rather than FAILED when tests fail
withCredentials([])	Bind secrets for a limited scope	Values are masked in the console output
input	Pause for human approval	Wrap it in timeout or it holds an executor indefinitely
submitter: 'release-managers'	Only that group may approve	Separation of duties for production
retry(5)	Retry a block up to 5 times	Right for health checks; wrong for deploys, which are not always idempotent

Line / block	What it does	Why it matters
archiveArtifacts	Store build output on the controller	fingerprint: true lets Jenkins trace which build produced a given file
cleanWs()	Wipe the workspace	Prevents stale files from a previous build affecting this one

SINGLE QUOTES VS DOUBLE QUOTES IN A JENKINSFILE

Groovy interpolates `${VAR}` in double-quoted strings but NOT in single-quoted ones. That is not a bug, it is the tool you use for secrets: `sh "echo ${PASSWORD}"` interpolates the secret into the command line, where it can appear in process listings and logs, while `sh 'echo $PASSWORD'` lets the shell expand it from the environment, keeping it out of the Groovy string. Use single quotes for anything secret.

6.5 The Complete CI/CD Flow

1 DEVELOPER	git push / open PR
2 GITHUB	webhook POST -> https://jenkins.example.com/github-webhook/
3 JENKINS	multibranch job for that branch is triggered, queued
4 AGENT	executor allocated, workspace prepared
5 CHECKOUT	checkout scm (shallow clone, credentials from the store)
6 BUILD	mvn package / npm run build -> artifact produced
7 TEST	unit + integration; junit publishes results -> UNSTABLE on failure
8 QUALITY	SonarQube / lint / dependency + image scan -> gate
9 PACKAGE	docker build -t image:BUILD_NUMBER (tag = immutable version)
10 PUSH	docker login (credential binding) -> docker push to registry
11 DEPLOY	kubectl set image / ansible-playbook / helm upgrade -> environment
12 VERIFY	kubectl rollout status + curl /health with retry
13 NOTIFY	Slack/email on success and failure, with BUILD_URL
14 ON FAILURE	rollback: deploy previous tag / kubectl rollout undo

Git to production - the flow to narrate in interviews

"WALK ME THROUGH YOUR PIPELINE"

"It starts with a webhook from GitHub - a push or a pull request triggers the multibranch job for that branch, so I am not polling. Jenkins allocates an executor on an agent and checks out that exact commit. Then build, producing a versioned artifact. Then tests, with results published so the build goes unstable rather than silently green if a test fails. Then quality and security gates. Then we package into a Docker image tagged with the build number, so the tag is immutable and traceable back to the commit. The image is pushed to the registry using credentials from the Jenkins credential store, never hardcoded. Deployment is a separate stage that updates the target environment and then waits for rollout status, followed by a health check with retries. The post block notifies on success and failure. Rollback is redeploying the previous image tag, because every build produces an immutable artifact."

6.6 Credentials and Security

Credential type	Use	Bound with
Secret text	API token, webhook secret	<code>string(credentialsId:, variable:)</code>
Username with password	Registry, Git over HTTPS, database	<code>usernamePassword(...)</code> - also gives <code>_USR/_PSW</code>
SSH username with private key	Git over SSH, SSH deployments	<code>sshUserPrivateKey(...)</code> or <code>sshagent([])</code>

Credential type	Use	Bound with
Secret file	kubeconfig, service account JSON, certificate	file(credentialsId:, variable:)
Certificate	Client certificates	

Credential binding patterns

```

withCredentials([string(credentialsId: 'slack-token', variable: 'SLACK_TOKEN')]) {
    sh 'curl -s -H "Authorization: Bearer $SLACK_TOKEN" https://slack.com/api/auth.test'
}

withCredentials([usernamePassword(credentialsId: 'nexus',
    usernameVariable: 'USER', passwordVariable: 'PASS')]) {
    sh 'mvn deploy -Dnexus.user=$USER -Dnexus.pass=$PASS'
}

withCredentials([file(credentialsId: 'kubeconfig-prod', variable: 'KUBECONFIG')]) {
    sh 'kubectl get nodes'
}

sshagent(['deploy-ssh-key']) {
    sh 'ssh -o StrictHostKeyChecking=accept-new deploy@app01 "systemctl restart app"'
}

```

Jenkins security control	What it does
Matrix / role-based authorisation	Per-user or per-role permissions on jobs and folders
Folder-scoped credentials	A team's credentials are only usable by that team's jobs
Script approval	Sandboxed Groovy; anything outside the sandbox needs admin approval
Agent-to-controller access control	Stops build scripts reading controller files
CSRF protection	On by default - keep it on
Disable "Anyone can do anything"	The single most common Jenkins misconfiguration
Audit Trail plugin	Who ran or changed what
Regular plugin updates	Plugins are the main source of Jenkins CVEs

SECRETS THAT LEAK ANYWAY

Even with credential binding, a secret leaks if you echo it, if you interpolate it into a Groovy double-quoted string that ends up in a command line, or if a tool prints it in a debug trace. Jenkins masks known credential values in console output, but only exact matches - base64 or URL-encode it and the mask fails. Rules: single quotes, never set -x around a secret, and rotate anything you suspect was printed.

6.7 Shared Libraries

When twenty repositories all need the same build logic, you do not copy the Jenkinsfile twenty times. A shared library puts the logic in one versioned repository.

Shared library layout

```

(root)
+- vars/
|   +- buildDockerApp.groovy    # a global step, callable as buildDockerApp(...)
|   +- notifySlack.groovy
+- src/
|   +- com/example/Deployer.groovy # full Groovy classes
+- resources/
|   +- com/example/deploy.sh      # files loaded with libraryResource

```

vars/buildDockerApp.groovy - a reusable step

```

def call(Map config) {
    pipeline {
        agent any
        stages {
            stage('Build') {
                steps { sh "docker build -t ${config.image}:${env.BUILD_NUMBER} ." }
            }
        }
    }
}

```

```

    }
    stage('Push') {
        steps {
            withCredentials([usernamePassword(credentialsId: config.credId,
                usernameVariable: 'U', passwordVariable: 'P')]) {
                sh 'echo "$P" | docker login -u "$U" --password-stdin'
                sh "docker push ${config.image}:${env.BUILD_NUMBER}"
            }
        }
    }
}
}
}
}
}

```

Consuming it - the whole Jenkinsfile becomes three lines

```

@Library('devops-shared@v2.1') _

buildDockerApp(image: 'registry.example.com/payments-api', credId: 'registry-credentials')

```

PIN THE LIBRARY VERSION

`@Library('devops-shared@v2.1')` pins to a tag. Using `@main` means someone else's commit can change every pipeline in the organisation without a single one of those repositories changing. Pinning, and promoting versions deliberately, is the mature answer.

6.8 Multibranch Pipelines, Webhooks and Triggers

Trigger	Configuration	Trade-off
GitHub webhook	GitHub posts to <code>/github-webhook/</code> on push/PR	Instant, no wasted polling - the correct choice
Poll SCM	<code>pollSCM('H/5 * * * *')</code>	Jenkins asks the repo every 5 minutes; simple, but adds latency and load. Use when Jenkins is not reachable from the internet
Build periodically	<code>cron('H 2 * * *')</code>	Time-based, regardless of changes - nightly jobs
Upstream	<code>upstream('build-lib', hudson.model.Result.SUCCESS)</code>	Chain jobs
Manual / parameterised	Operator clicks Build	Production deploys

Cron field	Meaning	
Minute	0-59	H = a hashed minute, stable per job
Hour	0-23	H(0-7) = a hashed hour within a window
Day of month	1-31	
Month	1-12	
Day of week	0-7 (0 and 7 are Sunday)	

WHY H EXISTS

`cron('0 2 * * *')` on fifty jobs starts fifty builds at exactly 02:00 and floods the agents. `cron('H 2 * * *')` hashes the job name to pick a consistent minute in that hour, spreading the load. Interviewers love this because it shows you have operated Jenkins at scale rather than just used it.

6.8.1 Multibranch behaviour

A multibranch pipeline scans the repository, finds branches containing a Jenkinsfile, and creates a job for each. It automatically removes jobs when branches are deleted, and it can build pull requests - crucially, it can build the PR's `*merge*` result, so you test what main will actually look like after the merge, not just the branch in isolation.

Branch-aware stages

```

stage('Deploy to staging') {
    when { branch 'develop' }
}

```

```

    steps { sh './deploy.sh staging' }
  }
  stage('Deploy to production') {
    when {
      allOf {
        branch 'main'
        expression { currentBuild.result == null || currentBuild.result == 'SUCCESS' }
      }
    }
    steps { sh './deploy.sh production' }
  }
  stage('PR checks only') {
    when { changeRequest() }
    steps { sh 'make lint test' }
  }
}

```

6.9 Agents, Including Docker and Kubernetes Agents

Agent selection patterns

```

pipeline {
  agent none // no global agent - each stage picks its own
  stages {
    stage('Unit tests') {
      agent { label 'linux && jdk17' } // label expression
      steps { sh 'mvn test' }
    }
    stage('Windows integration') {
      agent { label 'windows' } // your environment has both
      steps { bat 'run-tests.bat' }
    }
    stage('Build in a container') {
      agent {
        docker {
          image 'maven:3.9-eclipse-temurin-17'
          args '-v $HOME/.m2:/root/.m2' // cache dependencies between builds
        }
      }
      steps { sh 'mvn -B package' }
    }
    stage('Kubernetes agent') {
      agent {
        kubernetes {
          yaml '''
            apiVersion: v1
            kind: Pod
            spec:
              containers:
                - name: docker
                  image: docker:24-dind
                  securityContext:
                    privileged: true
            ...
          '''
        }
      }
      steps { container('docker') { sh 'docker version' } }
    }
  }
}

```

Agent connection method	How	When
SSH	Controller connects out to the agent	Agent has a fixed address and the controller can reach it
JNLP / inbound	Agent connects in to the controller	Agent is behind NAT or a firewall, or is ephemeral
Docker plugin	A container per build	Clean environment every time, no toolchain drift
Kubernetes plugin	A pod per build, destroyed afterwards	Elastic capacity, no idle agents - the modern standard

THE "WHY CONTAINERISED AGENTS?" ANSWER

"Static agents accumulate toolchain drift - one has Java 11, another Java 17, and builds mysteriously differ. With a Docker or Kubernetes agent the environment is declared in the Jenkinsfile, so it is identical every run and versioned with the code. It also means capacity is elastic: pods exist only while a build runs."

6.10 Jenkins Troubleshooting

AGENT IS OFFLINE

Symptoms: builds queue with "waiting for an available executor on <agent>".

1. Is the agent host up and reachable? `ping`, `ssh`.
2. Is the agent process running? On JNLP agents check the agent service and its log.
3. Java version mismatch between controller and agent - a very common cause after an upgrade.
4. Disk space on the agent: Jenkins takes a node offline automatically when free space falls below the threshold. `df -h` - this is the single most frequent cause.
5. Network/firewall: JNLP needs the agent port open from agent to controller.
6. Check Manage Jenkins -> Nodes -> the node's log for the actual exception.
7. Credentials for SSH agents - a rotated key silently breaks the connection.

WORKSPACE PROBLEMS

Symptoms: stale files, "file already exists", builds passing locally but failing in Jenkins, or the reverse.

- Add `cleanWs()` in `post { always { } }`, or use `deleteDir()` at the start.
- Two jobs sharing one workspace: enable `disableConcurrentBuilds()` or use `customWorkspace`.
- Permission errors: files created inside a Docker build run as root and the Jenkins user then cannot delete them. Fix by running the container with `-u $(id -u):$(id -g)`, or clean up inside the container.
- Disk filling on the agent: `buildDiscarder` on the job, and a cleanup job for orphaned workspaces.

CREDENTIALS FAILURE

Symptoms: "denied: authentication required", 401, "could not read Username".

1. Does the credential ID in the Jenkinsfile exactly match the one in the store? IDs are case-sensitive.
2. Is the credential in the right SCOPE? A credential in a folder is invisible to jobs outside it; a "System" scoped credential is invisible to builds entirely.
3. Has the token or key expired or been rotated?
4. Does the credential have the required permission on the target - push rights to the registry, write access to the repository?
5. Are you using single quotes so the secret is expanded by the shell rather than interpolated by Groovy?
6. For Docker: `docker login` state is per-user on the agent - make login part of the stage, not a manual one-off someone did months ago.

GIT CHECKOUT FAILURE

- `Host key verification failed` - the agent has never seen this host. Pre-seed `known_hosts`, or use `accept-new` deliberately.
- `Permission denied (publickey)` - wrong credential, or the deploy key lacks access to that repository.
- `couldn't find remote ref main` - the branch name changed (master vs main) or the branch was deleted.
- `fatal: unable to access ... Could not resolve host` - DNS or proxy on the agent.
- Detached HEAD confusing your script - use `checkout scm` and read `env.GIT_COMMIT` rather than assuming a branch name.
- Slow clones - use a shallow clone and reference repositories.

DOCKER STAGE FAILURES

- Cannot connect to the Docker daemon - the jenkins user is not in the docker group, or the socket is not mounted into the agent. Remember that adding a user to the docker group is effectively granting root.
- no space left on device - images and layers accumulate. `docker system df` then a scheduled `docker system prune -af --filter "until=72h"`.
- denied: requested access to the resource is denied - not logged in, wrong registry, or no push permission.
- Build works locally but not in Jenkins - almost always build context or a `.dockerignore` difference, or a base image tag that moved.
- `manifest unknown` on pull - the tag does not exist; check whether a previous push actually succeeded.

PIPELINE SYNTAX AND RUNTIME ERRORS

- `java.lang.NoSuchMethodError` / "expected to be within a step" - usually a declarative/scripted mix-up; wrap imperative Groovy in `script { }`.
- Scripts not permitted to use `method ...` - the Groovy sandbox blocked it; either approve it in Manage Jenkins -> In-process Script Approval or, better, avoid the call.
- Use the built-in Pipeline Syntax generator and Declarative Directive Generator to produce correct step syntax rather than guessing.
- Validate before committing: `curl -X POST -F "jenkinsfile=<Jenkinsfile" https://jenkins/pipeline-model-converter/validate`.
- Also: `hudson.AbortException` with no detail - read the full console log from the top; the real error is usually 50 lines above the stack trace.

DEPLOYMENT FAILURE AND ROLLBACK

1. Read which stage failed and its exit code - deploy failures usually surface as a non-zero exit from `kubectl`, `ansible` or a script.
2. `kubectl rollout status` timing out means the new pods never became Ready - go to `kubectl describe pod` and the container logs, not to Jenkins.
3. Roll back first: `kubectl rollout undo deployment/app`, or redeploy the previous image tag, or re-run the previous successful build.
4. Verify the rollback with the same health check the pipeline uses.
5. Then diagnose, with the failing image still available because you tagged it immutably.
6. Build the rollback INTO the pipeline: a `post { failure { } }` block that triggers the rollback removes the human delay.

6.11 Jenkins Interview Questions

Q. 1. What is Jenkins and what problem does it solve?

An open-source automation server that runs the build, test and deployment pipeline automatically on every change. It solves the problem of manual, inconsistent releases - instead of a person remembering a sequence of steps, the sequence is code, it runs identically every time, and it produces an auditable record of what was built and deployed.

Q. 2. Describe Jenkins architecture.

A controller and agents. The controller serves the UI and API, stores job configuration, build history and credentials, and schedules work. Agents execute the builds; each agent has a number of executors defining how many builds it runs concurrently, and a workspace per job. Agents connect by SSH from the controller or inbound over JNLP. Best practice is zero executors on the controller, because build scripts there have access to credentials and can take the instance down.

Q. 3. What is `$JENKINS_HOME` and what would you back up?

The directory holding all Jenkins state - `config.xml`, `jobs/` with definitions and history, `credentials.xml` with the secrets/ key material, `plugins/` and `users/`. I would back up everything except `workspace/` and `caches`. The critical detail is that `credentials.xml` is useless without `secrets/`, so they must be backed up together and protected.

Q. 4. Freestyle versus pipeline?

A freestyle job is configured through the UI, so its definition is not versioned, reviewed or portable. A pipeline is defined in a Jenkinsfile in the repository, so the build definition is code - reviewed in pull requests, branch-specific, and recoverable if Jenkins is rebuilt.

Q. 5. Declarative versus scripted pipeline?

Declarative uses a structured pipeline { } syntax that Jenkins validates before running, with post blocks for conditions - it is readable and hard to get wrong. Scripted is Groovy inside node { } with full programmatic control. I default to declarative and drop into script { } for the rare dynamic part.

Q. 6. What is a Jenkinsfile?

The pipeline definition as code, committed at the root of the repository. It defines agents, stages, steps and post-actions, and because it lives with the code, changing the build is a reviewable pull request.

Q. 7. What is an executor?

One concurrent build slot on an agent. An agent with four executors runs four builds at once. Setting it too high causes builds to contend for CPU and memory and everything slows down; a common rule of thumb is around one per core, lower if builds are memory-hungry.

Q. 8. How do you trigger a build automatically on commit?

A webhook from GitHub to /github-webhook/ on the Jenkins URL, with a multibranch or GitHub-integrated job. Polling with pollSCM is the fallback for when Jenkins is not reachable from the internet, but it adds latency and load - a webhook is push, polling is asking repeatedly.

Q. 9. Difference between webhook and poll SCM?

A webhook is pushed by the source system the moment a change happens - immediate and efficient. Poll SCM has Jenkins ask the repository on a schedule; it works behind firewalls but wastes requests and delays builds by up to the polling interval.

Q. 10. What does H mean in a Jenkins cron expression?

Hash. Jenkins hashes the job name to pick a consistent value within the allowed range, so jobs spread out instead of all firing at the same instant. H 2 * * * runs once between 02:00 and 02:59 at the same minute each day.

Q. 11. How do you handle secrets in Jenkins?

In the credentials store, scoped as narrowly as possible - folder-level rather than global - and bound into a build only for the block that needs them, using withCredentials or environment { X = credentials('id') }. In the shell I use single quotes so the shell expands the variable rather than Groovy interpolating it into the command string. I never echo them, and I rotate anything that might have been printed.

Q. 12. What is the difference between environment { X = credentials('id') } and withCredentials?

environment makes it available to the whole pipeline or stage; withCredentials scopes it to one block, which is tighter and therefore safer. withCredentials also supports more binding types such as secret files and SSH keys.

Q. 13. What are Jenkins parameters and when do you use them?

Inputs supplied at build time - choice, string, booleanParam, password. I use them for target environment, image tag overrides and toggles such as skipping tests. They are accessed as params.NAME.

Q. 14. What is the post section?

Actions that run after a stage or the whole pipeline, keyed by result - always, success, failure, unstable, changed, aborted. It is where I publish test results, archive artifacts, send notifications and clean the workspace, because those must happen even when the build fails.

Q. 15. What is the difference between FAILURE and UNSTABLE?

FAILURE means a step errored - the build could not complete. UNSTABLE means it completed but something reported a problem, most commonly failing tests published through junit. Distinguishing them lets you treat a compile failure differently from a flaky test.

Q. 16. How do you run stages in parallel?

A parallel block containing named branches, each with its own steps and optionally its own agent. It is how you cut pipeline time when unit tests, integration tests and static analysis are independent. failFast true aborts the others as soon as one fails.

Q. 17. What is a shared library?

A separate versioned repository containing reusable pipeline code - global steps in vars/, classes in src/, files in resources/. It stops the same hundred lines being copy-pasted into twenty Jenkinsfiles. I pin it to a tag so a change to the library cannot

silently alter every pipeline at once.

Q. 18. What is a multibranch pipeline?

A job that scans a repository and creates a pipeline for every branch containing a Jenkinsfile, removing them when branches are deleted. It also builds pull requests, and it can build the merge result rather than the branch alone, so you test what main will look like after merging.

Q. 19. How do you implement a manual approval before production?

An input step, wrapped in a timeout so it cannot hold an executor forever, and restricted with submitter so only the release group can approve. I also put the approval in its own stage so the pipeline visualisation makes the gate obvious.

Q. 20. Why should the input step be outside an agent block or wrapped in a timeout?

Because while it waits it occupies an executor. A pipeline waiting overnight for approval blocks a build slot. Wrapping it in a timeout and, in declarative, placing it in a stage with agent none, avoids holding the agent.

Q. 21. How do you archive artifacts and why fingerprint them?

archiveArtifacts artifacts: 'target/*.jar', fingerprint: true. The fingerprint is an MD5 record that lets Jenkins tell you which build produced a given file and where it was used, which is genuinely useful when tracing what is running in an environment.

Q. 22. How do you make a build fail when tests fail?

junit marks the build unstable by default. If I want a hard failure, I let the test command's non-zero exit propagate, or set the junit step's thresholds, or explicitly check the result and call error("..."). The important point is deciding deliberately rather than discovering that failing tests ship.

Q. 23. How do you version your Docker images from Jenkins?

With \$BUILD_NUMBER plus the short Git SHA, and a semantic version tag for releases. Never just latest, because latest is mutable - you cannot tell what is running and you cannot roll back to a specific build. I also push latest as a convenience but deploy by the immutable tag.

Q. 24. How do you roll back a deployment from Jenkins?

Re-deploy the previous immutable tag - either by re-running the last known-good build's deploy stage, or by a parameterised rollback job that takes a tag. In Kubernetes kubectl rollout undo is faster still. The prerequisite is immutable, versioned artifacts, which is why tagging matters so much.

Q. 25. How do you speed up a slow pipeline?

Measure first with timestamps() to find the slow stage. Then: parallelise independent stages, shallow-clone, cache dependencies (~/.m2, npm cache) on the agent or in a Docker volume, use Docker layer caching, only run heavy stages when relevant paths changed, split long test suites across parallel agents, and move to ephemeral Kubernetes agents so queueing stops being the bottleneck.

Q. 26. Your pipeline worked yesterday and fails today with no code change. How do you investigate?

Compare the two console logs side by side and find the first line that differs - that usually names the cause. Then check what else changed: a plugin update on the controller, an agent that was rebuilt or has a different tool version, a mutable base image or dependency tag that moved, an expired credential or certificate, disk filling on the agent, or an external service such as the registry or an artifact repository being down. Most "nothing changed" failures are a moving external dependency or a disk.

Q. 27. What is the Blue Ocean plugin?

A modern UI for pipeline visualisation that makes stage failures and logs much easier to read. It is a presentation layer - it changes nothing about how pipelines run.

Q. 28. How do you back up and restore Jenkins?

Back up \$JENKINS_HOME excluding workspaces and caches - either filesystem snapshots, or the ThinBackup plugin, or, better, keep configuration in code: job definitions as Jenkinsfiles, system configuration through Configuration as Code (JCasC), and plugins pinned in a list. Restore then becomes redeploying Jenkins and applying the YAML rather than restoring a blob.

Q. 29. What is Jenkins Configuration as Code (JCasC)?

A plugin that lets you define the whole controller configuration - security realm, agents, tools, credentials references, plugin settings - in a YAML file. It makes a Jenkins instance reproducible and reviewable, which is the same argument as Terraform for infrastructure.

Q. 30. How do you scale Jenkins?

Move all builds off the controller onto agents; add agents by workload type with labels; use ephemeral agents from Kubernetes or Docker so capacity scales with demand; keep `$JENKINS_HOME` on fast storage; prune build history with `buildDiscarder`; and split extremely large instances into several controllers by team, because a single controller has real limits.

Q. 31. What are the risks of plugins?

They are the main source of Jenkins vulnerabilities and breakage. An upgrade can change a step's behaviour and break every pipeline at once. Mitigations: pin versions, test upgrades on a staging controller, keep the plugin set minimal, and read changelogs before updating - especially for the Git, Docker and Credentials plugins.

Q. 32. How would you give a team access to only their jobs?

Folders per team, Role-Based Authorization Strategy granting each role rights only on its folder, and credentials scoped to the folder. That way one team cannot see or use another's credentials, which is the real risk.

Q. 33. What is the difference between `sh` and `bat`?

`sh` runs a shell command on a Unix agent; `bat` runs a Windows batch command; `powershell` runs PowerShell. In a mixed estate like the one I work in, stages are pinned to agents by label and use the right step for that platform.

Q. 34. How do you capture command output into a variable?

`def out = sh(script: 'git rev-parse --short HEAD', returnStdout: true).trim().returnStatus: true` gives the exit code instead of failing the build, which is how you handle an expected non-zero exit.

Q. 35. How do you stop two deployments running at the same time?

`disableConcurrentBuilds()` on the job, or a lock resource from the Lockable Resources plugin when several different jobs contend for the same environment. Concurrent deploys to one environment are a genuine outage cause.

Q. 36. What is a quality gate in the pipeline?

A stage that fails the build when a measured threshold is breached - SonarQube coverage or code smells, a Trivy scan finding HIGH or CRITICAL vulnerabilities, or a licence check. The point is that it fails automatically rather than depending on someone noticing a report.

Q. 37. How do you handle a flaky test in CI?

Short term, quarantine it so it does not block everyone, and make that visible. Medium term, fix it - flakiness is usually a timing assumption, shared state or a real race condition. Blanket retries are the wrong default because they hide genuine intermittent bugs; a retry with a report of how often it retried is acceptable as a stopgap.

Q. 38. How do you send notifications?

A post block with Slack, email or Teams steps, including the build URL, branch, commit and the failing stage. I notify on failure and on recovery - notifying on every success trains people to ignore the channel.

Q. 39. What happens if the controller restarts mid-build?

Pipeline builds are designed to be durable - the Pipeline plugin persists state so many builds resume after a restart, but anything mid-`sh` on an agent typically fails and must re-run. That is one more reason builds should be idempotent and re-runnable.

Q. 40. How do you test a Jenkinsfile change without breaking everyone?

Develop it on a feature branch - with a multibranch job, that branch gets its own pipeline automatically, so the change is exercised in isolation. Use the Replay feature to iterate quickly on a failing run, validate syntax with the pipeline-model-converter endpoint, and, for shared libraries, pin consumers to a tag so the change only reaches them when the tag is promoted.

Q. 41. What is the difference between `agent any` and `agent none`?

`agent any` allocates an executor for the whole pipeline. `agent none` allocates nothing globally and requires each stage to declare its own agent, which is better when stages need different environments, or when a stage is a pure approval gate that should not occupy an executor.

Q. 42. How do you cache dependencies between builds?

Mount a persistent directory into the build - for Maven `-v $HOME/.m2:/root/.m2` on a Docker agent, or a volume/PVC for Kubernetes agents. The trade-off is that a cache is shared mutable state, so the pipeline must tolerate a cold cache and you need a way to invalidate it.

Q. 43. What does `cleanWs()` do and when would you not use it?

It deletes the workspace after the build, preventing stale files and reclaiming disk. You would skip it when the next step needs the workspace, or when a full clean checkout is prohibitively slow and you deliberately rely on incremental builds -

but then you accept the risk of stale state.

Q. 44. How do you integrate Jenkins with Kubernetes?

The Kubernetes plugin, which runs each build in a dynamically created pod defined by a pod template - agents appear on demand and are destroyed afterwards, so there are no idle machines and no toolchain drift. Deployment then uses `kubectl` or Helm with a kubeconfig supplied as a secret file credential, ideally with a service account scoped to just the target namespace.

Q. 45. Jenkins versus GitHub Actions - when would you use each?

GitHub Actions is tightly integrated with the repository, requires no infrastructure to operate, and is excellent for building, testing and publishing where the code already lives on GitHub. Jenkins is better where you need to orchestrate across many systems, run on your own hardware or inside a private network, support complex approval workflows, or you already have significant investment in pipelines and shared libraries. I use both - GitHub Actions for repository-scoped CI and Jenkins for deployment orchestration.

Q. 46. How would you implement blue-green deployment in a Jenkins pipeline?

Deploy the new version to the idle environment, run smoke tests against it directly, then flip the router - an ALB target group, an nginx upstream, or a Kubernetes Service selector - in a separate stage. Keep the old environment running for a defined window, with a rollback stage that flips back. I would gate the flip behind an input for production.

Q. 47. What do you do if a build hangs?

The `timeout` option should abort it automatically - if there is none, that is the first fix. Then find why: a process waiting on input (Git prompting for credentials, apt asking a question), a network call with no timeout, a test waiting on an unavailable service, or a deadlock. Setting `GIT_TERMINAL_PROMPT=0` and passing `-y/--non-interactive` flags removes an entire class of hangs.

Q. 48. How do you ensure the artifact tested is the artifact deployed?

Build once, early, and tag it immutably with the build number and commit SHA. Every subsequent stage - test, scan, deploy to staging, deploy to production - references that same tag. Nothing is ever rebuilt per environment. Fingerprinting and pushing the image to a registry rather than rebuilding is what makes this real.

Q. 49. Your pipeline deploys successfully but the application still serves the old version. What now?

Confirm what is actually running rather than what the pipeline thinks: check the image digest on the running instances, not just the tag - a mutable tag like `latest` can mean the node reused a cached image. Check `imagePullPolicy`, whether the rollout actually created new pods, whether a load balancer is still routing to old targets, and whether a CDN or browser cache is serving stale content. Nine times out of ten it is a mutable tag plus `IfNotPresent`, or traffic still going to the old target group.

Q. 50. How would you design a pipeline for a brand-new service?

Trigger on PR and on merge to main. On PR: checkout, build, unit tests, lint, dependency scan - fast feedback, no deployment. On merge to main: build once, run the full test suite, scan the image, tag it with the build number and SHA, push to the registry, deploy to staging, run smoke tests and a health check, then require approval for production, deploy with a rolling update, verify rollout status and health, and notify. Failure at any stage stops the pipeline; a post `failure` block triggers rollback for deployment stages. Credentials come from the store, the pipeline itself is a Jenkinsfile in the repository, and common logic lives in a pinned shared library.

PART 6 RECALL POINTS

- Controller schedules, agents execute; zero executors on the controller.
- Jenkinsfile = build definition as code, reviewed like any other change.
- Declarative by default; `script { }` only when you must.
- `H` in cron spreads load; webhooks beat polling.
- Credentials from the store, bound with `withCredentials`, single-quoted in `sh`.
- `disableConcurrentBuilds()` on anything that deploys.
- Tag images with `BUILD_NUMBER` + SHA, never rely on `latest`.
- `buildDiscarder` and `cleanWs()` prevent the disk-full failure mode.
- Shared libraries pinned to a tag, never to `main`.
- Rollback = redeploy the previous immutable tag.

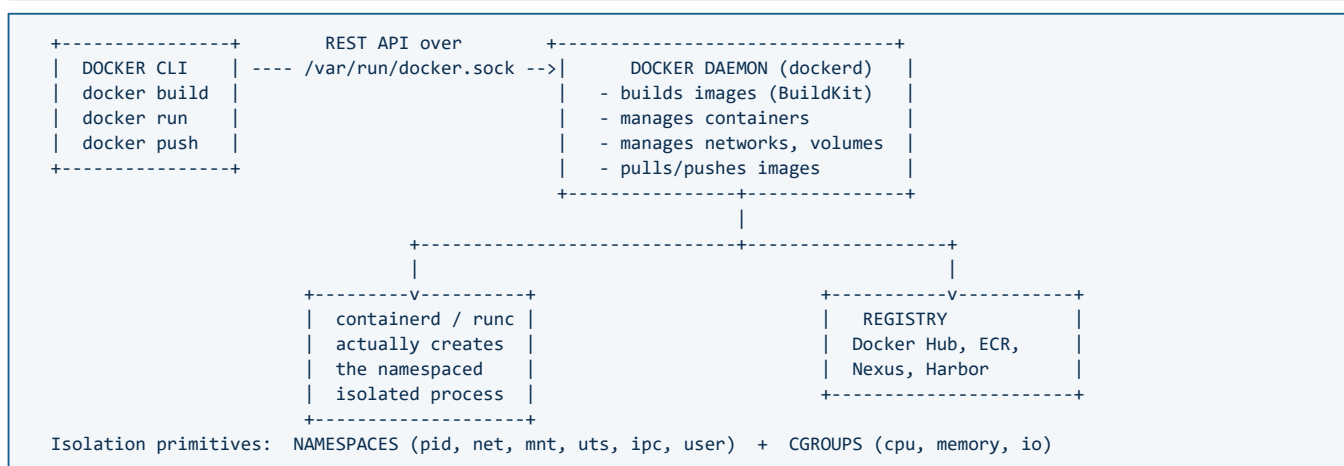
PART 7

DOCKER COMPLETE GUIDE

Architecture, images, Dockerfile, networking, volumes, Compose, troubleshooting

Your CV says you "containerized application components with Docker" and "packaged applications with Docker to standardize environments and reduce inconsistencies". That phrase - reducing inconsistency - is the right reason, and it is the answer interviewers want. This part makes sure you can back it up with Dockerfile detail, networking, volumes and failure diagnosis.

7.1 Docker Architecture



Client, daemon, registry

Component	Role
Docker CLI	Sends commands to the daemon over a Unix socket or TCP
Docker daemon (dockerd)	The long-running service that does everything
containerd	Container lifecycle manager the daemon delegates to
runc	The low-level runtime that actually creates the isolated process
Image	A read-only, layered template
Container	A running (or stopped) instance of an image, with a thin writable layer
Registry	Where images are stored and distributed
Dockerfile	The recipe for building an image

IMAGE VS CONTAINER - SAY IT EXACTLY LIKE THIS

"An image is the immutable, layered template - like a class. A container is a running instance of it with a thin writable layer on top - like an object. Many containers can run from one image, each with its own writable layer, and when a container is deleted that writable layer goes with it. That is why anything you need to keep goes in a volume."

ACCESS TO THE DOCKER SOCKET IS ROOT

Anyone in the docker group can run `docker run -v /:/host -it alpine chroot /host` and become root on the host. So "add the jenkins user to the docker group" is effectively granting root - a legitimate decision, but you should say it knowingly. Rootless Docker and Podman exist precisely to avoid it.

7.2 Images and Layers

Every instruction in a Dockerfile that changes the filesystem creates a layer. Layers are content-addressed and cached: if nothing above a layer has changed, Docker reuses it. That single fact drives every Dockerfile optimisation.

```

CONTAINER [ thin writable layer ] <- your runtime changes live here, and die with the container
-----
IMAGE     [ COPY app/ /app ] <- changes on every code commit
          [ RUN pip install -r ] <- changes only when requirements.txt changes
          [ COPY requirements.txt ]
          [ WORKDIR /app ]
          [ FROM python:3.12-slim ] <- base layer, shared by every image built on it
Shared, read-only layers are stored once on the host and reused by every container using them.

```

Layers and the writable container layer

THE DEPENDENCY-ORDERING RULE

Copy the dependency manifest and install dependencies BEFORE copying the application source. Source changes on every commit; dependencies change rarely. Get the order wrong and every build reinstalls everything, turning a 10-second build into a 4-minute one. This is the single most common Dockerfile interview question.

7.3 Dockerfile Instructions

Instruction	Purpose	Notes and traps
FROM	Base image	Pin a specific tag or digest; latest is mutable and destroys reproducibility
RUN	Execute a command at BUILD time, creating a layer	Chain related commands with && to reduce layers; clean package caches in the same RUN
COPY	Copy files from the build context into the image	The default choice
ADD	Like COPY, but also auto-extracts local tar archives and can fetch URLs	Prefer COPY; ADD's magic causes surprises
WORKDIR	Set the working directory (creates it if missing)	Use this instead of RUN cd, which does not persist
CMD	Default command/arguments, overridable at docker run	Only the last CMD counts
ENTRYPOINT	The executable that always runs	CMD becomes its default arguments
EXPOSE	Documents the port the app listens on	Documentation only - it does NOT publish the port
ENV	Environment variable available at build AND run time	Visible in docker inspect - never put secrets here
ARG	Build-time-only variable	Not present at runtime, but IS visible in image history - still not for secrets
USER	Switch to a non-root user	Security best practice; everything after runs as that user
VOLUME	Declare a mount point	Creates an anonymous volume if none is supplied
HEALTHCHECK	How Docker tests container health	Sets status to healthy/unhealthy, which orchestrators and Compose can act on
LABEL	Metadata	Useful for version, maintainer and provenance
ONBUILD	Trigger instructions in child images	Rare, and surprising - avoid

7.3.1 CMD vs ENTRYPOINT - the guaranteed question

	CMD	ENTRYPOINT
Purpose	Default command or default arguments	The executable that always runs
Overridden by docker run image arg	Yes, completely replaced	No - the arguments are APPENDED to it
Override at runtime	docker run image /bin/bash	Requires --entrypoint
Typical use	A default that users may replace	A container that behaves like a single binary

The three behaviours

```

# 1 - CMD only: fully overridable
CMD ["python", "app.py"]
# docker run img          -> python app.py
# docker run img /bin/bash -> /bin/bash

```

```
# 2 - ENTRYPOINT only: arguments are appended
ENTRYPOINT ["python", "app.py"]
# docker run img --debug -> python app.py --debug

# 3 - Both (the recommended pattern): ENTRYPOINT is the binary, CMD is the default args
ENTRYPOINT ["python", "app.py"]
CMD ["--port", "8080"]
# docker run img -> python app.py --port 8080
# docker run img --port 9090 -> python app.py --port 9090
```

EXEC FORM VS SHELL FORM

CMD ["python", "app.py"] is the exec form - the process runs as PID 1 and receives signals, so `docker stop` works cleanly. CMD `python app.py` is the shell form - it runs as `/bin/sh -c "python app.py"`, so the shell is PID 1, it does not forward SIGTERM, and `docker stop` waits 10 seconds then kills the container. Always use the exec form. This is the reason behind "my container takes 10 seconds to stop".

7.3.2 COPY vs ADD

	COPY	ADD
Copies local files	Yes	Yes
Extracts local tar archives automatically	No	Yes
Fetches remote URLs	No	Yes (but does not extract them, and produces a layer you cannot clean)
Recommendation	Use it by default	Use only when you genuinely want tar auto-extraction

7.3.3 A production Dockerfile, explained

Multi-stage build, non-root, cache-friendly

```
# ----- Stage 1: build -----
FROM node:20-alpine AS builder
WORKDIR /app

# Copy ONLY the manifests first so this layer caches until dependencies change
COPY package.json package-lock.json ./
RUN npm ci --omit=dev

# Now copy the source - this layer rebuilds on every commit, which is fine
COPY . .
RUN npm run build

# ----- Stage 2: runtime -----
FROM node:20-alpine
ENV NODE_ENV=production \
    PORT=8080

# Create an unprivileged user
RUN addgroup -S app && adduser -S -G app app

WORKDIR /app

# Copy ONLY the built output and production dependencies from the build stage.
# The compiler, dev dependencies and source code never reach the final image.
COPY --from=builder --chown=app:app /app/node_modules ./node_modules
COPY --from=builder --chown=app:app /app/dist ./dist

USER app
EXPOSE 8080

HEALTHCHECK --interval=30s --timeout=3s --start-period=20s --retries=3 \
    CMD wget -qO- http://127.0.0.1:8080/health || exit 1

ENTRYPOINT ["node"]
CMD ["dist/server.js"]
```

Python equivalent, with the RUN-chaining pattern

```

FROM python:3.12-slim AS base
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1          # unbuffered stdout so logs appear immediately

WORKDIR /app

# System packages: install and clean the cache in ONE layer, or the cache is in the image forever
RUN apt-get update \
    && apt-get install -y --no-install-recommends curl build-essential \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

RUN useradd -m -u 10001 appuser && chown -R appuser:appuser /app
USER appuser

EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=5s --retries=3 CMD curl -fsS http://localhost:8000/health || exit 1
ENTRYPOINT ["python", "-m", "gunicorn"]
CMD ["-b", "0.0.0.0:8000", "app:application"]

```

Optimisation	Effect
Multi-stage build	Final image excludes compilers, dev dependencies and source - often 10x smaller and a far smaller attack surface
Slim/alpine base	Smaller image, fewer CVEs. Caveat: alpine uses musl libc, which can break some Python/Node native wheels
Manifest before source	Dependency layer stays cached across code commits
<code>rm -rf /var/lib/apt/lists/*</code> in the SAME RUN	Deleting in a later layer does not shrink the image - the data is already in the earlier layer
<code>--no-cache-dir</code> for pip	No wheel cache left behind
<code>.dockerignore</code>	Keeps <code>.git</code> , <code>node_modules</code> and secrets out of the build context, speeding up builds
Non-root USER	Container escape does not immediately give root
HEALTHCHECK	Orchestrators know whether the app is actually up, not just running
Pin base image tags/digests	Reproducible builds

A .dockerignore that actually matters

```

.git
.gitignore
node_modules
__pycache__
*.pyc
.env
*.pem
*.key
.venv
dist
coverage
Dockerfile
docker-compose.yml
README.md

```

7.4 Docker Commands

Images

```

docker images          # list local images
docker images -a       # include intermediate layers
docker pull nginx:1.27-alpine  # download
docker build -t myapp:1.0 .    # build from ./Dockerfile with . as context
docker build -f docker/Dockerfile.prod -t myapp:prod .

```

```

docker build --no-cache -t myapp:1.0 . # ignore the layer cache
docker build --build-arg VERSION=1.0 -t myapp:1.0 .
docker tag myapp:1.0 registry.example.com/myapp:1.0
docker push registry.example.com/myapp:1.0
docker rmi myapp:1.0 # remove an image
docker history myapp:1.0 # layer-by-layer, with sizes - find what bloated the image
docker inspect myapp:1.0 # full metadata as JSON
docker save -o myapp.tar myapp:1.0 # export to a tarball (air-gapped transfer)
docker load -i myapp.tar

```

Containers

```

docker run nginx # foreground
docker run -d --name web nginx # detached, named
docker run -d -p 8080:80 nginx # publish hostPort:containerPort
docker run -d -p 127.0.0.1:8080:80 nginx # publish only on loopback
docker run -it ubuntu bash # interactive TTY
docker run --rm alpine echo hi # remove the container on exit
docker run -e APP_ENV=prod -e DB_HOST=db myapp # environment variables
docker run --env-file .env myapp
docker run -v /host/data:/data myapp # bind mount
docker run -v appdata:/var/lib/app myapp # named volume
docker run --network mynet --name api myapp # user-defined network
docker run --memory=512m --cpus=1.5 myapp # resource limits
docker run --restart=unless-stopped -d myapp # restart policy
docker run -u 1001:1001 myapp # run as a specific UID

docker ps # running containers
docker ps -a # including stopped - where exit codes live
docker ps -q # IDs only, for scripting
docker stop web # SIGTERM, then SIGKILL after the grace period
docker stop -t 30 web # 30-second grace period
docker start web
docker restart web
docker kill web # immediate SIGKILL
docker rm web # remove a stopped container
docker rm -f web # force: stop and remove
docker rm $(docker ps -aq) # remove all containers

```

Inspecting and debugging a running container

```

docker logs web # stdout/stderr of the main process
docker logs -f --tail 100 web # follow the last 100 lines
docker logs --since 10m web
docker exec -it web bash # shell inside a running container
docker exec -it web sh # alpine has no bash
docker exec web env # check the environment variables it actually has
docker exec web cat /etc/app/config.yml # check the config it actually got
docker inspect web # everything: mounts, network, env, state, exit code
docker inspect -f '{{.State.ExitCode}}' web
docker inspect -f '{{.NetworkSettings.IPAddress}}' web
docker inspect -f '{{json .Config.Env}}' web
docker stats # live CPU/memory/network per container
docker top web # processes inside the container
docker diff web # files changed since the image - what did it write?
docker port web # published port mappings
docker cp web:/app/logs/app.log ./ # copy a file out of a container

```

Housekeeping - the disk-full fix

```

docker system df # what is using disk: images, containers, volumes, cache
docker system df -v # per-object detail
docker image prune # remove dangling (untagged) images
docker image prune -a # remove all images not used by a container
docker container prune # remove stopped containers
docker volume prune # remove unused volumes (DATA LOSS - check first)
docker builder prune # clear the build cache
docker system prune # containers + networks + dangling images + build cache
docker system prune -a --volumes # everything unused, INCLUDING volumes - know what you are deleting
docker system prune -af --filter "until=72h" # safer: only objects older than 72 hours

```

7.5 Docker Networking

Network driver	Behaviour	Use
bridge (default)	Private internal network on the host; containers get 172.17.x.x; published ports are NATed	Single-host applications
user-defined bridge	Same, plus automatic DNS resolution by container name	The correct choice for multi-container apps
host	No network namespace - the container uses the host's stack directly, no port mapping	Maximum performance, or when you need the host's interfaces. Linux only
none	No networking at all	Batch jobs with no network needs
overlay	Multi-host network	Swarm; Kubernetes uses its own CNI instead
macvlan	Container gets its own MAC and appears directly on the physical network	Legacy applications needing a real LAN address

DEFAULT BRIDGE VS USER-DEFINED BRIDGE

On the DEFAULT bridge, containers can only reach each other by IP - there is no name resolution (the old `--link` flag is deprecated). On a USER-DEFINED bridge, Docker runs an embedded DNS server, so `http://api:8080` resolves by container name. That is why `docker network create` is the first step in any multi-container setup, and it is a frequent interview question.

Networking commands

```
docker network ls
docker network create appnet                # user-defined bridge, with DNS
docker network create --subnet 10.10.0.0/24 appnet
docker network inspect appnet               # which containers are attached, and their IPs
docker network connect appnet web           # attach a running container
docker network disconnect appnet web
docker run -d --name db --network appnet postgres:16
docker run -d --name api --network appnet -e DB_HOST=db myapi # "db" resolves by name
```

Port syntax	Meaning
<code>-p 8080:80</code>	Host port 8080 -> container port 80, on all host interfaces
<code>-p 127.0.0.1:8080:80</code>	Only reachable from the host itself
<code>-p 80</code>	Container port 80 mapped to a random high host port
<code>-P</code>	Publish every EXPOSEd port to random host ports
(none)	Reachable only from other containers on the same network

"THE CONTAINER RUNS BUT I CANNOT REACH IT"

Three causes, in order of frequency: (1) the application inside is bound to `127.0.0.1` instead of `0.0.0.0`, so it only accepts connections from inside the container's own namespace; (2) no `-p` mapping, so nothing is published; (3) the port in `-p host:container` is backwards, or the container port does not match what the app actually listens on. Diagnose with `docker exec web ss -tulpn` and `docker port web`.

7.6 Volumes and Persistence

	Named volume	Bind mount	tmpfs
Syntax	<code>-v appdata:/var/lib/app</code>	<code>-v /host/path:/container/path</code>	<code>--tmpfs /tmp</code>
Managed by	Docker, under <code>/var/lib/docker/volumes</code>	You - any host path	Memory only
Portability	Good - no host path assumptions	Tied to the host's filesystem layout	n/a
Permissions	Docker handles initial ownership	Host UID/GID must match - the usual source of permission errors	n/a

	Named volume	Bind mount	tmpfs
Backups	<code>docker run --rm -v vol:/d -v \$PWD:/b alpine tar czf /b/vol.tgz /d</code>	Normal filesystem backup	Not persisted
Best for	Databases and application state in production	Source code in development, config files	Secrets and scratch space, never written to disk

Volume operations

```

docker volume create pgdata
docker volume ls
docker volume inspect pgdata          # shows the Mountpoint on the host
docker volume rm pgdata
docker run -d --name db -v pgdata:/var/lib/postgresql/data postgres:16
docker run -d -v $(pwd)/src:/app/src:ro myapp      # read-only bind mount
docker run --rm -v pgdata:/data -v $(pwd)/backup alpine \
    tar czf /backup/pgdata-$(date +%F).tgz -C /data .

```

CONTAINER DATA IS EPHEMERAL BY DESIGN

The writable layer dies with the container. Anything that must survive a restart, an image update or a reschedule goes in a volume - or, better in a cloud context, in a managed database or object store. "Where does your container's data live?" is a question many candidates fail.

7.7 Docker Compose

docker-compose.yml with healthchecks, limits and dependencies

```

services:
  api:
    build:
      context: .
      dockerfile: Dockerfile
    args:
      VERSION: "1.4.2"
    image: registry.example.com/payments-api:1.4.2
    container_name: payments-api
    restart: unless-stopped
    ports:
      - "8080:8080"
    environment:
      APP_ENV: production
      DB_HOST: db
      DB_PORT: "5432"
    env_file:
      - .env
    depends_on:
      db:
        condition: service_healthy      # wait for the DB healthcheck, not just for it to start
    networks:
      - appnet
    healthcheck:
      test: ["CMD", "curl", "-fsS", "http://localhost:8080/health"]
      interval: 30s
      timeout: 5s
      retries: 3
      start_period: 20s
    deploy:
      resources:
        limits:
          cpus: "1.0"
          memory: 512M
    logging:
      driver: json-file
      options:
        max-size: "10m"
        max-file: "3"
  db:

```

```

image: postgres:16-alpine
restart: unless-stopped
environment:
  POSTGRES_DB: payments
  POSTGRES_USER: payments
  POSTGRES_PASSWORD_FILE: /run/secrets/db_password
volumes:
  - pgdata:/var/lib/postgresql/data
networks:
  - appnet
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U payments"]
  interval: 10s
  timeout: 5s
  retries: 5
secrets:
  - db_password

volumes:
  pgdata:

networks:
  appnet:
    driver: bridge

secrets:
  db_password:
    file: ./secrets/db_password.txt

```

Compose commands

```

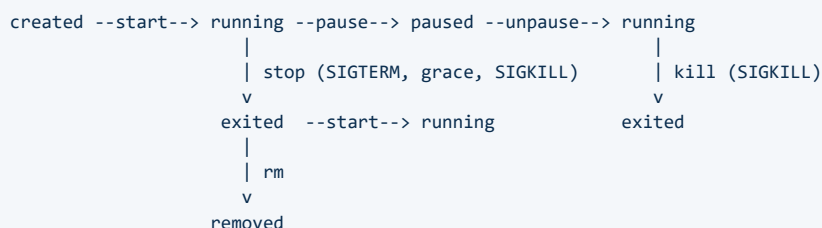
docker compose up -d          # build if needed, create and start everything
docker compose up -d --build  # force a rebuild
docker compose ps
docker compose logs -f api    # follow one service
docker compose exec api sh
docker compose stop           # stop but keep containers
docker compose down           # stop and REMOVE containers and networks
docker compose down -v        # ...and volumes - DATA LOSS
docker compose restart api
docker compose config          # render the final, merged configuration - use it to debug
docker compose pull            # refresh images
docker compose up -d --scale api=3 # multiple instances of one service

```

DEPENDS_ON ALONE DOES NOT WAIT FOR READINESS

Plain `depends_on` only waits for the container to START, not for the application inside to be ready. A database container is "started" long before Postgres accepts connections. The fix is `condition: service_healthy` with a real healthcheck, or application-level retry logic - which you need anyway, because dependencies restart in production.

7.8 Container Lifecycle and Restart Policies



States a container moves through

Restart policy	Behaviour
no (default)	Never restarts
on-failure[:max]	Restart only on a non-zero exit, optionally limited
always	Always restart, including after daemon restart - even if you stopped it manually

Restart policy	Behaviour
unless-stopped	Like always, but respects a manual stop - usually what you want

Exit code	Meaning
0	Clean exit - the main process finished. A container is not a VM: when PID 1 exits, it is over
1	Generic application error - check <code>docker logs</code>
125	The docker command itself failed (bad flag)
126	Command found but not executable - permissions or wrong architecture
127	Command not found - wrong path, or missing in a slim base image
137	Killed by SIGKILL (128+9) - usually the OOM killer or a <code>docker kill</code>
139	Segmentation fault (128+11)
143	Terminated by SIGTERM (128+15) - a normal <code>docker stop</code>

7.9 Docker Troubleshooting Scenarios

CONTAINER EXITS IMMEDIATELY

1. `docker ps -a` - read the exit code and the status.
2. `docker logs <container>` - the error is almost always here.
3. Exit 0 with no error usually means the main process finished. A container with `CMD ["nginx"]` exits immediately because nginx daemonises - it must run in the FOREGROUND (`nginx -g "daemon off;"`). Same class of bug: a shell script that ends, or `CMD ["bash"]` with no TTY.
4. Exit 127 - command not found. The binary is not in the slim base image, or the path is wrong. Check with `docker run --entrypoint sh -it image` and look.
5. Exit 126 - not executable, or an architecture mismatch (an amd64 binary on arm64, which is increasingly common on Apple hardware).
6. Exit 1 - read the application error; usually a missing environment variable or unreachable dependency at startup.
7. To inspect a container that will not stay up: `docker run --entrypoint sh -it image` to get a shell without running the app.

CONTAINER RESTARTING IN A LOOP (THE DOCKER EQUIVALENT OF CRASHLOOPBACKOFF)

1. `docker ps` shows "Restarting (1) 5 seconds ago" - the restart policy is masking a crash.
2. `docker logs --tail 100 <name>` after each crash; add `-f` to watch it fail live.
3. Common causes: a dependency such as the database is not reachable yet, a required environment variable is missing, a config file failed to parse, a port is already in use inside the container, or a permissions error on a mounted volume.
4. `docker inspect -f '{{.State.ExitCode}} {{.State.OOMKilled}}' <name>` - `OOMKilled: true` means it exceeded its memory limit.
5. Temporarily set `--restart=no` and run it in the foreground to see the failure clearly.

PORT IS NOT ACCESSIBLE FROM OUTSIDE

1. Is it actually published? `docker port <name>` and `docker ps` - look at the PORTS column.
2. What is the app bound to inside? `docker exec <name> ss -tulpn - 127.0.0.1:8080` means it will never be reachable; it must be `0.0.0.0:8080`.
3. Is the mapping the right way round? `-p HOSTPORT:CONTAINERPORT`.
4. Does the container port match what the app really listens on? EXPOSE documents, it does not publish or change anything.
5. Host firewall / security group open for the host port?
6. Test in layers: `docker exec <name> curl localhost:8080` (app is up) -> `curl localhost:8080` from the host (mapping works) -> from another machine (firewall).

CONTAINER CANNOT CONNECT TO THE DATABASE

1. Are they on the same user-defined network? On the default bridge there is no DNS, so `DB_HOST=db` will not resolve. `docker network inspect appnet` to confirm both are attached.
2. Is the app using `localhost` for the database? Inside a container, `localhost` is the CONTAINER itself, not the host and not the DB container. This is the single most common containerisation mistake.
3. To reach a service on the HOST from inside a container: `host.docker.internal` on Docker Desktop, or the bridge gateway IP on Linux (`ip route | grep default` from inside the container).
4. Test from inside: `docker exec -it api sh -c "nc -zv db 5432"` Or `getent hosts db`.
5. Is the database actually ready? Startup ordering - use healthchecks and retries.
6. Credentials and the database name - check with `docker exec api env`.

IMAGE BUILD FAILURE

- **COPY failed: file not found in build context** - the file is outside the context directory, or excluded by `.dockerignore`. The context is the path you passed to `docker build`, and nothing outside it can be copied.
- **Network failures during `apt-get/npm install`** - proxy settings, DNS inside the build, or a rate-limited registry. Docker Hub anonymous pull limits cause this in CI regularly; authenticate or use a pull-through cache.
- **A layer that used to work now fails** - a mutable base tag moved, or an upstream package version disappeared. Pin versions.
- **Out of disk during build** - `docker builder prune`.
- **RUN `cd /app` having no effect** - each RUN is a new layer with its own shell; use `WORKDIR`.
- **Debug by building up to the failing stage:** `docker build --target builder -t dbg .` then `docker run -it dbg sh`.

IMAGE PULL FAILURE

- **pull access denied / repository does not exist** - wrong registry path, typo, private repo without login, or the image was never pushed.
- **unauthorized: authentication required** - not logged in, or an expired token. For ECR the login token lasts 12 hours:
`aws ecr get-login-password | docker login --username AWS --password-stdin <acct>.dkr.ecr.<region>.amazonaws.com`
- **manifest unknown** - that tag does not exist. Check whether the push stage actually succeeded.
- **toomanyrequests** - Docker Hub rate limit. Authenticate, mirror, or use a private registry.
- **Timeouts on a private subnet** - the node has no route to the registry; it needs a NAT gateway or a VPC endpoint.
- **Architecture mismatch** - no matching manifest for `linux/arm64`. Build multi-arch with `buildx`.

PERMISSION DENIED INSIDE THE CONTAINER

- A bind-mounted host directory keeps its host UID/GID. If the container runs as UID 10001 and the host directory is owned by UID 1000, writes fail. Fixes: `chown` on the host to match, run the container with `-u $(id -u):$(id -g)`, or use a named volume, which Docker initialises with the right ownership.
- Files created by a root container on a bind mount end up root-owned on the host - which is why CI cannot clean its workspace afterwards.
- SELinux on RHEL blocks bind mounts unless you append `:z` or `:Z` to the volume flag.
- `docker: permission denied` while trying to connect to the Docker daemon socket on the host means your user is not in the `docker` group.

DISK FULL ON A DOCKER HOST

1. `docker system df` - is it images, containers, volumes or build cache?
2. `docker system prune -af --filter "until=72h"` for a safe cleanup; add `--volumes` only when you are certain nothing needed is unused.
3. Unbounded container logs are a classic cause - the `json-file` driver grows forever by default. Set `max-size` and `max-file` in `/etc/docker/daemon.json` so it applies to every container.
4. Exited containers holding writable layers - `docker container prune`.
5. Set a scheduled prune job on build hosts; on Kubernetes nodes, kubelet garbage collection handles images.

CONTAINER MEMORY ISSUES

- `docker stats` shows usage against the limit; `docker inspect -f '{{.State.OOMKilled}}'` confirms a kill.
- Exit code 137 means SIGKILL, which for a memory-limited container almost always means OOM.
- Without `--memory`, a container can consume all host memory and the kernel OOM killer may pick a different, innocent process. Always set limits in production.
- A JVM in a container must be told about the limit - modern JVMs read cgroup limits, but older ones see the host's total RAM and set a heap far larger than the container allows, guaranteeing an OOM kill. `-XX:MaxRAMPercentage=75` is the standard fix.

7.10 Docker Interview Questions

Q. 1. What is Docker and what problem does it solve?

A platform for packaging an application together with its dependencies into an image that runs identically anywhere the Docker engine runs. It solves environment inconsistency - the "works on my machine" problem - because the runtime, libraries and configuration travel with the application rather than being installed differently on each server.

Q. 2. Difference between a container and a virtual machine?

A VM virtualises hardware and runs a full guest operating system with its own kernel, so it is gigabytes in size and takes minutes to boot. A container virtualises the OS - it is a process on the host kernel, isolated with namespaces and limited with cgroups - so it is megabytes and starts in under a second. The trade-off is isolation strength: containers share the host kernel, so a kernel vulnerability is shared.

Q. 3. Difference between an image and a container?

An image is an immutable, layered template. A container is a running instance of that image with a thin writable layer on top. Many containers can run from one image; deleting a container deletes its writable layer, which is why persistent data belongs in a volume.

Q. 4. What is a Docker layer and why does it matter?

Each filesystem-changing instruction in a Dockerfile creates a read-only layer, cached by content. If nothing above a layer changed, Docker reuses it. That is why instruction order matters: copying dependency manifests and installing dependencies before copying source code keeps the expensive layer cached across code changes.

Q. 5. CMD versus ENTRYPOINT?

CMD provides a default command that is completely replaced if the user supplies one at `docker run`. ENTRYPOINT defines the executable that always runs, and any arguments at run time are appended to it. The recommended pattern is ENTRYPOINT for the binary and CMD for its default arguments.

Q. 6. What is the difference between exec form and shell form?

Exec form, CMD ["python","app.py"], runs the process directly as PID 1 so it receives signals and docker stop works immediately. Shell form, CMD python app.py, wraps it in /bin/sh -c, so the shell is PID 1 and does not forward SIGTERM - the container then takes the full grace period and gets SIGKilled. Always prefer exec form.

Q. 7. COPY versus ADD?

Both copy files into the image, but ADD additionally auto-extracts local tar archives and can download URLs. That implicit behaviour causes surprises, so the guidance is to use COPY unless you specifically want tar extraction.

Q. 8. What does EXPOSE actually do?

Nothing functional - it documents which port the application listens on, and it is used by -P to pick ports and by some tooling. Publishing is done with -p at run time, or by the orchestrator.

Q. 9. What is a multi-stage build and why use it?

A Dockerfile with several FROM stages where the final stage copies only the built artifacts from earlier stages. The compiler, build tools, dev dependencies and source never reach the final image, so it is dramatically smaller and has a much smaller attack surface, while the build still happens in one reproducible step.

Q. 10. How do you reduce image size?

Multi-stage builds, a slim or alpine base, chaining RUN commands and cleaning package caches within the same layer, .dockerignore to shrink the build context, avoiding unnecessary packages, and --no-install-recommends. docker history shows which layer is responsible when an image is unexpectedly large.

Q. 11. What is the difference between a volume and a bind mount?

A named volume is managed by Docker in its own storage area, is portable between hosts and handles initial permissions sensibly - it is the right choice for production state. A bind mount maps an arbitrary host path into the container, which is ideal for mounting source code in development but couples you to the host's filesystem layout and its UID/GID ownership.

Q. 12. How do containers communicate with each other?

On a user-defined bridge network, by container name - Docker provides embedded DNS. On the default bridge there is no name resolution, only IPs, which is why you create a network explicitly. Across hosts you would use an overlay network or, in practice, Kubernetes Services.

Q. 13. Inside a container, what is localhost?

The container itself. It is not the host and not another container. Connecting to a database on localhost from inside an app container is the most common containerisation mistake - the database must be addressed by its service name on the shared network, or via host.docker.internal when it genuinely runs on the host.

Q. 14. What are namespaces and cgroups?

Namespaces give isolation - separate views of process IDs, network stack, mounts, hostname, IPC and users. Cgroups give resource control - limits and accounting for CPU, memory, I/O and PIDs. Together they are what makes a container a container; Docker is the tooling around those kernel features.

Q. 15. How do you limit a container's resources?

--memory=512m and --cpus=1.5 at run time, or deploy.resources.limits in Compose, or requests and limits in Kubernetes. Without limits, one container can starve everything else on the host, and the kernel's OOM killer may kill an unrelated process.

Q. 16. What does exit code 137 mean?

128 + 9, so SIGKILL. In practice it means the container was killed for exceeding its memory limit, or someone ran docker kill. docker inspect showing OOMKilled: true confirms it.

Q. 17. How do you debug a container that will not start?

docker ps -a for the exit code, docker logs for the error, then docker run --entrypoint sh -it <image> to get a shell without running the failing command, so I can check the filesystem, permissions and whether the binary is even present. docker inspect gives the final configuration including mounts and environment.

Q. 18. How do you get a shell in a running container?

docker exec -it <name> bash, or sh for alpine-based images. If the image has no shell at all - a distroless or scratch image - I use docker debug where available, or run a sidecar/ephemeral container sharing the same namespaces.

Q. 19. Where do container logs go?

Whatever the main process writes to stdout and stderr is captured by the logging driver - by default json-file on the host. That is why applications in containers should log to stdout rather than to files: the platform collects it. Unbounded json-file

logs are a classic disk-full cause, so set `max-size` and `max-file`.

Q. 20. How do you pass configuration and secrets to a container?

Configuration as environment variables or mounted config files. Secrets should never be in the image or in ENV if avoidable, because `docker inspect` and image history expose them - use Docker secrets, a mounted `tmpfs` file, or a secrets manager, and in Kubernetes a Secret mounted as a file with the appropriate access controls.

Q. 21. Why should a container not run as root?

Because a container escape or an exploited application then has root on the host namespace, and a root process can write to bind-mounted host paths as root. Creating an unprivileged user and using `USER` costs nothing and removes an entire class of escalation.

Q. 22. What is a distroless image?

An image containing only the application and its runtime dependencies - no shell, no package manager, no utilities. It minimises the attack surface and CVE count. The trade-off is that debugging is harder because you cannot `exec` into a shell.

Q. 23. What is HEALTHCHECK and who uses the result?

An instruction defining a command that tests whether the application inside is actually working. Docker marks the container healthy or unhealthy, and Compose's `depends_on: condition: service_healthy` and various orchestrators act on it. Kubernetes ignores it - it has its own liveness and readiness probes.

Q. 24. How do you persist database data in Docker?

A named volume mounted at the database's data directory. In real production I would question running the database in a container at all on a single host - a managed service such as RDS removes the backup, failover and upgrade burden. If it must be containerised, it needs a volume, a backup job, and an understanding that the container can be rescheduled.

Q. 25. What happens when you run `docker run nginx`?

The CLI sends the request to the daemon; the daemon checks for the image locally and pulls it from the registry if missing; it creates a container with new namespaces and cgroups and a writable layer; it sets up networking on the bridge and, if requested, port mappings; then it starts the entrypoint process as PID 1 inside those namespaces and streams its output back.

Q. 26. What is the build context?

The directory you pass to `docker build`, which is sent to the daemon in its entirety before the build begins. That is why a large context - `.git`, `node_modules`, build output - makes builds slow, and why `.dockerignore` matters. Nothing outside the context can be `COPY`d.

Q. 27. How do you version and tag images?

With an immutable tag derived from the build - the CI build number plus the short Git SHA - and a semantic version for releases. `latest` is mutable and should never be what you deploy, because you cannot tell what is running or roll back precisely. Even better is deploying by digest.

Q. 28. What is the difference between `docker stop` and `docker kill`?

`stop` sends `SIGTERM`, waits for a grace period (10 seconds by default), then `SIGKILL` - so the application can shut down cleanly. `kill` sends `SIGKILL` immediately with no chance to clean up.

Q. 29. How would you scan images for vulnerabilities?

Trivy, Gype or the registry's own scanning such as ECR scan-on-push, run as a pipeline stage that fails the build on HIGH or CRITICAL findings. Just as important is rebuilding regularly, because a base image that was clean three months ago is not clean today.

Q. 30. What is Docker Compose for, and what are its limits?

Defining and running a multi-container application on a single host from one YAML file - ideal for local development, CI environments and small deployments. Its limit is exactly that: one host. There is no scheduling across nodes, no self-healing across machines and no rolling update with readiness gating, which is where Kubernetes takes over.

Q. 31. How do containers get their IP addresses?

The Docker daemon runs an embedded IPAM that assigns an address from the network's subnet - `172.17.0.0/16` for the default bridge - and connects the container's veth pair to the bridge. Those addresses are not stable across restarts, which is why you address containers by name on a user-defined network rather than by IP.

Q. 32. What is the difference between `docker run` and `docker start`?

`run` creates a NEW container from an image and starts it. `start` restarts an EXISTING stopped container with its original configuration. Repeatedly running `docker run` when you meant `start` is how hosts accumulate dozens of stopped

containers eating disk.

Q. 33. How do you copy files in and out of a container?

`docker cp container:/path ./local` and the reverse. It is useful for pulling out a log or a heap dump during an incident, but it is not a deployment mechanism - anything that needs to be in the image belongs in the Dockerfile.

Q. 34. What does `docker diff` tell you?

Which files have been added, changed or deleted in the container's writable layer compared with the image. It answers "what has this container actually written?" - useful for finding state that should have been in a volume, and for spotting a process writing logs inside the container.

Q. 35. How do you handle time zones and locales in containers?

Most base images are UTC, which is usually correct - logs and timestamps should be UTC everywhere and converted for display. If an application genuinely needs a local time zone, set `TZ` and install `tzdata`. Mismatched time zones between containers and hosts make log correlation during an incident much harder.

Q. 36. Your image builds on your laptop but fails in CI. Why?

Different build context or `.dockerignore`, a cached layer locally that CI does not have (so a broken step is hidden on your machine), different architecture - arm64 laptop versus amd64 CI, a mutable base tag that moved between the two builds, registry authentication or rate limits in CI, or network restrictions on the build agent. `--no-cache` locally usually reproduces it immediately.

Q. 37. How would you set up a private registry?

Use a managed one - ECR, Harbor, Nexus or Artifactory - with authentication, image scanning, retention policies to control storage, and immutable tags so a tag cannot be overwritten. CI authenticates with a scoped credential; for ECR that is a short-lived token from `get-login-password`, which is better than a long-lived secret.

Q. 38. What is the difference between `docker system prune` and `docker system prune -a --volumes`?

The plain form removes stopped containers, unused networks, dangling images and build cache. Adding `-a` also removes every image not currently used by a container, and `--volumes` removes unused volumes, which can destroy data that was merely not attached at that moment. I use a time filter and I always run `docker system df` first.

Q. 39. How do you make containers start automatically after a host reboot?

`--restart unless-stopped` (or `always`) so the daemon restarts them, and ensure the Docker service itself is enabled at boot. For anything that matters, an orchestrator or a systemd unit managing the container is more reliable than the restart policy alone.

Q. 40. What is the biggest mistake you see people make with Docker?

Treating a container like a virtual machine - SSHing into it, patching it by hand, storing state inside it, and running multiple services in one container with a supervisor. Containers should be immutable, single-purpose and disposable: to change something you rebuild the image and replace the container, and anything that must survive lives in a volume or an external service.

PART 7 RECALL POINTS

- Image = template, container = running instance with a writable layer.
- Manifest and dependencies BEFORE source, or you destroy the layer cache.
- ENTRYPOINT = the binary, CMD = its default arguments; always exec form.
- EXPOSE documents; `-p` publishes.
- Default bridge has no DNS - create a user-defined network.
- `localhost` inside a container is the container.
- 137 = OOMKilled; 127 = command not found; 126 = not executable.
- Bind mounts keep host UID/GID - the usual permission-denied cause.
- Never deploy `latest`; tag with build number + SHA.
- `docker system df` before any prune.

PART 8

AWS COMPLETE DEVOPS GUIDE

EC2, IAM, VPC, S3, CloudWatch, Route 53, load balancing, architecture, 60 questions

Your CV lists AWS as EC2, S3, IAM, VPC, CloudWatch, Route 53 and Security Groups. That is exactly the right scope for a 3-year DevOps engineer, and this part goes deep on those services rather than spreading thin across fifty. Anything outside that list is marked as preparation material - know the concept, never claim the experience.

8.1 EC2 - Elastic Compute Cloud

Concept	What it is	Interview detail
Instance	A virtual server	Identified by an instance ID; has a lifecycle: pending, running, stopping, stopped, terminated
AMI	Amazon Machine Image - the template an instance boots from	Region-specific; the basis of immutable infrastructure (bake an AMI, replace instances)
Instance type	The hardware profile - family, generation, size	t3.micro, m5.large, c5.xlarge, r5.large
EBS volume	Network-attached block storage	Persists independently of the instance; gp3 is the current general-purpose default
Instance store	Physically attached NVMe	EPHEMERAL - data is lost on stop or terminate. A classic exam trap
Security group	Instance-level stateful firewall	
Key pair	SSH public key injected at launch	AWS keeps the public key; lose the private key and you cannot SSH
User data	A script run at first boot	Bootstrapping - installs, configuration, joining a cluster
Instance metadata	http://169.254.169.254/latest/meta-data/	Identity, and temporary IAM role credentials
Elastic IP	A static public IPv4 you own	Charged when not attached to a running instance

Instance family	Optimised for	Typical workload
t (t3, t4g)	Burstable, cheap	Dev, small services, low steady CPU. Watch CPU credits - exhaustion causes sudden throttling
m (m5, m6i)	Balanced	General application servers
c (c5, c6i)	Compute	CPU-heavy: encoding, batch, build agents
r (r5, r6i)	Memory	Caches, in-memory databases
i, d	Storage	High local IOPS
g, p	GPU	ML, video transcoding

EBS type	Profile	Use
gp3	General purpose SSD, baseline 3000 IOPS independent of size	Default choice for almost everything
gp2	Older general purpose; IOPS scale with size	Legacy
io1 / io2	Provisioned IOPS SSD, highest durability	Demanding databases
st1	Throughput-optimised HDD	Big sequential reads - logs, data warehouse
sc1	Cold HDD	Infrequent access, lowest cost

Purchase option	Discount	Trade-off
On-Demand	None	Maximum flexibility
Reserved / Savings Plans	Up to ~72%	1 or 3-year commitment

Purchase option	Discount	Trade-off
Spot	Up to ~90%	Can be reclaimed with a 2-minute warning - ideal for CI agents and batch, never for a single stateful server
Dedicated Host	Premium	Licensing or compliance requirements

EC2 operations from the CLI

```
aws ec2 describe-instances --filters "Name=instance-state-name,Values=running" \
  --query "Reservations[].Instances[].[InstanceId,InstanceType,PrivateIpAddress,Tags[?Key=='Name'].Value][0]" \
  --output table

aws ec2 describe-instances --instance-ids i-0abc123
aws ec2 stop-instances --instance-ids i-0abc123
aws ec2 start-instances --instance-ids i-0abc123
aws ec2 reboot-instances --instance-ids i-0abc123
aws ec2 describe-instance-status --instance-ids i-0abc123 # system and instance status checks
aws ec2 get-console-output --instance-id i-0abc123 # boot log when SSH is impossible
aws ec2 create-image --instance-id i-0abc123 --name "app-2026-09-23"
aws ec2 describe-security-groups --group-ids sg-0abc
aws ec2 authorize-security-group-ingress --group-id sg-0abc --protocol tcp --port 443 --cidr 0.0.0.0/0
```

User data - bootstrapping at first boot

```
#!/bin/bash
set -euo pipefail
exec >>(tee /var/log/user-data.log) 2>&1 # capture everything; this log is your first debug stop

dnf update -y
dnf install -y docker amazon-cloudwatch-agent
systemctl enable --now docker
usermod -aG docker ec2-user

REGION=$(curl -s http://169.254.169.254/latest/meta-data/placement/region)
INSTANCE_ID=$(curl -s http://169.254.169.254/latest/meta-data/instance-id)
echo "Bootstrapping ${INSTANCE_ID} in ${REGION}"

aws ecr get-login-password --region "$REGION" | docker login --username AWS --password-stdin "$ECR_REGISTRY"
docker run -d --restart unless-stopped -p 80:8080 "$ECR_REGISTRY/app:$APP_VERSION"
```

USER DATA RUNS ONCE BY DEFAULT

It executes on first boot only. If it fails, the instance still launches - it just is not configured, which is why a health check that only tests "is the instance running" is useless. Debug with `/var/log/cloud-init-output.log` and `/var/log/user-data.log`. Note also that user data is visible to anyone who can read instance metadata, so it must never contain secrets - fetch them at boot from Secrets Manager or SSM Parameter Store using the instance's IAM role.

8.1.1 Auto Scaling

Component	Role
Launch Template	Defines what to launch: AMI, instance type, security groups, user data, IAM profile
Auto Scaling Group (ASG)	Maintains desired capacity across subnets/AZs, replaces unhealthy instances
Scaling policy	Target tracking (keep average CPU at 60%), step scaling, or scheduled
Health check type	EC2 (is it running) or ELB (is it passing the load balancer's health check) - use ELB
Cooldown / warm-up	Prevents thrashing while a new instance starts

THE ASG ANSWER THAT SHOWS DEPTH

"An ASG gives me two things: elasticity and self-healing. Elasticity is the scaling policy - I prefer target tracking on a meaningful metric rather than step scaling on raw CPU. Self-healing is the health check - and the important detail is setting the health check type to ELB rather than EC2, because an instance can be running perfectly while the application on it is dead. With EC2 health checks the ASG will happily keep a broken instance forever."

8.2 IAM - Identity and Access Management

Entity	What it is	Use
User	A person or a legacy application identity	Humans; avoid for applications
Group	A collection of users	Attach policies here, not to individual users
Role	An identity assumed temporarily, with no long-lived credentials	EC2 instances, Lambda, EKS pods, cross-account access, federated humans
Policy	A JSON document granting or denying permissions	Managed (AWS or customer) or inline
Identity-based policy	Attached to a user, group or role	"What can this identity do?"
Resource-based policy	Attached to the resource, e.g. an S3 bucket policy	"Who can act on this resource?"
Permissions boundary	A ceiling on what a policy can grant	Delegating admin safely
SCP (Organizations)	Guardrails across whole accounts	Can only restrict, never grant

Anatomy of an IAM policy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadAppArtifacts",
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:ListBucket"],
      "Resource": [
        "arn:aws:s3:::my-artifacts",
        "arn:aws:s3:::my-artifacts/*"
      ],
      "Condition": {
        "IpAddress": { "aws:SourceIp": "10.0.0.0/16" }
      }
    },
    {
      "Sid": "DenyUnencryptedUploads",
      "Effect": "Deny",
      "Action": "s3:PutObject",
      "Resource": "arn:aws:s3:::my-artifacts/*",
      "Condition": {
        "StringNotEquals": { "s3:x-amz-server-side-encryption": "AES256" }
      }
    }
  ]
}
```

IAM EVALUATION LOGIC - A VERY COMMON QUESTION

An explicit DENY always wins. Otherwise, an explicit ALLOW is required - the default is deny. So the order is: explicit deny beats everything; then, if any applicable policy allows it, it is allowed; if nothing allows it, it is denied. With Organizations, an SCP that does not allow an action blocks it even if the IAM policy allows it, because SCPs are a ceiling, not a grant.

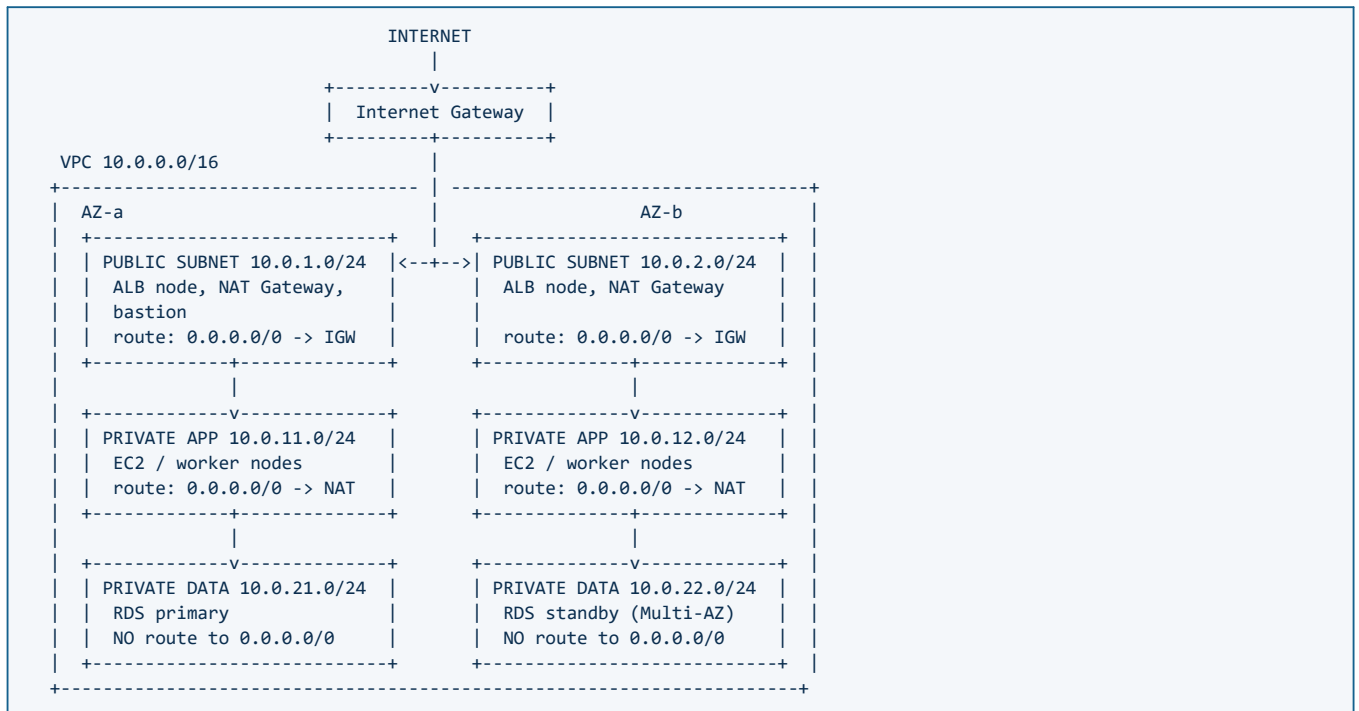
ROLES OVER ACCESS KEYS - THE SECURITY ANSWER

"For anything running on AWS I attach an IAM role rather than creating access keys. The instance gets temporary, automatically rotated credentials through the metadata service, so there is no long-lived secret to leak, commit to Git or forget to rotate. Access keys are for things outside AWS, and even then the modern answer is OIDC federation - for example GitHub Actions assuming a role - so there is still no stored secret."

Least privilege practice	Why
Grant specific actions, not "Action": "*"	Blast radius
Scope Resource to specific ARNs	A policy allowing s3:* on * is effectively admin over all data

Least privilege practice	Why
Use conditions - source IP, MFA, tags	Contextual restriction
Start from CloudTrail/Access Analyzer findings	Build the policy from what was actually used
Rotate and audit credentials	The IAM credential report lists key age and unused credentials
Enforce MFA on all human users	Especially the root account, which should then be locked away

8.3 VPC - Virtual Private Cloud



The standard production VPC

Layer	Why it exists
Internet Gateway	The only thing that makes a subnet "public" - a subnet is public because its route table sends 0.0.0.0/0 to an IGW, not because of a setting called "public"
Public subnet	Holds only what must be internet-facing: load balancer nodes, NAT gateways, bastion
NAT Gateway	Lets private instances reach the internet outbound for patches and API calls, while remaining unreachable inbound. One per AZ for high availability - a single NAT gateway is a single point of failure AND a cross-AZ data charge
Private app subnet	Application servers, never directly reachable from the internet; only the ALB's security group may reach them
Private data subnet	Databases; no internet route at all, reachable only from the app tier's security group
Two AZs	An AZ is a failure domain. ALB and RDS Multi-AZ both require two subnets in different AZs

VPC component	Purpose
CIDR block	The VPC's address space, e.g. 10.0.0.0/16. Cannot be shrunk later - size it generously
Subnet	A CIDR slice bound to ONE availability zone
Route table	Where traffic goes; associated with subnets
Internet Gateway	Two-way internet for resources with public IPs
NAT Gateway	One-way outbound internet for private resources
Security Group	Stateful, instance-level allow rules
NACL	Stateless, subnet-level allow and deny rules
VPC Endpoint (Gateway)	Private access to S3 and DynamoDB via a route table entry - free, and avoids NAT charges

VPC component	Purpose
VPC Endpoint (Interface/PrivateLink)	Private ENI for other AWS services or partner services
VPC Peering	Private one-to-one link between VPCs; not transitive; CIDRs must not overlap
Transit Gateway	Hub for many VPCs and on-premises connections; transitive routing
Flow Logs	Records accepted and rejected traffic - the tool for "why is this being blocked?"

THE S3 GATEWAY ENDPOINT IS A GREAT COST ANSWER

"If instances in private subnets pull large objects from S3 through a NAT gateway, you pay NAT data-processing charges for traffic that never needed to leave AWS. A gateway VPC endpoint for S3 is free, adds a route, and keeps that traffic on the AWS network - it improves security and removes the cost at the same time."

8.4 S3 - Simple Storage Service

Concept	Detail
Bucket	Globally unique name, lives in one region
Object	Key plus data plus metadata, up to 5 TB; use multipart upload above 100 MB
Durability	99.999999999% (11 nines) - designed for durability, not for availability
Consistency	Strong read-after-write for all operations since 2020
Versioning	Keeps every version; protects against overwrite and delete. A delete creates a delete marker
Lifecycle policy	Transition to cheaper classes or expire objects on a schedule
Encryption	SSE-S3 (AES256), SSE-KMS (audited, key-controlled), SSE-C, or client-side
Bucket policy	Resource-based JSON policy - who may do what to this bucket
Block Public Access	Account and bucket-level override that defeats accidental public policies - leave it ON
Pre-signed URL	Time-limited URL granting temporary access without credentials

Storage class	Use	Retrieval
Standard	Frequently accessed	Immediate
Intelligent-Tiering	Unknown or changing patterns	Immediate; moves objects automatically
Standard-IA	Infrequent but needs instant access	Immediate, retrieval fee
One Zone-IA	Re-creatable data	Immediate; lost if that AZ is lost
Glacier Instant Retrieval	Archive, instant access	Immediate
Glacier Flexible	Archive	Minutes to hours
Glacier Deep Archive	Long-term compliance	Up to 12 hours; cheapest

S3 operations

```
aws s3 ls
aws s3 ls s3://my-artifacts/releases/ --recursive --human-readable --summarize
aws s3 cp app.jar s3://my-artifacts/releases/1.4.2/app.jar
aws s3 cp s3://my-artifacts/releases/1.4.2/app.jar .
aws s3 sync ./build s3://my-site --delete # mirror, removing extras
aws s3 rm s3://my-artifacts/old/ --recursive
aws s3 presign s3://my-artifacts/report.pdf --expires-in 3600

aws s3api get-bucket-versioning --bucket my-artifacts
aws s3api put-bucket-versioning --bucket my-artifacts --versioning-configuration Status=Enabled
aws s3api get-bucket-policy --bucket my-artifacts
aws s3api list-object-versions --bucket my-artifacts --prefix releases/
aws s3api head-object --bucket my-artifacts --key releases/1.4.2/app.jar
```

A lifecycle policy that controls cost

```
{
  "Rules": [{
    "ID": "archive-and-expire-logs",
    "Status": "Enabled",
```

```

"Filter": { "Prefix": "logs/" },
"Transitions": [
  { "Days": 30, "StorageClass": "STANDARD_IA" },
  { "Days": 90, "StorageClass": "GLACIER" }
],
"Expiration": { "Days": 365 },
"NoncurrentVersionExpiration": { "NoncurrentDays": 30 },
"AbortIncompleteMultipartUpload": { "DaysAfterInitiation": 7 }
}]
}

```

THE TWO S3 DETAILS THAT CATCH PEOPLE OUT

1) Versioning plus no lifecycle rule for noncurrent versions means your bill grows forever - every overwrite keeps the old object. 2) Incomplete multipart uploads are invisible in the console's object list but you are billed for them; always add `AbortIncompleteMultipartUpload`. Mentioning either of these marks you as someone who has actually owned an S3 bill.

8.5 CloudWatch

Feature	What it is	Use
Metrics	Time-series numeric data, 1-minute (or 1-second detailed) resolution	CPUUtilization, NetworkIn, ALB TargetResponseTime, custom metrics
Logs	Log groups containing log streams	Application and system logs via the CloudWatch agent
Alarms	Threshold evaluation on a metric, with actions	Notify SNS, trigger Auto Scaling, recover an instance
Dashboards	Visualisation	Operational views
Logs Insights	Query language over log groups	Incident investigation without a separate stack
Events / EventBridge	React to AWS events	Trigger automation on state changes
Metric filter	Turn a log pattern into a metric	Count ERROR lines and alarm on the rate

MEMORY AND DISK ARE NOT DEFAULT EC2 METRICS

CloudWatch sees the hypervisor's view: CPU, network and disk I/O. It cannot see inside the guest OS, so memory used and disk space used require the CloudWatch agent to be installed and configured. "Why is there no memory metric for my EC2 instance?" is a standard interview question and the answer is exactly this.

CloudWatch from the CLI

```

aws cloudwatch get-metric-statistics \
  --namespace AWS/EC2 --metric-name CPUUtilization \
  --dimensions Name=InstanceId,Value=i-0abc123 \
  --start-time 2026-09-23T00:00:00Z --end-time 2026-09-23T06:00:00Z \
  --period 300 --statistics Average Maximum

aws cloudwatch put-metric-alarm \
  --alarm-name app-high-5xx \
  --namespace AWS/ApplicationELB --metric-name HTTPCode_Target_5XX_Count \
  --statistic Sum --period 300 --threshold 50 \
  --comparison-operator GreaterThanThreshold --evaluation-periods 2 \
  --alarm-actions arn:aws:sns:eu-west-1:123456789012:ops-alerts

aws logs describe-log-groups
aws logs tail /aws/app/payments --follow --since 10m
aws logs start-query --log-group-name /aws/app/payments \
  --start-time $(date -d '1 hour ago' +%s) --end-time $(date +%s) \
  --query-string 'fields @timestamp, @message | filter @message like /ERROR/ | sort @timestamp desc | limit 50'

```

8.6 Route 53

Feature	Detail
Hosted zone	A container for records for one domain; public or private (VPC-scoped)
Record types	A, AAAA, CNAME, MX, TXT, NS, SOA, PTR, SRV
ALIAS record	AWS-specific; works at the zone apex, points at AWS resources, free queries
Health check	Monitors an endpoint and can remove records from responses
Routing policies	Simple, weighted, latency, failover, geolocation, geoproximity, multivalue
TTL	Caching duration - lower it before a planned migration

Scenario	Policy
Send 10% of traffic to a new version	Weighted
Serve users from the closest region	Latency-based
Active-passive DR	Failover with a health check
Comply with data residency	Geolocation
Point the apex at an ALB	ALIAS - a CNAME is not permitted at the apex

8.7 Load Balancing on AWS

	ALB	NLB	CLB (legacy)
Layer	7	4	4 and 7
Routing	Host, path, header, method, query, source IP	IP and port	Basic
TLS	Terminates with ACM	Terminates or passes through	Terminates
Static IP	No (DNS name)	Yes - one Elastic IP per AZ	No
Source IP preserved	No - see X-Forwarded-For	Yes	No
WebSockets / HTTP/2	Yes	Pass-through	Limited
Targets	Instances, IPs, Lambda	Instances, IPs, ALB	Instances
Use	Web apps, microservices, containers	Non-HTTP protocols, extreme throughput, static IP requirements	Do not use for new work

Target group setting	Meaning	Common mistake
Health check path	e.g. /health	Pointing it at / which requires auth, so everything is unhealthy
Healthy / unhealthy threshold	Consecutive checks to change state	Too aggressive: a slow start flaps targets out of service
Interval and timeout	How often, how long to wait	Timeout longer than interval causes overlapping checks
Deregistration delay	Drain time before removing a target	Too short cuts live connections during a deploy
Stickiness	Cookie-based session affinity	Masking a stateful application that should be stateless

8.8 A Complete Architecture, and Why Each Layer Exists

```

USERS
|
[ Route 53 ]    DNS: apex ALIAS -> ALB. Health checks enable regional failover.
                WHY: a name that can be repointed without changing clients.
                |
                v
[ CloudFront ]  (optional) CDN: caches static content at the edge, absorbs spikes,
                terminates TLS close to users, provides DDoS buffering.
                |
                v
[ WAF ]         Filters malicious requests - SQLi, XSS, bad bots - before the app.
                |
                v
[ ALB ]         Public subnets, multiple AZs. Terminates TLS with an ACM cert,
                routes by host/path, health-checks targets, spreads load.
                WHY: the single entry point, so app servers are never exposed.
                |
                v
[ ASG: EC2 app ] Private subnets, multiple AZs. Security group accepts traffic
                ONLY from the ALB's security group. Scales on demand,
                replaces unhealthy instances automatically.
                WHY: elasticity plus self-healing, with no public exposure.
                |
                v
[ RDS Multi-AZ ] Private data subnets, no internet route. Synchronous standby in
                a second AZ with automatic failover; automated backups.
                WHY: the database is the hardest thing to rebuild - protect it most.
+--> [ ElastiCache ] Reduces read load and latency.
+--> [ S3 ]          Static assets, artifacts, backups - via a gateway endpoint.

Cross-cutting: CloudWatch metrics/logs/alarms, CloudTrail audit, IAM roles per tier,
NAT Gateway per AZ for outbound patching, Secrets Manager for credentials.

```

Internet to database - the standard three-tier design

"WHY DOES EACH LAYER EXIST?" - THE ANSWER THAT GETS YOU HIRED

"Every layer exists to remove a single point of failure or a class of exposure. Route 53 gives me a stable name and a failover mechanism. The load balancer is the only public entry point, so the application servers have no public IPs and cannot be attacked directly. The Auto Scaling Group gives elasticity and self-healing, so losing an instance is not an incident. Spanning two availability zones means losing an AZ is not an outage. The database sits in a subnet with no internet route and only accepts connections from the application tier's security group. And NAT gateways let private instances patch themselves without becoming reachable. If you take any one of those out, you can name exactly which failure it stops protecting you from."

8.9 AWS Troubleshooting Scenarios

EC2 UNREACHABLE / SSH FAILURE

1. Instance state and status checks: `aws ec2 describe-instance-status`. A failed SYSTEM status check is AWS's hardware - stop and start the instance to move it to new hardware. A failed INSTANCE status check is inside the OS - check the console output.
2. `aws ec2 get-console-output --instance-id i-xxx` - the boot log. A bad `/etc/fstab` entry blocking boot shows here, and it is the number one cause of "instance is running but nothing responds".
3. Security group: inbound TCP 22 from your IP. NACL: inbound 22 AND outbound on the ephemeral range, because NACLs are stateless.
4. Route table: does the subnet have a route to the IGW, and does the instance have a public IP or an Elastic IP?
5. Key pair: correct `.pem`, `chmod 600`, correct default user - `ec2-user`, `ubuntu`, `admin` or `centos` depending on the AMI.
6. `ssh -v` and read where it stops: "Connection timed out" is a network/firewall problem; "Permission denied (publickey)" means you reached `sshd` and the key or user is wrong. Never confuse those two.
7. Disk full on the instance can break `sshd` - the console output or SSM Session Manager will tell you.
8. If SSH is broken beyond repair: use SSM Session Manager, or stop the instance, detach the root volume, attach it to a working instance, fix the file, and reattach.

HTTP UNAVAILABLE BEHIND AN ALB

1. Target group health: are ANY targets healthy? An empty or all-unhealthy target group returns 503.
2. If unhealthy, read the reason - "Health checks failed with these codes: [404]" means the health check path is wrong; a timeout means the security group or the app is not responding.
3. The target's security group must allow the ALB's security group on the target port. Referencing the ALB's SG rather than a CIDR is the correct pattern.
4. Is the app listening on the port the target group uses, and bound to 0.0.0.0?
5. Listener rules: is there a rule matching this host/path, or is the default action returning a fixed 404?
6. TLS: certificate valid, and attached to the listener? Is the certificate in the same region as the ALB?
7. ALB access logs and the `HTTPCode_ELB_5XX` versus `HTTPCode_Target_5XX` metrics tell you whether the ALB or the application produced the error - that distinction saves a lot of time.

IAM ACCESSDENIED

1. Read the error message - it names the principal, the action and the resource. That is usually 80% of the answer.
2. Use the IAM Policy Simulator, or `aws sts get-caller-identity` to confirm WHICH identity you are actually using (a stale profile or environment variable is a common cause).
3. Check for an explicit Deny - in the identity policy, a permissions boundary, an SCP at the organisation level, or a resource policy. Deny always wins.
4. Is the resource ARN in the policy exactly right? `arn:aws:s3:::bucket` and `arn:aws:s3:::bucket/*` are DIFFERENT - bucket-level actions such as `ListBucket` need the first, object actions need the second. This is the single most common S3 policy mistake.
5. KMS: if the object is encrypted with a customer-managed key, the principal also needs `kms:Decrypt` on that key. The error still says S3 AccessDenied, which is confusing.
6. Cross-account: both the resource policy and the caller's identity policy must allow it.
7. CloudTrail shows the exact denied call with the full request context.

HIGH CPU ON AN INSTANCE

1. CloudWatch CPUUtilization - is it sustained or spiky, and when did it start? Correlate with a deploy or a traffic change.
2. On the instance: `top` with `1` for per-core, then identify the process. High `st` (steal) on a t-family instance means CPU credits are exhausted and you are being throttled - the fix is a different instance family or unlimited mode, not application tuning.
3. High `wa` means it is really disk - check EBS metrics (VolumeQueueLength, BurstBalance) before blaming the CPU.
4. Decide: scale out through the ASG for a real load increase, or fix the application if a single process is looping.
5. Prevent: a target-tracking scaling policy, plus an alarm before saturation rather than at it.

APPLICATION CANNOT REACH THE INTERNET FROM A PRIVATE SUBNET

1. Does the private subnet's route table have `0.0.0.0/0 -> nat-xxxxx`? This is the usual cause.
2. Is the NAT gateway in a PUBLIC subnet with a route to the IGW? A NAT gateway in a private subnet cannot work.
3. Security group outbound rules - default is allow all, but a hardened SG may not be.
4. NACL outbound AND the inbound ephemeral range for the return traffic.
5. Is it only S3 or DynamoDB failing? Then a gateway endpoint's route may be missing or misconfigured.
6. Confirm from the instance: `curl -sI https://example.com` and `dig example.com` to separate DNS from routing.

8.10 AWS Interview Questions

Q. 1. What is the difference between a region and an availability zone?

A region is a geographic area, such as eu-west-1, containing multiple availability zones. An AZ is one or more discrete data centres with independent power, cooling and networking, connected to the other AZs in the region by low-latency links. You design across AZs for high availability and across regions for disaster recovery or latency.

Q. 2. What makes a subnet public?

Its route table has a route for 0.0.0.0/0 pointing at an Internet Gateway. There is no checkbox called "public" - it is purely the route, plus the resource having a public IP.

Q. 3. Internet Gateway versus NAT Gateway?

An IGW provides two-way internet connectivity for resources with public IPs and is what makes a subnet public. A NAT Gateway provides outbound-only connectivity for private resources - they can reach the internet, but nothing on the internet can initiate a connection to them.

Q. 4. How many IP addresses does AWS reserve in a subnet?

Five - the network address, the VPC router, the DNS server, one reserved for future use, and the broadcast address. So a /24 gives 251 usable addresses.

Q. 5. Security group versus NACL?

Security groups are stateful, operate at the instance/ENI level, support allow rules only, and can reference other security groups. NACLs are stateless, operate at the subnet level, support allow and deny, and are evaluated in numeric order. Because NACLs are stateless, you must explicitly allow the ephemeral return ports - which is the cause of many mysterious timeouts.

Q. 6. Can a security group reference another security group? Why does that matter?

Yes, and it is the correct pattern for tiering: the database's security group allows 5432 from the application tier's security group rather than from a CIDR. It then keeps working as instances are replaced and IPs change, and it documents the intended relationship.

Q. 7. What is an IAM role and when do you use one instead of a user?

A role is an identity with permissions but no long-lived credentials, assumed temporarily. I use roles for anything running on AWS - EC2, Lambda, containers - so credentials are temporary and auto-rotated through the metadata service, and for cross-account access and federated human access. Users with access keys are a last resort for workloads outside AWS.

Q. 8. Explain IAM policy evaluation.

Default deny. An explicit Deny in any applicable policy always wins. Otherwise the request is allowed only if some policy explicitly allows it. Service Control Policies act as a ceiling across the account - if an SCP does not permit an action, no IAM policy can grant it. Permissions boundaries work the same way for an individual principal.

Q. 9. What is the instance metadata service and why does IMDSv2 exist?

A link-local endpoint at 169.254.169.254 that returns instance identity and the temporary credentials for the attached role. IMDSv1 was a simple GET, which meant a server-side request forgery vulnerability in an application could be used to read credentials. IMDSv2 requires a PUT to obtain a session token first and enforces a hop limit, which defeats that class of attack. Enforcing IMDSv2 is a standard hardening step.

Q. 10. What is the difference between EBS and instance store?

EBS is network-attached and persists independently of the instance's lifecycle - it survives a stop, and can be snapshot and reattached. Instance store is physically attached NVMe: very fast, but the data is lost when the instance stops or terminates. Anything you care about goes on EBS or in a managed service.

Q. 11. How do you back up EBS?

Snapshots, which are incremental and stored in S3. In practice I would use Data Lifecycle Manager or AWS Backup to schedule and retain them automatically rather than relying on a manual process, and I would test a restore, because an untested backup is a hypothesis.

Q. 12. What happens to data when you stop versus terminate an instance?

On stop, EBS volumes persist and instance store is lost; the public IPv4 changes unless it is an Elastic IP. On terminate, the root volume is deleted if DeleteOnTermination is true, which is the default, and the instance is gone permanently.

Q. 13. How do you resize an EC2 instance?

Stop it, change the instance type, start it again - which means downtime for that instance. That is exactly why you run behind a load balancer with an Auto Scaling Group: you change the launch template and roll instances, so the resize is not an outage.

Q. 14. What is an AMI and how do you use one in a DevOps workflow?

A template containing the root volume snapshot and launch configuration. The immutable infrastructure pattern is to bake an AMI in the pipeline - with Packer, for example - containing the OS, agents and the application, then roll the ASG onto the new AMI. Deployment becomes instance replacement and rollback becomes rolling back to the previous AMI.

Q. 15. What is user data and what are its limits?

A script that runs at first boot for bootstrapping. Limits: it runs once by default, it is capped at 16 KB, it runs as root, and it is readable from instance metadata - so it must never contain secrets. Debug it through `/var/log/cloud-init-output.log`.

Q. 16. How does an Auto Scaling Group decide an instance is unhealthy?

Either EC2 status checks, or the load balancer's health check if you set the health check type to ELB. ELB health checks are the right choice because an instance can pass EC2 checks while the application on it is completely broken.

Q. 17. What is a launch template and how does it differ from a launch configuration?

Both describe what to launch. Launch templates are the current mechanism: they support versioning, mixed instance types and newer features; launch configurations are immutable, legacy and no longer recommended. Versioning matters because rolling forward and back is just selecting a template version.

Q. 18. Difference between scaling out and scaling up?

Scaling out adds more instances - horizontal, near-unbounded, no downtime, requires a stateless application. Scaling up moves to a larger instance type - vertical, bounded by the largest type, and requires a restart. Cloud architecture prefers scaling out.

Q. 19. ALB versus NLB?

ALB is layer 7, routes on host, path and headers, terminates TLS, and suits web applications. NLB is layer 4, handles TCP and UDP for any protocol, offers static IPs per AZ, preserves the client source IP and handles extreme throughput. For a non-HTTP protocol such as RTSP, or a static-IP requirement, NLB is the answer.

Q. 20. How does an ALB know a target is healthy?

It sends a configured health check - a path, port and protocol - at an interval, and moves the target to healthy or unhealthy after consecutive successes or failures crossing the thresholds. Misconfiguring the path or setting the thresholds too aggressively is a common self-inflicted outage.

Q. 21. What is connection draining / deregistration delay?

When a target is removed, the load balancer stops sending it new requests but allows in-flight requests to complete for the deregistration delay, 300 seconds by default. Without it, a deployment cuts live connections. Setting it too high slows deployments, so it should roughly match your longest normal request.

Q. 22. What is S3 durability versus availability?

Durability is the probability that an object survives - 11 nines, achieved by replicating across devices and AZs. Availability is the probability you can access it at a given moment, which is lower. People conflate them: S3 will almost never lose your data, but it can have an availability event.

Q. 23. How do you make an S3 bucket private and be sure of it?

Enable Block Public Access at the account and bucket level - it overrides any bucket policy or ACL that would grant public access - disable ACLs with Object Ownership set to bucket owner enforced, and write a bucket policy that grants only the specific principals needed. Then verify with IAM Access Analyzer rather than by eye.

Q. 24. What is S3 versioning and what does it cost you?

Every overwrite or delete keeps the previous version, protecting against accidental loss, with deletes creating a delete marker. The cost is that storage grows indefinitely unless you add a lifecycle rule to expire noncurrent versions - a very common bill surprise.

Q. 25. How would you serve a static website with low latency globally?

S3 for storage with the bucket private, CloudFront in front with Origin Access Control so the bucket is only readable by the distribution, an ACM certificate on CloudFront, and Route 53 with an ALIAS at the apex. Caching at the edge handles latency and traffic spikes, and invalidations handle deployments.

Q. 26. What is a pre-signed URL?

A time-limited URL that grants temporary access to a specific object without the recipient having AWS credentials. It is the standard way to let a user upload or download a private object directly, keeping large transfers off your application servers.

Q. 27. Why can't CloudWatch show memory usage on EC2 by default?

Because CloudWatch collects metrics from the hypervisor, which sees CPU, network and disk I/O but has no visibility into the guest operating system. Memory and disk space used are inside the OS, so you install the CloudWatch agent to publish them as custom metrics.

Q. 28. What is a CloudWatch alarm and what states does it have?

A rule evaluating a metric against a threshold over a number of periods. States are OK, ALARM and INSUFFICIENT_DATA. Actions can notify SNS, trigger Auto Scaling, or recover an instance. Configuring what happens on INSUFFICIENT_DATA matters - it is often the state you get when the thing you are monitoring has died entirely.

Q. 29. How do you get application logs into CloudWatch?

Install and configure the CloudWatch agent to tail the log files into a log group, or have the application write to stdout and let the container platform's log driver forward it. Then use metric filters to turn patterns such as ERROR into metrics you can alarm on, and Logs Insights to query during an incident.

Q. 30. What is CloudTrail and how is it different from CloudWatch?

CloudTrail records API calls - who did what, from where, when - for audit and forensics. CloudWatch records operational telemetry - metrics and logs about how things are performing. During an incident I use CloudWatch to see what is broken and CloudTrail to see what someone changed.

Q. 31. What is the difference between a CNAME and an ALIAS record in Route 53?

A CNAME points a name at another name and cannot be used at the zone apex. An ALIAS is an AWS extension that behaves like a CNAME but is resolved server-side, works at the apex, can point at AWS resources such as ALBs and CloudFront distributions, and is not charged for queries.

Q. 32. How would you do a blue-green deployment on AWS?

Two target groups behind one ALB. Deploy green, health-check and smoke-test it directly through a test listener, then switch the production listener's default action to the green target group. Rollback is switching back. Alternatively, weighted target groups let you shift 10% first, which turns it into a canary. At the DNS level, weighted Route 53 records do the same thing more slowly and with the TTL caveat.

Q. 33. What is the difference between horizontal scaling of the app tier and the database?

The application tier is usually stateless, so adding instances behind a load balancer is straightforward. A relational database is stateful: you scale reads with read replicas and scale writes vertically or by sharding, which is a significant architectural change. That asymmetry is why the database is normally the first bottleneck.

Q. 34. What is RDS Multi-AZ and how is it different from a read replica?

Multi-AZ maintains a synchronous standby in another AZ purely for availability - you cannot read from it, and failover is automatic via a DNS change. A read replica is asynchronous and exists to serve read traffic, and it can be promoted manually. Multi-AZ is for availability; read replicas are for scale.

Q. 35. How do you store database credentials for an application on EC2?

In Secrets Manager or SSM Parameter Store as a SecureString, retrieved at runtime using the instance's IAM role, with the KMS key permission scoped to that role. Secrets Manager adds managed rotation. What I would not do is put them in user data, environment variables baked into an AML, or a file in the repository.

Q. 36. What is a VPC endpoint and why use one?

A private connection to an AWS service that keeps traffic on the AWS network. Gateway endpoints for S3 and DynamoDB are free and work through route tables; interface endpoints create an ENI in your subnet for other services. They improve security by removing the need for internet egress, and they cut NAT gateway data-processing costs.

Q. 37. Two VPCs are peered but cannot communicate. What do you check?

Route tables on both sides must have routes to the other VPC's CIDR - peering does not create routes automatically. Security groups and NACLs on both sides. Overlapping CIDRs, which make peering impossible. And remember peering is not transitive: A-B and B-C does not give A-C.

Q. 38. How do you reduce AWS cost without reducing reliability?

Right-size instances based on actual CloudWatch metrics rather than guesses; use Savings Plans or Reserved Instances for steady baseline load and Spot for interruptible work such as CI agents; S3 lifecycle policies and expiring noncurrent versions; delete unattached EBS volumes and old snapshots; release unused Elastic IPs; consolidate NAT gateways carefully while keeping AZ redundancy in mind; use S3 gateway endpoints to avoid NAT data charges; and set budgets and anomaly detection so surprises are caught early.

Q. 39. What is the AWS shared responsibility model?

AWS is responsible for security OF the cloud - the hardware, the hypervisor, the physical facilities and managed service infrastructure. The customer is responsible for security IN the cloud - IAM, security groups, patching guest operating systems, encryption choices, and application security. The boundary moves with the service: for EC2 you patch the OS, for RDS AWS does, but you still own access control.

Q. 40. How would you handle secrets in a CI/CD pipeline deploying to AWS?

Ideally no stored secrets at all: GitHub Actions or Jenkins assumes an IAM role via OIDC, receiving short-lived credentials. Where that is not possible, an access key scoped tightly to the deployment actions, stored in the CI credential store, rotated regularly, with CloudTrail monitoring its use.

Q. 41. What is an Elastic IP and when should you avoid it?

A static public IPv4 allocated to your account. You avoid it when a DNS name would do, because Elastic IPs are a limited resource, you are charged for unattached ones, and pinning to an IP encourages designs that break when instances are replaced. Legitimate uses are NAT gateways and whitelisting requirements where a partner needs a fixed source IP.

Q. 42. How do you patch a fleet of EC2 instances?

Preferably by replacement - bake a new AMI with the patches and roll the ASG, so instances are immutable. Where in-place patching is necessary, Systems Manager Patch Manager with maintenance windows and patch baselines gives controlled, auditable patching without SSH access.

Q. 43. What is Systems Manager Session Manager and why is it better than SSH?

It gives shell access through the SSM agent and the AWS API, so there is no open inbound port 22, no bastion host and no SSH keys to manage, and every session is logged to CloudTrail and optionally recorded to S3. It removes an entire attack surface, which is why it is now the recommended access pattern.

Q. 44. How would you design for disaster recovery?

Start with the business requirements - RTO, how quickly you must recover, and RPO, how much data you can afford to lose. Then choose the pattern: backup and restore for the cheapest and slowest, pilot light with a minimal standby, warm standby with a scaled-down running copy, or active-active multi-region for near-zero RTO. The critical part is testing the failover regularly, because an untested DR plan does not work.

Q. 45. What is the difference between a snapshot and an AMI?

A snapshot is a point-in-time backup of a single EBS volume. An AMI is a launchable template that references one or more snapshots plus the metadata needed to boot - block device mapping, architecture, virtualisation type.

Q. 46. Your EC2 instance is running but the website is down. Walk me through it.

First, is it the instance or the path to it? I check the ALB target group health - if the target is unhealthy, the ALB is doing its job and the application is the problem. On the instance, `systemctl status` for the service, `ss -tlnp` to confirm it is listening and on which address, then application logs. If the service is fine, I work outward: security group, target group port, listener rule, DNS. In parallel I check the boring causes - `df -h` for a full disk and `free -h` for memory, because both silently break services. And I ask what changed in the last hour.

Q. 47. What is the difference between Application Auto Scaling and EC2 Auto Scaling?

EC2 Auto Scaling manages the number of EC2 instances in an ASG. Application Auto Scaling is the generic service that scales other resources - ECS services, DynamoDB capacity, Aurora replicas - using the same target-tracking concepts.

Q. 48. What is an SCP and how does it interact with IAM?

A Service Control Policy applied at the AWS Organizations level, acting as a permissions ceiling for every principal in an account, including the root user. It can only restrict, never grant. So if an SCP denies an action, the most permissive IAM policy in the world will not allow it - and that is exactly the confusion behind many AccessDenied tickets.

Q. 49. How do you monitor and alert on AWS costs?

AWS Budgets with alerts at percentage thresholds, Cost Explorer for trend analysis, Cost Anomaly Detection for unexpected spikes, and consistent resource tagging so cost can be attributed to a team or environment. Tagging discipline is the part that usually fails, and without it cost allocation is guesswork.

Q. 50. What is the difference between SNS and SQS?

SNS is publish-subscribe: one message is pushed to many subscribers. SQS is a queue: messages are pulled by consumers and each message is processed once. A common pattern is SNS fanning out to several SQS queues so each consumer gets its own durable buffer. SQS also gives you retries and a dead-letter queue, which is why it is used to decouple components.

Q. 51. What happens if you delete a NAT gateway?

Every private subnet whose route table pointed at it loses outbound internet access immediately - package installs, API calls and image pulls start failing with timeouts. Since a NAT gateway is also charged hourly, people do delete them to save money and then cannot work out why the application "randomly" cannot reach external services.

Q. 52. How do you give a pod or an application on EC2 access to S3 without keys?

An IAM role - an instance profile for EC2, or IAM Roles for Service Accounts on EKS. The SDK automatically retrieves temporary credentials from the metadata service or the token file, so there is nothing to store or rotate.

Q. 53. What is CloudFormation and how does it compare with Terraform?

CloudFormation is AWS's native IaC service using JSON or YAML templates, with managed state, drift detection and rollback on failure, but it only manages AWS. Terraform is provider-agnostic with its own state file and a strong module ecosystem, and it can manage AWS alongside other providers in one workflow. I use Terraform, and the main practical difference is that I own the state file and its locking, whereas CloudFormation manages that for me.

Q. 54. How would you enforce tagging?

Tag policies in Organizations, SCPs that deny resource creation without required tags, Config rules that flag non-compliant resources, and enforcing tags in the Terraform modules themselves with `default_tags` on the provider so nothing is created untagged in the first place. Enforcement in the module is the one that actually works, because it is the path of least resistance.

Q. 55. What is the difference between vertical and horizontal scaling of RDS?

Vertical is changing the instance class, which requires a brief failover or downtime. Horizontal for reads is adding read replicas and directing read traffic at them, which requires the application to distinguish reads from writes. Writes cannot be scaled horizontally without sharding or moving to a distributed engine such as Aurora.

Q. 56. Your S3 upload fails with AccessDenied but the IAM policy looks correct. What else?

The bucket policy may explicitly deny. Block Public Access may be interfering if you are trying to set an ACL. The bucket may require server-side encryption with a specific header, and a Deny condition rejects uploads without it. If it is a KMS-encrypted bucket, the principal also needs `kms:GenerateDataKey`. And check that the resource ARN includes `/*` for object-level actions. Finally, confirm which identity is actually being used with `aws sts get-caller-identity`.

Q. 57. What are CloudWatch Logs Insights used for?

Querying log groups with a purpose-built query language - filtering, parsing fields, aggregating and sorting - without exporting to another system. During an incident it lets me answer "how many 500s by endpoint in the last hour?" in seconds, which is otherwise a grep across many instances.

Q. 58. How do you handle an AWS service limit being hit?

Recognise the symptom - errors mentioning limit exceeded or throttling. Check Service Quotas for the current limit and usage, request an increase, and in the meantime work around it. The real lesson is proactive: monitor quota usage for things that grow, such as Elastic IPs, VPCs and instances per region, because hitting a limit mid-incident is a bad time to discover it.

Q. 59. What is the purpose of a bastion host and what replaces it today?

A hardened jump box in a public subnet as the only SSH entry point to private instances. Today it is largely replaced by SSM Session Manager, which removes the open port, the key management and the instance to patch, while adding full session logging.

Q. 60. If you could make one change to improve the security of a typical AWS account, what would it be?

Eliminate long-lived IAM access keys - move workloads to roles and human access to federated SSO with MFA - because leaked keys are the most common root cause of real AWS breaches. Close behind that: enable CloudTrail in all regions with log file validation, and turn on Block Public Access for S3 at the account level, because those two cover the next most common failure modes.

PART 8 RECALL POINTS

- A subnet is public because its route table points 0.0.0.0/0 at an IGW.
- AWS reserves 5 IPs per subnet.
- Security groups stateful and can reference other SGs; NACLs stateless.
- Roles, not access keys - IMDSv2 to protect them.
- Explicit Deny wins; SCPs are a ceiling.
- `arn:...:bucket` versus `arn:...:bucket/*` - the classic S3 policy bug.
- CloudWatch cannot see memory or disk usage without the agent.
- ALIAS at the apex, not CNAME.
- ASG health check type should be ELB, not EC2.
- Instance store is ephemeral; EBS persists.

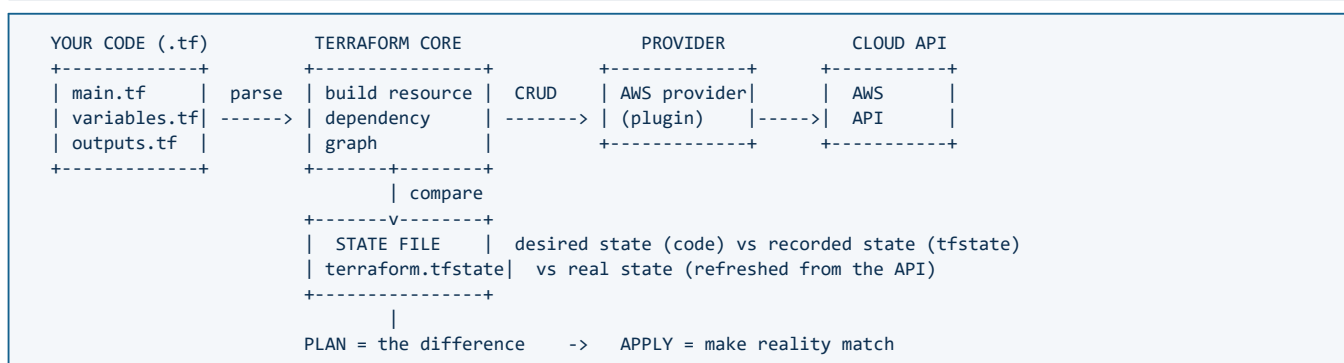
PART 9

TERRAFORM COMPLETE GUIDE

HCL, state, modules, workspaces, CI/CD, drift, 40 questions

Your CV claims Terraform, Terraform modules and infrastructure provisioning in both roles, plus a named project: "Built reusable Terraform modules to provision AWS infrastructure, enabling repeatable environment creation and reducing manual configuration." Expect the interviewer to test state, modules and the plan/apply lifecycle hardest - those are where real experience shows.

9.1 Terraform Architecture



How an apply actually works

Concept	Definition
Provider	A plugin that knows how to talk to a platform's API - aws, azure, kubernetes, github
Resource	An infrastructure object Terraform creates and manages
Data source	A read-only lookup of something Terraform does NOT manage
Variable	Input to a configuration or module
Local	A named expression, computed once, used for readability
Output	A value exported from a configuration or module
Module	A reusable, parameterised group of resources
State	Terraform's record of which real resources correspond to which configuration addresses
Backend	Where state is stored - local, S3, Terraform Cloud
Provisioner	A last-resort escape hatch to run scripts - avoid

9.2 HCL - The Building Blocks

providers.tf - pin everything

```

terraform {
  required_version = ">= 1.6.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.40"      # allows 5.x, blocks 6.0 - pessimistic constraint
    }
  }

  backend "s3" {
    bucket        = "acme-terraform-state"
    key           = "prod/network/terraform.tfstate"
    region        = "eu-west-1"
    dynamodb_table = "terraform-locks"
    encrypt       = true
  }
}
  
```

```

    }
  }

  provider "aws" {
    region = var.region

    default_tags {
      tags = {
        Environment = var.environment
        ManagedBy   = "terraform"
        Project      = var.project
      }
    }
  }
}

```

variables.tf - typed, validated, documented

```

variable "region" {
  description = "AWS region to deploy into"
  type        = string
  default     = "eu-west-1"
}

variable "environment" {
  description = "Environment name"
  type        = string

  validation {
    condition     = contains(["dev", "staging", "prod"], var.environment)
    error_message = "environment must be one of dev, staging, prod."
  }
}

variable "instance_count" {
  description = "Number of application instances"
  type        = number
  default     = 2
}

variable "subnet_cidrs" {
  description = "CIDR blocks for the application subnets"
  type        = list(string)
  default     = ["10.0.11.0/24", "10.0.12.0/24"]
}

variable "instance_config" {
  description = "Per-environment instance settings"
  type = object({
    type        = string
    volume_size = number
    monitoring   = bool
  })
  default = {
    type        = "t3.micro"
    volume_size = 20
    monitoring   = false
  }
}

variable "db_password" {
  description = "Database master password"
  type        = string
  sensitive   = true          # redacted from plan output and CLI display
}

```

main.tf - resources, data sources, locals, meta-arguments

```

locals {
  name_prefix = "${var.project}-${var.environment}"

  common_tags = {
    Name = local.name_prefix
  }
}

```

```

}

# Data source: look up something we do NOT manage
data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]

  filter {
    name   = "name"
    values = ["al2023-ami-*-x86_64"]
  }
}

data "aws_availability_zones" "available" {
  state = "available"
}

resource "aws_vpc" "main" {
  cidr_block      = var.vpc_cidr
  enable_dns_hostnames = true
  enable_dns_support = true

  tags = merge(local.common_tags, { Name = "${local.name_prefix}-vpc" })
}

# for_each over a map: stable addressing by key
resource "aws_subnet" "private" {
  for_each = {
    for idx, cidr in var.subnet_cidrs :
      data.aws_availability_zones.available.names[idx] => cidr
  }

  vpc_id            = aws_vpc.main.id
  cidr_block        = each.value
  availability_zone = each.key

  tags = merge(local.common_tags, { Name = "${local.name_prefix}-private-${each.key}" })
}

resource "aws_security_group" "app" {
  name_prefix = "${local.name_prefix}-app-"
  description = "Application tier"
  vpc_id      = aws_vpc.main.id

  dynamic "ingress" {
    for_each = var.allowed_ports
    content {
      from_port = ingress.value
      to_port   = ingress.value
      protocol  = "tcp"
      security_groups = [aws_security_group.alb.id]
    }
  }

  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  lifecycle {
    create_before_destroy = true # avoids "resource in use" when replacing
  }
}

resource "aws_instance" "app" {
  count = var.instance_count # count: simple numeric repetition

  ami           = data.aws_ami.amazon_linux.id
  instance_type = var.instance_config.type
  subnet_id     = values(aws_subnet.private)[count.index % length(aws_subnet.private)].id
}

```

```

vpc_security_group_ids = [aws_security_group.app.id]
monitoring              = var.instance_config.monitoring

root_block_device {
  volume_size = var.instance_config.volume_size
  volume_type = "gp3"
  encrypted   = true
}

user_data = templatefile("${path.module}/templates/userdata.sh.tftpl", {
  environment = var.environment
  app_version = var.app_version
})

tags = merge(local.common_tags, { Name = "${local.name_prefix}-app-${count.index + 1}" })

lifecycle {
  ignore_changes = [ami]      # do not replace instances just because a new AMI was published
}

depends_on = [aws_nat_gateway.main] # explicit dependency where it is not implied
}

```

outputs.tf

```

output "vpc_id" {
  description = "ID of the created VPC"
  value       = aws_vpc.main.id
}

output "private_subnet_ids" {
  value = [for s in aws_subnet.private : s.id]
}

output "app_private_ips" {
  value = aws_instance.app[*].private_ip # splat expression
}

output "db_endpoint" {
  value       = aws_db_instance.main.endpoint
  sensitive   = true
}

```

9.2.1 count vs for_each - a frequent question

	count	for_each
Input	A number	A map or a set of strings
Address	aws_instance.app[0]	aws_instance.app["web-a"]
Removing an item	Re-indexes everything after it - Terraform destroys and recreates unrelated resources	Only the removed key is affected
Best for	Truly identical, interchangeable copies; conditional creation with count = var.enabled ? 1 : 0	Anything with a natural identity - subnets per AZ, users, buckets

THE COUNT RE-INDEX TRAP

With `count`, removing the first element of a list of three shifts index 1 to 0 and 2 to 1, so Terraform plans to destroy and recreate resources that did not actually change. With `for_each`, each resource is keyed by a stable name, so removing one touches only that one. Prefer `for_each` for anything with identity. Interviewers use this to separate people who have read the docs from people who have been burned.

9.3 The Command Workflow

The lifecycle, in the order you will use it

```

terraform init          # download providers, configure the backend, install modules
terraform init -upgrade  # update providers within their constraints
terraform init -reconfigure # change backend without migrating state
terraform init -migrate-state # move existing state to a new backend

```

```

terraform fmt -recursive      # canonical formatting - run it in CI as a check
terraform validate            # syntax and internal consistency (no API calls)

terraform plan                # show what WOULD change
terraform plan -out=tfplan    # save the plan - what CI should apply
terraform plan -var-file=prod.tfvars
terraform plan -target=aws_instance.app    # limit scope - emergency use only
terraform plan -destroy      # preview a destroy

terraform apply tfplan        # apply the SAVED plan - no re-evaluation, no surprises
terraform apply -auto-approve # CI only, and only after a reviewed plan
terraform destroy             # tear everything down

terraform output              # all outputs
terraform output -raw vpc_id  # unquoted, for scripting
terraform output -json

terraform show                # human-readable current state
terraform show -json tfplan   # machine-readable plan, for policy checks
terraform graph | dot -Tpng > graph.png

```

State commands - handle with care

```

terraform state list          # every managed resource address
terraform state show aws_instance.app[0]    # attributes of one resource
terraform state mv aws_instance.web aws_instance.app    # rename without destroying
terraform state rm aws_instance.legacy      # stop managing it (does NOT delete the real resource)
terraform state pull > backup.tfstate       # always back up before surgery
terraform state push backup.tfstate         # last resort

terraform import aws_instance.app i-0abc123def # adopt an existing resource into state
terraform refresh                # deprecated; use: terraform apply -refresh-only

terraform force-unlock <LOCK_ID>           # ONLY when you are certain no apply is running
terraform taint aws_instance.app[0]        # legacy; modern: terraform apply -replace=...
terraform apply -replace="aws_instance.app[0]" # force recreation of one resource

```

9.4 Terraform State - Explained Deeply

State is a JSON file mapping each configuration address (`aws_instance.app[0]`) to the real resource (`i-0abc123`), along with the attribute values Terraform last saw. Without it, Terraform cannot tell the difference between "create this" and "this already exists".

State does	Why it is needed
Maps config addresses to real resource IDs	Terraform has no other way to know which instance is <code>aws_instance.app[0]</code>
Stores attribute values	Enables diffing without querying every attribute
Tracks dependencies	Correct destroy ordering even after the code is deleted
Caches metadata	Performance on large configurations
Holds outputs	So other configurations can consume them via <code>terraform_remote_state</code>

STATE IS A SECRET

The state file contains every attribute of every resource in plaintext - including database passwords, private keys and any variable marked sensitive. `sensitive = true` only hides it from CLI output, not from state. Therefore: encrypt the backend, restrict access with IAM, never commit `.tfstate` to Git, and add `*.tfstate*` to `.gitignore`. This is one of the most important things you can say about Terraform in an interview.

9.4.1 Remote state and locking

The standard AWS backend

```

terraform {
  backend "s3" {
    bucket = "acme-terraform-state"
    key    = "prod/app/terraform.tfstate" # one key per environment/component
    region = "eu-west-1"
  }
}

```

```

    encrypt      = true
    dynamodb_table = "terraform-locks"          # state locking
  }
}

```

Backend feature	Why
S3 with versioning	Durable, and you can roll back to a previous state version after a bad apply
encrypt = true plus SSE-KMS	State contains secrets
DynamoDB table for locking	Prevents two engineers or two pipeline runs applying at once and corrupting state
Separate key per environment	Blast radius - a mistake in dev cannot destroy prod
Bucket policy denying delete	Protects against accidental state loss

WHY LOCKING MATTERS

Two simultaneous applies both read the same state, both make changes, and the second write overwrites the first - so Terraform's record no longer matches reality, and the next plan proposes to destroy resources it has "lost". DynamoDB-based locking makes the second apply wait. If a process crashes mid-apply the lock persists, which is what `force-unlock` is for - and you only use it after confirming nothing is actually running.

9.4.2 Drift

Drift is when reality no longer matches state - somebody changed a security group in the console, or an autoscaling process altered something.

Detecting and handling drift

```

terraform plan                # any unexpected change is drift
terraform plan -refresh-only   # show ONLY what changed outside Terraform
terraform apply -refresh-only  # accept reality into state without changing infrastructure
# Then decide: codify the change, or re-apply to revert reality back to the code.

```

THE DRIFT ANSWER INTERVIEWERS WANT

"Drift happens when someone changes infrastructure outside Terraform. I detect it by running `terraform plan` on a schedule in CI and alerting if the plan is non-empty. Then it is a decision, not an automatic action: if the manual change was correct, I codify it and use `-refresh-only` to reconcile; if it was not, I re-apply to bring reality back to the code. The real fix is preventing it - remove or restrict console write access in production, so Terraform is the only path."

9.5 Modules

Standard module layout

```

modules/
  vpc/
    main.tf          # the resources
    variables.tf      # inputs
    outputs.tf        # exported values
    versions.tf       # provider constraints
    README.md         # what it does, inputs, outputs, example
  environments/
    dev/
      main.tf
      terraform.tfvars
      backend.tf
    prod/
      main.tf
      terraform.tfvars
      backend.tf

```

modules/vpc/main.tf - a reusable module

```

variable "name"      { type = string }
variable "cidr_block" { type = string }
variable "az_count"   { type = number, default = 2 }
variable "enable_nat" { type = bool,   default = true }
variable "tags"       { type = map(string), default = {} }

```

```

data "aws_availability_zones" "this" { state = "available" }

locals {
  azs = slice(data.aws_availability_zones.this.names, 0, var.az_count)
}

resource "aws_vpc" "this" {
  cidr_block      = var.cidr_block
  enable_dns_hostnames = true
  tags            = merge(var.tags, { Name = var.name })
}

resource "aws_subnet" "public" {
  for_each      = toset(local.azs)
  vpc_id        = aws_vpc.this.id
  cidr_block    = cidrsubnet(var.cidr_block, 8, index(local.azs, each.key))
  availability_zone = each.key
  map_public_ip_on_launch = true
  tags          = merge(var.tags, { Name = "${var.name}-public-${each.key}", Tier = "public" })
}

output "vpc_id" { value = aws_vpc.this.id }
output "public_subnet_ids" { value = [for s in aws_subnet.public : s.id] }

```

environments/prod/main.tf - consuming the module

```

module "network" {
  source = "../../modules/vpc"          # local path
  # source = "git::https://github.com/acme/tf-modules.git//vpc?ref=v1.4.0" # pinned git
  # source = "terraform-aws-modules/vpc/aws"                               # registry
  # version = "5.5.1"

  name      = "acme-prod"
  cidr_block = "10.0.0.0/16"
  az_count  = 3
  tags      = { Environment = "prod" }
}

module "app" {
  source = "../../modules/app-tier"

  name            = "acme-prod-app"
  vpc_id          = module.network.vpc_id      # module outputs wire modules together
  subnet_ids      = module.network.private_subnet_ids
  instance_count  = 6
  instance_type   = "m5.large"
}

```

Module practice	Reason
Pin the source version (?ref=v1.4.0)	An unpinned module can change under you and alter every environment
No hardcoded values inside a module	That is what makes it reusable
No provider block inside a module	Providers are configured by the root and inherited; declaring them in a module breaks composition
Outputs for everything a caller might need	Otherwise callers reach into internals or duplicate lookups
README with inputs, outputs and an example	Modules without docs do not get reused
Keep modules small and single-purpose	A module that creates "everything" is just a configuration with extra steps

"TELL ME ABOUT THE TERRAFORM MODULES YOU BUILT"

Tie it to your CV wording and keep it mechanical: "I built reusable modules so that creating an environment was supplying variable values rather than writing new resource blocks. The module takes inputs like name, CIDR, AZ count and instance sizing, creates the resources consistently with tagging applied centrally, and exports the IDs that other modules need - so the app module consumes the network module's outputs rather than hardcoding subnet IDs. The point was repeatability: the same code produces dev and production with different variables, so environments do not drift apart, and a change to the pattern is made once in the module rather than in every environment."

9.6 Workspaces and Environment Separation

Workspaces

```
terraform workspace list
terraform workspace new staging
terraform workspace select prod
terraform workspace show
# In code: terraform.workspace -> "prod"
```

Approach	Pros	Cons
Workspaces	One codebase, one backend key prefix, quick to switch	Same code for every environment; easy to apply to the wrong one - <code>terraform workspace show</code> is not a safety mechanism
Directory per environment	Explicit separation, separate state keys, per-environment variables and backends, different code where needed	Some duplication
Terragrunt / wrapper	DRY plus explicit separation	Another tool to learn

THE SAFE ANSWER

"Workspaces are fine for lightweight variations, but for dev versus production I prefer separate directories with separate state keys and separate backends. The reason is blast radius and human error: with workspaces, a forgotten `workspace select` applies dev changes to production, and nothing about the command you type tells you which one you are on."

9.7 Terraform in CI/CD

```

PULL REQUEST                                MERGE TO MAIN
|                                           |
| terraform fmt -check                    fails on formatting |
| terraform init                         |                     |
| terraform validate                     |                     |
| tfsec / checkov                        policy + security gate |
| terraform plan -out=tfplan              |                     |
|                                         |                     |
| post the plan as a PR comment <- HUMAN REVIEWS THIS |
|                                         |                     |
| approve + merge -----> terraform apply tfplan
|                                         |                     |
|                                         (prod: manual approval gate first)
```

The standard pipeline

GitHub Actions - plan on PR, apply on merge

```
name: terraform
on:
  pull_request:
    paths: ["infra/**"]
  push:
    branches: [main]
    paths: ["infra/**"]

permissions:
  id-token: write      # OIDC - no stored AWS keys
  contents: read
  pull-requests: write

jobs:
  terraform:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: infra/prod
    steps:
      - uses: actions/checkout@v4

      - uses: aws-actions/configure-aws-credentials@v4
        with:
```

```

    role-to-assume: arn:aws:iam::123456789012:role/github-terraform
    aws-region: eu-west-1

- uses: hashicorp/setup-terraform@v3
  with:
    terraform_version: 1.7.5

- run: terraform fmt -check -recursive
- run: terraform init
- run: terraform validate

- name: Plan
  id: plan
  run: terraform plan -out=tfplan -no-color

- name: Apply
  if: github.ref == 'refs/heads/main' && github.event_name == 'push'
  run: terraform apply -auto-approve tfplan

```

WHY YOU APPLY A SAVED PLAN

terraform apply with no plan file re-evaluates everything at apply time, so what it does may differ from the plan a human reviewed minutes earlier. terraform plan -out=tfplan followed by terraform apply tfplan guarantees that exactly what was reviewed is what runs. In an interview this is the answer that shows you have run Terraform in a pipeline rather than only on a laptop.

9.8 What Actually Happens During terraform apply

Step	What Terraform does
1. Load	Parses all .tf files in the directory into a configuration
2. Init check	Verifies providers and modules are installed and the backend is configured
3. Lock	Acquires the state lock (DynamoDB) so no one else can apply concurrently
4. Read state	Downloads the current state from the backend
5. Refresh	Queries the provider API for the real current attributes of managed resources
6. Build graph	Constructs a dependency graph from implicit references and depends_on
7. Diff	Compares desired (code) with actual (refreshed state) to produce the plan
8. Execute	Walks the graph, creating, updating or destroying, in parallel where dependencies allow (10 by default)
9. Write state	Updates the state file after each resource operation
10. Unlock	Releases the lock and prints outputs

Plan symbol	Meaning
+	create
-	destroy
~	update in place
-/+	destroy and recreate - forced by an immutable attribute change
<=	read (data source)

ALWAYS READ THE PLAN FOR -/+

An in-place update is usually safe; a destroy-and-recreate on a database, an EBS volume or an Elastic IP is an outage. The plan tells you why: "forces replacement" appears next to the attribute responsible. Anyone who runs terraform apply -auto-approve on production without reading the plan will eventually delete something important - and interviewers ask this specifically.

9.9 Terraform Interview Questions

Q. 1. What is Terraform and why use it?

An infrastructure-as-code tool that provisions infrastructure declaratively across providers. You describe the desired end state and Terraform works out the actions. The value is reproducibility - the same code creates identical environments - reviewability, because changes go through pull requests, and safety, because `plan` shows exactly what will change before anything happens.

Q. 2. Terraform versus CloudFormation?

Terraform is provider-agnostic and can manage AWS alongside Kubernetes, GitHub and others in one workflow, with a strong module ecosystem; you own and manage the state file. CloudFormation is AWS-native, with AWS-managed state, built-in rollback on failure and drift detection, but it only manages AWS. I use Terraform primarily because of the module reuse and because not everything we manage is AWS.

Q. 3. Terraform versus Ansible?

Terraform provisions infrastructure - it creates the VPC, the instances, the load balancer - and tracks them in state so it knows what it owns. Ansible configures existing systems - it installs packages, renders config files and manages services - with no state file, checking reality per task each run. The usual pattern is Terraform builds the house, Ansible furnishes it.

Q. 4. What is the state file and why is it essential?

A JSON record mapping configuration addresses to real resource IDs, along with the attributes last observed. Without it Terraform cannot tell whether a resource in the code already exists, so it would try to create duplicates and could never update or destroy correctly. It also stores dependency information used for correct destroy ordering.

Q. 5. Why store state remotely?

So a team shares one source of truth, so it survives a lost laptop, so it can be encrypted and access-controlled, and - most importantly - so it can be locked. Local state means two engineers can apply simultaneously and corrupt each other's work.

Q. 6. How does state locking work in the S3 backend?

A DynamoDB table holds a lock item keyed on the state path. Terraform writes the lock before applying and removes it afterwards; a second apply sees the lock and waits or fails. If a run crashes, the lock stays and must be cleared with `terraform force-unlock` - but only after confirming nothing is genuinely running.

Q. 7. Is it safe to put the state file in Git?

No. It contains every resource attribute in plaintext, including passwords and keys, so committing it leaks secrets. It also breaks concurrency - Git has no locking semantics for this - and merge conflicts in state are effectively unrecoverable. State belongs in an encrypted, versioned, locked backend.

Q. 8. What does `terraform init` do?

Initialises the working directory: downloads the required provider plugins, installs modules, configures the backend and creates the dependency lock file. It must be run before `plan` or `apply`, and re-run when providers, modules or the backend change.

Q. 9. What is the difference between `terraform plan` and `terraform apply`?

`plan` computes and displays the difference between the desired state in code and the actual state, changing nothing. `apply` executes those changes. In CI I always save the plan with `-out` and apply that exact file, so what was reviewed is what runs.

Q. 10. What does `terraform refresh` do and why is it deprecated?

It updated state from the real infrastructure without changing anything. It was deprecated as a standalone command because it silently rewrote state; the explicit replacement is `terraform apply -refresh-only`, which shows you what changed outside Terraform and asks you to approve reconciling it.

Q. 11. What is drift and how do you handle it?

Reality diverging from state because something was changed outside Terraform. I detect it with a scheduled `terraform plan` in CI that alerts on a non-empty plan. Then I decide: codify a legitimate change, or re-apply to revert an illegitimate one. Prevention is better - restrict console write access in production so Terraform is the only path.

Q. 12. What is `terraform import` for?

Bringing an existing resource that was created manually under Terraform management. You write the matching configuration, then import it into state by ID. It does not generate the configuration for you - though newer versions support import blocks and config generation - and the workflow is: import, then plan, then adjust the code until the plan is empty.

Q. 13. How do you move a resource without destroying it?

`terraform state mv old.address new.address`, which updates the state mapping only. That is how you rename a resource or move resources into a module without Terraform destroying and recreating them. Newer Terraform has `moved` blocks, which do the same thing declaratively and reviewably in code - which is safer because it goes through a pull request.

Q. 14. What does `terraform state rm` do?

Removes a resource from state so Terraform stops managing it - the real resource is untouched. It is used when handing ownership to another configuration or team. The danger is forgetting that it is now unmanaged and orphaned.

Q. 15. Difference between `count` and `for_each`?

`count` takes a number and addresses resources by index; `for_each` takes a map or set and addresses by key. The practical difference is that removing an item from a `count` list re-indexes everything after it, causing Terraform to destroy and recreate unrelated resources, whereas `for_each` keys are stable. I use `for_each` for anything with identity and `count` mainly for conditional creation.

Q. 16. How do you create a resource conditionally?

`count = var.enabled ? 1 : 0`, then reference it as `resource.name[0]`, usually guarded with `one()` or a `try()` so references do not explode when it is absent. For modules, the same pattern with `count` on the module block.

Q. 17. What is a data source?

A read-only lookup of something Terraform does not manage - an existing VPC, the latest AMI, the current account ID. It lets you reference existing infrastructure without importing it, which is how you compose with resources owned by other teams.

Q. 18. What is the difference between a variable and a local?

A variable is an input supplied from outside - `tfvars`, CLI, environment. A local is a named expression computed inside the configuration for readability and to avoid repetition. You cannot override a local from outside, which is the point.

Q. 19. How are variables supplied, and what is the precedence?

Defaults in the declaration, then `terraform.tfvars` and `*.auto.tfvars`, then `TF_VAR_name` environment variables, then `-var-file`, then `-var` on the command line - with later sources winning. In CI I pass a `-var-file` per environment and inject secrets as environment variables.

Q. 20. How do you handle secrets in Terraform?

Never in the code or in a committed `tfvars` file. Mark variables `sensitive` so they are redacted from output, source them from environment variables or a secret store, and prefer having Terraform reference a secret rather than set it - for example creating an RDS instance with a password generated and stored in Secrets Manager. And remember that regardless of all that, the value lands in state, so the state backend must be encrypted and access-controlled.

Q. 21. What are provisioners and why avoid them?

`local-exec` and `remote-exec` run scripts as part of resource creation. They are discouraged because they are imperative, they break Terraform's declarative model, they only run at create time so they do not converge, and a failure leaves the resource tainted. The recommended alternatives are user data or cloud-init for bootstrapping and a configuration management tool such as Ansible for ongoing configuration.

Q. 22. What is the dependency graph and how does Terraform build it?

Terraform infers dependencies from references - if resource B uses `aws_vpc.main.id`, B depends on the VPC - and builds a directed acyclic graph, then executes it with parallelism where possible. `depends_on` adds an explicit edge for dependencies that are not expressed through a reference, such as an IAM policy that must exist before an instance can use a role.

Q. 23. What does `lifecycle { create_before_destroy = true }` do?

It makes Terraform create the replacement before destroying the original, which avoids downtime and avoids "resource in use" errors - for example when replacing a security group that is attached to instances. It requires that names do not collide, which is why you use `name_prefix` instead of `name`.

Q. 24. What does `ignore_changes` do and when is it appropriate?

It tells Terraform to ignore drift on specific attributes. It is appropriate when something legitimately changes outside Terraform - an autoscaling desired count, or tags added by another system, or an AMI you do not want to force a replacement for. It is inappropriate as a way to silence a plan you do not understand.

Q. 25. What is `prevent_destroy`?

A lifecycle setting that makes Terraform error rather than destroy a resource. I put it on databases and state buckets. It is a guard rail against a bad plan or a mistaken `terraform destroy`, and it forces a deliberate code change to remove.

Q. 26. How do you structure a Terraform repository?

Reusable modules in `modules/`, one directory per environment under `environments/` with its own backend key and tfvars, shared conventions enforced in the modules - tagging through `default_tags`, naming through a prefix variable. Small state files per component rather than one enormous state, because a huge state is slow, risky and has a large blast radius.

Q. 27. Why split state into multiple files?

Blast radius, speed and ownership. A single state for everything means every plan refreshes every resource, every apply risks everything, and every team contends for one lock. Splitting by environment and by component - network, data, application - keeps applies fast and failures contained. The components then reference each other through data sources or remote state outputs.

Q. 28. What is `terraform_remote_state` used for?

Reading outputs from another configuration's state, so the application configuration can consume the VPC ID produced by the network configuration without hardcoding it. The alternative, which I prefer where possible, is looking the resource up with a data source by tag, because it avoids coupling to another team's state file layout.

Q. 29. What happens if two people run apply at the same time?

With locking, the second waits or fails with a lock error. Without locking, both read the same state, both write, and the last write wins - so state no longer reflects reality, and the next plan may propose destroying resources it has lost track of. That is exactly why the DynamoDB lock table is not optional.

Q. 30. Your apply failed halfway. What is the state?

Terraform writes state incrementally, so resources created before the failure ARE in state; nothing is rolled back - Terraform has no transactions. The recovery is to fix the cause and run apply again; it will create only what is missing. If a resource was created but not recorded because of a crash, you may need to import it or delete it manually to unstick the plan.

Q. 31. How do you force recreation of a single resource?

`terraform apply -replace="aws_instance.app[0]"`. The older mechanism was `terraform taint`, which is now deprecated because `-replace` is explicit and visible in the plan rather than being hidden state.

Q. 32. What is `-target` and why is it discouraged?

It restricts an operation to specific resources. It is discouraged because it skips the dependency graph, so you can leave infrastructure in an inconsistent state where the code and reality only partially agree. It is legitimate as an emergency tool to recover from a broken apply, not as normal workflow.

Q. 33. How do you version modules?

Git tags with a semantic version, referenced as `?ref=v1.4.0`, or registry versions with a version constraint. Pinning matters because an unpinned module reference means someone else's commit can change every environment that consumes it the next time you run `init`.

Q. 34. What is the dependency lock file?

`.terraform.lock.hcl` records the exact provider versions and their checksums selected during `init`. It should be committed so every engineer and the CI pipeline use identical provider versions - without it, two people can generate different plans from the same code.

Q. 35. How do you test Terraform code?

Static checks first - `fmt`, `-check`, `validate`, and policy scanning with `tfsec` or `Checkov` in CI. Then the plan itself as a review artifact posted to the pull request. For modules, integration testing with `Terratest` or the native `terraform test` framework, which provisions into a sandbox account, asserts, and destroys. Plus a scheduled plan against production to catch drift.

Q. 36. How do you handle multiple AWS regions or accounts?

Provider aliases - multiple `provider "aws"` blocks with `alias` and different regions or assumed roles - and then `provider = aws.eu` on the resources that belong to the second region. For multiple accounts, `assume_role` in the provider configuration, with the pipeline's role trusted by each target account.

Q. 37. What is the difference between `terraform destroy` and removing the code?

`destroy` deletes the real resources and clears state. Deleting the code and applying also destroys them, because Terraform sees resources in state that are no longer in the configuration. Neither is reversible, which is why `prevent_destroy` on critical resources and reading the plan matter.

Q. 38. How would you introduce Terraform into an environment that was built by hand?

Incrementally. Start with new resources so everything new is managed from the start. Then import existing resources component by component, writing the configuration and iterating until `plan` is empty - an empty plan is the proof that the code matches reality. Meanwhile, restrict console write access so the manual drift stops growing. Trying to import an entire estate in one pass is how these projects fail.

Q. 39. What are the most common Terraform mistakes you have seen?

Committing state or tfvars containing secrets; running `-auto-approve` on production without reading the plan; one giant state file for the whole estate; unpinned provider and module versions so builds are not reproducible; using `count` where `for_each` was needed and then recreating live resources; and using provisioners for configuration instead of a proper configuration management tool.

Q. 40. Terraform wants to destroy production infrastructure. What do you do?

Stop - do not apply. Read the plan carefully and find which resources and which attribute forces replacement; the plan says "forces replacement" against the specific attribute. Then work out why: someone may have changed an immutable field such as an availability zone or a name, or a module version bump changed a default, or the state was rolled back or partially removed, or I am pointed at the wrong workspace or state key. I verify which state I am using and which account with `aws sts get-caller-identity`. If it is a genuine immutable change I plan a safe migration - create before destroy, or a manual cutover - rather than letting a single apply do it. And this is exactly why `prevent_destroy` on databases and a required human review of plans exist.

PART 9 RECALL POINTS

- State maps code addresses to real resources - it is what makes update and destroy possible.
- State contains secrets in plaintext: encrypt it, lock it, never commit it.
- DynamoDB locking prevents concurrent applies corrupting state.
- `plan -out=tfplan` then `apply tfplan` - apply exactly what was reviewed.
- `for_each` for identity, `count` for interchangeable copies and conditionals.
- Pin providers, modules and the lock file.
- `-/+` in a plan means destroy and recreate - read why.
- Drift: detect with a scheduled plan, reconcile with `-refresh-only`.
- Terraform provisions, Ansible configures.

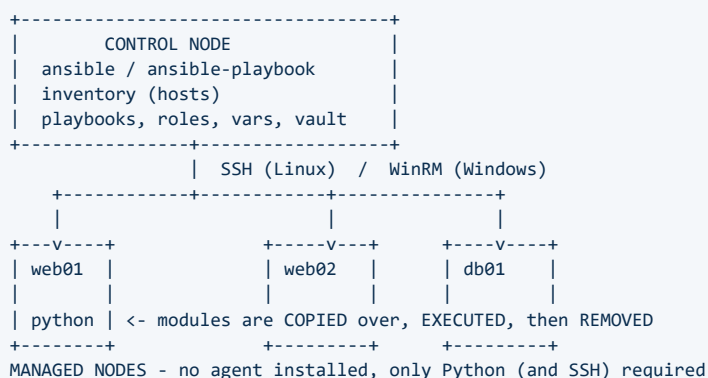
PART 10

ANSIBLE COMPLETE GUIDE

Inventory, playbooks, roles, variables, vault, idempotency, 30 questions

Your CV says you "automated server and application configuration with Ansible for repeatable, controlled environments" in both roles. The two things you must be able to defend are idempotency - why running a playbook twice is safe - and the division of labour between Terraform and Ansible.

10.1 Ansible Architecture



Agentless push model

Component	Role
Control node	Where Ansible runs. Linux only - Ansible does not run its control process on Windows
Managed node	The target. Needs SSH and Python for Linux; WinRM or SSH plus PowerShell for Windows
Inventory	The list of hosts and groups
Module	A unit of work - apt, copy, service, user. Written mostly in Python
Task	One invocation of a module
Play	A mapping of a host group to a list of tasks
Playbook	A YAML file containing one or more plays
Role	A standard directory structure packaging tasks, handlers, templates, files and variables
Handler	A task triggered only when notified by a changed task - typically a service restart
Facts	Data gathered from the target at the start of a run
Collection	A distributable bundle of roles, modules and plugins (Ansible Galaxy)

AGENTLESS IS THE HEADLINE FEATURE

"Ansible is agentless - it connects over SSH, copies the module to the target, executes it with Python, collects the JSON result and removes it. There is nothing to install or keep patched on the managed nodes, which is a large operational advantage over agent-based tools like Puppet or Chef. The trade-offs are that it is push-based rather than continuously enforcing, and that it is slower at very large scale because everything is SSH."

10.2 Inventory

Static inventory - INI format

```

[webservers]
web01 ansible_host=10.0.11.10
web02 ansible_host=10.0.11.11

[dbservers]
db01 ansible_host=10.0.21.10
  
```

```
[windows]
win01 ansible_host=10.0.11.50

[production:children]
webservers
dbservers

[webservers:vars]
http_port=8080
app_env=production

[windows:vars]
ansible_connection=winrm
ansible_winrm_transport=ntlm
ansible_port=5985

[all:vars]
ansible_user=deploy
ansible_ssh_private_key_file=~/.ssh/deploy.pem
ansible_python_interpreter=/usr/bin/python3
```

Static inventory - YAML format (preferred for complex structures)

```
all:
  vars:
    ansible_user: deploy
  children:
    production:
      children:
        webservers:
          hosts:
            web01: { ansible_host: 10.0.11.10 }
            web02: { ansible_host: 10.0.11.11 }
          vars:
            http_port: 8080
        dbservers:
          hosts:
            db01: { ansible_host: 10.0.21.10 }
```

	Static inventory	Dynamic inventory
Source	A file you maintain	A plugin or script querying a live source
Suits	Fixed, long-lived servers	Cloud, where instances come and go
AWS example	-	aws_ec2 plugin grouping by tag, region, ASG
Risk	Goes stale silently	Depends on the source's availability and permissions

aws_ec2.yml - dynamic inventory keyed on tags

```
plugin: amazon.aws.aws_ec2
regions:
  - eu-west-1
filters:
  tag:Environment: production
  instance-state-name: running
keyed_groups:
  - key: tags.Role          # creates groups like tag_Role_webserver
    prefix: role
  - key: placement.availability_zone
    prefix: az
hostnames:
  - private-ip-address
compose:
  ansible_host: private_ip_address
```

Inventory commands

```
ansible-inventory -i inventory.ini --list          # full JSON view
ansible-inventory -i inventory.ini --graph         # tree of groups and hosts
ansible-inventory -i aws_ec2.yml --graph          # verify the dynamic plugin works
ansible all -i inventory.ini --list-hosts
```

10.3 Ad-Hoc Commands

One-off operations across a fleet

```
ansible all -m ping                                # connectivity + Python check
ansible webserver -m command -a "uptime"
ansible webserver -m shell -a "df -h | grep -v tmpfs" # shell allows pipes and redirection
ansible webserver -m setup                          # dump all gathered facts
ansible webserver -m setup -a "filter=ansible_memory_mb"
ansible webserver -m apt -a "name=nginx state=present" --become
ansible webserver -m service -a "name=nginx state=restarted" --become
ansible webserver -m copy -a "src=./app.conf dest=/etc/app/app.conf mode=0644" --become
ansible webserver -m user -a "name=deploy groups=docker append=yes" --become
ansible all -m shell -a "systemctl is-active nginx" --one-line
ansible all -i inv -u ec2-user --private-key ~/.ssh/k.pem -m ping
```

COMMAND VERSUS SHELL

command runs the binary directly - no shell, so no pipes, redirects, globs or environment variable expansion, which makes it safer. shell runs through /bin/sh, so all of that works but you inherit shell injection risks. The interview answer: "Use command unless you specifically need shell features, and prefer a purpose-built module over both - apt rather than command: apt-get install, because the module is idempotent and command is not."

10.4 Playbooks

A complete, realistic playbook

```
---
- name: Configure and deploy the application tier
  hosts: webserver
  become: yes                # privilege escalation (sudo)
  gather_facts: yes
  serial: 2                  # rolling: 2 hosts at a time, not all at once
  max_fail_percentage: 0

  vars:
    app_name: payments-api
    app_dir: /opt/{{ app_name }}
    app_port: 8080

  vars_files:
    - vars/common.yml
    - vars/{{ ansible_facts['distribution'] | lower }}.yml

  pre_tasks:
    - name: Ensure the package cache is fresh
      ansible.builtin.apt:
        update_cache: yes
        cache_valid_time: 3600
      when: ansible_facts['os_family'] == "Debian"

  tasks:
    - name: Install required packages
      ansible.builtin.package:
        name:
          - nginx
          - python3
          - python3-pip
        state: present

    - name: Create the application group
      ansible.builtin.group:
        name: "{{ app_name }}"
        state: present

    - name: Create the application user
      ansible.builtin.user:
        name: "{{ app_name }}"
        group: "{{ app_name }}"
        home: "{{ app_dir }}"
```

```

    shell: /usr/sbin/nologin
    system: yes

- name: Create the application directories
  ansible.builtin.file:
    path: "{{ item }}"
    state: directory
    owner: "{{ app_name }}"
    group: "{{ app_name }}"
    mode: "0750"
  loop:
    - "{{ app_dir }}"
    - "{{ app_dir }}/releases"
    - /var/log/{{ app_name }}

- name: Render the application configuration
  ansible.builtin.template:
    src: app.conf.j2
    dest: "{{ app_dir }}/app.conf"
    owner: "{{ app_name }}"
    mode: "0640"
    backup: yes                # keep the previous version
    validate: "/usr/bin/app --check-config %s"
  notify: Restart application   # handler fires ONLY if this task changed something

- name: Copy the systemd unit
  ansible.builtin.template:
    src: app.service.j2
    dest: /etc/systemd/system/{{ app_name }}.service
    mode: "0644"
  notify:
    - Reload systemd
    - Restart application

- name: Ensure the service is running and enabled
  ansible.builtin.systemd:
    name: "{{ app_name }}"
    state: started
    enabled: yes

- name: Wait for the application port to accept connections
  ansible.builtin.wait_for:
    port: "{{ app_port }}"
    host: 127.0.0.1
    timeout: 60

- name: Verify the health endpoint
  ansible.builtin.uri:
    url: "http://127.0.0.1:{{ app_port }}/health"
    status_code: 200
  register: health
  retries: 5
  delay: 5
  until: health.status == 200

handlers:
- name: Reload systemd
  ansible.builtin.systemd:
    daemon_reload: yes

- name: Restart application
  ansible.builtin.systemd:
    name: "{{ app_name }}"
    state: restarted

```

Playbook keyword	Purpose
hosts	Which inventory group or pattern this play targets
become	Escalate privileges; become_user chooses the target user
gather_facts	Collect system facts. Setting it to no speeds up runs that do not need facts

Playbook keyword	Purpose
serial	Batch size for rolling updates - the key to zero-downtime configuration changes
vars, vars_files	Variable definitions
pre_tasks / post_tasks	Run before / after roles and tasks
roles	Include reusable roles
handlers	Tasks that run once at the end, only if notified
register	Capture a task's result into a variable
when	Conditional execution
loop	Repeat a task over a list
tags	Selectively run or skip parts of a playbook
ignore_errors	Continue despite failure - use sparingly
block / rescue / always	Structured error handling, like try/catch/finally

10.5 Idempotency - The Core Concept

THE DEFINITION TO GIVE

"Idempotency means running the playbook a second time produces no further changes, because each task checks the current state and only acts if reality does not already match the desired state. That is what makes automation safe to re-run - I can apply the same playbook to a fresh server or an existing one and get the same result. The place it breaks is command and shell tasks, which have no way to know the desired state, so they report 'changed' every run and may actually re-do work. Where I have to use them I add creates:, removes: or a when: guard so they become conditionally idempotent."

Non-idempotent versus idempotent

```
# BAD - runs every time, always reports changed, appends duplicates
- name: Add a line to a config
  ansible.builtin.shell: echo "max_connections=200" >> /etc/app/app.conf

# GOOD - declarative module, only changes if needed
- name: Ensure the setting is present
  ansible.builtin.lineinfile:
    path: /etc/app/app.conf
    regexp: '^max_connections='
    line: 'max_connections=200'
    state: present

# BAD - extracts every run
- name: Unpack the release
  ansible.builtin.shell: tar xzf /tmp/app.tgz -C /opt/app

# GOOD - guarded, so it runs only once
- name: Unpack the release
  ansible.builtin.unarchive:
    src: /tmp/app.tgz
    dest: /opt/app
    remote_src: yes
    creates: /opt/app/VERSION
```

Task result	Meaning
ok	Already in the desired state - nothing done
changed	The module made a change - this is what triggers handlers
failed	The task errored; the play stops for that host by default
skipped	A when condition was false
unreachable	Could not connect - SSH, key or network problem

10.6 Variables, Facts, Templates

Variable source	Precedence (later wins)
Role defaults (defaults/main.yml)	Lowest - intended to be overridden
Inventory group vars	
Inventory host vars	
Playbook vars	
vars_files	
Registered variables, facts	
Role vars (vars/main.yml)	High - not intended to be overridden
--extra-vars on the command line	Highest - always wins

Facts and conditionals

```

- name: Show gathered facts
  ansible.builtin.debug:
    msg: >
      {{ ansible_facts['hostname'] }} is {{ ansible_facts['distribution'] }}
      {{ ansible_facts['distribution_version'] }} with
      {{ ansible_facts['memtotal_mb'] }} MB RAM and
      {{ ansible_facts['processor_vcpus'] }} vCPUs

- name: Install the Debian-family package
  ansible.builtin.apt: { name: nginx, state: present }
  when: ansible_facts['os_family'] == "Debian"

- name: Install the RedHat-family package
  ansible.builtin.yum: { name: nginx, state: present }
  when: ansible_facts['os_family'] == "RedHat"

- name: Fail fast on insufficient memory
  ansible.builtin.fail:
    msg: "This host needs at least 4 GB RAM, found {{ ansible_facts['memtotal_mb'] }} MB"
  when: ansible_facts['memtotal_mb'] < 3800

- name: Set a custom fact that persists on the host
  ansible.builtin.set_fact:
    deploy_stamp: "{{ ansible_date_time.iso8601 }}"

```

templates/app.conf.j2 - Jinja2 templating

```

# Managed by Ansible - do not edit by hand
# Rendered {{ ansible_date_time.iso8601 }} for {{ inventory_hostname }}

[server]
bind    = 0.0.0.0
port    = {{ app_port }}
workers = {{ (ansible_facts['processor_vcpus'] | int) * 2 }}

[database]
host     = {{ db_host }}
port     = {{ db_port | default(5432) }}
name     = {{ db_name }}

{% if app_env == 'production' %}
log_level = WARN
debug     = false
{% else %}
log_level = DEBUG
debug     = true
{% endif %}

[allowed_hosts]
{% for host in groups['webserver'] %}
allow = {{ hostvars[host]['ansible_host'] }}
{% endfor %}

```

Jinja2 filter	Use
default('x')	Fallback when a variable is undefined
int, float, bool	Type conversion
upper, lower, trim	String manipulation
join(',')	List to string
to_json, from_json, to_nice_yaml	Serialisation
b64encode, b64decode	Kubernetes secrets, basic auth
regex_replace('a','b')	Pattern substitution
length	Count
password_hash('sha512')	Hash a password for the user module
mandatory	Fail explicitly if a variable is not defined

10.7 Roles

Standard role structure - created by ansible-galaxy init

```
roles/
  nginx/
    tasks/main.yml      # the work
    handlers/main.yml   # restart/reload handlers
    templates/          # Jinja2 templates (.j2)
    files/              # static files to copy
    vars/main.yml        # high-precedence variables
    defaults/main.yml   # low-precedence, overridable defaults
    meta/main.yml        # dependencies and metadata
    tests/              # test playbook
    README.md
```

Using roles, with variables and tags

```
---
- name: Build the web tier
  hosts: webservers
  become: yes

  roles:
    - role: common
      tags: [base]

    - role: nginx
      vars:
        nginx_worker_processes: 4
        nginx_upstream_servers: "{{ groups['appservers'] }}"
      tags: [web]

    - role: monitoring
      vars:
        node_exporter_version: "1.8.1"
      tags: [monitoring]
```

Galaxy and role management

```
ansible-galaxy init roles/myrole      # scaffold a new role
ansible-galaxy install geerlingguy.nginx # install a community role
ansible-galaxy install -r requirements.yml
ansible-galaxy collection install amazon.aws
```

10.8 Ansible Vault

Encrypting secrets at rest

```
ansible-vault create secrets.yml      # create a new encrypted file
ansible-vault edit secrets.yml        # decrypt, open in $EDITOR, re-encrypt
ansible-vault view secrets.yml        # read without editing
ansible-vault encrypt vars/prod.yml   # encrypt an existing file
ansible-vault decrypt vars/prod.yml   # permanently decrypt - rarely what you want
```

```

ansible-vault rekey secrets.yml                # change the vault password

# Encrypt a single value to embed inline in a normal YAML file
ansible-vault encrypt_string 'S3cr3tP@ss' --name 'db_password'

# Running playbooks that use vault
ansible-playbook site.yml --ask-vault-pass
ansible-playbook site.yml --vault-password-file ~/.vault_pass      # CI, file mode 600
ansible-playbook site.yml --vault-id prod@~/.vault_prod            # multiple vault identities

```

VAULT VERSUS A SECRETS MANAGER

"Vault encrypts secrets at rest inside the repository, which is far better than plaintext, but it is still a static secret shared by everyone who can decrypt it, and rotating it means re-encrypting files. In production I would prefer Ansible reading from AWS Secrets Manager or SSM Parameter Store at run time using the control node's IAM role, so nothing sensitive is in Git at all. Ansible Vault is a good fit for things like a bootstrap credential where a lookup is not possible."

10.9 Running Playbooks

Commands and flags that matter

```

ansible-playbook site.yml
ansible-playbook -i inventory/prod site.yml
ansible-playbook site.yml --check              # DRY RUN - predicts changes, changes nothing
ansible-playbook site.yml --check --diff       # dry run plus the actual file differences
ansible-playbook site.yml --diff               # show what changed in files
ansible-playbook site.yml --limit web01        # one host only
ansible-playbook site.yml --limit 'webserver:!web03'
ansible-playbook site.yml --tags "config,deploy"
ansible-playbook site.yml --skip-tags "slow"
ansible-playbook site.yml --start-at-task "Render the application configuration"
ansible-playbook site.yml --step               # confirm each task interactively
ansible-playbook site.yml -e "app_version=1.4.2" # extra vars win over everything
ansible-playbook site.yml -e "@prod-vars.yml"
ansible-playbook site.yml -v                   # -v -vv -vvv -vvvv increasing verbosity
ansible-playbook site.yml --syntax-check
ansible-playbook site.yml --list-tasks
ansible-playbook site.yml --list-hosts
ansible-playbook site.yml -f 20                # forks: 20 hosts in parallel

```

--CHECK --DIFF IS YOUR PRODUCTION SAFETY NET

"Before running a playbook against production I run it with `--check --diff`. Check mode predicts what would change without changing anything, and diff shows the exact file differences. It is not perfect - tasks whose result depends on an earlier task's change can report inaccurately in check mode, and `command/shell` tasks are skipped - but it catches most mistakes. It is the Ansible equivalent of terraform `plan`."

10.10 Practical Playbook Examples

1. Install and configure a package with a service

```

- name: Install and configure nginx
  hosts: webserver
  become: yes
  tasks:
    - name: Install nginx
      ansible.builtin.package: { name: nginx, state: present }

    - name: Deploy the site configuration
      ansible.builtin.template:
        src: site.conf.j2
        dest: /etc/nginx/conf.d/site.conf
        validate: 'nginx -t -c %s'
        notify: Reload nginx

    - name: Ensure nginx is running and enabled
      ansible.builtin.service: { name: nginx, state: started, enabled: yes }

```

```
handlers:
  - name: Reload nginx
    ansible.builtin.service: { name: nginx, state: reloaded }
```

2. Create users with SSH keys and sudo

```
- name: Manage operations users
  hosts: all
  become: yes
  vars:
    ops_users:
      - { name: aasim, key: "ssh-ed25519 AAAA...", sudo: true }
      - { name: qa, key: "ssh-ed25519 BBBB...", sudo: false }
  tasks:
    - name: Create users
      ansible.builtin.user:
        name: "{{ item.name }}"
        shell: /bin/bash
        groups: "{{ 'sudo' if item.sudo else omit }}"
        append: yes
        state: present
        loop: "{{ ops_users }}"

    - name: Install authorised keys
      ansible.posix.authorized_key:
        user: "{{ item.name }}"
        key: "{{ item.key }}"
        state: present
        loop: "{{ ops_users }}"

    - name: Grant restricted sudo for service restarts
      ansible.builtin.lineinfile:
        path: /etc/sudoers.d/ops
        line: "{{ item.name }} ALL=(root) NOPASSWD: /bin/systemctl restart payments-api"
        create: yes
        mode: "0440"
        validate: 'visudo -cf %s'
        loop: "{{ ops_users }}"
        when: item.sudo
```

3. Zero-downtime rolling application deployment

```
- name: Rolling deploy
  hosts: webservers
  become: yes
  serial: 1 # one host at a time
  max_fail_percentage: 0 # stop the whole run if any host fails

  tasks:
    - name: Remove this host from the load balancer
      ansible.builtin.uri:
        url: "http://lb.internal/api/drain/{{ inventory_hostname }}"
        method: POST
        delegate_to: localhost # run on the control node, not the target

    - name: Wait for connections to drain
      ansible.builtin.pause: { seconds: 20 }

    - name: Pull the new image
      community.docker.docker_image:
        name: "registry.example.com/payments-api:{{ app_version }}"
        source: pull

    - name: Restart the container with the new image
      community.docker.docker_container:
        name: payments-api
        image: "registry.example.com/payments-api:{{ app_version }}"
        state: started
        restart_policy: unless-stopped
        recreate: yes
        published_ports: ["8080:8080"]

    - name: Wait for the health endpoint
```

```

ansible.builtin.uri:
  url: "http://127.0.0.1:8080/health"
  status_code: 200
register: health
retries: 12
delay: 5
until: health.status == 200

- name: Return the host to the load balancer
  ansible.builtin.uri:
    url: "http://lb.internal/api/enable/{{ inventory_hostname }}"
    method: POST
  delegate_to: localhost

```

4. Error handling with block/rescue/always

```

- name: Deploy with rollback
  hosts: appservers
  become: yes
  tasks:
    - block:
        - name: Deploy the new release
          ansible.builtin.unarchive:
            src: "app-{{ app_version }}.tgz"
            dest: /opt/app/releases/{{ app_version }}

        - name: Point current at the new release
          ansible.builtin.file:
            src: /opt/app/releases/{{ app_version }}
            dest: /opt/app/current
            state: link

        - name: Restart the service
          ansible.builtin.systemd: { name: app, state: restarted }

        - name: Health check
          ansible.builtin.uri: { url: "http://127.0.0.1:8080/health", status_code: 200 }
          retries: 6
          delay: 5

    rescue:
        - name: Roll back the symlink to the previous release
          ansible.builtin.file:
            src: "/opt/app/releases/{{ previous_version }}"
            dest: /opt/app/current
            state: link

        - name: Restart on the previous release
          ansible.builtin.systemd: { name: app, state: restarted }

        - name: Report the failure
          ansible.builtin.fail:
            msg: "Deployment of {{ app_version }} failed; rolled back to {{ previous_version }}"

    always:
        - name: Record the outcome
          ansible.builtin.lineinfile:
            path: /var/log/deploy-history.log
            line: "{{ ansible_date_time.iso8601 }} {{ app_version }} {{ 'OK' if not ansible_failed_task is
defined else 'ROLLED BACK' }}"
            create: yes

```

10.11 Terraform vs Ansible - Say This Clearly

	Terraform	Ansible
Category	Provisioning / IaC	Configuration management
Model	Declarative with a state file	Task-based; each task idempotent, no state file
Knows what it owns	Yes - the state file	No - it checks reality each run

	Terraform	Ansible
Creates infrastructure	Yes - VPCs, instances, load balancers	Can, through cloud modules, but it is not its strength and there is no state to reconcile
Configures an OS	Not its job (provisioners are discouraged)	Yes - packages, files, services, users
Drift handling	Detects and can revert, because it has state	Re-running converges each task, but it will not remove things it never created
Agent	None	None (agentless over SSH/WinRM)
Typical order	Runs first	Runs second, against what Terraform created

THE COMBINED-WORKFLOW ANSWER

"They are complementary. Terraform creates the infrastructure - VPC, subnets, security groups, EC2 instances - and because it holds state, it knows exactly what it manages and can show me a plan before changing anything. Ansible then configures those machines: installs packages, renders configuration from templates, manages services and deploys the application. In practice Terraform outputs the instance details, either into a dynamic inventory or through the `aws_ec2` inventory plugin keyed on tags, and Ansible picks them up from there. Using Terraform provisioners to do configuration, or using Ansible to build a whole VPC, is possible but it fights each tool's design."

10.12 Ansible Interview Questions

Q. 1. What is Ansible and how does it differ from Puppet or Chef?

An agentless configuration management and automation tool that connects over SSH or WinRM, pushes modules to the target, executes them and removes them. Puppet and Chef are agent-based and pull-based - an agent on each node periodically fetches and applies the catalogue. Ansible's advantage is nothing to install or patch on managed nodes and a very low barrier to entry; the trade-offs are that it is push-based so it does not continuously enforce state, and SSH becomes a bottleneck at very large scale.

Q. 2. What are the requirements on a managed node?

SSH access and Python for Linux targets; WinRM (or SSH) plus PowerShell for Windows. No Ansible agent. The control node must be Linux - Ansible does not run its control process natively on Windows.

Q. 3. What is idempotency and why does it matter?

Running the same playbook repeatedly produces no further changes because each task checks current state and only acts if reality differs from the desired state. It matters because it makes automation safe to re-run - I can apply the same playbook to a new server or an existing one. It breaks with `command` and `shell`, which always report changed, so I guard them with `creates:`, `removes:` or `when:`.

Q. 4. What is the difference between a playbook and a role?

A playbook is the top-level YAML file mapping hosts to tasks - it is the thing you run. A role is a reusable, standard directory structure packaging tasks, handlers, templates, files and defaults, which a playbook includes. Roles are how you avoid copying the same fifty tasks into ten playbooks.

Q. 5. What is a handler?

A task that runs only when notified by another task that actually changed something, and only once at the end of the play regardless of how many tasks notified it. The classic use is restarting a service after a configuration file changed - so the service restarts only if the config genuinely changed, not on every run.

Q. 6. What if you need a handler to run immediately rather than at the end?

`meta: flush_handlers` forces all pending handlers to run at that point. You need it when a later task depends on the service having already been restarted - for example reloading `systemd` before starting a newly installed unit.

Q. 7. What are facts?

System information gathered from the target at the start of a run - OS family, distribution and version, memory, CPU count, network interfaces, IP addresses. They let a playbook adapt to the host, for example installing with `apt` or `yum` depending on `ansible_facts['os_family']`. Disabling fact gathering with `gather_facts: no` noticeably speeds up playbooks that do not need them.

Q. 8. What is the variable precedence order, roughly?

From lowest to highest: role defaults, inventory group vars, inventory host vars, playbook vars, vars_files, role vars, and finally --extra-vars, which always wins. The practical rule is to put overridable values in defaults/ and things that must not be overridden in vars/.

Q. 9. What is Ansible Vault?

Encryption for sensitive data at rest - whole files or individual values with `encrypt_string` - decrypted at run time with a vault password. It stops secrets sitting in plaintext in Git. For production I would prefer looking secrets up from a secrets manager at run time, since vault passwords are static shared secrets that are awkward to rotate.

Q. 10. What is the difference between `copy` and `template`?

`copy` transfers a file as-is. `template` renders a Jinja2 template with variables and facts before transferring it, so one template produces host-specific configuration. Templates also support `validate:`, which runs a syntax check before the file is put in place - that is how you avoid deploying a config that breaks the service.

Q. 11. What is check mode?

--check is a dry run: Ansible reports what would change without changing anything. Combined with --diff it shows the exact file differences. It is the closest equivalent to `terraform plan`, with the caveat that tasks depending on earlier changes can report inaccurately, and `command/shell` tasks are skipped unless marked `check_mode: no`.

Q. 12. How do you run a playbook on just one host?

--limit hostname, which also accepts patterns and exclusions such as --limit 'webservers:!web03'. Combined with --tags and --start-at-task, it is how you re-run part of a playbook after fixing a failure instead of starting over.

Q. 13. What are tags used for?

Selectively running or skipping parts of a playbook - --tags deploy or --skip-tags slow. In a long playbook it turns a 20-minute full run into a 30-second targeted change.

Q. 14. How do you achieve a rolling update with Ansible?

`serial:` controls how many hosts are processed at a time - `serial: 1` for one at a time or a percentage for batches - combined with removing the host from the load balancer, deploying, health-checking, and putting it back. `max_fail_percentage: 0` stops the rollout as soon as one host fails, so a bad release does not propagate across the fleet.

Q. 15. What does `delegate_to` do?

Runs a task on a different host from the one currently being processed - typically `delegate_to: localhost` to call a load balancer API or update DNS while iterating over target hosts. `run_once: true` combined with it runs a task a single time for the whole play rather than per host.

Q. 16. How do you handle errors?

`block/rescue/always` for structured handling - deploy in the block, roll back in rescue, log in always. `failed_when` and `changed_when` to define what actually constitutes failure or change for a command task. `ignore_errors: yes` only where a failure is genuinely acceptable, because it hides real problems.

Q. 17. How do you register and use a command's output?

`register:` `result` captures the return structure, then you use `result.stdout`, `result.rc` or `result.stdout_lines` in later tasks and conditions - often with `until`, `retries` and `delay` to poll until a condition is met, which is how health checks are written.

Q. 18. What is the difference between `include_tasks` and `import_tasks`?

`import_tasks` is static - processed at parse time, so tags and conditionals apply to each imported task individually and the tasks are visible to --list-tasks. `include_tasks` is dynamic - evaluated at run time, so it can use variables set during the run and loop, but it is opaque to some pre-run tooling. The rule of thumb is import when it is static, include when the decision depends on runtime data.

Q. 19. What is dynamic inventory and why do you need it in the cloud?

An inventory generated by querying a live source rather than maintained by hand. In AWS the `aws_ec2` plugin queries EC2 and groups instances by tag, region or availability zone. It is necessary because instances in an Auto Scaling Group come and go - a static file goes stale immediately and you end up configuring hosts that no longer exist while missing ones that do.

Q. 20. How do you connect Ansible to Windows hosts?

Over WinRM, with `ansible_connection: winrm` and an appropriate transport such as NTLM or Kerberos, plus WinRM enabled and listening on the target. Then you use the `win_*` modules - `win_feature`, `win_service`, `win_copy` - rather than the Linux ones. This is directly relevant to my current role, where we run both Linux and Windows servers.

Q. 21. What is `ansible.cfg` and what do you put in it?

The configuration file controlling defaults - inventory path, remote user, private key, forks, whether host key checking is enabled, roles path, and performance settings such as SSH pipelining. Committing it with the project means everyone runs with the same settings rather than depending on personal environments.

Q. 22. How do you speed up Ansible?

Increase forks for more parallelism, enable SSH pipelining to reduce round trips, turn off fact gathering where it is not needed or use fact caching, use `async` with `poll: 0` for long-running independent tasks, avoid unnecessary loops over large lists, and use `--limit` and `--tags` so you are not re-running everything.

Q. 23. What is the difference between `apt/yum` and the `package` module?

`package` is a generic wrapper that dispatches to the right package manager based on the target's OS family, so one task works on Debian and RedHat. The specific modules expose options the generic one does not, such as `update_cache` and `cache_valid_time` for `apt`. I use `package` for portability and the specific module when I need its features.

Q. 24. How do you ensure a config change actually restarts the service?

notify a handler from the task that writes the config. The handler runs only if that task reported changed, and only once at the end. That is the whole point of handlers - a naive `service: restarted` task would restart on every run, causing needless disruption.

Q. 25. What is `lineinfile` and when would you use `blockinfile` instead?

`lineinfile` ensures a single line is present or absent, matched by a regex. `blockinfile` manages a whole marked block of text, which is better for multi-line insertions because it wraps the block in markers it can find again. Both are for editing files you do not own; if you own the file, a template is cleaner because it declares the whole desired content.

Q. 26. How would you test Ansible code?

`--syntax-check` and `ansible-lint` in CI; `--check --diff` against a staging host; and Molecule for role testing, which spins up a container or VM, applies the role, asserts with Testinfra, and verifies idempotency by running it twice and requiring zero changes on the second pass. That idempotency test is the one that catches the most real bugs.

Q. 27. How does Ansible fit into CI/CD?

The pipeline builds and publishes the artifact, then triggers the playbook for the target environment, with credentials injected from the CI credential store and the inventory generated dynamically from cloud tags. The playbook does the deploy and the health check, and its exit code determines whether the pipeline stage passes. Rolling deployment with `serial` and rollback with `block/rescue` live inside the playbook rather than in the pipeline.

Q. 28. What is the difference between `become` and `become_user`?

`become`: yes escalates privileges, by default to root using `sudo`. `become_user` specifies a different target user - for example becoming the application user to run a database migration. `become_method` chooses the mechanism: `sudo`, `su`, `doas`, `runas` on Windows.

Q. 29. What happens when a task fails on one host out of ten?

By default that host is removed from the play and the remaining hosts continue; at the end the recap shows the failure. `any_errors_fatal: true` aborts the entire play immediately, and `max_fail_percentage` sets a tolerance. For a rolling deployment I set these so a failure stops the rollout instead of continuing to break the rest of the fleet.

Q. 30. What are the most common Ansible mistakes?

Using `shell` and `command` where a module exists, which destroys idempotency; not using handlers, so services restart on every run; hardcoding values instead of using variables and defaults; secrets in plaintext; no `--check` before production; monolithic playbooks with no roles; and ignoring the idempotency test - if the second run reports changes, the playbook is not doing what the author thinks it is.

PART 10 RECALL POINTS

- Agentless: SSH/WinRM, push modules, execute, remove.
- Idempotency is the core promise; `command/shell` break it unless guarded.
- Handlers fire only on change, once, at the end.
- `serial`: is how you do rolling, zero-downtime configuration changes.
- `--check --diff` is the Ansible equivalent of `terraform plan`.
- Vault for secrets at rest; a secrets manager is better in production.
- `--extra-vars` beats every other variable source.
- Terraform provisions, Ansible configures - and Ansible has no state file.

PART 11

KUBERNETES COMPLETE GUIDE

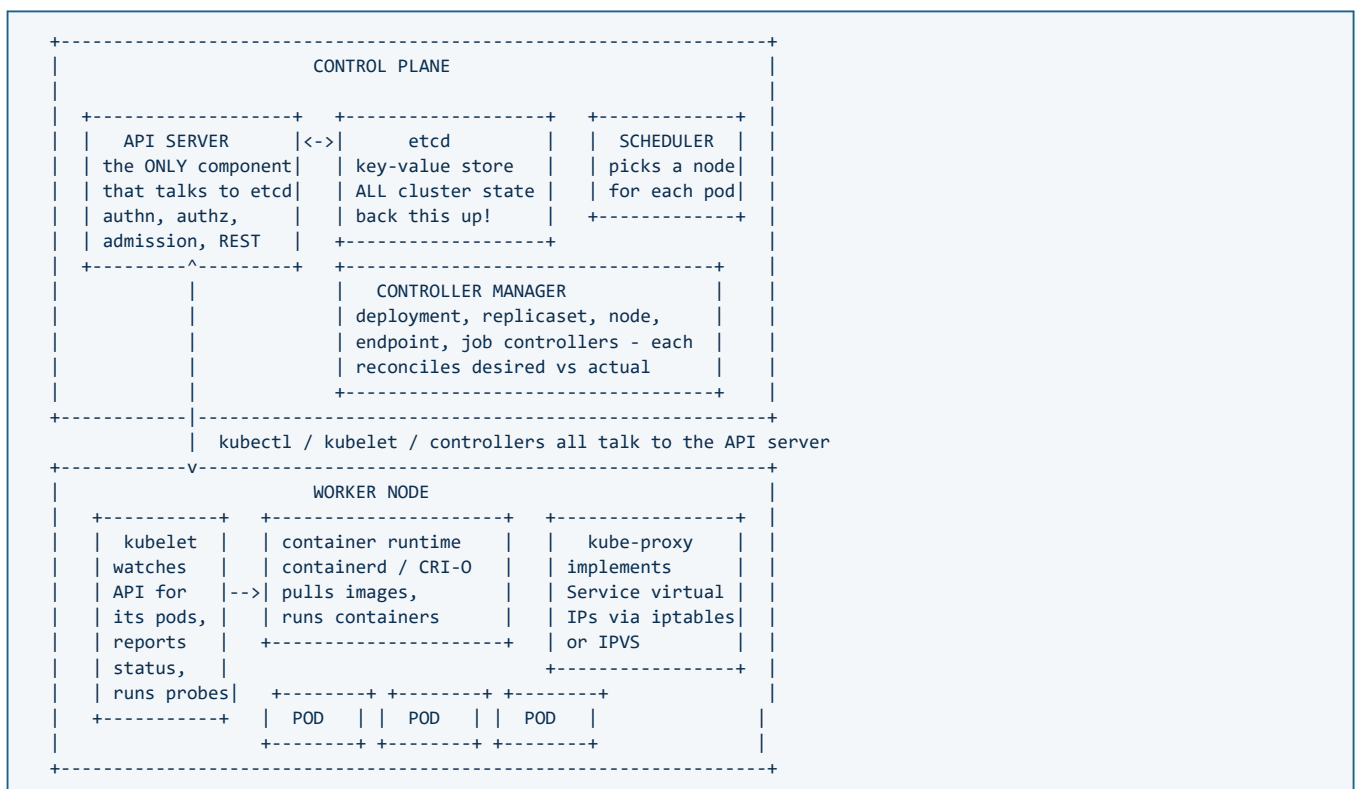
Architecture, objects, YAML, kubectl, 15 troubleshooting scenarios, 70 questions

Your CV says you "support Kubernetes deployments, troubleshooting Pods, Deployments, Services, ConfigMaps and application connectivity issues", and it lists Kubernetes administration as something you are currently strengthening. That is an honest position and you should hold it: you troubleshoot workloads, you do not administer control planes. This part gives you enough depth to be strong on workloads and credible on architecture.

HOW TO ANSWER KUBERNETES QUESTIONS HONESTLY

For workload questions - pods, deployments, services, configmaps, secrets, probes, rollouts, debugging - answer with confidence; that is your CV's claim. For cluster administration - etcd backups, upgrading control planes, CNI selection, cluster autoscaler tuning - answer with the concept and then say: "I have not administered a cluster in production; my Kubernetes work is at the workload level - deploying and troubleshooting. This is my understanding of how it works." That combination scores far better than a bluff.

11.1 Architecture



Control plane and worker nodes

Component	Responsibility	If it fails
API server	Front door: REST API, authentication, authorisation, admission control, the only writer to etcd	Nothing can be changed; running workloads keep running
etcd	Consistent key-value store holding all cluster state	Catastrophic - this is what you back up
Scheduler	Assigns pending pods to nodes based on resources, taints, affinity	New pods stay Pending; existing pods unaffected
Controller manager	Runs the reconciliation loops - deployment, replicaset, node, endpoints	Self-healing stops; nothing replaces failed pods

Component	Responsibility	If it fails
cloud-controller-manager	Cloud-specific integration - load balancers, node lifecycle, volumes	LoadBalancer Services and node cleanup stop working
kubelet	Node agent: starts and stops containers, runs probes, reports node and pod status	That node goes NotReady; its pods are eventually evicted
Container runtime	Actually runs containers via the CRI	Pods on that node fail
kube-proxy	Programmes iptables/IPVS so Service virtual IPs route to pod IPs	Service networking breaks on that node
CoreDNS	Cluster DNS - resolves Service names	In-cluster name resolution fails; a very common incident

THE RECONCILIATION LOOP IS THE CORE IDEA

"Kubernetes is a set of controllers each running a loop: read the desired state from the API server, observe the actual state, and take action to close the gap. That is why it is declarative - I do not tell it to start three pods, I declare that three should exist, and the ReplicaSet controller makes that true and keeps it true. Self-healing is not a feature bolted on; it is a consequence of the architecture."

11.2 Pods

A pod is the smallest deployable unit: one or more containers that share a network namespace (so they share an IP and can reach each other on localhost), share storage volumes, and are always scheduled together on one node.

A pod with the settings that matter in production

```
apiVersion: v1
kind: Pod
metadata:
  name: payments-api
  labels:
    app: payments-api
    tier: backend
spec:
  serviceAccountName: payments-api
  securityContext:
    runAsNonRoot: true
    runAsUser: 10001
    fsGroup: 10001

  initContainers:
    # runs to completion BEFORE app containers start
    - name: wait-for-db
      image: busybox:1.36
      command: ['sh', '-c', 'until nc -z postgres 5432; do echo waiting; sleep 2; done']

  containers:
    - name: api
      image: registry.example.com/payments-api:1.4.2
      imagePullPolicy: IfNotPresent
      ports:
        - containerPort: 8080
          name: http

      env:
        - name: APP_ENV
          value: production
        - name: DB_HOST
          valueFrom:
            configMapKeyRef: { name: payments-config, key: db_host }
        - name: DB_PASSWORD
          valueFrom:
            secretKeyRef: { name: payments-secret, key: db_password }

  resources:
    requests:
      # used for SCHEDULING - guaranteed
      memory: "256Mi"
      cpu: "250m"
```

```

limits:                                # enforced ceiling
  memory: "512Mi"
  cpu: "1000m"

livenessProbe:                          # failing -> container is RESTARTED
  httpGet: { path: /healthz, port: 8080 }
  initialDelaySeconds: 30
  periodSeconds: 10
  timeoutSeconds: 3
  failureThreshold: 3

readinessProbe:                         # failing -> pod removed from Service ENDPOINTS
  httpGet: { path: /ready, port: 8080 }
  initialDelaySeconds: 5
  periodSeconds: 5
  failureThreshold: 3

startupProbe:                           # protects slow starters from the liveness probe
  httpGet: { path: /healthz, port: 8080 }
  failureThreshold: 30
  periodSeconds: 10

volumeMounts:
- name: config
  mountPath: /etc/app
  readOnly: true
- name: tmp
  mountPath: /tmp

securityContext:
  allowPrivilegeEscalation: false
  readOnlyRootFilesystem: true
  capabilities: { drop: ["ALL"] }

volumes:
- name: config
  configMap: { name: payments-config }
- name: tmp
  emptyDir: {}

```

Pod phase	Meaning
Pending	Accepted but not running - waiting for scheduling, image pull, or a volume
Running	Bound to a node, at least one container running
Succeeded	All containers exited 0 and will not restart (Jobs)
Failed	All containers terminated, at least one with a non-zero exit
Unknown	The node cannot be reached

Probe	Failure action	Purpose	Trap
Liveness	Restart the container	Detect a deadlocked or wedged process	Too aggressive = restart loop on a healthy but slow app. Never point it at a dependency
Readiness	Remove the pod from Service endpoints	Do not send traffic to a pod that cannot serve	The correct place to check dependencies
Startup	Delay liveness until the app has started	Slow-starting applications, e.g. large JVMs	Without it you must inflate initialDelaySeconds, which delays real failure detection

THE LIVENESS/READINESS DISTINCTION WINS INTERVIEWS

"Readiness controls traffic; liveness controls restarts. If a pod cannot reach its database, readiness should fail so it stops receiving traffic - but liveness should NOT fail, because restarting the container will not fix the database and you will just churn the whole deployment into a restart loop while the database recovers. That is the most common probe mistake I have seen."

11.3 Deployments and ReplicaSets

```

DEPLOYMENT payments-api (desired: 3 replicas, image 1.4.2)
|
| manages, one per pod-template revision
v
REPLICASET payments-api-7d4f8 (image 1.4.2, desired 3) <- current
REPLICASET payments-api-6b2c1 (image 1.4.1, desired 0) <- kept for rollback
|
| creates
v
POD payments-api-7d4f8-abc12  POD ...-def34  POD ...-ghi56

```

Rolling update: the Deployment scales the NEW ReplicaSet up and the OLD one down, step by step, honouring maxSurge and maxUnavailable, waiting for readiness at each step. Rollback = scale the old ReplicaSet back up. That is why old ReplicaSets are retained.

The ownership chain, and how a rollout works

A Deployment with a controlled rollout

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: payments-api
  labels: { app: payments-api }
spec:
  replicas: 3
  revisionHistoryLimit: 5
  minReadySeconds: 10                # a pod must stay ready this long before counting
  progressDeadlineSeconds: 600        # rollout fails after this -> "progress deadline exceeded"

  selector:
    matchLabels: { app: payments-api } # IMMUTABLE - must match the template labels

  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1                    # 1 extra pod allowed above replicas
      maxUnavailable: 0              # never go below 3 ready - zero downtime

  template:
    metadata:
      labels: { app: payments-api }
      annotations:
        prometheus.io/scrape: "true"
        prometheus.io/port: "8080"
    spec:
      containers:
        - name: api
          image: registry.example.com/payments-api:1.4.2
          ports: [{ containerPort: 8080 }]
          resources:
            requests: { memory: "256Mi", cpu: "250m" }
            limits: { memory: "512Mi", cpu: "1000m" }
          readinessProbe:
            httpGet: { path: /ready, port: 8080 }
            initialDelaySeconds: 5
            periodSeconds: 5
      terminationGracePeriodSeconds: 45

```

Rollout management

```

kubectl rollout status deployment/payments-api      # wait for it to finish - use in CI
kubectl rollout history deployment/payments-api
kubectl rollout history deployment/payments-api --revision=3
kubectl rollout undo deployment/payments-api        # back to the previous revision
kubectl rollout undo deployment/payments-api --to-revision=3
kubectl rollout pause deployment/payments-api       # freeze mid-rollout (manual canary)
kubectl rollout resume deployment/payments-api
kubectl rollout restart deployment/payments-api     # recreate all pods (pick up a new Secret)
kubectl set image deployment/payments-api api=registry.example.com/payments-api:1.4.3
kubectl scale deployment/payments-api --replicas=6

```

KUBECTL ROLLOUT RESTART IS A USEFUL THING TO KNOW

Changing a ConfigMap or Secret does NOT restart pods automatically - environment variables from them are injected at pod start, so the running pods keep the old values. `kubectl rollout restart deployment/x` triggers a rolling recreation. The production-grade alternative is annotating the pod template with a checksum of the config so any config change naturally produces a new pod template and a rollout.

11.4 Services and Networking

Service type	What it does	Use
ClusterIP (default)	A stable virtual IP reachable only inside the cluster	Internal service-to-service traffic
NodePort	Opens the same high port (30000-32767) on every node	Development, or behind an external load balancer
LoadBalancer	Provisions a cloud load balancer pointing at the nodes	Exposing a service on a cloud provider
ExternalName	Returns a CNAME to an external DNS name	Pointing at a managed database or an external API
Headless (clusterIP: None)	No virtual IP; DNS returns the pod IPs directly	StatefulSets, client-side load balancing

Service and Ingress

```

apiVersion: v1
kind: Service
metadata:
  name: payments-api
spec:
  type: ClusterIP
  selector:
    app: payments-api          # THIS is what wires the Service to pods - labels, not names
  ports:
    - name: http
      port: 80                 # the Service's port
      targetPort: 8080         # the container's port
      protocol: TCP
---
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: payments-api
  annotations:
    nginx.ingress.kubernetes.io/ssl-redirect: "true"
    nginx.ingress.kubernetes.io/proxy-body-size: "10m"
    cert-manager.io/cluster-issuer: letsencrypt-prod
spec:
  ingressClassName: nginx
  tls:
    - hosts: [api.example.com]
      secretName: api-tls
  rules:
    - host: api.example.com
      http:
        paths:
          - path: /payments
            pathType: Prefix
            backend:
              service:
                name: payments-api
                port: { number: 80 }

```

```

Internet
|
[ Cloud LB ] (created by a LoadBalancer Service, or by the ingress controller's Service)
|
[ Ingress Controller pod ] reads Ingress objects, routes by host and path, terminates TLS
|
[ Service (ClusterIP) ] a virtual IP; kube-proxy programmes iptables/IPVS rules
                        the Service's ENDPOINTS list = the READY pods matching its selector
|
[ Pod ] the container's targetPort

DNS inside the cluster: payments-api.default.svc.cluster.local -> the Service's ClusterIP
Short forms that work from a pod in the same namespace: payments-api or payments-api.default

```

How a request reaches a pod

ENDPOINTS ARE THE DIAGNOSTIC

"A Service does not point at pods by name - it selects them by label, and the set of READY matching pods becomes its Endpoints. So when a Service returns nothing, the first command is `kubectl get endpoints <svc>`. Empty endpoints means either the selector does not match any pod's labels, or the pods exist but are failing their readiness probe. That one command splits the problem in half immediately."

11.5 ConfigMaps and Secrets

Both, and the three ways to consume them

```

apiVersion: v1
kind: ConfigMap
metadata: { name: payments-config }
data:
  db_host: "postgres.data.svc.cluster.local"
  log_level: "INFO"
  app.properties: |
    server.port=8080
    server.threads=32
---
apiVersion: v1
kind: Secret
metadata: { name: payments-secret }
type: Opaque
data:
  db_password: c3VwZXJzZWNyZXQ=      # base64 - ENCODING, not encryption
  stringData:
    api_key: "plain-text-here"        # Kubernetes base64-encodes this for you
---
# Consuming them
spec:
  containers:
    - name: api
      env:
        - name: LOG_LEVEL
          valueFrom: { configMapKeyRef: { name: payments-config, key: log_level } }
        - name: DB_PASSWORD
          valueFrom: { secretKeyRef: { name: payments-secret, key: db_password } }
      envFrom:
        - configMapRef: { name: payments-config }      # every key becomes an env var
      volumeMounts:
        - name: config
          mountPath: /etc/app                          # each key becomes a file
      volumes:
        - name: config
          configMap: { name: payments-config }

```

	ConfigMap	Secret
For	Non-sensitive configuration	Passwords, tokens, keys, certificates
Storage	Plaintext in etcd	base64 in etcd - encrypted at rest only if encryption-at-rest is enabled
Size limit	1 MiB	1 MiB

	ConfigMap	Secret
Mounted as a volume	Updates propagate after a delay	Same
As environment variables	Does NOT update - pod restart required	Same

SECRETS ARE NOT ENCRYPTED BY DEFAULT

base64 is encoding, and anyone who can run `kubectl get secret -o yaml` can decode it instantly. Real protection requires: encryption at rest for etcd, RBAC restricting who can read Secrets in that namespace, and ideally an external store - AWS Secrets Manager via the Secrets Store CSI driver, or External Secrets Operator. Saying "Secrets are base64, not encrypted" is a strong signal in an interview; assuming they are secure is a weak one.

11.6 The Other Objects

Object	Purpose	Key property
Namespace	Logical partition of a cluster	Scope for names, RBAC, quotas and network policies
ReplicaSet	Maintains N identical pods	You rarely create one directly - Deployments own them
StatefulSet	Pods with stable identity and storage	Ordered creation, stable names (app-0, app-1), a PVC per pod, stable DNS
DaemonSet	One pod per node (or per matching node)	Node agents: log shippers, node_exporter, CNI, CSI drivers
Job	Run to completion	<code>completions</code> , <code>parallelism</code> , <code>backoffLimit</code>
CronJob	Scheduled Job	Cron syntax, <code>concurrencyPolicy</code> , <code>startingDeadlineSeconds</code>
PersistentVolume (PV)	A piece of storage in the cluster	Provisioned by an admin or dynamically by a StorageClass
PersistentVolumeClaim (PVC)	A request for storage by a pod	Binds to a PV; the pod references the claim
StorageClass	A template for dynamic provisioning	Defines the provisioner, parameters and reclaim policy
HorizontalPodAutoscaler	Scales replicas on metrics	CPU, memory or custom metrics
NetworkPolicy	Pod-level firewall	Default is allow-all; a policy selecting a pod makes it default-deny for that direction
ResourceQuota / LimitRange	Namespace capacity control and per-pod defaults	Prevents one team consuming the cluster
PodDisruptionBudget	Minimum availability during voluntary disruption	Protects you during node drains and upgrades
ServiceAccount	Identity for a pod when calling the API	Bound to Roles via RoleBindings

StatefulSet, DaemonSet, CronJob, HPA, PVC - the shapes to recognise

```

apiVersion: apps/v1
kind: StatefulSet
metadata: { name: postgres }
spec:
  serviceName: postgres          # the required headless Service
  replicas: 3
  selector: { matchLabels: { app: postgres } }
  template:
    metadata: { labels: { app: postgres } }
    spec:
      containers:
        - name: postgres
          image: postgres:16
          volumeMounts: [{ name: data, mountPath: /var/lib/postgresql/data }]
  volumeClaimTemplates:          # each pod gets its OWN PVC: data-postgres-0, data-postgres-1...
    - metadata: { name: data }
      spec:
        accessModes: ["ReadWriteOnce"]
        storageClassName: gp3
        resources: { requests: { storage: 20Gi } }
---
```

```

apiVersion: apps/v1
kind: DaemonSet
metadata: { name: node-exporter }
spec:
  selector: { matchLabels: { app: node-exporter } }
  template:
    metadata: { labels: { app: node-exporter } }
    spec:
      tolerations:
        - operator: Exists          # run on EVERY node, including tainted control-plane nodes
      hostNetwork: true
      containers:
        - name: node-exporter
          image: prom/node-exporter:v1.8.1
          ports: [{ containerPort: 9100 }]
---
apiVersion: batch/v1
kind: CronJob
metadata: { name: db-backup }
spec:
  schedule: "0 2 * * *"
  concurrencyPolicy: Forbid        # never overlap with a still-running backup
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 3
  jobTemplate:
    spec:
      backoffLimit: 2
      template:
        spec:
          restartPolicy: OnFailure
          containers:
            - name: backup
              image: postgres:16
              command: ["/bin/sh", "-c", "pg_dump -h postgres -U app app | gzip > /backup/db.gz"]
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: { name: payments-api }
spec:
  scaleTargetRef: { apiVersion: apps/v1, kind: Deployment, name: payments-api }
  minReplicas: 3
  maxReplicas: 20
  metrics:
    - type: Resource
      resource:
        name: cpu
        target: { type: Utilization, averageUtilization: 70 }
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 300    # avoid flapping

```

HPA REQUIRES RESOURCE REQUESTS

The HPA computes utilisation as usage divided by the **request**. If a container has no CPU request, there is nothing to compute a percentage against and the HPA reports <unknown> and never scales. "My HPA is not scaling" is almost always a missing resource request or a missing metrics-server.

11.7 kubectl - The Commands You Must Know Cold

Inspecting

```

kubectl get pods
kubectl get pods -A          # all namespaces
kubectl get pods -o wide     # node, pod IP, nominated node
kubectl get pods -w          # watch changes live
kubectl get pods -l app=payments-api    # by label
kubectl get pods --field-selector status.phase=Pending
kubectl get pods --sort-by=.status.containerStatuses[0].restartCount
kubectl get deploy,svc,ing,cm,secret -n prod
kubectl get all -n prod

```

```
kubectl get endpoints payments-api          # THE Service debugging command
kubectl get events --sort-by=.lastTimestamp # what actually happened, in order
kubectl get events -w --field-selector type=Warning
kubectl get nodes -o wide
kubectl top nodes                          # requires metrics-server
kubectl top pods --sort-by=memory

kubectl describe pod payments-api-7d4f8-abc12 # events, probes, mounts, reasons - read this FIRST
kubectl describe node worker-3                # conditions, allocatable, pressure, taints
kubectl describe svc payments-api
```

Logs and execution

```
kubectl logs payments-api-7d4f8-abc12
kubectl logs -f payments-api-7d4f8-abc12      # follow
kubectl logs payments-api-7d4f8-abc12 -c api   # a specific container
kubectl logs payments-api-7d4f8-abc12 --previous # the CRASHED container's logs - vital for
    CrashLoopBackOff
kubectl logs -l app=payments-api --tail=100 --all-containers
kubectl logs deploy/payments-api --since=15m

kubectl exec -it payments-api-7d4f8-abc12 -- sh
kubectl exec payments-api-7d4f8-abc12 -- env
kubectl exec payments-api-7d4f8-abc12 -- cat /etc/app/app.properties
kubectl exec -it payments-api-7d4f8-abc12 -- nslookup postgres
kubectl debug -it payments-api-7d4f8-abc12 --image=busybox --target=api # ephemeral debug container

kubectl port-forward pod/payments-api-7d4f8-abc12 8080:8080 # test a pod directly, bypassing the Service
kubectl port-forward svc/payments-api 8080:80 # test the Service, bypassing Ingress
kubectl cp payments-api-7d4f8-abc12:/tmp/heap.hprof ./heap.hprof
```

Changing things

```
kubectl apply -f deployment.yaml          # declarative - the correct default
kubectl apply -f ./manifests/             # a whole directory
kubectl apply -k ./overlays/prod         # kustomize
kubectl diff -f deployment.yaml           # what WOULD change - the plan equivalent
kubectl delete -f deployment.yaml
kubectl delete pod payments-api-7d4f8-abc12 # the Deployment immediately replaces it
kubectl edit deployment payments-api       # live edit - fine for debugging, never for real change
kubectl scale deployment payments-api --replicas=5
kubectl set image deployment/payments-api api=image:1.4.3
kubectl annotate deployment payments-api kubernetes.io/change-cause="release 1.4.3"
kubectl label pod x env=test

kubectl cordon worker-3                   # mark unschedulable
kubectl drain worker-3 --ignore-daemonsets --delete-emptydir-data # evict pods for maintenance
kubectl uncordon worker-3

kubectl auth can-i create deployments -n prod
kubectl auth can-i '*' '*' --as system:serviceaccount:prod:payments-api
kubectl explain deployment.spec.strategy # built-in schema documentation
kubectl api-resources                     # every object kind and its short name
```

11.8 The 15 Troubleshooting Scenarios

1. CRASHLOOPBACKOFF

Meaning: the container starts, exits, and Kubernetes restarts it with exponential backoff (10s, 20s, 40s ... capped at 5 minutes). The pod is not stuck - it is failing repeatedly.

Steps:

1. `kubectl logs <pod> --previous` - the logs of the container that just died. This is the single most important command and most candidates forget `--previous`.
2. `kubectl describe pod <pod>` - look at Last State, the exit code and Reason.
3. Exit code 1 - application error: missing env var, unreachable dependency at startup, bad config, failed migration.
4. Exit code 137 with `OOMKilled` - the memory limit is too low, or there is a leak. Compare actual usage with the limit.
5. Exit code 0 repeatedly - the process finishes and exits; the command is wrong, or the app daemonises instead of staying in the foreground.
6. Exit code 127 - command not found; wrong path or missing in a slim image.
7. Config/Secret problem - `kubectl exec` is impossible while crashing, so check with `kubectl get cm/secret -o yaml` and re-read the manifest.
8. A liveness probe failing during startup also produces this - fix it with a `startupProbe` rather than by disabling liveness.

Isolate: run the image manually with the same env and command (`kubectl run debug --image=... --command -- sleep 3600`, then `exec` in) to separate application failure from Kubernetes configuration.

2. IMAGEPULLBACKOFF / ERRIMAGEPULL

Meaning: the kubelet cannot pull the image.

Steps:

1. `kubectl describe pod <pod>` - the Events section names the exact registry error.
2. "manifest unknown" / "not found" - the tag does not exist. Check the tag and whether the CI push actually succeeded.
3. "unauthorized" / "authentication required" - a private registry with no or wrong `imagePullSecrets`. Verify the secret exists in the SAME namespace and is referenced by the pod or its `ServiceAccount`.
4. "toomanyrequests" - Docker Hub rate limit; authenticate or use a mirror.
5. Timeout - the node has no network route to the registry: missing NAT gateway for private subnets, or a missing VPC endpoint for ECR.
6. Wrong registry host or a typo in the image path.
7. Architecture mismatch - an arm64 image on amd64 nodes.

Verify: pull the same image manually on a node, or from a debug pod, with the same credentials.

3. POD STUCK IN PENDING

Meaning: the scheduler cannot place it, or it is placed but cannot start.

Steps:

1. `kubectl describe pod <pod>` - the Events say exactly why. This scenario is almost always solved by reading that message.
2. "Insufficient cpu/memory" - no node has enough ALLOCATABLE capacity for the pod's REQUESTS. Check with `kubectl describe node` and `kubectl top nodes`. Either lower the requests, add nodes, or let the cluster autoscaler do it.
3. "node(s) had taint that the pod didn't tolerate" - add a toleration or target different nodes.
4. "didn't match Pod's node affinity/selector" - the `nodeSelector` or affinity matches no node.
5. "pod has unbound immediate PersistentVolumeClaims" - the PVC is not bound. Check the PVC, the `StorageClass` and whether the zone of the volume matches the node.
6. "0/5 nodes are available: 5 Insufficient pods" - the per-node pod limit is reached.
7. No nodes Ready at all - a cluster-level problem, not a pod problem.

4. POD IS RUNNING BUT THE APPLICATION IS UNAVAILABLE

1. Is it actually Ready? `kubectl get pod - 1/1` versus `0/1`. Running is not Ready.
2. `kubectl logs` - is the application actually serving, or stuck in startup?
3. Is it listening on the expected port and interface? `kubectl exec <pod> -- ss -tulpn - 127.0.0.1:8080` means it will never receive traffic from the Service.
4. Test the pod directly: `kubectl port-forward pod/<pod> 8080:8080` then curl locally. That removes Service and Ingress from the equation.
5. If the pod works directly, the problem is the Service, the Ingress or DNS - move to scenario 5.

5. SERVICE HAS NO ENDPOINTS

1. `kubectl get endpoints <svc>` - empty is the diagnosis.
2. Compare the Service's selector with the pods' actual labels: `kubectl get pods --show-labels`. A single character difference means no match.
3. If labels match, the pods are not READY - only ready pods are added to endpoints. Fix the readiness probe or the underlying failure.
4. Check `targetPort` against the container's actual `containerPort`.
5. Cross-namespace: a Service only selects pods in its own namespace.
6. `kubectl describe svc <svc>` shows both the selector and the resolved endpoints together.

6. DNS FAILURE INSIDE THE CLUSTER

1. From a pod: `kubectl exec -it <pod> -- nslookup payments-api` and `nslookup kubernetes.default`.
2. If nothing resolves, check CoreDNS: `kubectl get pods -n kube-system -l k8s-app=kube-dns` and its logs.
3. Check the Service exists and is in the namespace you think: the FQDN is `<svc>.<namespace>.svc.cluster.local`.
4. `cat /etc/resolv.conf` inside the pod - it should point at the kube-dns ClusterIP with an `ndots:5` search path.
5. A NetworkPolicy blocking egress to kube-system on port 53 is a classic cause of "DNS suddenly broke for one namespace".
6. External name resolution failing but internal working points at CoreDNS's upstream forwarder or node-level DNS.

7. CONFIGMAP PROBLEM

- Pod stuck in `CreateContainerConfigError`: the referenced ConfigMap or key does not exist. `kubectl describe pod` names it.
- The application reads an old value: environment variables from a ConfigMap are injected at pod START. Changing the ConfigMap does not update running pods - `kubectl rollout restart` is required. Mounted volumes DO update, after a propagation delay, but only if the application re-reads the file.
- Mounting a ConfigMap over a directory replaces the entire directory's contents; use `subPath` to mount a single file into an existing directory - but note that `subPath` mounts do NOT receive updates.
- Wrong namespace: a ConfigMap is namespaced and must live alongside the pod.

8. SECRET PROBLEM

- Same `CreateContainerConfigError` if the Secret or key is missing.
- Wrong base64: `echo -n "value" | base64` - a trailing newline from `echo` without `-n` corrupts the value and produces a confusing authentication failure. Use `stringData` to avoid the whole problem.
- RBAC: the ServiceAccount may not have permission to read the Secret, which appears as a mount failure.
- The value looks right but the app rejects it: decode and compare exactly - `kubectl get secret x -o jsonpath='{.data.password}' | base64 -d`.

9. READINESS PROBE FAILURE

- `kubectl describe pod` shows "Readiness probe failed: ..." with the HTTP status or connection error.
- Wrong path or port - a `/ready` endpoint that does not exist returns 404, which fails the probe.
- The probe runs from the kubelet to the pod IP, so the app must be listening on 0.0.0.0, not 127.0.0.1.
- `initialDelaySeconds` too short for a slow-starting app, so it never gets to ready before the failure threshold.
- The probe endpoint checks a dependency that is genuinely down - which is correct behaviour; fix the dependency.
- Consequence: the pod is removed from endpoints, and if all pods fail readiness you get a 503 at the Ingress.

10. LIVENESS PROBE FAILURE

- Symptom: containers restarting periodically with `Liveness probe failed` in the events.
- Most common cause is an over-aggressive probe: a timeout shorter than the app's worst-case response time under load, so a busy-but-healthy app gets killed, which makes it busier - a death spiral.
- Second most common: pointing liveness at an endpoint that checks the database. The database blips, every pod restarts, and a small dependency problem becomes a total outage.
- Fix: a liveness endpoint that only tests the process itself, generous timeout and `failureThreshold`, and a `startupProbe` for slow starts.

11. OOMKILLED

1. `kubectl describe pod` - Last State: Terminated, Reason: OOMKilled, Exit Code: 137.
2. Compare actual usage with the limit: `kubectl top pod <pod>`, and historical memory from Prometheus if available.
3. Decide: is the limit too low, or is there a leak? A limit too low shows steady usage just above the limit; a leak shows usage climbing steadily until it hits the limit, repeatedly.
4. For a JVM or Node app, the runtime must respect the container limit - set `-XX:MaxRAMPercentage=75` or `--max-old-space-size` so the heap fits inside the container limit. A JVM that sizes its heap from the host's RAM will always be OOMKilled.
5. Raise the limit as a mitigation, then fix the cause. Remember that the memory limit is a hard kill - unlike CPU, memory is not throttled.

12. NODE NOTREADY

1. `kubectl describe node <node>` - read the Conditions: MemoryPressure, DiskPressure, PIDPressure, NetworkUnavailable, and the kubelet's last heartbeat.
2. DiskPressure is the most common in practice - the node's disk is full of images and logs; kubelet garbage collection should handle it but often needs help.
3. On the node: is kubelet running? `systemctl status kubelet` and `journalctl -u kubelet -n 200`.
4. Is the container runtime healthy? `systemctl status containerd`.
5. Network: can the node reach the API server? Certificate expiry on the kubelet is a classic cause of a whole cluster going NotReady at once.
6. Effects: pods on a NotReady node are marked for eviction after the toleration period (5 minutes by default) and rescheduled elsewhere - unless they cannot be, for example a StatefulSet with a zone-bound volume.
7. Remediation: `kubectl drain` the node, fix or replace it, `uncordon`. In a cloud environment, replacing the node is usually faster than repairing it.

13. DEPLOYMENT ROLLOUT STUCK

1. `kubectl rollout status deployment/x` hangs, then reports "progress deadline exceeded".
2. `kubectl get pods -l app=x` - what state are the NEW pods in? The answer is always one of the earlier scenarios: Pending, ImagePullBackOff, CrashLoopBackOff, or Running-but-never-Ready.
3. `kubectl describe rs <new-replicaset>` for events about why pods could not be created - quota exceeded, for example.
4. With `maxUnavailable: 0` and insufficient cluster capacity, the new pod cannot be scheduled and the rollout stalls forever at the first step.
5. A `PodDisruptionBudget` can also block progress.
6. Action: `kubectl rollout undo` to restore service, then fix the underlying cause offline.

14. OLD VERSION STILL SERVING AFTER A SUCCESSFUL DEPLOY

1. Verify what is actually running: `kubectl get pods -o jsonpath='{..image}'` and, better, check the image DIGEST, not the tag.
2. Mutable tag plus `imagePullPolicy: IfNotPresent` - the node already had an image called `myapp:latest` and never pulled the new one. This is the number one cause. Fix: immutable tags per build; Always only papers over it.
3. Did the pods actually restart? `kubectl get pods` - check AGE. If they are old, the Deployment's pod template never changed, so no rollout happened - which is exactly what occurs when only a `ConfigMap` changed.
4. Are old pods still in the Service endpoints? `kubectl get endpoints`.
5. Caching further out: CDN, browser, or a client holding a keep-alive connection to an old pod.
6. Multiple Deployments or a stale `ReplicaSet` serving under the same Service selector.

15. PERSISTENTVOLUME PROBLEMS

1. PVC stuck Pending: `kubectl describe pvc <name>`. Common causes - no `StorageClass` and no matching static PV, the `StorageClass` provisioner is broken or missing, or the requested size or `accessMode` cannot be satisfied.
2. `ReadWriteOnce` means one NODE at a time. A Deployment with 3 replicas sharing one RWO volume will leave two pods stuck in `ContainerCreating` with a multi-attach error. Either use a single replica, or `ReadWriteMany` with a filesystem that supports it (EFS, NFS), or a `StatefulSet` where each pod gets its own volume.
3. Zone mismatch: an EBS volume exists in one availability zone and the pod is scheduled in another, so it can never attach. `volumeBindingMode: WaitForFirstConsumer` on the `StorageClass` prevents this by provisioning the volume only once the pod's node is chosen.
4. Pod stuck Terminating with a volume still attached - often a node failure; the volume detach times out.
5. Data disappeared after deleting a PVC: the `StorageClass reclaimPolicy` was `Delete`. Use `Retain` for anything that matters.

11.9 Kubernetes Interview Questions

Q. 1. What is Kubernetes and what problem does it solve?

A container orchestration platform. Running one container is easy; running hundreds across many machines, keeping the right number alive, replacing failures, rolling out new versions without downtime and giving them stable network identity is not. Kubernetes solves scheduling, self-healing, service discovery, scaling, configuration and rollout, all through a declarative API.

Q. 2. Explain the control plane components.

The API server is the front door and the only component that writes to etcd - everything goes through it, including authentication, authorisation and admission control. etcd stores all cluster state. The scheduler assigns pending pods to nodes based on requests, taints, affinity and constraints. The controller manager runs the reconciliation loops - deployment, replicaset, node, endpoints - each closing the gap between desired and actual state. On a cloud there is also a cloud controller manager handling load balancers, node lifecycle and volumes.

Q. 3. What runs on a worker node?

The kubelet, which watches the API server for pods assigned to it, tells the runtime to start them, runs the probes and reports status. The container runtime - containerd or CRI-O - which actually runs containers. And kube-proxy, which

programmes iptables or IPVS so Service virtual IPs route to pod IPs.

Q. 4. What is etcd and why does it matter?

A distributed, consistent key-value store holding all cluster state. It matters because losing it means losing the cluster - every object definition is there. So it is backed up on a schedule, restores are tested, and it runs with an odd number of members for quorum.

Q. 5. What is a pod and why not just a container?

A pod is one or more containers that share a network namespace and storage and are always scheduled together. The abstraction exists for the sidecar pattern - a log shipper, a proxy or an init container running alongside the main application, able to reach it on localhost and share its volumes. In practice most pods contain a single application container.

Q. 6. What is the relationship between a Deployment, a ReplicaSet and a Pod?

A Deployment manages ReplicaSets; a ReplicaSet manages Pods. Every change to the pod template creates a new ReplicaSet, and the Deployment scales the new one up and the old one down according to the rollout strategy. The old ReplicaSets are retained with zero replicas, which is exactly what makes `rollout undo` instantaneous.

Q. 7. What happens when you run `kubectl apply -f deployment.yaml`?

`kubectl` sends the manifest to the API server, which authenticates and authorises the request, runs admission controllers, validates the object and persists it to etcd. The deployment controller notices a new desired state and creates a ReplicaSet; the replicaset controller creates pod objects; the scheduler assigns each pod to a node; the kubelet on that node pulls the image and starts the containers and reports status back. Nothing is imperative - each controller is just closing a gap.

Q. 8. Difference between a Deployment and a StatefulSet?

A Deployment treats pods as interchangeable - random names, any order, shared or no storage. A StatefulSet gives each pod a stable ordinal identity, a stable DNS name, its own PersistentVolumeClaim, and ordered creation and deletion. You use it for things that care which replica they are - databases, message brokers, anything doing leader election or clustering.

Q. 9. What is a DaemonSet for?

Running exactly one pod per node, automatically including new nodes as they join. It is how node-level agents are deployed - log shippers, `node_exporter`, CNI and CSI drivers - and it typically includes tolerations so it runs on tainted nodes too.

Q. 10. What is the difference between a Job and a CronJob?

A Job runs pods until a specified number complete successfully and then stops; a CronJob creates Jobs on a schedule. For CronJobs, `concurrencyPolicy` matters - `Forbid` stops a slow run overlapping with the next one, which is essential for backups.

Q. 11. Explain the Service types.

`ClusterIP` gives a stable internal virtual IP - the default, for service-to-service traffic. `NodePort` opens a fixed high port on every node. `LoadBalancer` asks the cloud provider for an external load balancer. `ExternalName` returns a CNAME to an external hostname. A headless Service with `clusterIP: None` returns pod IPs directly, which StatefulSets use for stable per-pod DNS.

Q. 12. How does a Service know which pods to send traffic to?

By label selector. The set of ready pods matching the selector becomes the Service's Endpoints, maintained by the endpoints controller. kube-proxy programmes rules to load-balance the virtual IP across those endpoints. That is why `kubectl get endpoints` is the first debugging command for Service problems.

Q. 13. Service versus Ingress?

A Service provides a stable virtual IP and load-balances at layer 4 across pods. An Ingress is layer 7 HTTP routing - host and path based - with TLS termination, implemented by an ingress controller running in the cluster. The Ingress object is just configuration; without a controller it does nothing. The practical value is that one load balancer can front many services instead of one per service.

Q. 14. How does DNS work inside a cluster?

CoreDNS runs as a Deployment and is exposed as a Service; pods are configured to use it via `/etc/resolv.conf`. A Service gets a record at `<service>.<namespace>.svc.cluster.local`, with the search path allowing short names within the same namespace. Headless Services additionally get per-pod A records.

Q. 15. What is the difference between requests and limits?

The request is what the scheduler reserves and guarantees - it decides placement. The limit is the enforced ceiling. Exceeding a CPU limit causes throttling; exceeding a memory limit causes the container to be OOMKilled, because memory cannot be throttled. Requests without limits risks noisy neighbours; limits without requests means the scheduler has no idea what your pod needs.

Q. 16. What are QoS classes?

Guaranteed, where every container has requests equal to limits for both CPU and memory; Burstable, where requests are set and lower than limits; and BestEffort, where nothing is set. Under node memory pressure the kubelet evicts BestEffort first, then Burstable, and Guaranteed last - so the class directly determines what survives.

Q. 17. Explain liveness, readiness and startup probes.

Liveness determines whether the container should be restarted - it detects a wedged process. Readiness determines whether the pod should receive traffic - it controls Service endpoints. Startup delays liveness checks until the application has finished starting, which is how you support slow starters without setting a huge initialDelaySeconds. The critical design rule is that readiness may check dependencies but liveness must not, otherwise a dependency blip restarts the entire fleet.

Q. 18. What is CrashLoopBackOff and how do you debug it?

The container repeatedly starts and exits, so Kubernetes backs off exponentially between restarts. I start with `kubectl logs <pod> --previous` to see the dead container's output, then `kubectl describe pod` for the exit code and reason. Exit 1 is an application error, 137 with OOMKilled is memory, 0 means the process simply finished, and 127 means the command was not found. If it is config-related I check the ConfigMap and Secret references, since I cannot exec into a crashing container.

Q. 19. What causes ImagePullBackOff?

The kubelet cannot pull the image. `kubectl describe pod` gives the exact registry error - a tag that does not exist, missing or wrong `imagePullSecrets` for a private registry, a rate limit, no network route from the node to the registry, or an architecture mismatch.

Q. 20. A pod is Pending. What do you check?

`kubectl describe pod` and read the scheduler's message in Events, which states the reason directly - insufficient CPU or memory for the pod's requests, an untolerated taint, a nodeSelector or affinity matching no node, an unbound PVC, or the node pod limit. Each has a different fix, so reading the message first saves a lot of guessing.

Q. 21. What are taints and tolerations?

A taint on a node repels pods that do not tolerate it; a toleration on a pod allows it to be scheduled there. They are used to reserve nodes - GPU nodes, control-plane nodes - and to mark nodes as unhealthy so pods are evicted. They are the opposite of node affinity: affinity attracts pods, taints repel them.

Q. 22. What is the difference between nodeSelector, node affinity and pod affinity?

nodeSelector is a simple exact label match. Node affinity is the richer form with required and preferred rules and set-based operators. Pod affinity and anti-affinity schedule relative to other pods - anti-affinity spreading replicas across nodes or zones is the standard way to avoid all your replicas landing on one machine.

Q. 23. What is a PodDisruptionBudget?

A minimum availability guarantee during VOLUNTARY disruptions such as a node drain or cluster upgrade. It does not protect against a node crashing - that is involuntary. Without one, draining a node can take down every replica of a service at once.

Q. 24. How does a rolling update work and what controls it?

The Deployment creates a new ReplicaSet and shifts replicas from old to new gradually, governed by `maxSurge` - how many extra pods above the desired count - and `maxUnavailable` - how many may be unavailable. It waits for new pods to pass readiness before continuing. `maxUnavailable: 0` with `maxSurge: 1` gives a strictly zero-downtime rollout at the cost of needing capacity for one extra pod.

Q. 25. How do you roll back?

`kubectl rollout undo deployment/<name>`, optionally with `--to-revision`. It works because the previous ReplicaSets are retained with zero replicas, so rollback is just scaling them back up. The caveat is that it rolls back the pod template only - ConfigMaps, Secrets and database migrations are not reverted.

Q. 26. What is the difference between `kubectl apply` and `kubectl create`?

`create` is imperative and fails if the object already exists. `apply` is declarative - it creates or updates to match the manifest and records the configuration in an annotation for three-way merges. `apply` is the correct default because it is idempotent and works with version-controlled manifests.

Q. 27. How do you update configuration without downtime?

Change the ConfigMap or Secret and trigger a rolling restart, because environment variables are injected at pod start and do not update in place. The clean pattern is to annotate the pod template with a checksum of the config, so any config change produces a new pod template and therefore a normal rolling update with readiness gating.

Q. 28. Are Kubernetes Secrets secure?

Not by default - the data is base64-encoded, which is encoding, not encryption, and anyone who can read Secrets in the namespace can decode them. Making them secure requires enabling encryption at rest for etcd, tight RBAC on Secret read access, and ideally an external secret store integrated through the Secrets Store CSI driver or External Secrets Operator so the real secret never lives in etcd at all.

Q. 29. What is a namespace and what does it isolate?

A logical partition for names, and a scope for RBAC, resource quotas and network policies. It does not isolate networking by default - pods in different namespaces can reach each other unless a NetworkPolicy says otherwise - and it does not isolate nodes. So it is an organisational and policy boundary, not a security boundary on its own.

Q. 30. What is RBAC in Kubernetes?

Role-based access control using four objects: Role and ClusterRole define permissions, RoleBinding and ClusterRoleBinding grant them to a user, group or ServiceAccount. Roles are namespaced, ClusterRoles are cluster-wide. `kubectl auth can-i` is how you test it, including impersonating a ServiceAccount.

Q. 31. What is a ServiceAccount?

The identity a pod uses when calling the Kubernetes API. Every pod gets one - `default` if unspecified - and a token is projected into the pod. You create dedicated ServiceAccounts with narrow RBAC for anything that needs API access, and on EKS you bind them to IAM roles so a pod can call AWS APIs without credentials.

Q. 32. What is a NetworkPolicy?

A pod-level firewall defined by selectors. The default is allow-all; as soon as a policy selects a pod for a direction, that direction becomes default-deny for that pod and only the listed traffic is permitted. It requires a CNI that implements it - Calico or Cilium, for example - and with a CNI that does not, the policy object exists and silently does nothing, which is a dangerous false sense of security.

Q. 33. How does the HPA decide to scale?

It compares the average utilisation across pods against a target, where utilisation is measured against the container's resource REQUEST, and adjusts the replica count. It needs metrics-server for resource metrics and requests must be set, otherwise it reports unknown and never scales. It also has stabilisation windows to prevent flapping.

Q. 34. What is the difference between HPA, VPA and Cluster Autoscaler?

HPA changes the number of pods. VPA changes the CPU and memory requests of pods, which generally requires restarting them. Cluster Autoscaler changes the number of nodes when pods cannot be scheduled or nodes are underused. They solve different layers, and HPA and VPA on the same metric conflict with each other.

Q. 35. What is a PersistentVolume and a PersistentVolumeClaim?

A PV is a piece of storage in the cluster, provisioned statically by an admin or dynamically by a StorageClass. A PVC is a request for storage by a workload; it binds to a PV, and the pod references the claim rather than the volume. The indirection is what lets the same manifest run in different clusters with different storage backends.

Q. 36. What are access modes and why do they cause problems?

ReadWriteOnce means one node can mount it read-write, ReadOnlyMany means many nodes read-only, and ReadWriteMany means many nodes read-write. The problem is people assume RWO means one pod - it means one node - and then try to run a multi-replica Deployment on a single EBS volume, which leaves pods stuck with multi-attach errors. Shared write access needs a filesystem like EFS or NFS.

Q. 37. What is a StorageClass and what is `volumeBindingMode`?

A StorageClass describes how to dynamically provision storage - the provisioner, the parameters and the reclaim policy. `volumeBindingMode: WaitForFirstConsumer` delays provisioning until a pod is scheduled, so the volume is created in the same availability zone as the node. Without it you get volumes that can never attach because they are in the wrong zone.

Q. 38. What is the reclaim policy?

What happens to the underlying volume when the PVC is deleted - Delete removes it, Retain keeps it for manual recovery. Delete is often the default, and it is how people permanently lose data by deleting a namespace. For anything stateful, Retain.

Q. 39. What is an init container?

A container that runs to completion before the application containers start, in order. It is used for waiting on a dependency, running a migration, or fetching configuration - and because it can have a different image and different permissions, it keeps that tooling out of the main application image.

Q. 40. What is a sidecar?

A container running alongside the application in the same pod, sharing its network and volumes - a log shipper, a metrics exporter, a service mesh proxy. The value is adding capability without modifying the application image.

Q. 41. How do you debug a pod with no shell?

`kubectl debug` with an ephemeral container attached to the pod's namespaces, using an image that does have tools. Failing that, `kubectl logs`, `kubectl describe`, and `kubectl port-forward` to test it directly. Distless images make debugging harder on purpose, and ephemeral containers are the supported answer.

Q. 42. What is the difference between `kubectl delete pod` and `kubectl drain`?

Deleting a pod removes that one pod; if it is owned by a controller it is immediately recreated, possibly on the same node. Draining a node cordons it so nothing new is scheduled and then evicts all its pods, respecting `PodDisruptionBudgets`, so the node can be taken out of service safely.

Q. 43. What happens when a node dies?

The node controller stops receiving heartbeats and marks the node `NotReady`, then after the toleration period taints it so pods are evicted and rescheduled elsewhere. Pods with volumes that cannot be detached, or `StatefulSet` pods with zone-bound storage, may not reschedule cleanly - which is why stateful workloads need more care than stateless ones.

Q. 44. How do you expose an application to the internet?

For a single service, a `LoadBalancer Service`, which provisions a cloud load balancer. For many services, an ingress controller behind one load balancer, with Ingress objects routing by host and path and terminating TLS - that is cheaper and gives layer-7 routing. `NodePort` is really only for development or when something else fronts the nodes.

Q. 45. What is the difference between `imagePullPolicy: Always` and `IfNotPresent`?

`Always` pulls on every pod start; `IfNotPresent` uses the node's cached image if the tag is already present. `IfNotPresent` with a mutable tag such as `latest` is the classic cause of "we deployed but the old version is still running". The real fix is immutable tags per build, not `Always`.

Q. 46. How would you deploy to multiple environments?

`Kustomize overlays` - a base set of manifests with per-environment patches for replicas, resources, image tags and config - or Helm with per-environment values files. Both keep one source of truth and make the differences between environments explicit and reviewable, rather than maintaining three divergent copies of the YAML.

Q. 47. What is Helm?

A package manager for Kubernetes. A chart is a templated set of manifests with a values file; installing it renders the templates and applies them, and Helm tracks releases so you can upgrade and roll back as a unit. It is how you install third-party components like ingress controllers and Prometheus without hand-maintaining hundreds of manifests.

Q. 48. What is the difference between Helm and `Kustomize`?

Helm templates with Go templating and manages releases with revision history; it is a full package manager, which is what you want for distributing software. `Kustomize` patches plain YAML declaratively with no templating language, and is built into `kubectl`; it is often preferred for your own applications because the manifests stay valid YAML and diffs are readable.

Q. 49. What is GitOps?

The cluster's desired state lives in Git, and a controller in the cluster - Argo CD or Flux - continuously reconciles the cluster to match it. Deployment becomes a merge, not a pipeline pushing `kubectl` commands; drift is detected and corrected automatically; and the audit trail is the Git history. It also removes the need to hand cluster credentials to the CI system.

Q. 50. How do you handle database migrations in Kubernetes?

Usually a Job or an init container that runs the migration before the new version starts, with migrations written to be backward compatible so old and new pods can run simultaneously during a rolling update - expand and contract. Running migrations in every application replica's startup is a race condition, and irreversible migrations make rollback impossible, which is the real risk to plan around.

Q. 51. What is a `ResourceQuota`?

A namespace-level limit on total CPU, memory, storage and object counts. It stops one team consuming the whole cluster. A `LimitRange` complements it by setting default and maximum requests and limits per pod, so workloads without explicit resources still get sensible values and cannot be created with absurd ones.

Q. 52. How do you back up a Kubernetes cluster?

`etcd` snapshots for the cluster state, plus volume snapshots for persistent data - and in practice a tool like Velero that backs up both object definitions and volumes per namespace and can restore into another cluster. The more important point is that manifests should be in Git anyway, so the cluster is reproducible; backup then mainly covers stateful data.

Q. 53. What is the difference between ClusterIP and headless services for StatefulSets?

A ClusterIP Service load-balances across pods, which is wrong when a client needs to reach a specific replica. A headless Service returns the individual pod IPs and gives each pod a stable DNS name, which is how database clients discover and address individual members.

Q. 54. What happens during pod termination?

The pod is marked Terminating and removed from Service endpoints; a preStop hook runs if defined; the container gets SIGTERM; after `terminationGracePeriodSeconds` any remaining process gets SIGKILL. The common bug is an application that ignores SIGTERM, so every deployment drops in-flight requests. A short preStop sleep is a standard trick to let endpoint removal propagate before the process starts shutting down.

Q. 55. Why might a pod still receive traffic after it starts terminating?

Because endpoint removal and the kube-proxy rule update on every node are asynchronous with the SIGTERM. For a brief window, rules may still point at the terminating pod. That is exactly why a preStop sleep of a few seconds plus graceful shutdown in the application is the recommended pattern.

Q. 56. How do you troubleshoot slow application performance in Kubernetes?

Check whether it is the pod or the platform. `kubect1 top pod` against limits - CPU throttling from hitting a CPU limit is a very common and invisible cause of latency. Then node-level pressure with `kubect1 top nodes` and `describe node`. Then dependencies - database latency, DNS resolution time, downstream services. Then the application itself with its own metrics and traces. The Kubernetes-specific traps are CPU throttling and DNS latency from `ndots` search-path expansion.

Q. 57. What is CPU throttling and why is it sneaky?

When a container exceeds its CPU limit the kernel throttles it via cgroup quotas - the application does not crash, it just gets slower, in a way that does not appear in any error log. You see it in the `container_cpu_cfs_throttled_seconds_total` metric. It is a frequent cause of "the app is slow but everything looks healthy".

Q. 58. What is a static pod?

A pod defined by a manifest file on a node and managed directly by the kubelet rather than the API server. Control-plane components are often run this way, which is why they exist before the API server does. You cannot delete one with `kubect1` - you remove the manifest file.

Q. 59. What is the difference between a ReplicaSet and a ReplicationController?

The ReplicationController is the original object; the ReplicaSet replaced it and supports set-based label selectors. In practice you do not create either directly - Deployments manage ReplicaSets for you, which is what gives you rollouts and rollbacks.

Q. 60. How do you give a pod access to AWS resources?

IAM Roles for Service Accounts on EKS - the ServiceAccount is annotated with a role ARN, and the pod receives a projected token it exchanges for temporary AWS credentials. No access keys are stored anywhere. The older approaches - node instance profiles shared by every pod, or mounted access keys - are both worse because they over-grant or create long-lived secrets.

Q. 61. What is an admission controller?

A plugin in the API server request path that can validate or mutate objects before they are persisted - adding default labels, injecting sidecars, rejecting privileged pods. Policy engines such as OPA Gatekeeper and Kyverno are implemented as admission webhooks, which is how organisations enforce rules like "every image must come from our registry".

Q. 62. How does Kubernetes handle secrets rotation?

Not automatically for environment variables - those are fixed at pod start, so rotation needs a rolling restart. Mounted secret volumes are refreshed periodically, so an application that re-reads the file can pick up a new value. The cleanest answer is an external secret store with a CSI driver or operator handling rotation and triggering a rollout.

Q. 63. What is the Container Network Interface (CNI)?

The plugin interface Kubernetes uses to set up pod networking - assigning IPs and wiring the pod to the network. Calico, Cilium, Flannel and the AWS VPC CNI are implementations. The choice matters for whether NetworkPolicy is enforced, how IPs are allocated, and performance. On EKS the VPC CNI gives pods real VPC IPs, which means pod density is limited by ENI capacity per instance type.

Q. 64. What is the difference between kube-proxy iptables and IPVS mode?

iptables mode creates a chain of rules per Service, which is simple but scales poorly - rule evaluation is linear and updates get slow with thousands of Services. IPVS uses the kernel's load balancer with hash tables, giving better performance at scale and more balancing algorithms. Most clusters run iptables mode and it is fine until you have a very large number of Services.

Q. 65. How would you perform a canary deployment in Kubernetes?

The simple way is two Deployments behind one Service - a stable one with nine replicas and a canary with one - so roughly 10% of traffic hits the canary, and you watch its error rate and latency before scaling up. Proper traffic splitting needs an ingress controller with canary annotations or a service mesh, which lets you set an exact percentage independent of replica counts and shift it gradually. Either way, the deciding factor is having per-version metrics.

Q. 66. What are the most common mistakes you see in Kubernetes manifests?

No resource requests or limits; no readiness probe, so traffic hits pods that are not ready; liveness probes that check dependencies; mutable image tags; secrets in plain manifests committed to Git; a single replica with no PodDisruptionBudget; and no anti-affinity, so every replica lands on one node and one node failure is a full outage.

Q. 67. How do you find out why a pod was killed?

`kubectl describe pod` shows Last State with a reason and exit code; `kubectl get events` shows eviction and OOM events with timestamps; and node-level `dmesg` confirms a kernel OOM kill. If the pod object is already gone, the events and your monitoring system are all you have - which is a good argument for shipping events to your logging stack.

Q. 68. What is the difference between eviction and deletion?

Deletion is a direct API call removing the pod. Eviction goes through the eviction API and respects PodDisruptionBudgets, which is what `kubectl drain` uses. The kubelet also evicts pods under node resource pressure, choosing victims by QoS class - BestEffort first.

Q. 69. How do you monitor a Kubernetes cluster?

Prometheus scraping kube-state-metrics for object state, node_exporter for node metrics and cAdvisor through the kubelet for container metrics, with Grafana for dashboards and Alertmanager for alerts. Key signals are pod restart rate, pods not Ready, CPU throttling, memory approaching limits, node conditions, PVC usage and rollout status. Logs go to a central store, because pod logs disappear with the pod.

Q. 70. If I gave you a cluster where one application is intermittently failing, how would you approach it?

I would first establish the pattern - which pods, which nodes, what times, and whether it correlates with deploys or load. `kubectl get pods -o wide` to see whether failures cluster on one node, which points at a node problem rather than the application. Then restart counts and `--previous` logs for the failing pods, and events for OOM or eviction. Then resource usage against limits, looking for memory creeping up or CPU throttling. Then dependencies - DNS resolution time, database latency - because intermittent failures are very often a dependency or a timeout rather than the application itself. Intermittent almost always means resource pressure, a bad node, or a dependency, and the pattern tells you which.

PART 11 RECALL POINTS

- Controllers reconcile desired versus actual - that is the whole model.
- `kubectl describe pod` and `kubectl logs --previous` solve most pod problems.
- `kubectl get endpoints` is the first Service debugging command.
- Readiness controls traffic, liveness controls restarts - never check dependencies in liveness.
- Requests schedule, limits enforce; memory over limit = OOMKilled, CPU over limit = throttled.
- 137 = OOMKilled; Pending = read the scheduler's message.
- Secrets are base64, not encrypted.
- ConfigMap env vars need a rollout restart; mounted files update in place.
- RWO means one NODE, not one pod.
- Mutable tags + `IfNotPresent` = old version still serving.

PART 12

PROMETHEUS AND GRAFANA

Metrics, scraping, PromQL, dashboards, alerting

Your CV says you implemented monitoring with Prometheus, Grafana and CloudWatch to track application, infrastructure and service health. Interviewers will test three things: how Prometheus actually collects data, whether you can write a PromQL query, and whether you understand what makes an alert useful rather than noisy.

12.1 Metrics and Time Series

Term	Meaning
Metric	A named measurement, e.g. <code>http_requests_total</code>
Label	A key-value dimension, e.g. <code>{method="GET", status="200", instance="web01"}</code>
Time series	A unique combination of metric name AND label values, with timestamped samples
Sample	One value at one timestamp
Cardinality	The number of distinct time series - the single biggest scaling concern

CARDINALITY IS HOW YOU BREAK PROMETHEUS

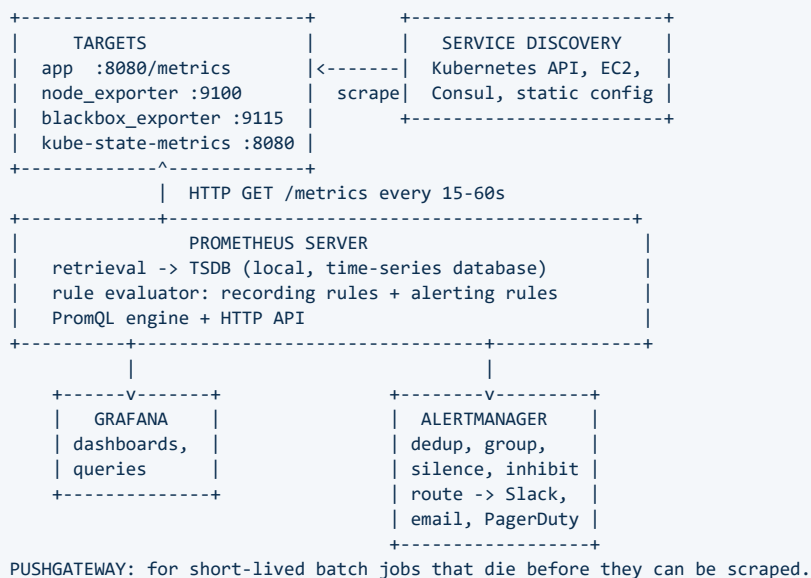
Every unique combination of label values creates a separate time series stored in memory. A label like `user_id`, `request_id`, `session_id` or a raw URL with IDs in it produces millions of series and will exhaust memory on the Prometheus server. Labels must be bounded and low-cardinality: `method`, `status code`, `endpoint TEMPLATE (/users/:id, not /users/12345)`, `environment`, `instance`. Saying this unprompted is a strong signal.

Metric type	What it does	Example	Query with
Counter	Only ever increases (resets to 0 on restart)	<code>http_requests_total</code> , <code>node_network_receive_bytes_total</code>	<code>rate()</code> / <code>increase()</code> - never use the raw value
Gauge	Goes up and down	<code>node_memory_MemAvailable_bytes</code> , <code>kube_deployment_status_replicas</code>	Directly, or <code>avg_over_time()</code>
Histogram	Samples bucketed into configurable ranges, plus sum and count	<code>http_request_duration_seconds_bucket</code>	<code>histogram_quantile()</code>
Summary	Client-computed quantiles	<code>rpc_duration_seconds{quantile="0.99"}</code>	Read directly; cannot be aggregated across instances

COUNTER VERSUS GAUGE, AND WHY `RATE()` EXISTS

"A counter only goes up, so its absolute value is meaningless - what matters is how fast it is increasing. `rate(http_requests_total[5m])` gives per-second requests averaged over 5 minutes, and it correctly handles counter resets when a process restarts. A gauge is a current value like memory in use, so you read it directly. The most common PromQL mistake is graphing a counter raw and wondering why it is a straight line going up."

12.2 Prometheus Architecture



Pull-based collection

Component	Role
Prometheus server	Scrapes targets, stores samples in its local TSDB, evaluates rules, serves PromQL
Exporter	Translates something's state into Prometheus metrics - node_exporter for hosts, mysqld_exporter, blackbox_exporter for probes
Instrumentation	Metrics exposed directly by your application via a client library
kube-state-metrics	Exposes Kubernetes OBJECT state - desired vs ready replicas, pod phase, restarts
Alertmanager	Receives alerts from Prometheus and handles grouping, deduplication, silencing, inhibition and routing
Pushgateway	Accepts pushed metrics from short-lived jobs. Use sparingly - it becomes a stale-metric trap
Grafana	Visualisation across Prometheus, CloudWatch, Loki and other data sources

PULL VERSUS PUSH - A STANDARD QUESTION

"Prometheus pulls: it scrapes an HTTP endpoint on each target on a schedule. The advantages are that the scrape itself tells you whether the target is up - the up metric is free health checking - you control the collection rate centrally, and targets do not need to know where Prometheus is. The disadvantage is short-lived jobs that finish before a scrape, which is what Pushgateway exists for, and that Prometheus must be able to reach every target on the network."

12.3 Configuration

prometheus.yml

```

global:
  scrape_interval: 15s
  evaluation_interval: 15s
  external_labels:
    cluster: prod-eu-west-1

alerting:
  alertmanagers:
    - static_configs:
        - targets: ["alertmanager:9093"]

rule_files:
  - /etc/prometheus/rules/*.yaml

scrape_configs:
  - job_name: prometheus
    static_configs:
      - targets: ["localhost:9090"]

  - job_name: node
  
```

```

static_configs:
  - targets: ["10.0.11.10:9100", "10.0.11.11:9100"]
    labels:
      env: production

- job_name: application
  metrics_path: /metrics
  scrape_interval: 10s
  static_configs:
    - targets: ["payments-api:8080"]

- job_name: kubernetes-pods
  kubernetes_sd_configs:
    - role: pod
  relabel_configs:
    # only scrape pods with prometheus.io/scrape: "true"
    - source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_scrape]
      action: keep
      regex: "true"
    # use the port from the annotation
    - source_labels: [__address__, __meta_kubernetes_pod_annotation_prometheus_io_port]
      action: replace
      regex: ([^:]+)(?::\d+)?(\d+)
      replacement: $1:$2
      target_label: __address__
    # copy useful metadata onto the series as labels
    - source_labels: [__meta_kubernetes_namespace]
      target_label: namespace
    - source_labels: [__meta_kubernetes_pod_name]
      target_label: pod

```

12.4 PromQL

Selectors and the basics

up	# 1 if the last scrape succeeded, 0 if it failed
up{job="node"}	# label filter
up{job=~"node application"}	# regex match
up{env!="dev"}	# negative match
node_memory_MemAvailable_bytes	# instant vector: current value per series
node_memory_MemAvailable_bytes[5m]	# range vector: all samples in the last 5 minutes
node_memory_MemAvailable_bytes offset 1h	# the value one hour ago

Rate, aggregation and the queries you will actually be asked for

```

# Requests per second over 5 minutes
rate(http_requests_total[5m])

# Total RPS across all instances
sum(rate(http_requests_total[5m]))

# RPS broken down by status code
sum by (status) (rate(http_requests_total[5m]))

# ERROR RATE as a percentage of all requests <- the single most useful alert query
100 * sum(rate(http_requests_total{status=~"5.."}[5m]))
    / sum(rate(http_requests_total[5m]))

# p95 latency from a histogram
histogram_quantile(0.95, sum by (le) (rate(http_request_duration_seconds_bucket[5m])))

# p99 latency per endpoint
histogram_quantile(0.99,
  sum by (le, endpoint) (rate(http_request_duration_seconds_bucket[5m])))

# Average request duration (sum / count - cheaper than a quantile)
rate(http_request_duration_seconds_sum[5m]) / rate(http_request_duration_seconds_count[5m])

```

Infrastructure queries

```

# CPU utilisation % per instance (100 minus the idle time)
100 - (avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)

```

```
# Memory used %
100 * (1 - node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes)

# Disk used % for real filesystems
100 - (node_filesystem_avail_bytes{fstype!~"tmpfs|overlay"}
      / node_filesystem_size_bytes{fstype!~"tmpfs|overlay"} * 100)

# Predict disk full within 4 hours, based on the last 6 hours of trend
predict_linear(node_filesystem_avail_bytes{mountpoint="/" }[6h], 4*3600) < 0

# Network receive throughput in bits per second
rate(node_network_receive_bytes_total{device!~"lo|veth.*"}[5m]) * 8

# Load average relative to core count
node_load5 / count by (instance) (node_cpu_seconds_total{mode="idle"})
```

Kubernetes queries

```
# Pod restarts in the last hour
increase(kube_pod_container_status_restarts_total[1h]) > 0

# Pods not in Running phase
kube_pod_status_phase{phase!="Running"} == 1

# Container memory usage as a fraction of its limit
container_memory_working_set_bytes{container!=""}
  / on (pod, container) kube_pod_container_resource_limits{resource="memory"}

# CPU throttling - the invisible latency cause
rate(container_cpu_cfs_throttled_seconds_total[5m]) > 0

# Deployment replica mismatch (rollout stuck or pods failing)
kube_deployment_status_replicas_available != kube_deployment_spec_replicas

# Nodes not Ready
kube_node_status_condition{condition="Ready", status="true"} == 0
```

PromQL function	Use
rate(v[5m])	Per-second average increase of a counter, reset-aware
irate(v[5m])	Instantaneous rate from the last two samples - spiky, for fast-moving graphs only
increase(v[1h])	Total increase over the window - "how many restarts in the last hour"
sum, avg, min, max, count	Aggregations, with by (label) or without (label)
topk(5, v) / bottomk	The five largest series
histogram_quantile(0.95, ...)	Percentile from histogram buckets
predict_linear(v[6h], 3600)	Linear extrapolation - disk-full prediction
avg_over_time(v[1h])	Average of a gauge over a window
absent(v)	1 if the series does not exist - alert when a target disappears entirely
delta(v[1h])	Difference for gauges
changes(v[1h])	How many times a gauge changed

CHOOSING THE RATE WINDOW

The window must be at least four times the scrape interval so a counter reset or a missed scrape does not distort the result - with a 15-second scrape interval, [1m] is the practical minimum and [5m] is the usual choice. Too short and the graph is noisy; too long and you smooth away the incident you are trying to see.

12.5 Alerting

Alert rules that are actually useful

```
groups:
- name: availability
  rules:
    - alert: TargetDown
```

```

expr: up == 0
for: 2m
labels: { severity: critical }
annotations:
  summary: "{{ $labels.job }}" target "{{ $labels.instance }}" is down"
  runbook: "https://wiki.internal/runbooks/target-down"

- alert: HighErrorRate
  expr: |
    100 * sum by (service) (rate(http_requests_total{status=~"5.."}[5m]))
      / sum by (service) (rate(http_requests_total[5m])) > 2
  for: 5m
  labels: { severity: critical }
  annotations:
    summary: "{{ $labels.service }}" 5xx rate is {{ $value | printf "%.1f" }}%"

- alert: HighLatency
  expr: |
    histogram_quantile(0.95,
      sum by (le, service) (rate(http_request_duration_seconds_bucket[5m]))) > 1
  for: 10m
  labels: { severity: warning }

- name: capacity
  rules:
    - alert: DiskWillFillIn4Hours
      expr: predict_linear(node_filesystem_avail_bytes{mountpoint="/"}[6h], 4*3600) < 0
      for: 15m
      labels: { severity: warning }
      annotations:
        summary: "{{ $labels.instance }}" root filesystem will fill within 4 hours"

    - alert: MemoryPressure
      expr: 100 * (1 - node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes) > 90
      for: 10m
      labels: { severity: warning }

- name: kubernetes
  rules:
    - alert: PodCrashLooping
      expr: increase(kube_pod_container_status_restarts_total[15m]) > 3
      for: 5m
      labels: { severity: critical }
      annotations:
        summary: "{{ $labels.namespace }}" / "{{ $labels.pod }}" restarted {{ $value }} times in 15m"

    - alert: DeploymentReplicasMismatch
      expr: kube_deployment_status_replicas_available != kube_deployment_spec_replicas
      for: 15m
      labels: { severity: warning }

```

Alert design rule	Why
Alert on symptoms users feel, not on causes	"Error rate above 2%" is actionable; "CPU 85%" often is not
Always use for:	Prevents a transient spike paging someone at 3am
Every alert links a runbook	The person woken up should not have to invent a procedure
Severity tiers: page vs ticket	Only page for things that need a human right now
Predict where you can	predict_linear on disk gives hours of warning instead of an outage
Alert on absence too	up == 0 and absent() catch the target that stopped reporting entirely
Review and delete noisy alerts	Alert fatigue is how real incidents get ignored

Alertmanager routing

```

route:
  receiver: slack-default
  group_by: [alertname, cluster, service]
  group_wait: 30s           # wait to batch related alerts from the same incident
  group_interval: 5m
  repeat_interval: 4h

```

```

routes:
- matchers: [severity="critical"]
  receiver: pagerduty
  continue: true
- matchers: [severity="warning"]
  receiver: slack-default

inhibit_rules:
# If the whole target is down, do not also page about its error rate
- source_matchers: [alertname="TargetDown"]
  target_matchers: [severity="warning"]
  equal: [instance]

receivers:
- name: slack-default
  slack_configs:
    - channel: "#ops-alerts"
      title: "{{ .CommonLabels.alertname }}"
      text: "{{ range .Alerts }}{{ .Annotations.summary }}\n{{ end }}"
- name: pagerduty
  pagerduty_configs:
    - service_key: "<key>"

```

WHAT ALERTMANAGER ADDS

"Prometheus decides WHEN an alert fires; Alertmanager decides WHO hears about it and HOW. It groups related alerts so one incident is one notification instead of fifty, deduplicates alerts from multiple Prometheus servers, applies silences during planned maintenance, and uses inhibition so a cause alert suppresses its symptom alerts - if a node is down, you do not also want twenty alerts about the pods on it."

12.6 Grafana

Concept	Detail
Data source	Prometheus, CloudWatch, Loki, Elasticsearch, SQL - one dashboard can mix them
Panel	One visualisation: time series, stat, gauge, table, heatmap, logs
Variable	A dashboard dropdown, e.g. \$namespace, \$instance - built from a label_values() query
Dashboard as code	JSON in Git, provisioned automatically - dashboards built by clicking are lost when the pod restarts
Annotations	Mark deploys on a graph - the fastest way to correlate an incident with a release
Alerting	Grafana has its own alerting, but with Prometheus the usual choice is Prometheus rules plus Alertmanager

	Prometheus	Grafana
What it is	Metrics collection, storage and alerting	Visualisation and dashboarding
Stores data	Yes, its own TSDB	No - it queries data sources
Query language	PromQL	Uses each data source's language
Alerting	Yes, rules plus Alertmanager	Yes, its own engine
Can replace the other	No	No - they are complementary

THE ONE-LINE ANSWER

"Prometheus collects and stores the metrics and evaluates alert rules; Grafana is the visualisation layer that queries Prometheus and displays it. They solve different problems - people ask this to check you are not treating them as alternatives."

12.7 What to Monitor

Layer	Signals
Host	CPU, memory available, disk used and inode used, disk I/O wait, network throughput and errors, load average, uptime

Layer	Signals
Container / pod	CPU throttling, memory working set vs limit, restart count, pod phase, OOM events
Application	Request rate, error rate, latency percentiles, saturation of thread and connection pools, queue depth, business metrics
Dependencies	Database connections and query latency, cache hit rate, external API latency and error rate
Pipeline	Build duration, failure rate, deploy frequency, rollback count
Certificates	Days until expiry - via blackbox_exporter; a genuinely common outage cause

MONITORING TROUBLESHOOTING

- **Target down in Prometheus but the app works:** the scrape endpoint is wrong, the port is not exposed, a network policy blocks Prometheus, or the metrics path is not `/metrics`. Check the Targets page - it shows the exact scrape error.
- **No data in a Grafana panel:** first run the query in Prometheus directly. If it returns data there, it is a Grafana data source, time range or variable problem, not a metrics problem.
- **Prometheus using huge memory / OOMKilled:** cardinality. Find the worst offenders with `topk(10, count by (__name__)(__name__=~".+"))` and drop high-cardinality labels with relabelling.
- **Gaps in graphs:** scrapes failing intermittently - check scrape duration against the interval, and target restarts.
- **Alerts firing but nobody notified:** Alertmanager routing, a silence still active, or a broken receiver. Check the Alertmanager UI, not Prometheus.
- **Counter graph is a straight line up:** you forgot `rate()`.

12.8 Monitoring Interview Questions

Q. 1. What is Prometheus and how does it collect data?

A time-series database and monitoring system that PULLS metrics by scraping an HTTP endpoint, usually `/metrics`, on each target at a fixed interval, and stores them in a local TSDB. Targets are found through static configuration or service discovery - the Kubernetes API, EC2, Consul.

Q. 2. Why pull instead of push?

The scrape itself is a health check - the up metric tells you whether the target responded. Collection frequency is controlled centrally, targets do not need to know about the monitoring system, and it is easy to run a second Prometheus for testing without affecting anything. The gap is short-lived jobs, which is what Pushgateway covers.

Q. 3. What is the difference between a counter and a gauge?

A counter only increases and resets to zero on restart, so you always query it with `rate()` or `increase()` - the absolute value is meaningless. A gauge goes up and down and represents a current value, so you read it directly.

Q. 4. What is a histogram and how do you get a percentile from it?

A histogram counts observations into cumulative buckets plus a sum and a count. `histogram_quantile(0.95, sum by (le) (rate(..._bucket[5m])))` estimates the 95th percentile. The key advantage over a summary is that histograms can be aggregated across instances, because the buckets are additive - summaries compute quantiles client-side and cannot be meaningfully averaged.

Q. 5. What is cardinality and why does it matter?

The number of distinct time series, which is the product of all label value combinations. Each series consumes memory, so a high-cardinality label such as user ID or request ID can create millions of series and take the server down. Labels must be bounded - method, status, endpoint template, environment.

Q. 6. Write a query for error rate as a percentage.

`100 * sum(rate(http_requests_total{status=~"5.."}[5m])) / sum(rate(http_requests_total[5m]))`. I aggregate numerator and denominator the same way, and I usually add by `(service)` so the alert names the affected service.

Q. 7. How do you monitor CPU usage with node_exporter?

`node_exporter` exposes `node_cpu_seconds_total` per mode, so you compute usage as the inverse of idle: `100 - (avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)`. There is no direct "CPU percent" metric because the counter model gives you the raw seconds and you derive the rate.

Q. 8. What does the `for:` clause do in an alert rule?

The condition must remain true continuously for that duration before the alert fires. It filters out transient spikes. Without it, a 20-second CPU blip pages someone, and the alert becomes noise people learn to ignore.

Q. 9. What does Alertmanager do that Prometheus does not?

Routing and notification management - grouping related alerts into one notification, deduplicating across Prometheus replicas, silencing during maintenance, inhibiting symptom alerts when a cause alert is firing, and delivering to the right receiver by severity or team.

Q. 10. What is the difference between Prometheus and Grafana?

Prometheus collects, stores and alerts; Grafana visualises by querying data sources. Grafana stores no metrics of its own. They are complementary, not alternatives.

Q. 11. How do you monitor a short-lived batch job?

Pushgateway - the job pushes its metrics before exiting, and Prometheus scrapes the gateway. The caveat is that Pushgateway never forgets a metric, so a job that stops running leaves a stale value that looks fine forever; you need a deletion strategy or a separate "last success timestamp" alert.

Q. 12. How do you alert on a job that failed to run at all?

On the age of its last success: `time() - job_last_success_timestamp_seconds > 86400`. Alerting on failure only catches the job when it runs - the dangerous case is when it stops running entirely, and only an absence or staleness check catches that. `absent()` serves the same purpose for a missing series.

Q. 13. What are the four golden signals?

Latency, traffic, errors and saturation. They are the minimum set for a user-facing service, and importantly latency should be measured separately for successful and failed requests, because fast errors otherwise make your latency graph look healthy.

Q. 14. What is the USE method?

For each resource, check Utilisation, Saturation and Errors. It is the counterpart to the golden signals - golden signals for services, USE for resources like CPU, memory, disk and network.

Q. 15. How would you monitor certificate expiry?

`blackbox_exporter` probing the endpoint exposes `probe_ssl_earliest_cert_expiry`, and you alert when it is less than, say, 21 days away. It is worth calling out because expired certificates cause outages that are completely predictable and therefore inexcusable.

Q. 16. How do you monitor Kubernetes with Prometheus?

`kube-state-metrics` for object state - replicas desired versus available, pod phase, restart counts; `cAdvisor` via the kubelet for container CPU and memory; `node_exporter` as a DaemonSet for node metrics; and the applications' own instrumentation scraped via pod annotations. The `kube-prometheus-stack` Helm chart wires all of that together with default dashboards and alerts.

Q. 17. What is a recording rule?

A rule that precomputes an expensive query at evaluation time and stores it as a new series. It makes dashboards fast and keeps alert expressions simple, and it is the standard fix when a dashboard with heavy aggregations takes ten seconds to load.

Q. 18. How long does Prometheus keep data and what if you need longer?

Local retention defaults to about 15 days and is bounded by disk. For long-term storage you use remote write to Thanos, Cortex, Mimir or a managed service, which also solves global querying across multiple Prometheus servers and downsampling.

Q. 19. Is Prometheus highly available?

Not by clustering - the usual approach is running two identical Prometheus servers scraping the same targets, with Alertmanager deduplicating their alerts. For a single global view and long retention you put Thanos or Mimir in front. A single Prometheus is a single point of failure for alerting, which is worth acknowledging.

Q. 20. How do you reduce alert noise?

Alert on symptoms rather than causes, use `for:` durations, group related alerts in Alertmanager, add inhibition so a node-down alert suppresses its pod alerts, set severity tiers so only genuine emergencies page, and regularly review which alerts fired and what action was actually taken - any alert that never leads to action should be deleted or demoted to a dashboard.

Q. 21. What is the `up` metric?

A metric Prometheus generates itself for every scrape: 1 if the scrape succeeded, 0 if it failed. It is the simplest and most important alert you can have, and it is free - you do not have to instrument anything for it.

Q. 22. How do you find what is consuming memory in Prometheus?

The `/status` page and TSDB stats show the highest-cardinality metrics and labels, and `topk(10, count by (__name__){__name__=~".+"})` finds the worst metric names. The fix is usually dropping a label with `metric_relabel_configs` at scrape time rather than asking the application team to change the code immediately.

Q. 23. What would you put on an application dashboard?

Top row: request rate, error rate and p95/p99 latency - the signals that tell you whether users are affected. Second row: saturation - CPU, memory against limits, connection pool usage, queue depth. Third row: dependencies - database latency, cache hit rate, downstream error rates. Plus deploy annotations, so the first question in any incident - "did we just release?" - is answered by looking at the graph.

Q. 24. What is the difference between `rate` and `irate`?

`rate` averages over the whole window, giving a smooth line suitable for alerting. `irate` uses only the last two samples, so it is very responsive but spiky. The rule is `rate` for alerts and most graphs, `irate` only for high-resolution graphs of fast-moving counters.

Q. 25. How do monitoring and logging differ, and when do you use each?

Metrics are numeric, cheap, aggregatable and answer "is something wrong and how bad" - they are what you alert on. Logs are detailed events and answer "what exactly happened" - they are what you investigate with. Metrics get you to the right service in seconds; logs get you to the root cause. You need both, and in practice I would start from a dashboard and end in the logs.

PART 12 RECALL POINTS

- Prometheus pulls; up is free health checking.
- Counters need `rate()`; gauges are read directly.
- Cardinality kills Prometheus - never label with IDs.
- `histogram_quantile(0.95, sum by (le) (rate(..._bucket[5m])))`.
- `100 * sum(rate(5xx)) / sum(rate(all))` is the error-rate alert.
- `for:` prevents flapping; every alert needs a runbook.
- Alertmanager groups, dedupes, silences, inhibits and routes.
- Alert on absence, not just failure.
- Prometheus stores, Grafana displays.

PART 13

AWS CLOUDWATCH

Metrics, logs, alarms, the agent, and where it fits alongside Prometheus

CloudWatch appears in both roles on your CV, alongside Prometheus and Grafana. The question you will be asked is not "what is CloudWatch" but "you already have Prometheus - why do you also use CloudWatch?" This part gives you that answer and the operational detail behind it.

13.1 The Components

Component	What it is	Typical use in your stack
Metrics	Time-series data, namespaced, with up to 30 dimensions	EC2 CPU, ALB request count and target response time, RDS connections
Custom metrics	Metrics you publish yourself	Memory and disk from the agent; application counters
Logs	Log groups containing log streams of events	Application logs, system logs, VPC flow logs, Lambda output
Log group	A container with a retention setting and optional KMS encryption	/aws/app/payments, /var/log/messages
Log stream	A sequence of events from one source, usually one instance or container	One per instance ID
Metric filter	Turns a log pattern into a metric	Count of ERROR lines, which you can then alarm on
Alarm	Threshold evaluation over a metric with actions	Notify SNS, scale an ASG, recover an instance
Composite alarm	Combines several alarms with AND/OR	Reduce noise: page only when both error rate AND latency are bad
Dashboard	Visualisation, shareable, definable as JSON	Environment overview
Logs Insights	Query language over log groups	Incident investigation
EventBridge	Event bus reacting to AWS state changes	Trigger automation when an instance changes state
Synthetics Canary	Scripted checks from outside	End-user-perspective availability
Container Insights	Curated metrics and dashboards for ECS/EKS	Cluster and pod-level metrics without running Prometheus

WHAT CLOUDWATCH CAN AND CANNOT SEE BY DEFAULT

CloudWatch collects EC2 metrics from the hypervisor, which sees CPU, network and disk I/O (operations and bytes) - but NOT memory usage, NOT disk space used, and NOT per-process information, because those live inside the guest OS. Publishing them requires the CloudWatch agent. This is one of the most frequently asked AWS monitoring questions and the reason so many teams discover they have no memory metrics during an incident.

13.2 Key Metrics by Service

Service	Metric	Why it matters
EC2	CPUUtilization	Load; note that on t-family instances you must also watch CPUCreditBalance
EC2	StatusCheckFailed_System / _Instance	System = AWS hardware (stop/start to migrate); Instance = inside the OS
EC2	NetworkIn / NetworkOut	Traffic, and unexpected egress
EBS	VolumeQueueLength, BurstBalance	I/O saturation; an exhausted burst balance on gp2 looks exactly like an application slowdown

Service	Metric	Why it matters
ALB	RequestCount, TargetResponseTime	Traffic and latency
ALB	HTTPCode_Target_5XX_Count	Errors the APPLICATION produced
ALB	HTTPCode_ELB_5XX_Count	Errors the LOAD BALANCER produced - usually no healthy targets
ALB	HealthyHostCount / UnHealthyHostCount	The single most useful availability metric
ALB	RejectedConnectionCount	Capacity limits reached
RDS	CPUUtilization, DatabaseConnections	Saturation and connection leaks
RDS	FreeableMemory, FreeStorageSpace	Both cause outages when exhausted
RDS	ReadLatency / WriteLatency, ReplicaLag	Performance and replication health
ASG	GroupInServiceInstances vs GroupDesiredCapacity	A persistent gap means instances are failing to come into service
S3	BucketSizeBytes, NumberOfObjects	Daily granularity - for cost and growth, not real time
Lambda	Duration, Errors, Throttles, ConcurrentExecutions	Throttles are the one people miss

ELB 5XX VERSUS TARGET 5XX - A GREAT ANSWER

"HTTPCode_Target_5XX_Count means the application returned an error - I go to the application logs. HTTPCode_ELB_5XX_Count means the load balancer itself generated the error, which almost always means there were no healthy targets or the target connection failed - so I go to the target group health, not the application. Separating those two on a dashboard saves a lot of time in an incident."

13.3 The CloudWatch Agent

amazon-cloudwatch-agent.json - metrics AND logs

```
{
  "agent": {
    "metrics_collection_interval": 60,
    "run_as_user": "cwagent"
  },
  "metrics": {
    "namespace": "CustomEC2",
    "append_dimensions": {
      "InstanceId": "${aws:InstanceId}",
      "AutoScalingGroupName": "${aws:AutoScalingGroupName}"
    },
    "aggregation_dimensions": [ ["AutoScalingGroupName"], ["InstanceId"] ],
    "metrics_collected": {
      "mem": {
        "measurement": ["mem_used_percent", "mem_available_mb"]
      },
      "disk": {
        "measurement": ["used_percent", "inodes_free"],
        "resources": ["/", "/var"],
        "ignore_file_system_types": ["sysfs", "devtmpfs", "tmpfs", "overlay"]
      },
      "diskio": {
        "measurement": ["io_time", "read_bytes", "write_bytes"]
      },
      "swap": { "measurement": ["swap_used_percent"] },
      "procstat": [
        { "exe": "nginx", "measurement": ["cpu_usage", "memory_rss", "pid_count"] }
      ]
    }
  },
  "logs": {
    "logs_collected": {
      "files": {
        "collect_list": [
          {
            "file_path": "/var/log/payments-api/app.log",
            "log_group_name": "/aws/app/payments",

```

```

        "log_stream_name": "{instance_id}",
        "timestamp_format": "%Y-%m-%d %H:%M:%S",
        "multi_line_start_pattern": "^\\d{4}-\\d{2}-\\d{2}",
        "retention_in_days": 30
    },
    {
        "file_path": "/var/log/messages",
        "log_group_name": "/aws/ec2/system",
        "log_stream_name": "{instance_id}"
    }
  ]
}
}
}
}
}

```

Installing and running the agent

```

sudo dnf install -y amazon-cloudwatch-agent
sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl \
    -a fetch-config -m ec2 -s -c file:/opt/aws/amazon-cloudwatch-agent/etc/config.json
sudo systemctl status amazon-cloudwatch-agent
sudo tail -f /opt/aws/amazon-cloudwatch-agent/logs/amazon-cloudwatch-agent.log # first debug stop

```

THE AGENT NEEDS AN IAM ROLE

The instance profile requires CloudWatchAgentServerPolicy - specifically cloudwatch:PutMetricData, logs:CreateLogGroup, logs:CreateLogStream, logs:PutLogEvents and ec2:DescribeTags. "The agent is installed but no metrics appear" is nearly always either a missing IAM permission or the agent config not being loaded - and the agent's own log file tells you which.

13.4 Alarms

Creating the alarms that matter

```

# Application errors - symptom-based
aws cloudwatch put-metric-alarm \
  --alarm-name payments-5xx-high \
  --namespace AWS/ApplicationELB --metric-name HTTPCode_Target_5XX_Count \
  --dimensions Name=LoadBalancer,Value=app/payments-alb/abc123 \
  --statistic Sum --period 300 --evaluation-periods 2 --threshold 50 \
  --comparison-operator GreaterThanThreshold \
  --treat-missing-data notBreaching \
  --alarm-actions arn:aws:sns:eu-west-1:123456789012:ops-critical

# No healthy targets - the availability alarm
aws cloudwatch put-metric-alarm \
  --alarm-name payments-no-healthy-hosts \
  --namespace AWS/ApplicationELB --metric-name HealthyHostCount \
  --dimensions Name=TargetGroup,Value=targetgroup/payments/abc \
    Name=LoadBalancer,Value=app/payments-alb/abc123 \
  --statistic Minimum --period 60 --evaluation-periods 2 --threshold 1 \
  --comparison-operator LessThanThreshold \
  --treat-missing-data breaching \
  --alarm-actions arn:aws:sns:eu-west-1:123456789012:ops-critical

# Disk from the agent's custom metric
aws cloudwatch put-metric-alarm \
  --alarm-name web-disk-80 \
  --namespace CustomEC2 --metric-name disk_used_percent \
  --dimensions Name=InstanceId,Value=i-0abc123 Name=path,Value=/ \
  --statistic Average --period 300 --evaluation-periods 2 --threshold 80 \
  --comparison-operator GreaterThanThreshold \
  --alarm-actions arn:aws:sns:eu-west-1:123456789012:ops-warning

# Auto-recover an instance that fails its SYSTEM status check
aws cloudwatch put-metric-alarm \
  --alarm-name i-0abc123-system-recover \
  --namespace AWS/EC2 --metric-name StatusCheckFailed_System \
  --dimensions Name=InstanceId,Value=i-0abc123 \
  --statistic Maximum --period 60 --evaluation-periods 2 --threshold 1 \

```

```
--comparison-operator GreaterThanOrEqualToThreshold \
--alarm-actions arn:aws:automate:eu-west-1:ec2:recover
```

Alarm setting	Meaning	Getting it wrong
Period	The aggregation window of each data point	Too long and you find out late; too short and it flaps
Evaluation periods	How many periods are assessed	1 period means a single blip pages you
Datapoints to alarm	M out of N periods must breach	The tool for tolerating occasional spikes
Statistic	Sum, Average, Minimum, Maximum, percentile	Average hides spikes; use Maximum or p99 for latency
treat-missing-data	breaching, notBreaching, ignore, missing	For HealthyHostCount, missing data should be BREACHING - no data often means everything is gone
Actions	SNS, Auto Scaling, EC2 recover	An alarm with no action is just a coloured square on a dashboard

THE MISSING-DATA TRAP

The default for missing data is `missing`, which keeps the alarm in its previous state. If a service dies so completely that it stops publishing metrics, an alarm configured this way never fires - your monitoring goes quiet exactly when the outage is worst. For availability metrics, set `treat-missing-data` to `breaching`.

13.5 Logs Insights

Queries to have ready

```
# Recent errors
fields @timestamp, @message
| filter @message like /ERROR/
| sort @timestamp desc
| limit 100

# Error count per 5-minute bucket - shows when it started
fields @timestamp
| filter @message like /ERROR/
| stats count() as errors by bin(5m)

# Parse structured logs and group by endpoint
fields @timestamp, @message
| parse @message '"status":*, "path":*"' as status, path
| filter status >= 500
| stats count() as errors by path
| sort errors desc

# Slowest requests
fields @timestamp, @message
| parse @message /duration=(?<duration>\d+)/
| filter duration > 2000
| sort duration desc
| limit 50

# ALB access logs: 5xx by target
fields @timestamp, target_status_code, target_ip, request_url
| filter target_status_code >= 500
| stats count() by target_ip

# VPC flow logs: what is being REJECTED - the firewall debugging query
fields @timestamp, srcAddr, dstAddr, dstPort, action
| filter action = "REJECT"
| stats count() as rejects by srcAddr, dstPort
| sort rejects desc
```

Logs from the CLI

```
aws logs tail /aws/app/payments --follow --since 15m
aws logs tail /aws/app/payments --filter-pattern "ERROR"
aws logs describe-log-groups --query "logGroups[].[logGroupName,retentionInDays,storedBytes]" --output table
aws logs put-retention-policy --log-group-name /aws/app/payments --retention-in-days 30
aws logs filter-log-events --log-group-name /aws/app/payments \
```

```
--start-time $(date -d '1 hour ago' +%s000) --filter-pattern "ERROR"
```

LOG GROUPS DEFAULT TO NEVER EXPIRE

Every log group created without an explicit retention policy keeps data forever and you pay for it forever. Setting retention is a five-second action that is routinely forgotten, and CloudWatch Logs storage is a common surprise on an AWS bill. Set it in your IaC so it can never be missed.

A metric filter - turning logs into an alarm

```
aws logs put-metric-filter \
  --log-group-name /aws/app/payments \
  --filter-name app-errors \
  --filter-pattern "ERROR" \
  --metric-transformations \
    metricName=AppErrorCount,metricNamespace=CustomApp,metricValue=1,defaultValue=0
# Then alarm on CustomApp/AppErrorCount as you would any other metric.
```

13.6 CloudWatch Alongside Prometheus

	CloudWatch	Prometheus + Grafana
Covers	AWS-managed services you cannot instrument - ALB, RDS, NAT, Lambda, S3	Applications, containers, anything you can scrape
Model	Push from AWS services and the agent	Pull by scraping
Query	Metric Math and Logs Insights	PromQL - far more expressive
Retention	15 months, automatically rolled up	Local retention is short; long term needs Thanos or similar
Cost	Per metric, per alarm, per GB ingested and stored	Infrastructure you run yourself
Operations	Fully managed - nothing to run	You operate it, including its own availability
Granularity	1 minute standard, 1 second for high resolution	Whatever scrape interval you choose

"YOU HAVE PROMETHEUS - WHY ALSO CLOUDWATCH?"

"Because they cover different things. I cannot install an exporter inside an ALB, an RDS instance or a NAT gateway - those metrics only exist in CloudWatch, and they are exactly the ones that tell me whether the problem is the platform or my application. Prometheus covers everything I run and instrument, with much better query and alerting capability. In practice I point Grafana at both, so one dashboard shows the ALB's target response time and healthy host count from CloudWatch next to the application's own request rate and error rate from Prometheus. There is also a resilience argument: if my cluster is in trouble, Prometheus may be in trouble with it, whereas CloudWatch is managed and still reporting."

13.7 CloudWatch Interview Scenarios

Q. 1. Your EC2 dashboard shows no memory metric. Why, and what do you do?

CloudWatch collects EC2 metrics from the hypervisor, which cannot see inside the guest OS, so memory and disk-space-used are not available by default. I install and configure the CloudWatch agent to publish them as custom metrics, and make sure the instance role has CloudWatchAgentServerPolicy. Ideally the agent is baked into the AMI or installed by user data so it is never forgotten.

Q. 2. An alarm is in INSUFFICIENT_DATA. What does that mean and is it safe?

The metric has not reported enough data points in the evaluation window. It is not safe to treat it as OK - for an availability metric it often means the source has stopped reporting entirely, which is the worst case, not a benign one. That is why I set treat-missing-data to breaching for availability alarms.

Q. 3. How would you alert on application errors that only appear in logs?

Create a metric filter on the log group matching the error pattern, which produces a CloudWatch metric, then alarm on that metric's Sum over a window. That turns unstructured log output into something alertable. If the application can be

instrumented instead, a proper error counter metric is better, because pattern matching on log text is brittle.

Q. 4. CloudWatch costs have risen sharply. Where do you look?

Custom metrics count - every unique metric-plus-dimension combination is charged, so a dimension containing an instance ID or a request ID multiplies cost the same way cardinality does in Prometheus. Then log ingestion volume, especially debug logging left on in production. Then log storage where retention was never set. And high-resolution metrics at one-second granularity, which cost considerably more.

Q. 5. How do you get application logs from containers into CloudWatch?

The awslogs log driver for ECS or plain Docker, or Fluent Bit as a DaemonSet for Kubernetes with the CloudWatch output plugin. The application writes to stdout and the platform ships it - the container should never manage log files itself.

Q. 6. What is a composite alarm and why use one?

An alarm whose condition is a boolean expression over other alarms. It reduces noise - for example, page only when the error-rate alarm AND the healthy-host alarm are both in ALARM, or suppress downstream alarms when an upstream alarm is already firing. It is CloudWatch's equivalent of Alertmanager inhibition.

Q. 7. How would you monitor an ALB properly?

HealthyHostCount as the primary availability signal, with missing data treated as breaching. TargetResponseTime at p95 or p99 rather than average. HTTPCode_Target_5XX and HTTPCode_ELB_5XX as separate metrics, because they point at completely different causes. RequestCount for traffic context, and RejectedConnectionCount for capacity. Plus ALB access logs to S3 and queryable, because that is where you find which client, path and target were involved.

Q. 8. How does CloudWatch integrate with Auto Scaling?

An alarm action can trigger a scaling policy, and target-tracking policies create and manage their own alarms behind the scenes. The important design point is scaling on a metric that reflects real demand - request count per target or a queue depth is usually better than raw CPU, because CPU can be high for reasons that adding instances will not fix.

Q. 9. What is CloudWatch Synthetics and when is it worth it?

Canaries - scripted checks that call your endpoints from outside on a schedule, including multi-step flows like a login. They are worth it because all your internal metrics can look perfect while DNS, a certificate or a CDN configuration makes the service unusable for real users. It is monitoring from the user's side rather than the server's.

Q. 10. How would you investigate "the application was slow between 14:00 and 14:30 yesterday"?

Start at the edge and work in. ALB TargetResponseTime and RequestCount for that window - was it a traffic spike or did latency rise without more traffic? HealthyHostCount to see whether capacity dropped. Then the compute layer - CPU, memory from the agent, and EBS BurstBalance and VolumeQueueLength, because exhausted burst credits look exactly like a slow application. Then RDS - CPU, connections, read and write latency, and slow queries. Then Logs Insights over that window to find slow endpoints and errors. And finally CloudTrail and deployment history to see whether anything changed at 14:00, because that is the most common answer.

PART 13 RECALL POINTS

- No memory or disk-used metrics without the CloudWatch agent.
- The agent needs an IAM role; check its own log first when it is silent.
- ELB 5XX = load balancer / no healthy targets; Target 5XX = the application.
- `treat-missing-data` breaching for availability alarms.
- Log groups never expire unless you set retention - a real cost trap.
- Metric filters turn log patterns into alarmable metrics.
- CloudWatch for managed AWS services, Prometheus for what you run - both into Grafana.

PART 14

PYTHON FOR DEVOPS

Only what you need, plus nine complete scripts you can defend

Your CV says you "developed Python and Bash scripts for deployment support, system checks, log analysis and network troubleshooting". This part teaches DevOps Python - not software engineering Python. Every construct here appears in the scripts, and every script is one you should be able to write on a whiteboard and explain line by line.

14.1 The Language Subset You Actually Need

Variables, types and string handling

```
name = "payments-api"          # str
port = 8080                     # int
threshold = 85.5                # float
enabled = True                  # bool
missing = None                  # NoneType

# f-strings - the only formatting style you need
print(f"{name} is listening on port {port}")
print(f"Disk used: {threshold:.1f}%")      # 85.5
print(f"{'service':<15}{'state':>10}")      # padding for aligned output

# String operations used constantly in log parsing
line = " 2026-09-23 14:02:11 ERROR db timeout after 30s "
line.strip()                     # remove surrounding whitespace
line.split()                     # split on any whitespace -> list of tokens
line.split(maxsplit=3)           # keep the message intact as the last field
"ERROR" in line                  # membership test - the cheapest filter
line.lower().startswith("2026") # case handling
line.replace("ERROR", "ERR")
",".join(["a", "b", "c"])        # "a,b,c"
```

Lists, dicts, sets and comprehensions

```
servers = ["web01", "web02", "db01"]
servers.append("web03")
servers[0]                # "web01"
servers[-1]               # last element
servers[0:2]              # slice
len(servers)
sorted(servers)
"web01" in servers

config = {"host": "10.0.1.5", "port": 5432, "ssl": True}
config["host"]             # KeyError if missing
config.get("timeout", 30)  # safe access with a default - use this in scripts
config["timeout"] = 30
for key, value in config.items():
    print(f"{key} = {value}")

unique_ips = set()         # sets deduplicate automatically
unique_ips.add("10.0.1.5")

# Comprehensions - the most useful Python feature for log work
errors = [l for l in lines if "ERROR" in l]
ips = [l.split()[0] for l in lines]
hosts_up = {h: check(h) for h in servers}
big_files = [f for f in files if f.size > 100_000_000]
```

Conditions, loops and functions

```
used = 87
if used > 90:
    level = "CRITICAL"
elif used > 80:
    level = "WARNING"
```

```

else:
    level = "OK"

for server in servers:
    print(server)

for index, server in enumerate(servers, start=1):
    print(f"{index}. {server}")

for host, port in zip(hosts, ports):
    check(host, port)

attempt = 0
while attempt < 5:
    if probe():
        break
    attempt += 1
else:
    print("never succeeded")    # the else on a while runs if no break happened

def check_disk(path="/", threshold=80):
    """Return (used_percent, is_breach). Docstrings matter in shared scripts."""
    usage = shutil.disk_usage(path)
    pct = usage.used / usage.total * 100
    return pct, pct > threshold

pct, breach = check_disk("/var", threshold=90)

```

Exceptions - the difference between a script and a tool

```

try:
    response = requests.get(url, timeout=5)
    response.raise_for_status()    # raises on 4xx/5xx
except requests.exceptions.Timeout:
    log.error("timed out after 5s")
    sys.exit(2)
except requests.exceptions.ConnectionError as e:
    log.error(f"cannot connect: {e}")
    sys.exit(2)
except requests.exceptions.HTTPError as e:
    log.error(f"http error: {e.response.status_code}")
    sys.exit(1)
except Exception as e:
    # last resort - always log the type
    log.exception(f"unexpected: {type(e).__name__}: {e}")
    sys.exit(3)
else:
    log.info("request succeeded")    # runs only if no exception
finally:
    log.debug("done")    # always runs - cleanup goes here

```

EXIT CODES MAKE A SCRIPT AUTOMATABLE

A script that prints "ERROR" and exits 0 is invisible to cron, Jenkins and Kubernetes - all of which decide success or failure from the exit code alone. Convention: 0 success, 1 a handled failure, 2 a usage or connection error, and distinct codes for distinct conditions so the caller can branch. Always `sys.exit(n)` deliberately.

Files, JSON, CSV

```

# Always use a context manager - the file closes even on exception
with open("/var/log/app/app.log") as f:
    for line in f:
        # streams line by line - safe for huge files
        if "ERROR" in line:
            print(line.rstrip())

with open("report.txt", "w") as f:
    # "w" truncates, "a" appends
    f.write("line one\n")
    f.writelines(["a\n", "b\n"])

import json
with open("config.json") as f:
    config = json.load(f)    # file -> dict
data = json.loads(response.text)    # string -> dict

```

```
print(json.dumps(data, indent=2))          # dict -> pretty string

import csv
with open("servers.csv", newline="") as f:
    for row in csv.DictReader(f):          # each row is a dict keyed by header
        print(row["hostname"], row["environment"])

with open("out.csv", "w", newline="") as f:
    w = csv.DictWriter(f, fieldnames=["host", "status", "latency_ms"])
    w.writeheader()
    w.writerows(results)
```

os, pathlib, subprocess, logging, argparse

```
import os, sys, shutil, socket, logging, argparse, subprocess
from pathlib import Path
from datetime import datetime, timedelta

os.environ.get("APP_ENV", "dev")          # env var with a default
os.getenv("DB_HOST")                     # None if unset
os.path.exists("/opt/app")
os.makedirs("/opt/app/logs", exist_ok=True)
shutil.disk_usage("/")                   # (total, used, free) in bytes
socket.gethostname()

p = Path("/var/log/app")                  # pathlib is cleaner than os.path
for logfile in p.glob("*.log"):
    print(logfile.name, logfile.stat().st_size)
(p / "archive").mkdir(parents=True, exist_ok=True)

# subprocess - ALWAYS pass a list, never shell=True with untrusted input
r = subprocess.run(["systemctl", "is-active", "nginx"],
    capture_output=True, text=True, timeout=10)
print(r.returncode, r.stdout.strip(), r.stderr.strip())
if r.returncode != 0:
    ...

# logging - never use print() in anything scheduled
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s %(levelname)-8s %(name)s %(message)s",
    handlers=[logging.FileHandler("/var/log/devops-scripts.log"),
              logging.StreamHandler(sys.stdout)],
)
log = logging.getLogger("healthcheck")
log.info("starting"); log.warning("disk at 85%"); log.error("service down")

# argparse - makes a script a tool other people can use
parser = argparse.ArgumentParser(description="Check service health")
parser.add_argument("host", help="target hostname")
parser.add_argument("-p", "--port", type=int, default=443)
parser.add_argument("-t", "--threshold", type=float, default=80.0)
parser.add_argument("--dry-run", action="store_true")
parser.add_argument("-v", "--verbose", action="count", default=0)
args = parser.parse_args()
```

SUBPROCESS WITH SHELL=True

`subprocess.run(f"ping -c1 {host}", shell=True)` lets anyone who controls host run arbitrary commands - "8.8.8.8; rm -rf /". Pass a list instead: `subprocess.run(["ping", "-c1", host])`. There is no shell, so there is no injection. Only use `shell=True` when you genuinely need pipes or redirection, and never with input you did not construct yourself.

14.2 Script 1 - Disk Space Check

disk_check.py

```
#!/usr/bin/env python3
"""Check filesystem usage and exit non-zero when any mount exceeds the threshold."""
import shutil, sys, argparse, socket

MOUNTS = ["/", "/var", "/opt"]
```

```

def check(path, warn, crit):
    try:
        total, used, free = shutil.disk_usage(path)
    except FileNotFoundError:
        return None
    pct = used / total * 100
    state = "CRIT" if pct >= crit else "WARN" if pct >= warn else "OK"
    return {"path": path, "pct": pct, "free_gb": free / 1024**3, "state": state}

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--warn", type=float, default=80.0)
    ap.add_argument("--crit", type=float, default=90.0)
    ap.add_argument("--paths", nargs="*", default=MOUNTS)
    args = ap.parse_args()

    host = socket.gethostname()
    worst = 0
    for path in args.paths:
        r = check(path, args.warn, args.crit)
        if r is None:
            continue
        print(f"{host} {r['path']:<8} {r['pct']:5.1f}% used, "
              f"{r['free_gb']:6.1f} GB free [{r['state']}]")
        worst = max(worst, {"OK": 0, "WARN": 1, "CRIT": 2}[r["state"]])

    sys.exit(worst)    # 0 OK, 1 warning, 2 critical - usable by cron and Nagios-style checks

if __name__ == "__main__":
    main()

```

EXPLAINING THIS SCRIPT

"It reads usage for each mount with `shutil.disk_usage`, which is the standard library rather than parsing `df` output - parsing command output is fragile across distributions. It classifies against a warning and critical threshold, prints a line per mount, and exits with the worst severity so cron or a monitoring agent can act on the exit code rather than the text."

14.3 Script 2 - Memory Check

memory_check.py

```

#!/usr/bin/env python3
"""Report memory pressure from /proc/meminfo - no external dependencies."""
import sys

def meminfo():
    info = {}
    with open("/proc/meminfo") as f:
        for line in f:
            key, value = line.split(":", 1)
            info[key] = int(value.split()[0])    # values are in kB
    return info

def main(threshold=90.0):
    m = meminfo()
    total = m["MemTotal"]
    available = m["MemAvailable"]    # the number that matters, not MemFree
    used_pct = (total - available) / total * 100

    swap_total = m.get("SwapTotal", 0)
    swap_used = swap_total - m.get("SwapFree", 0)
    swap_pct = (swap_used / swap_total * 100) if swap_total else 0.0

    print(f"Memory: {used_pct:.1f}% used "
          f"f({(total-available)/1024**2:.1f} of {total/1024**2:.1f} GiB)")
    print(f"Available: {available/1024**2:.1f} GiB")
    print(f"Swap: {swap_pct:.1f}% used")

```

```

if used_pct >= threshold:
    print("CRITICAL: memory pressure")
    sys.exit(2)
if swap_pct > 50:
    print("WARNING: significant swap usage")
    sys.exit(1)
sys.exit(0)

if __name__ == "__main__":
    main(float(sys.argv[1]) if len(sys.argv) > 1 else 90.0)

```

WHY MEMAVAILABLE, NOT MEMFREE

Linux uses spare RAM as page cache, so MemFree is almost always low on a healthy system. MemAvailable is the kernel's own estimate of how much a new process could actually get, including reclaimable cache. A script that alerts on MemFree will page you constantly about a perfectly healthy machine - and being able to explain that difference is a genuine differentiator.

14.4 Script 3 - Process Check

process_check.py

```

#!/usr/bin/env python3
"""Verify expected processes are running, reading directly from /proc."""
import os, sys

EXPECTED = ["nginx", "python3", "containerd"]

def running_processes():
    procs = {}
    for pid in filter(str.isdigit, os.listdir("/proc")):
        try:
            with open(f"/proc/{pid}/comm") as f:
                name = f.read().strip()
            with open(f"/proc/{pid}/cmdline") as f:
                cmdline = f.read().replace("\0", " ").strip()
            procs.setdefault(name, []).append((pid, cmdline))
        except (FileNotFoundError, ProcessLookupError, PermissionError):
            continue # the process exited between listdir and open - normal, ignore it
    return procs

def main():
    procs = running_processes()
    missing = []
    for name in EXPECTED:
        instances = procs.get(name, [])
        if instances:
            print(f"OK      {name}: {len(instances)} process(es), pid {instances[0][0]}")
        else:
            print(f"MISSING {name}")
            missing.append(name)

    if missing:
        print(f"\nFAILED: not running -> {' '.join(missing)}")
        sys.exit(1)
    sys.exit(0)

if __name__ == "__main__":
    main()

```

14.5 Script 4 - TCP Port Check

port_check.py

```

#!/usr/bin/env python3
"""Check TCP reachability for a list of host:port pairs, in parallel."""
import socket, sys, argparse
from concurrent.futures import ThreadPoolExecutor

def check_port(target, timeout=3.0):

```

```

host, port = target.rsplit(":", 1)
port = int(port)
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.settimeout(timeout)
try:
    start = __import__("time").perf_counter()
    s.connect((host, port))
    ms = (__import__("time").perf_counter() - start) * 1000
    return target, True, f"open ({ms:.0f} ms)"
except socket.timeout:
    return target, False, "TIMEOUT - filtered, dropped, or host down"
except ConnectionRefusedError:
    return target, False, "REFUSED - host reachable, nothing listening"
except socket.gaierror:
    return target, False, "DNS resolution failed"
except OSError as e:
    return target, False, f"error: {e}"
finally:
    s.close()

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("targets", nargs="+", help="host:port ...")
    ap.add_argument("-t", "--timeout", type=float, default=3.0)
    args = ap.parse_args()

    failures = 0
    with ThreadPoolExecutor(max_workers=20) as pool:
        for target, ok, msg in pool.map(lambda t: check_port(t, args.timeout), args.targets):
            print(f"{'OK' if ok else 'FAIL'} {target:<30} {msg}")
            failures += 0 if ok else 1

    sys.exit(1 if failures else 0)

if __name__ == "__main__":
    main()

```

WHY THIS SCRIPT IS WORTH DISCUSSING

It encodes the single most useful piece of network troubleshooting knowledge: REFUSED and TIMEOUT mean completely different things. Refused proves the network path works and the service is not listening; timeout means a firewall, security group or routing problem. A check script that reports which one you got saves the next engineer ten minutes.

14.6 Script 5 - HTTP API Call with Retry

api_check.py

```

#!/usr/bin/env python3
"""Call an HTTP endpoint with retries and exponential backoff; report status and latency."""
import sys, time, json, argparse, urllib.request, urllib.error

def call(url, timeout=5, headers=None):
    req = urllib.request.Request(url, headers=headers or {})
    start = time.perf_counter()
    with urllib.request.urlopen(req, timeout=timeout) as r:
        body = r.read().decode()
        return r.status, body, (time.perf_counter() - start) * 1000

def call_with_retry(url, attempts=5, timeout=5, headers=None):
    delay = 1
    last = None
    for i in range(1, attempts + 1):
        try:
            status, body, ms = call(url, timeout, headers)
            return status, body, ms, i
        except urllib.error.HTTPError as e:
            last = f"HTTP {e.code}"
            if 400 <= e.code < 500:
                # client errors will not fix themselves - stop
                break

```

```

except urllib.error.URLError as e:
    last = f"connection error: {e.reason}"
except Exception as e:
    last = f"{type(e).__name__}: {e}"

    if i < attempts:
        print(f" attempt {i} failed ({last}); retrying in {delay}s", file=sys.stderr)
        time.sleep(delay)
        delay = min(delay * 2, 30)    # exponential backoff with a cap
    raise RuntimeError(f"failed after {attempts} attempts: {last}")

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("url")
    ap.add_argument("--attempts", type=int, default=5)
    ap.add_argument("--timeout", type=float, default=5)
    ap.add_argument("--token")
    ap.add_argument("--expect", type=int, default=200)
    args = ap.parse_args()

    headers = {"Authorization": f"Bearer {args.token}"} if args.token else {}
    try:
        status, body, ms, attempts = call_with_retry(args.url, args.attempts, args.timeout, headers)
    except RuntimeError as e:
        print(f"FAIL {e}")
        sys.exit(2)

    print(f"OK {args.url} -> {status} in {ms:.0f} ms (attempt {attempts})")
    try:
        print(json.dumps(json.loads(body), indent=2)[:500])
    except json.JSONDecodeError:
        print(body[:500])

    sys.exit(0 if status == args.expect else 1)

if __name__ == "__main__":
    main()

```

DO NOT RETRY A 4XX

Retrying is only correct for errors that might resolve themselves - timeouts, connection failures, 502, 503, 504. A 400 or 401 will return exactly the same result five times and just delays the failure. Exponential backoff with a cap, and ideally jitter, prevents a wave of clients all retrying in lockstep and turning a brief blip into a sustained outage.

14.7 Script 6 - Log Parser

log_parser.py

```

#!/usr/bin/env python3
"""Summarise an application log: error counts, top messages, and a per-hour timeline."""
import re, sys, gzip, argparse
from collections import Counter, defaultdict

LINE = re.compile(
    r"^(?P<ts>\d{4}-\d{2}-\d{2}[ T]\d{2}:\d{2}:\d{2})\s+"
    r"(?P<level>DEBUG|INFO|WARN|WARNING|ERROR|FATAL|CRITICAL)\s+"
    r"(?P<msg>.*)$"
)

def open_any(path):
    return gzip.open(path, "rt", errors="replace") if path.endswith(".gz") \
        else open(path, errors="replace")

def parse(paths, since=None):
    levels = Counter()
    messages = Counter()
    by_hour = defaultdict(Counter)
    unparsed = 0

    for path in paths:

```

```

with open_any(path) as f:
    for line in f:
        m = LINE.match(line)
        if not m:
            unparsed += 1
            continue
        ts, level, msg = m.group("ts"), m.group("level").upper(), m.group("msg")
        if since and ts < since:
            continue
        level = "WARN" if level == "WARNING" else level
        levels[level] += 1
        by_hour[ts[:13]][level] += 1
        if level in ("ERROR", "FATAL", "CRITICAL"):
            # Normalise variable parts so similar errors group together
            key = re.sub(r"\b\d+\b", "N", msg)
            key = re.sub(r"\b[0-9a-f]{8,}\b", "HEX", key)
            messages[key[:120]] += 1
    return levels, messages, by_hour, unparsed

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("logs", nargs="+")
    ap.add_argument("--since", help="only lines at or after this timestamp prefix")
    ap.add_argument("--top", type=int, default=10)
    args = ap.parse_args()

    levels, messages, by_hour, unparsed = parse(args.logs, args.since)
    total = sum(levels.values())

    print(f"Parsed {total} lines ({unparsed} unparsed)\n")
    print("LEVELS")
    for level, count in levels.most_common():
        print(f" {level:<9} {count:>8} {count/total*100:5.1f}%")

    print(f"\nTOP {args.top} ERROR MESSAGES")
    for msg, count in messages.most_common(args.top):
        print(f" {count:>6} {msg}")

    print("\nERRORS PER HOUR")
    for hour in sorted(by_hour):
        errs = by_hour[hour]["ERROR"] + by_hour[hour]["FATAL"] + by_hour[hour]["CRITICAL"]
        if errs:
            print(f" {hour}:00 {errs:>6} {'#' * min(errs, 60)}")

    sys.exit(1 if levels.get("ERROR", 0) or levels.get("FATAL", 0) else 0)

if __name__ == "__main__":
    main()

```

THE DETAIL THAT MAKES THIS SCRIPT GOOD

Normalising numbers and hex strings out of error messages before counting is what turns 4,000 unique-looking errors into three real problems. Without it, "connection timeout to 10.0.1.5:5432 after 30012ms" and the same error with a different millisecond count are two different messages and the summary is useless. Mention this if you are asked to design a log analysis tool.

14.8 Script 7 - File Cleanup

cleanup.py

```

#!/usr/bin/env python3
"""Delete or archive files older than N days, with a mandatory dry-run default."""
import sys, time, gzip, shutil, logging, argparse
from pathlib import Path

logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s")
log = logging.getLogger("cleanup")

def find_old(root, pattern, days):
    cutoff = time.time() - days * 86400

```

```

for path in Path(root).rglob(pattern):
    if path.is_file() and path.stat().st_mtime < cutoff:
        yield path

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("root")
    ap.add_argument("--pattern", default="*.log")
    ap.add_argument("--days", type=int, default=30)
    ap.add_argument("--action", choices=["delete", "compress"], default="compress")
    ap.add_argument("--apply", action="store_true",
                    help="actually do it; without this flag it is a dry run")
    args = ap.parse_args()

    root = Path(args.root)
    if not root.is_dir():
        log.error(f"{root} is not a directory"); sys.exit(2)

    total_bytes = 0
    count = 0
    for path in find_old(root, args.pattern, args.days):
        size = path.stat().st_size
        total_bytes += size
        count += 1

    if not args.apply:
        log.info(f"[DRY RUN] would {args.action} {path} ({size/1024**2:.1f} MB)")
        continue

    try:
        if args.action == "delete":
            path.unlink()
            log.info(f"deleted {path}")
        else:
            gz = path.with_suffix(path.suffix + ".gz")
            with open(path, "rb") as src, gzip.open(gz, "wb") as dst:
                shutil.copyfileobj(src, dst)
            path.unlink()
            log.info(f"compressed {path} -> {gz.name}")
    except OSError as e:
        log.error(f"failed on {path}: {e}")

    log.info(f"'Would process' if not args.apply else 'Processed' "
            f"{count} files, {total_bytes/1024**3:.2f} GB")

if __name__ == "__main__":
    main()

```

DRY RUN BY DEFAULT, FOR ANYTHING DESTRUCTIVE

The `--apply` flag means the safe behaviour is the default and destruction is opt-in. Any script that deletes files should behave this way, log every action with the full path, handle per-file errors without aborting the whole run, and report a total. Saying "my cleanup scripts are dry-run by default" is a small detail that tells an interviewer you have been burned and learned.

14.9 Script 8 - Service Check

service_check.py

```

#!/usr/bin/env python3
"""Check systemd services and optionally restart failed ones."""
import sys, subprocess, logging, argparse

logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s")
log = logging.getLogger("servicecheck")

def run(cmd, timeout=15):
    return subprocess.run(cmd, capture_output=True, text=True, timeout=timeout)

def is_active(service):

```

```

return run(["systemctl", "is-active", "--quiet", service]).returncode == 0

def is_enabled(service):
    return run(["systemctl", "is-enabled", "--quiet", service]).returncode == 0

def last_logs(service, lines=20):
    return run(["journalctl", "-u", service, "-n", str(lines), "--no-pager"]).stdout

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("services", nargs="+")
    ap.add_argument("--restart", action="store_true", help="attempt to restart failed services")
    args = ap.parse_args()

    failed = []
    for svc in args.services:
        if is_active(svc):
            log.info(f"OK      {svc} (enabled={is_enabled(svc)})")
            continue

        log.error(f"DOWN    {svc}")
        log.error("last log lines:\n" + last_logs(svc))
        failed.append(svc)

        if args.restart:
            log.warning(f"restarting {svc}")
            r = run(["systemctl", "restart", svc], timeout=60)
            if r.returncode == 0 and is_active(svc):
                log.info(f"RECOVERED {svc}")
                failed.remove(svc)
            else:
                log.error(f"restart failed for {svc}: {r.stderr.strip()}")

    sys.exit(1 if failed else 0)

if __name__ == "__main__":
    main()

```

AUTO-RESTART WITHOUT A LIMIT IS DANGEROUS

A script that restarts a service every time it finds it down will happily restart a crashing service every minute forever, hiding the real problem and possibly amplifying it - a service that crashes under load will crash faster when restarted repeatedly. If you automate restarts, cap the attempts, back off between them, and always raise an alert so a human knows it happened. Restarting is mitigation, not a fix.

14.10 Script 9 - Composite Health Check

healthcheck.py

```

#!/usr/bin/env python3
"""Run all checks and emit a JSON summary suitable for a pipeline or a dashboard."""
import sys, json, socket, shutil, subprocess, urllib.request, urllib.error
from datetime import datetime, timezone

def check_disk(path="/", threshold=85):
    total, used, _ = shutil.disk_usage(path)
    pct = used / total * 100
    return {"name": f"disk:{path}", "value": round(pct, 1),
            "ok": pct < threshold, "detail": f"{pct:.1f}% used"}

def check_memory(threshold=90):
    info = {}
    with open("/proc/meminfo") as f:
        for line in f:
            k, v = line.split(":", 1)
            info[k] = int(v.split()[0])
    pct = (info["MemTotal"] - info["MemAvailable"]) / info["MemTotal"] * 100
    return {"name": "memory", "value": round(pct, 1),
            "ok": pct < threshold, "detail": f"{pct:.1f}% used"}

```

```

def check_service(name):
    rc = subprocess.run(["systemctl", "is-active", "--quiet", name]).returncode
    return {"name": f"service:{name}", "value": rc, "ok": rc == 0,
            "detail": "active" if rc == 0 else "inactive"}

def check_port(host, port, timeout=3):
    s = socket.socket(); s.settimeout(timeout)
    try:
        s.connect((host, port)); ok, detail = True, "open"
    except ConnectionRefusedError:
        ok, detail = False, "refused"
    except Exception as e:
        ok, detail = False, type(e).__name__
    finally:
        s.close()
    return {"name": f"port:{host}:{port}", "value": None, "ok": ok, "detail": detail}

def check_http(url, expect=200, timeout=5):
    try:
        with urllib.request.urlopen(url, timeout=timeout) as r:
            ok = r.status == expect
            return {"name": f"http:{url}", "value": r.status, "ok": ok,
                    "detail": f"HTTP {r.status}"}
    except urllib.error.HTTPError as e:
        return {"name": f"http:{url}", "value": e.code, "ok": False, "detail": f"HTTP {e.code}"}
    except Exception as e:
        return {"name": f"http:{url}", "value": None, "ok": False, "detail": type(e).__name__}

def main():
    checks = [
        check_disk("/"),
        check_disk("/var"),
        check_memory(),
        check_service("nginx"),
        check_service("docker"),
        check_port("127.0.0.1", 8080),
        check_http("http://127.0.0.1:8080/health"),
    ]

    failed = [c for c in checks if not c["ok"]]
    report = {
        "host": socket.gethostname(),
        "timestamp": datetime.now(timezone.utc).isoformat(),
        "status": "healthy" if not failed else "unhealthy",
        "failed_count": len(failed),
        "checks": checks,
    }

    print(json.dumps(report, indent=2))
    sys.exit(0 if not failed else 1)

if __name__ == "__main__":
    main()

```

WHY JSON OUTPUT MATTERS

Emitting JSON means the same script serves a human reading the terminal, a pipeline step checking the exit code, and a monitoring system parsing the fields - without three different scripts. Combined with a non-zero exit on failure, it drops straight into Jenkins, a cron job, or a Kubernetes readiness command.

14.11 Python Interview Questions

Q. 1. Why use Python for DevOps rather than Bash?

Bash is better for short command orchestration - piping tools together, a few conditionals. Python takes over when there is real logic: parsing structured data such as JSON or CSV, calling APIs, error handling with different behaviour per failure type, or anything longer than about fifty lines. Python is also far more readable and testable when someone else has to maintain it at 3am.

Q. 2. What is the difference between a list and a tuple?

A list is mutable, a tuple is immutable and therefore hashable, so it can be a dictionary key or a set member. In practice I use tuples for fixed records like (host, port, status) and lists for collections I build up.

Q. 3. What is a dictionary and how do you access it safely?

A key-value mapping. `d["key"]` raises `KeyError` if the key is missing; `d.get("key", default)` returns a default instead. In scripts reading external data I always use `.get()` with a default, because a missing field should not crash a monitoring script.

Q. 4. What is a list comprehension?

A concise way to build a list from an iterable with optional filtering - `[l for l in lines if "ERROR" in l]`. It is faster and clearer than an append loop for simple transformations, though I would use an explicit loop if the logic needs more than a condition and an expression.

Q. 5. How do you read a very large log file without running out of memory?

Iterate over the file object directly - `for line in f:` - which streams one line at a time. `f.read()` or `f.readlines()` loads the entire file into memory, which fails on a multi-gigabyte log. I also use `gzip.open` for rotated logs so I do not have to decompress them first.

Q. 6. How do you handle exceptions properly?

Catch specific exceptions and handle each meaningfully - a timeout is different from a connection refusal, which is different from an HTTP 500. I log with enough context to act on, use `finally` for cleanup, and exit with a distinct code per failure class. Catching bare `Exception` and continuing silently is how failures become invisible.

Q. 7. What is a context manager and why use `with`?

An object with `enter` and `exit` methods that guarantees cleanup. `with open(...) as f:` closes the file even if an exception is raised inside the block. Without it, an exception leaves the descriptor open, and a long-running script leaks them until it hits the open-files limit.

Q. 8. How do you run a shell command from Python?

`subprocess.run(["systemctl", "is-active", "nginx"], capture_output=True, text=True, timeout=10)`, then check `returncode`, `stdout` and `stderr`. I always pass a list rather than a string with `shell=True`, because that removes command injection entirely, and I always set a timeout so a hung command cannot hang the script.

Q. 9. What is the difference between `os` and `pathlib`?

`os.path` is the older string-based API; `pathlib.Path` is an object-oriented API where paths support operators and methods - `p / "sub" / "file.log"`, `p.glob("*.log")`, `p.stat()`, `p.unlink()`. I prefer `pathlib` for new code because it is far more readable and handles platform differences.

Q. 10. How do you parse JSON?

`json.loads(string)` and `json.load(file_object)` to read, `json.dumps(obj, indent=2)` to write. I wrap it in a `try/except` for `json.JSONDecodeError`, because an API returning an HTML error page instead of JSON is a common and confusing failure.

Q. 11. How do you make an HTTP request?

`requests` if it is available - `requests.get(url, timeout=5)` then `raise_for_status()`. If I cannot install dependencies, `urllib.request` from the standard library. The non-negotiable part is always setting a timeout; without one, a request can hang forever and a scheduled script silently stops running.

Q. 12. What is `if __name__ == "__main__":` for?

It only runs the block when the file is executed directly, not when it is imported. That makes a script importable and testable - you can import its functions into a test without running the whole thing.

Q. 13. How do you handle command-line arguments?

`argparse` - it gives typed arguments, defaults, flags, help text and validation for free, and produces a proper `--help`. `sys.argv` is fine for a two-line throwaway, but anything another person will run should use `argparse`.

Q. 14. Why use logging instead of `print`?

Logging gives levels so you can filter noise, timestamps, the ability to write to a file and `stdout` simultaneously, and structured formatting. For anything scheduled, print output usually goes nowhere, which means when it fails at 2am there is no record of why.

Q. 15. What is a virtual environment and why does it matter?

An isolated Python environment with its own packages - `python3 -m venv .venv`, then activate and `pip install -r requirements.txt`. It matters because installing packages system-wide can break OS tooling that depends on specific versions, and because pinning requirements is what makes a script reproducible on another machine.

Q. 16. How do you make a script idempotent?

Check state before acting - create a directory only if it does not exist, add a config line only if it is absent, skip a download if the checksum matches. That makes the script safe to re-run, which is essential for anything driven by cron or a pipeline that may retry.

Q. 17. How would you retry a failing API call?

A loop with a bounded number of attempts and exponential backoff with a cap and ideally jitter, retrying only errors that can succeed later - timeouts, connection errors, 5xx - and failing fast on 4xx. Unbounded retries turn a transient blip into a resource leak, and synchronised retries across many clients turn it into a self-inflicted denial of service.

Q. 18. How do you check whether a port is open in Python?

A socket with a timeout: create, `settimeout`, `connect`, and catch the exceptions separately - `ConnectionRefusedError` means reachable but nothing listening, `socket.timeout` means filtered or down, `socket.gaierror` means DNS failed. Reporting which one occurred is the actually useful part.

Q. 19. How do you read system metrics without external libraries?

From `/proc` - `/proc/meminfo` for memory, `/proc/loadavg` for load, `/proc/stat` for CPU, and per-process directories under `/proc/PID/`. Plus `shutil.disk_usage` for filesystems. That keeps scripts dependency-free, which matters on minimal images and locked-down servers where you cannot pip install.

Q. 20. How would you run checks against 100 hosts quickly?

`concurrent.futures.ThreadPoolExecutor` with a bounded worker count, since these checks are I/O bound and threads are the right tool - the GIL is not a constraint when you are waiting on sockets. I set a per-check timeout so one dead host cannot stall the run, and collect results rather than printing from threads.

Q. 21. What is the difference between threads and processes in Python?

Threads share memory and are limited by the GIL for CPU-bound work, but they are ideal for I/O-bound work like network checks. Processes have separate memory and genuinely parallelise CPU work via `multiprocessing`. Almost all DevOps scripting is I/O bound, so threads are usually the right answer.

Q. 22. How do you write output that both humans and machines can use?

JSON to stdout with a meaningful exit code, and optionally a human-readable mode behind a flag. That way the same script works in a terminal, in a pipeline that checks the exit code, and in a monitoring system that parses the fields.

Q. 23. How do you handle secrets in a Python script?

Read them from environment variables or a secret store at runtime - never hardcode them, never commit them, and never log them. On AWS I would use boto3 to fetch from Secrets Manager or SSM using the instance or pod role, so there is no static credential at all.

Q. 24. How would you test a DevOps script?

Separate the logic from the I/O so the logic is testable - a function that takes a usage percentage and returns a severity can be unit-tested without touching a disk. Then `pytest` with mocked subprocess and HTTP calls for the boundaries. At minimum, run it with a dry-run flag against a real environment before scheduling it.

Q. 25. What does `sys.exit()` do and why does it matter?

It terminates the script with a specific exit code. It matters because cron, Jenkins, systemd and Kubernetes all determine success from the exit code, not from what was printed. A script that reports a problem in text but exits 0 will never trigger anything.

Q. 26. How do you parse a timestamped log line?

A compiled regular expression with named groups for timestamp, level and message, applied per line while streaming. I compile the pattern once outside the loop, and I count lines that do not match rather than discarding them silently, because a sudden rise in unparsed lines usually means the log format changed.

Q. 27. What is `collections.Counter` and why is it useful here?

A dict subclass for counting hashable items, with `most_common(n)`. It turns "which errors happen most" or "which IPs hit us most" into two lines instead of manual dictionary bookkeeping. `defaultdict` is its companion for grouping.

Q. 28. How would you interact with AWS from Python?

boto3, using the instance or pod IAM role so no credentials are stored. I use `paginator`s for any list operation because the default page size will silently truncate results, handle `ClientError` explicitly, and rely on boto3's built-in retry behaviour for throttling rather than writing my own.

Q. 29. What would you check in a code review of someone's DevOps script?

Does it set timeouts on every network and subprocess call? Does it exit non-zero on failure? Does it log rather than print? Is it safe to re-run? Does it have a dry-run for anything destructive? Does it avoid `shell=True`? Are secrets read from the environment rather than hardcoded? Are exceptions specific rather than a bare except that swallows everything?

Q. 30. Tell me about a script you wrote and why.

Anchor this to your CV - system checks, log analysis and network troubleshooting. For example: "I wrote health-check scripts that verify disk, memory, the services we depend on and the application's own health endpoint, emit a JSON summary and exit non-zero on failure, so the same script works from the terminal during an incident and from a scheduled job. The log parser was the more useful one - it normalises variable parts out of error messages before counting, which turns thousands of near-identical lines into a short list of actual distinct problems with a per-hour timeline showing exactly when it started."

PART 14 RECALL POINTS

- Stream files line by line; never `read()` a large log.
- `.get(key, default)` for external data.
- `subprocess.run([list], capture_output=True, text=True, timeout=N)` - never `shell=True`.
- Always set a timeout on network calls.
- Exit codes are what automation reads; logging is what humans read.
- Retry timeouts and 5xx, never 4xx; exponential backoff with a cap.
- Dry run by default for anything destructive.
- `/proc` and `shutil.disk_usage` give you metrics with zero dependencies.
- `ThreadPoolExecutor` for I/O-bound fan-out.

PART 15

BASH SCRIPTING

The patterns that matter, and ten production scripts

Bash is the language of the deployment step, the cron job and the container entrypoint. This part covers the constructs that appear in real DevOps scripts and then gives you ten complete, defensible scripts. Part 3 covered the commands; this part covers turning them into something reliable.

15.1 The Safety Header

Start every script with this

```
#!/usr/bin/env bash
set -euo pipefail
IFS=$'\n\t'
```

Option	Effect	Why it matters	
set -e	Exit immediately on any command returning non-zero	Without it, a failed <code>cd</code> is followed by the rest of the script running in the wrong directory	
set -u	Error on an undefined variable	Turns <code>rm -rf "\$DIR"/</code> with an empty <code>DIR</code> from a disaster into an error	
set -o pipefail	A pipeline fails if ANY command in it fails	Without it <code>false \</code>	<code>tee log` succeeds, because only the last exit code counts</code>
IFS=\$'\n\t'	Word-split only on newline and tab	Stops filenames containing spaces being split into multiple arguments	
set -x	Print each command before executing	Debugging only - and never around a secret	

THE THREE HABITS THAT SEPARATE A SAFE SCRIPT FROM A DANGEROUS ONE

1) Quote every variable expansion: `"$var"`, not `$var`. An unquoted empty variable disappears entirely and an unquoted path with a space becomes two arguments. 2) Use `${var}` braces when concatenating: `"${name}_backup"`. 3) Prefer `[[ ]]` over `[ ]` in bash - it handles empty variables safely and supports pattern and regex matching.

15.2 Variables, Arguments and Defaults

The patterns you will use constantly

```
APP_NAME="payments-api"
readonly LOG_DIR="/var/log/${APP_NAME}"          # constant
TODAY=$(date +%F)                                # command substitution - prefer $() over backticks
COUNT=$(( 3 + 4 ))                              # arithmetic

# Positional parameters
echo "script      : $0"
echo "first arg   : ${1:-}"                       # ':-' avoids an error under set -u
echo "all args    : $*"                           # one word
echo "all args    : @$@"                          # separate words - use "$@" almost always
echo "arg count   : $#"
```

```
echo "last exit   : $?"
echo "this PID    : $$"
```

```
# Defaults and required values
ENVIRONMENT="${1:-staging}"                      # default when unset or empty
: "${API_TOKEN:?API_TOKEN must be set}"           # abort with a message if unset
VERSION="${VERSION:-$(git describe --tags 2>/dev/null || echo dev)}"
```

```
# Parameter expansion - string manipulation without calling external tools
path="/var/log/app/app.log"
echo "${path##*/}"                               # app.log - strip longest leading match (basename)
```

```

echo "${path%/*}"      # /var/log/app - strip shortest trailing match (dirname)
echo "${path%.log}"    # ...app      - strip the extension
echo "${path/log/LOG}" # replace first occurrence
echo "${#path}"        # string length

# Arrays
SERVERS=(web01 web02 db01)
echo "${SERVERS[0]}"
echo "${SERVERS[@]}"   # all elements
echo "${#SERVERS[@]}"  # count
SERVERS+=(web03)
for s in "${SERVERS[@]"; do echo "$s"; done

```

15.3 Conditions

Test operators worth memorising

```

# Files
[[ -e path ]]      # exists
[[ -f path ]]      # is a regular file
[[ -d path ]]      # is a directory
[[ -L path ]]      # is a symlink
[[ -r path ]]      # readable      -w writable      -x executable
[[ -s path ]]      # exists AND is non-empty
[[ file1 -nt file2 ]] # newer than

# Strings
[[ -z "$var" ]]     # empty
[[ -n "$var" ]]     # non-empty
[[ "$a" == "$b" ]]  # equal
[[ "$a" != "$b" ]]  # not equal
[[ "$name" == app* ]] # glob pattern match
[[ "$line" =~ ^ERROR ]] # regex match

# Numbers
[[ "$x" -eq 5 ]]    # -eq -ne -lt -le -gt -ge
(( x > 5 ))         # arithmetic context - more natural for numbers

# Combining
if [[ "$USED" -gt 80 && "$ENV" == "prod" ]]; then ... fi
if [[ ! -f "$CONFIG" ]]; then echo "missing config"; exit 1; fi

# Short-circuit idioms
command -v docker >/dev/null 2>&1 || { echo "docker not installed"; exit 1; }
[[ -d "$DIR" ]] || mkdir -p "$DIR"

case "${1:-}" in
  start)  systemctl start "$APP_NAME" ;;
  stop)   systemctl stop  "$APP_NAME" ;;
  status) systemctl status "$APP_NAME" ;;
  *)      echo "Usage: $0 {start|stop|status}" >&2; exit 1 ;;
esac

```

15.4 Loops, Functions and Traps

Structure

```

for host in "${SERVERS[@]"; do
  ssh -o ConnectTimeout=5 "$host" "uptime" || echo "FAILED: $host"
done

for f in /var/log/*.log; do
  [[ -e "$f" ]] || continue      # guard against no matches - the glob stays literal
  gzip "$f"
done

for i in {1..5}; do echo "attempt $i"; done
for ((i = 0; i < 5; i++)); do echo "$i"; done

# Reading a file line by line - the CORRECT form
while IFS= read -r line; do      # IFS= preserves whitespace, -r preserves backslashes

```

```

    echo "processing: $line"
done < servers.txt

# Reading command output without a subshell losing your variables
while IFS= read -r pod; do
    kubectl delete pod "$pod"
done < <(kubectl get pods -o name | grep Evicted)

# Functions
log() { printf '%s [%s] %s\n' "$(date +%F %T)" "${2:-INFO}" "$1" | tee -a "$LOGFILE"; }
die() { log "$1" ERROR; exit "${2:-1}"; }

require() {
    local cmd="$1"
    command -v "$cmd" >/dev/null 2>&1 || die "required command not found: $cmd" 127
}

retry() {
    local attempts="$1"; shift
    local delay=1 n=1
    until "$@"; do
        (( n >= attempts )) && return 1
        log "attempt $n failed, retrying in ${delay}s" WARN
        sleep "$delay"
        delay=$(( delay * 2 ))
        n=$(( n + 1 ))
    done
}

retry 5 curl -sf http://localhost:8080/health

# trap - cleanup that runs even on failure or interrupt
TMPDIR=$(mktemp -d)
cleanup() {
    local rc=$?
    rm -rf "$TMPDIR"
    (( rc != 0 )) && log "script failed with exit code $rc" ERROR
    exit $rc
}
trap cleanup EXIT INT TERM

```

TRAP ... EXIT IS WHAT MAKES A SCRIPT SAFE

Without it, a script that fails halfway leaves temporary directories, lock files and a service stopped. With it, cleanup runs on success, on failure and on Ctrl+C. Any deployment script that stops a service should have a trap that restarts it if the deployment aborts - mentioning this in an interview shows you have thought about partial failure.

15.5 Script 1 - Health Check with Retry

healthcheck.sh

```

#!/usr/bin/env bash
set -euo pipefail

URL="${1:?usage: $0 <url> [attempts]}"
ATTEMPTS="${2:-10}"
DELAY=2
LOGFILE="/var/log/healthcheck.log"

log() { printf '%s [%s] %s\n' "$(date -u +%FT%TZ)" "$*" | tee -a "$LOGFILE"; }

for (( i = 1; i <= ATTEMPTS; i++ )); do
    code=$(curl -s -o /dev/null -w '%{http_code}' --max-time 5 "$URL" || echo 000)
    if [[ "$code" == "200" ]]; then
        log "OK $URL returned 200 on attempt $i"
        exit 0
    fi
    log "attempt $i/$ATTEMPTS: $URL returned $code"
    sleep "$DELAY"
    (( DELAY = DELAY < 30 ? DELAY * 2 : 30 ))
done

```

```
log "FAIL $URL never returned 200 after $ATTEMPTS attempts"
exit 1
```

15.6 Script 2 - Disk Space Alert

disk_alert.sh

```
#!/usr/bin/env bash
set -euo pipefail

WARN=80
CRIT=90
HOST=$(hostname -f)
status=0

# Skip pseudo-fileystems; -P forces single-line POSIX output
df -hP -x tmpfs -x devtmpfs -x overlay -x squashfs | tail -n +2 | while read -r fs size used avail pct
do
    mount; do
        used_pct="${pct%}"
        if (( used_pct >= CRIT )); then
            echo "CRITICAL $HOST $mount ${used_pct}% used (${avail} free)"
            status=2
        elif (( used_pct >= WARN )); then
            echo "WARNING $HOST $mount ${used_pct}% used (${avail} free)"
            (( status < 1 )) && status=1
        fi
    done

# Inodes matter too - "no space left on device" with free blocks
df -iP | tail -n +2 | awk -v h="$HOST" '$5+0 > 85 {print "WARNING " h " " $6 " inodes " $5 " used"}'

# The largest consumers, to make the alert actionable
echo "--- largest directories under /var ---"
du -xh --max-depth=1 /var 2>/dev/null | sort -h | tail -5

exit "$status"
```

THE PIPELINE-SUBSHELL GOTCHA IN THIS SCRIPT

`df | while read ...` runs the loop body in a SUBSHELL, so `status=2` set inside it is lost when the pipeline ends. That is a genuine bash trap. The fixes are process substitution - `while read ...; do ...; done < <(df ...)` - or `shopt -s lastpipe`. Knowing this, and knowing why, is a strong bash answer.

15.7 Script 3 - Log Rotation and Archive

rotate_logs.sh

```
#!/usr/bin/env bash
set -euo pipefail

LOG_DIR="${1:-/var/log/myapp}"
ARCHIVE_DIR="${LOG_DIR}/archive"
COMPRESS_AFTER_DAYS=1
DELETE_AFTER_DAYS=30
DRY_RUN="${DRY_RUN:-true}"

run() {
    if [[ "$DRY_RUN" == "true" ]]; then
        echo "[DRY RUN] $"
    else
        "$@"
    fi
}

[[ -d "$LOG_DIR" ]] || { echo "no such directory: $LOG_DIR" >&2; exit 2; }
run mkdir -p "$ARCHIVE_DIR"

echo "== compressing logs older than ${COMPRESS_AFTER_DAYS} day(s) =="
while IFS= read -r -d '' file; do
```

```

    echo "compress: $file"
    run gzip -9 "$file"
    run mv "${file}.gz" "$ARCHIVE_DIR/"
done < <(find "$LOG_DIR" -maxdepth 1 -type f -name '*.log' -mtime "+${COMPRESS_AFTER_DAYS}" -print0)

echo "== deleting archives older than ${DELETE_AFTER_DAYS} days =="
while IFS= read -r -d '' file; do
    echo "delete: $file"
    run rm -f "$file"
done < <(find "$ARCHIVE_DIR" -type f -name '*.gz' -mtime "+${DELETE_AFTER_DAYS}" -print0)

echo "remaining usage: $(du -sh "$LOG_DIR" | cut -f1)"
[[ "$DRY_RUN" == "true" ]] && echo "NOTE: dry run - set DRY_RUN=false to apply"
exit 0

```

-PRINT0 AND READ -R -D ''

Filenames can contain spaces and newlines. `find -print0` separates results with null bytes and `read -d ''` reads them the same way, so no filename can ever be split. Using `for f in $(find ...)` is the classic bug that eventually deletes the wrong thing.

15.8 Script 4 - Deployment with Rollback

deploy.sh

```

#!/usr/bin/env bash
set -euo pipefail

APP="payments-api"
RELEASES="/opt/${APP}/releases"
CURRENT="/opt/${APP}/current"
VERSION="${1:?usage: $0 <version>}"
ARTIFACT_URL="https://artifacts.example.com/${APP}/${APP}-${VERSION}.tgz"
HEALTH_URL="http://127.0.0.1:8080/health"
LOGFILE="/var/log/${APP}-deploy.log"

log() { printf '%s %s\n' "$(date -u +%FT%TZ)" "$*" | tee -a "$LOGFILE"; }

PREVIOUS=""
[[ -L "$CURRENT" ]] && PREVIOUS=$(readlink -f "$CURRENT")

rollback() {
    [[ -z "$PREVIOUS" ]] && { log "ERROR no previous release to roll back to"; return; }
    log "ROLLING BACK to $PREVIOUS"
    ln -sf "$PREVIOUS" "$CURRENT"
    systemctl restart "$APP"
    sleep 5
    if curl -sf --max-time 5 "$HEALTH_URL" >/dev/null; then
        log "rollback successful"
    else
        log "CRITICAL rollback did not restore health - manual intervention required"
    fi
}

on_error() {
    log "ERROR deployment of $VERSION failed at line $1"
    rollback
    exit 1
}
trap 'on_error $LINENO' ERR

log "=== deploying ${APP} ${VERSION} (previous: ${PREVIOUS:-none}) ==="

TARGET="${RELEASES}/${VERSION}"
mkdir -p "$TARGET"

log "downloading artifact"
curl -sL --max-time 120 "$ARTIFACT_URL" -o "/tmp/${APP}-${VERSION}.tgz"

log "extracting"

```

```

tar xzf "/tmp/${APP}-${VERSION}.tgz" -C "$TARGET"
chown -R "${APP}:${APP}" "$TARGET"

log "switching symlink"
ln -sf "$TARGET" "$CURRENT"          # atomic swap

log "restarting service"
systemctl restart "$APP"

log "waiting for health"
for i in {1..12}; do
    if curl -sf --max-time 5 "$HEALTH_URL" >/dev/null; then
        log "healthy after ${i} checks"
        log "=== deployment of ${VERSION} SUCCEEDED ==="
        # keep the five most recent releases
        ls -ldt "${RELEASES}"/*/* | tail -n +6 | xargs -r rm -rf
        rm -f "/tmp/${APP}-${VERSION}.tgz"
        exit 0
    fi
    sleep 5
done

log "health check never passed"
false      # triggers the ERR trap -> rollback

```

WHY THIS SCRIPT IS A GOOD INTERVIEW ARTEFACT

It demonstrates four things interviewers look for: an atomic switch via symlink so there is no half-deployed state, a recorded previous version so rollback is possible, an ERR trap so rollback happens automatically on any failure, and a health check that gates success rather than assuming the restart worked. Walk through those four points rather than the code line by line.

15.9 Script 5 - Service Watchdog

watchdog.sh

```

#!/usr/bin/env bash
set -uo pipefail          # deliberately NOT -e: we want to continue past a failed check

SERVICES=(nginx docker "payments-api")
MAX_RESTARTS=3
STATE_DIR=/var/lib/watchdog
LOGFILE=/var/log/watchdog.log

mkdir -p "$STATE_DIR"
log() { printf '%s %s\n' "$(date -u +%FT%TZ)" "$*" | tee -a "$LOGFILE"; }

for svc in "${SERVICES[@]}; do
    if systemctl is-active --quiet "$svc"; then
        echo 0 > "${STATE_DIR}/${svc}.restarts"    # healthy: reset the counter
        continue
    fi

    log "DOWN $svc"
    count=$(cat "${STATE_DIR}/${svc}.restarts" 2>/dev/null || echo 0)

    if (( count >= MAX_RESTARTS )); then
        log "GIVING UP on $svc after ${count} restarts - escalating"
        journalctl -u "$svc" -n 50 --no-pager >> "$LOGFILE"
        # notify here: curl to Slack / SNS / Alertmanager
        continue
    fi

    log "restarting $svc (attempt $(( count + 1 ))/${MAX_RESTARTS})"
    systemctl restart "$svc"
    sleep 5

    if systemctl is-active --quiet "$svc"; then
        log "RECOVERED $svc"
    else

```

```

        log "restart FAILED for $svc"
        journalctl -u "$svc" -n 30 --no-pager >> "$LOGFILE"
    fi
    echo $(( count + 1 )) > "${STATE_DIR}/${svc}.restarts"
done

```

15.10 Script 6 - Backup with Verification

backup.sh

```

#!/usr/bin/env bash
set -euo pipefail

SOURCE_DIRS=(/opt/app/config /etc/nginx /var/lib/app/data)
BACKUP_ROOT=/backup
S3_BUCKET="s3://acme-backups/${hostname -s}"
RETENTION_DAYS=14
STAMP=$(date -u +%Y%m%dT%H%M%SZ)
ARCHIVE="${BACKUP_ROOT}/backup-${STAMP}.tar.gz"
LOGFILE=/var/log/backup.log

log() { printf '%s %s\n' "$(date -u +%FT%TZ)" "$*" | tee -a "$LOGFILE"; }
trap 'log "ERROR backup failed at line $LINENO"; exit 1' ERR

mkdir -p "$BACKUP_ROOT"

log "=== backup started ==="
log "creating archive $ARCHIVE"
tar czf "$ARCHIVE" "${SOURCE_DIRS[@]} 2>/dev/null

log "verifying archive integrity"
tar tzf "$ARCHIVE" >/dev/null || { log "ARCHIVE IS CORRUPT"; exit 1; }

SIZE=$(du -h "$ARCHIVE" | cut -f1)
CHECKSUM=$(sha256sum "$ARCHIVE" | cut -d' ' -f1)
log "archive OK: ${SIZE}, sha256 ${CHECKSUM}"
echo "$CHECKSUM $(basename "$ARCHIVE")" > "${ARCHIVE}.sha256"

log "uploading to S3"
aws s3 cp "$ARCHIVE" "${S3_BUCKET}/" --storage-class STANDARD_IA
aws s3 cp "${ARCHIVE}.sha256" "${S3_BUCKET}/"

log "verifying remote object exists"
aws s3api head-object --bucket "${S3_BUCKET#s3://}" \
    --key "$(basename "$ARCHIVE")" >/dev/null 2>&1 || {
    # bucket/key split for buckets with a prefix
    aws s3 ls "${S3_BUCKET}/${(basename "$ARCHIVE")}" >/dev/null || {
        log "REMOTE VERIFICATION FAILED"; exit 1; }
    }

log "pruning local backups older than ${RETENTION_DAYS} days"
find "$BACKUP_ROOT" -name 'backup-*.tar.gz*' -mtime "+${RETENTION_DAYS}" -delete

log "=== backup completed: ${ARCHIVE} ==="

```

AN UNVERIFIED BACKUP IS NOT A BACKUP

This script does three things most backup scripts do not: it verifies the archive is readable after creating it, it records a checksum, and it confirms the remote object exists after upload. The only thing better is periodically restoring from it - and "we test our restores" is a sentence that will impress any interviewer, because almost nobody does it.

15.11 Script 7 - Parallel Remote Command

fleet_exec.sh

```

#!/usr/bin/env bash
set -uo pipefail

HOSTS_FILE="${1:?usage: $0 <hosts-file> <command>}"
shift

```

```

COMMAND="$*"
PARALLEL=10
TIMEOUT=15
FAILED=0

run_one() {
    local host="$1"
    local out
    if out=$(timeout "$TIMEOUT" ssh -o BatchMode=yes -o ConnectTimeout=5 \
        -o StrictHostKeyChecking=accept-new "$host" "$COMMAND" 2>&1); then
        printf '[ OK ] %-20s %s\n' "$host" "$(echo "$out" | head -1)"
    else
        printf '[FAIL] %-20s %s\n' "$host" "$(echo "$out" | head -1)"
        return 1
    fi
}
export -f run_one
export COMMAND TIMEOUT

# xargs -P gives parallelism without needing GNU parallel installed
grep -vE '^\\s*(#|$)' "$HOSTS_FILE" \
    | xargs -P "$PARALLEL" -I{} bash -c 'run_one "$@"' _ {} \
    || FAILED=1

exit "$FAILED"

```

15.12 Script 8 - Docker Cleanup

docker_cleanup.sh

```

#!/usr/bin/env bash
set -euo pipefail

THRESHOLD=75
KEEP_HOURS=72
DRY_RUN="${DRY_RUN:-true}"

usage=$(df --output=pcent /var/lib/docker | tail -1 | tr -dc '0-9')
echo "docker filesystem usage: ${usage}%"
docker system df

if (( usage < THRESHOLD )); then
    echo "below ${THRESHOLD}% - nothing to do"
    exit 0
fi

echo "above threshold - cleaning objects older than ${KEEP_HOURS}h"

if [[ "$DRY_RUN" == "true" ]]; then
    echo "[DRY RUN] would run: docker system prune -af --filter until=${KEEP_HOURS}h"
    echo "[DRY RUN] would run: docker builder prune -af --filter until=${KEEP_HOURS}h"
    echo "Exited containers that would be removed:"
    docker ps -a --filter status=exited --format ' {{.Names}}\t{{.Status}}'
    exit 0
fi

docker container prune -f --filter "until=${KEEP_HOURS}h"
docker image prune -af --filter "until=${KEEP_HOURS}h"
docker builder prune -af --filter "until=${KEEP_HOURS}h"
# NOTE: volumes are deliberately NOT pruned here - that is where data lives.

new_usage=$(df --output=pcent /var/lib/docker | tail -1 | tr -dc '0-9')
echo "usage after cleanup: ${new_usage}% (reclaimed $(( usage - new_usage )) percentage points)"
docker system df

```

15.13 Script 9 - SSL Certificate Expiry Check

cert_check.sh

```

#!/usr/bin/env bash
set -uo pipefail

```

```

WARN_DAYS=30
CRIT_DAYS=7
ENDPOINTS=("api.example.com:443" "www.example.com:443" "internal.example.com:8443")
status=0

for endpoint in "${ENDPOINTS[@]"; do
    host="${endpoint%:*}"
    port="${endpoint#*:}"

    end_date=$(echo | timeout 10 openssl s_client -connect "$endpoint" -servername "$host" 2>/dev/null \
        | openssl x509 -noout -enddate 2>/dev/null | cut -d= -f2)

    if [[ -z "$end_date" ]]; then
        printf '%-35s UNKNOWN - could not retrieve certificate\n' "$endpoint"
        status=2
        continue
    fi

    end_epoch=$(date -d "$end_date" +%s)
    days_left=$(( (end_epoch - $(date +%s)) / 86400 ))

    if (( days_left < 0 )); then level="EXPIRED "; status=2
    elif (( days_left <= CRIT_DAYS )); then level="CRITICAL"; status=2
    elif (( days_left <= WARN_DAYS )); then level="WARNING "; (( status < 1 )) && status=1
    else level="OK "
    fi

    printf '%-35s %s %4d days (expires %s)\n' "$endpoint" "$level" "$days_left" "$end_date"
done

exit "$status"

```

15.14 Script 10 - Container Entrypoint

entrypoint.sh - the pattern every containerised app needs

```

#!/usr/bin/env sh
set -eu

# 1. Validate required configuration up front, with a clear message.
: "${DB_HOST:?DB_HOST is required}"
: "${DB_PORT:=5432}"
: "${APP_PORT:=8080}"

# 2. Read secrets from files if the _FILE convention is used (Docker/K8s secrets).
if [ -n "${DB_PASSWORD_FILE:-}" ] && [ -f "$DB_PASSWORD_FILE" ]; then
    DB_PASSWORD=$(cat "$DB_PASSWORD_FILE")
    export DB_PASSWORD
fi

# 3. Wait for dependencies - do NOT assume ordering.
echo "waiting for ${DB_HOST}:${DB_PORT}"
i=0
until nc -z "$DB_HOST" "$DB_PORT" 2>/dev/null; do
    i=$(( i + 1 ))
    if [ "$i" -ge 60 ]; then
        echo "database not reachable after 60s" >&2
        exit 1
    fi
    sleep 1
done
echo "database reachable"

# 4. Run migrations once, guarded, if this instance is designated to.
if [ "${RUN_MIGRATIONS:-false}" = "true" ]; then
    echo "running migrations"
    ./manage.py migrate --noinput
fi

# 5. exec replaces the shell, so the application becomes PID 1 and receives SIGTERM.

```

```
echo "starting application on port ${APP_PORT}"
exec "$@"
```

EXEC "\$@" IS THE WHOLE POINT OF AN ENTRYPOINT SCRIPT

Without `exec`, the shell stays as PID 1 and does not forward SIGTERM to your application, so `docker stop` and Kubernetes pod termination wait the full grace period and then SIGKILL - dropping in-flight requests on every single deployment. With `exec`, the application replaces the shell and receives signals directly. This is a high-value detail that very few candidates know.

15.15 Bash Interview Questions

Q. 1. What does `set -euo pipefail` do?

`-e` exits on any command failure, `-u` errors on an unset variable, and `pipefail` makes a pipeline return failure if any stage failed rather than only the last. Together they turn silent partial failure into a loud stop, which is what you want in a deployment script.

Q. 2. Why quote variables?

An unquoted variable is word-split and glob-expanded. A path containing a space becomes two arguments, and an empty variable disappears entirely - which is how `rm -rf $DIR/` becomes `rm -rf /`. Quoting removes both problems, so the rule is to quote every expansion unless you specifically want splitting.

Q. 3. Difference between `[` and `[[`?

`[` is a command and follows normal word-splitting rules, so an unquoted empty variable produces a syntax error. `[[` is a bash keyword that does not word-split, and it supports pattern matching with `==` and regex with `~=`. In bash scripts I use `[[`.

Q. 4. Difference between `*$*` and `@$@`?

Quoted, `"$*"` joins all arguments into a single word and `"$@"` preserves them as separate words. `"$@"` is almost always what you want when passing arguments through to another command.

Q. 5. What does `exec "$@"` do in an entrypoint script?

It replaces the shell process with the given command, so the application becomes PID 1 and receives signals directly. Without it the shell stays PID 1 and does not forward SIGTERM, so the container takes the full grace period to stop and then gets killed, dropping connections on every deploy.

Q. 6. How do you read a file line by line correctly?

`while IFS= read -r line; do ...; done < file`. `IFS=` preserves leading and trailing whitespace and `-r` stops backslash interpretation. Using `for line in $(cat file)` splits on whitespace instead of newlines, which breaks any line containing spaces.

Q. 7. What is the pipeline subshell problem?

Each stage of a pipeline runs in a subshell, so variables assigned inside `cmd | while read ...` are lost when the pipeline ends. The fix is process substitution - `while read ...; done < <(cmd)` - which keeps the loop in the current shell, or `shopt -s lastpipe`.

Q. 8. How do you handle filenames with spaces in `find`?

`find ... -print0` with `xargs -0`, or `while IFS= read -r -d '' file; do ...; done < <(find ... -print0)`. Null separation is the only separator that cannot appear in a filename.

Q. 9. What is a trap and why use one?

A handler for signals and for script exit. `trap cleanup EXIT INT TERM` guarantees `cleanup` runs whether the script succeeds, fails or is interrupted - removing temp files, releasing locks, or rolling back a partially completed deployment. Without it, an aborted script leaves the system in a half-changed state.

Q. 10. How do you implement retry with backoff?

A loop with a maximum attempt count, a delay that doubles each time up to a cap, and a check that the error is worth retrying. `until command; do ... sleep $delay; delay=$((delay*2)); done` with a guard that returns failure once attempts are exhausted. Unbounded retries hide failures rather than handling them.

Q. 11. What is the difference between `$(command)` and backticks?

They both substitute command output, but `$( )` nests cleanly and handles quoting properly, while backticks require escaping when nested. `$( )` is the modern form and the only one I use.

Q. 12. How do you check whether a command exists?

`command -v docker >/dev/null 2>&1 || { echo "docker required"; exit 1; }. I prefer command -v over which because it is a shell builtin, is POSIX, and does not depend on an external binary that may not be installed.`

Q. 13. What does `${VAR:?message}` do?

Exits with that error message if `VAR` is unset or empty. It is the concise way to validate required configuration at the top of a script rather than failing confusingly twenty lines later.

Q. 14. What is the difference between `>` and `>>`, and what is `2>&1`?

`>` truncates, `>>` appends. `2>&1` redirects `stderr` to wherever `stdout` currently points - and order matters, so `> file 2>&1` sends both to the file while `2>&1 > file` does not.

Q. 15. Why does a cron job fail when the script works interactively?

Cron runs with a minimal environment - a short `PATH`, no `~/.bashrc`, a different working directory, and often a different user. Scripts calling `aws`, `kubectl` or `docker` by bare name fail with "command not found". The fixes are absolute paths, setting `PATH` at the top of the crontab, and always redirecting output to a log so the failure is visible.

Q. 16. How do you prevent two copies of a script running at once?

`flock` - either `flock -n /var/lock/myscript.lock -c ./script.sh` or acquiring the lock inside the script on a file descriptor. It is essential for cron jobs that can overrun their interval, otherwise a slow backup starts overlapping with itself and the load spirals.

Q. 17. How do you make a script safe to re-run?

Check state before acting - `mkdir -p` rather than `mkdir`, test before adding a config line, skip a download if the checksum already matches. Anything driven by cron or a pipeline may be retried, so idempotency is a requirement, not a nicety.

Q. 18. What is the difference between `sh` and `bash` in a shebang?

`#!/bin/sh` is POSIX shell, which on many systems is `dash` - no arrays, no `[[`, no `local` in some versions. `#!/usr/bin/env bash` gets `bash` wherever it is installed. Using `bash` features under a `sh` shebang is a classic source of "works on my machine, fails in the alpine container".

Q. 19. How do you debug a bash script?

`bash -x script.sh` or `set -x` around the suspect section to trace each command with expansions applied; `set -e` to stop at the first failure; `PS4='+${BASH_SOURCE}:${LINENO}: '` to include the line number in the trace. And `shellcheck` before any of that - it catches most bugs statically, including unquoted variables.

Q. 20. When would you choose Python over Bash?

When there is real logic - parsing JSON or CSV, branching differently on different error types, calling APIs with retries, or anything beyond roughly fifty lines. Bash is excellent for chaining commands and for deployment steps; it becomes unmaintainable once you need data structures or structured error handling.

PART 15 RECALL POINTS

- `set -euo pipefail` and quote every variable.
- `trap cleanup EXIT INT TERM` for anything that changes state.
- `while IFS= read -r line; do ... done < file` - never `for x in $(cat)`.
- Pipelines create subshells; use process substitution to keep variables.
- `find -print0` with `read -r -d ''` for filenames with spaces.
- `exec "$@"` in entrypoints so the app gets `SIGTERM`.
- `flock` for cron jobs that could overlap.
- Dry run by default; log with timestamps; exit non-zero on failure.
- Run `shellcheck` on everything.

PART 16

PRODUCTION TROUBLESHOOTING HANDBOOK

20 incidents, each with the full method

Your CV's strongest differentiator is production support: "Investigate production incidents using application logs, Windows services, Linux commands, database connectivity checks and network diagnostics; perform RCA, coordinate fixes with development and QA." This part turns that into twenty runbooks. Each one follows the same structure, and that structure is itself the answer to "how do you troubleshoot?"

16.1 The Universal Method

SYMPTOM	What is the user-visible effect? Scope: everyone or some? Since when?
POSSIBLE CAUSES	List them before touching anything - stops tunnel vision
FIRST CHECK	The cheapest test that eliminates the most causes
COMMANDS	Specific, in order, least invasive first
LOGS	Application, system, proxy, platform - correlated by timestamp
ISOLATION	Cut the problem in half: which layer, which host, which dependency?
FIX	Mitigate FIRST (restore service), then correct
VALIDATION	Prove it is fixed with the same signal that showed it broken
RCA	Why it happened, why it was not caught, how long detection took
PREVENTION	An action item with an owner and a date

The shape of every incident response

THE FOUR QUESTIONS THAT START EVERY INCIDENT

- 1) **What changed?** Deploy, config, certificate, DNS, scaling event, a dependency's release. Most incidents follow a change.
- 2) **What is the blast radius?** One user, one host, one AZ, everyone? That alone eliminates most causes.
- 3) **When did it start?** Correlate with deploys, cron schedules and traffic patterns.
- 4) **Is it getting worse?** A trend decides whether you mitigate immediately or investigate first.

MITIGATE BEFORE YOU DIAGNOSE

Say this out loud in an interview: "In a live incident my first goal is restoring service, not understanding the cause. If it started at a deploy, I roll back first and investigate afterwards, because the rollback both restores users and confirms the hypothesis. I preserve evidence before restarting anything - logs, a heap dump, the failing container image - so the RCA is still possible."

16.2 Incident 1 - Server Down / Unreachable

Stage	Detail
SYMPTOM	No SSH, no HTTP, monitoring shows the host as down, or all checks stale
POSSIBLE CAUSES	Instance stopped or terminated; kernel panic or OOM of the whole host; disk full preventing sshd; network/security group change; failed boot after a reboot; AWS hardware failure; certificate or auth breakage (that is NOT "down")
FIRST CHECK	Is it down, or unreachable? Cloud console instance state and status checks decide this in ten seconds
COMMANDS	<code>aws ec2 describe-instance-status --instance-ids i-xxx, aws ec2 get-console-output --instance-id i-xxx, ping, nc -zv host 22, mtr host</code>

Stage	Detail
LOGS	Console output for boot failures; after recovery, <code>journalctl -b -1</code> for the previous boot, <code>dmesg -T</code>
ISOLATION	Can another host in the same subnet reach it? If yes, it is the host. If no host in the subnet is reachable, it is network/routing. If only one AZ is affected, suspect the AZ
FIX	System status check failed = AWS hardware: stop and start to migrate to new hardware (do NOT reboot - reboot keeps the same host). Instance check failed = inside the OS: use the console output, or detach the root volume, mount it elsewhere and fix <code>/etc/fstab</code> or the full disk
VALIDATION	Status checks pass, SSH works, the service answers, monitoring returns
RCA	Why did it fail, and why did nothing catch it earlier - was there a disk alert at 80%?
PREVENTION	Run behind an ASG with ELB health checks so a single instance failure is not an incident; CloudWatch auto-recover alarm on <code>StatusCheckFailed_System</code> ; alert on disk before it fills

16.3 Incident 2 - CPU at 100%

Stage	Detail	
SYMPTOM	Latency up, timeouts, load average far above core count	
POSSIBLE CAUSES	Traffic spike; a runaway process or infinite loop; a cron/batch job; GC thrashing; CPU credits exhausted on a t-family instance; CPU steal from the hypervisor; the load is actually I/O wait, not CPU	
FIRST CHECK	<code>top</code> and press 1 - is it user, system, iowait or steal? Those are four different problems	
COMMANDS	<code>top -o %CPU, `ps -eo pid,%cpu,etime,cmd --sort=-%cpu \</code>	<code>head, vmstat 1 5, pidstat 1, top -H -p PID for threads, uptime` for trend</code>
LOGS	Application logs around the start time; deployment history; cron logs	
ISOLATION	All instances or one? All = traffic or a release. One = that host or an uneven load balancer. Does it correlate with a cron schedule?	
FIX	Mitigate: scale out, or kill/restart the offending process, or roll back the release. High <code>st</code> means the instance type is the problem - resize or move off burstable. High <code>wa</code> means go to disk, not CPU	
VALIDATION	CPU back to baseline, latency recovered, queue drained	
RCA	What consumed the CPU and why it was not bounded	
PREVENTION	Autoscaling on a meaningful metric; alerting at 70-80% not 100%; CPU limits on containers; avoid burstable instance types for sustained load	

16.4 Incident 3 - Memory Exhausted / OOM

Stage	Detail		
SYMPTOM	Processes killed unexpectedly, service restarts, severe slowness, swap thrashing		
POSSIBLE CAUSES	Memory leak; a limit set too low; an unbounded cache or query result; too many workers configured; JVM heap sized from host RAM rather than the container limit; a legitimate load increase		
FIRST CHECK	<code>free -h</code> - look at available , not free - and <code>`dmesg -T`</code>	<code>grep -i oom` for evidence of a kill</code>	
COMMANDS	<code>free -h, vmstat 1 5 (si/so columns), `ps -eo pid,rss,%mem,cmd --sort=-rss \</code>	<code>head, dmesg -T \</code>	<code>grep -i "killed process", in Kubernetes <code>kubectl describe pod` for <code>OOMKilled` and exit 137</code></code></code>

Stage	Detail		
LOGS	Kernel log for the OOM killer's choice and scores; application logs for allocation failures		
ISOLATION	Does RSS grow steadily over hours and never fall? That is a leak. Does it spike with traffic? That is sizing. Does it start after a release? That is the release		
FIX	Immediate: restart the process to reclaim, raise the limit, add memory, reduce worker count. Then fix the cause - for a JVM set <code>-XX:MaxRAMPercentage=75</code> so the heap respects the container limit		
VALIDATION	Memory stable over several hours under normal traffic; no further OOM events		
RCA	Leak versus sizing, and why memory was not monitored - CloudWatch has no memory metric without the agent, which is a very common gap		
PREVENTION	Memory metrics and alerting; container limits set from measured usage; heap sized against the limit; alert on restart count, not just on memory		

16.5 Incident 4 - Disk Full

Stage	Detail		
SYMPTOM	"No space left on device", writes fail, database refuses connections, services will not start, sshd may fail		
POSSIBLE CAUSES	Unrotated logs; a log level left at DEBUG; Docker images and stopped containers; core dumps; a runaway temp file; deleted-but-open files; inode exhaustion with blocks free		
FIRST CHECK	<code>df -h</code> AND <code>df -i</code> - inodes are the case people miss		
COMMANDS	<code>df -h, df -i, du -xh --max-depth=1 / \</code>	<code>sort -h, then descend; lsof +L1 or lsof \</code>	<code>grep deleted; docker system df; journalctl --disk-usage`</code>
LOGS	Which directory grew, and since when - <code>ls -ltr</code> in the offender		
ISOLATION	One filesystem or all? One host or the whole fleet? A fleet-wide fill usually means a logging change shipped in a release		
FIX	Reclaim safely: compress and archive logs rather than deleting live ones; <code>journalctl --vacuum-size=200M</code> ; <code>docker system prune -af --filter "until=72h"</code> ; clear package caches. If space does not return after deleting, a process holds the descriptor - restart it. Then grow the volume: <code>aws ec2 modify-volume, growpart, resize2fs</code>		
VALIDATION	<code>df -h</code> shows headroom, services recovered, writes succeed		
RCA	What produced the data and why nothing bounded it		

Stage	Detail		
PREVENTION	logrotate with size and count limits; Docker log driver max-size/max-file; retention on log groups; alert at 80% AND a <code>predict_linear</code> alert giving hours of warning; separate partitions for <code>/var/log</code> so filling it does not take down the root filesystem		

16.6 Incident 5 - Application Down

Stage	Detail	
SYMPTOM	Users report errors; health checks failing; 5xx at the load balancer	
POSSIBLE CAUSES	Process crashed or never started after a deploy; dependency unavailable; config or secret error; port conflict; resource exhaustion; certificate expiry	
FIRST CHECK	Is the process running and listening? <code>systemctl status app</code> and <code>ss -tulpn \</code>	<code>grep :8080`</code>
COMMANDS	<code>systemctl status app -l, journalctl -u app -n 200 --no-pager, ss -tulpn, curl -v localhost:8080/health, kubectl get pods and kubectl logs --previous</code>	
LOGS	Application log first, then the proxy's error log, then the platform's events	
ISOLATION	Works on localhost but not remotely = network/bind address. Fails on localhost = the application. One instance or all?	
FIX	If it followed a deploy, roll back first. Otherwise restart with evidence preserved, fix the config, or restore the dependency	
VALIDATION	Health endpoint 200 from the load balancer's perspective, error rate back to baseline, and confirm with real traffic not just a local curl	
RCA	The failure and the detection gap - how long between failure and alert?	
PREVENTION	Readiness probes and load balancer health checks; alert on healthy host count; automated rollback on a failed deployment health check	

16.7 Incident 6 - Database Unavailable

Stage	Detail
SYMPTOM	Application errors mentioning connection refused, timeout, too many connections, or authentication failure
POSSIBLE CAUSES	Database down or failing over; connection pool exhausted or leaked; <code>max_connections</code> reached; credentials rotated; security group change; disk full on the database; a long-running lock; DNS pointing at the old endpoint after failover
FIRST CHECK	Can the application host reach the database port at all? <code>nc -zv db-host 5432</code> - that separates network from database
COMMANDS	<code>nc -zv, dig db-host, psql -h host -U user -c "select 1" or mysql -h host -e "select 1", RDS CloudWatch DatabaseConnections, FreeStorageSpace, CPUUtilization, ReplicaLag, and SHOW PROCESSLIST / pg_stat_activity</code>
LOGS	Application connection errors with exact messages; RDS logs; slow query log
ISOLATION	Refused = reachable, not listening or not accepting. Timeout = network/security group. Auth error = credentials. Too many connections = pooling
FIX	Restart the application to release leaked connections as mitigation; raise <code>max_connections</code> or add a pooler like PgBouncer; kill a blocking query; fail over; correct the security group or endpoint
VALIDATION	Connections succeed, error rate falls, connection count stable rather than climbing
RCA	Pool leak versus real load versus a change; also why the application did not degrade gracefully
PREVENTION	Connection pooling with sane limits and timeouts; alert on connection count approaching max and on replica lag; retry with backoff in the application; Multi-AZ with tested failover

16.8 Incident 7 - Jenkins Failure

Stage	Detail
SYMPTOM	Builds failing, queued forever, or the UI is unavailable
POSSIBLE CAUSES	Agent offline; disk full on controller or agent; expired credentials; plugin upgrade; Git host unreachable; a moved base image or dependency; JVM heap exhaustion on the controller
FIRST CHECK	Compare the console log of the last good build with the first bad one and find the first line that differs
COMMANDS	On the agent: <code>df -h</code> , <code>systemctl status jenkins</code> , agent log. In Jenkins: Manage Jenkins -> Nodes, System Log; <code>journalctl -u jenkins</code>
LOGS	Build console output from the top; the node's log; <code>\$JENKINS_HOME/logs</code>
ISOLATION	One job or all jobs? One agent or all? A single job means the job or its dependency; all jobs means the controller, the network or a plugin
FIX	Free disk (the number one cause of an agent going offline), reconnect or replace the agent, roll back the plugin, rotate the credential, pin the dependency that moved
VALIDATION	Re-run the failing build and a known-good build; confirm agents are online with free executors
RCA	Usually "something external moved" - an unpinned dependency or plugin - or disk
PREVENTION	<code>buildDiscarder</code> and <code>cleanWs()</code> ; disk alerts on agents; pinned plugin versions tested on a staging controller; pinned shared library versions; ephemeral agents so state cannot accumulate

16.9 Incident 8 - Docker Failure

Stage	Detail
SYMPTOM	Containers will not start, restart repeatedly, or the daemon is unresponsive
POSSIBLE CAUSES	Image pull failure; bad configuration or missing env var; port conflict; OOM; disk full on <code>/var/lib/docker</code> ; permissions on a bind mount; daemon crash
FIRST CHECK	<code>docker ps -a</code> for exit codes, then <code>docker logs --tail 100 <name></code>
COMMANDS	<code>docker ps -a</code> , <code>docker logs --tail 200 <c></code> , <code>docker inspect -f '{{.State.ExitCode}} {{.State.OOMKilled}}' <c></code> , <code>docker system df</code> , <code>systemctl status docker</code> , <code>journalctl -u docker</code>
LOGS	Container logs; the daemon's journal
ISOLATION	Exit 137 = OOM. 127 = command not found. 126 = not executable. 1 = application error. 0 repeatedly = the process finishes (a daemonising process that needs foreground mode)
FIX	Correct the image tag or registry credentials; fix the env/config; raise the memory limit; prune disk; correct bind-mount ownership or run with the right UID
VALIDATION	Container stays up through several minutes and passes its health check
RCA	Configuration, resources or image
PREVENTION	Immutable image tags; health checks; memory limits; log driver size limits; scheduled prune on build hosts

16.10 Incident 9 - Kubernetes Failure

Stage	Detail
SYMPTOM	Pods not running, service unreachable, rollout stuck
POSSIBLE CAUSES	See Part 11 - every Kubernetes incident reduces to one of: scheduling, image, crash, readiness, endpoints, DNS, resources or storage
FIRST CHECK	<code>kubectl get pods -o wide</code> then <code>kubectl describe pod <pod></code> - the Events section states the cause
COMMANDS	<code>kubectl get pods -o wide</code> , <code>kubectl describe pod</code> , <code>kubectl logs --previous</code> , <code>kubectl get events --sort-by=.lastTimestamp</code> , <code>kubectl get endpoints</code> , <code>kubectl top pods</code> , <code>kubectl describe node</code>
LOGS	Container logs including <code>--previous</code> ; cluster events; kubelet journal on the node
ISOLATION	Failures on one node = node problem. Across all nodes = application or config. Pending = scheduling. CrashLoop = application. Running but unreachable = readiness or Service
FIX	<code>kubectl rollout undo</code> to restore service first, then fix. Cordon and drain a bad node
VALIDATION	Pods Ready, endpoints populated, <code>rollout status</code> completes, health endpoint responds through the Service

Stage	Detail
RCA	Which of the eight categories, and why the pipeline did not catch it
PREVENTION	Resource requests and limits; correct probes; PodDisruptionBudgets; anti-affinity; a deployment gate that waits on rollout status and rolls back on failure

16.11 Incident 10 - Network Timeout

Stage	Detail
SYMPTOM	Requests hang then fail; intermittent failures; slow responses from one dependency
POSSIBLE CAUSES	Security group or NACL; route table; NAT gateway problem; packet loss on a path; MTU on a tunnel; DNS returning a dead IP; the target genuinely overloaded
FIRST CHECK	Timeout or refused? Timeout means dropped packets, refused means reachable-but-not-listening
COMMANDS	<code>nc -zv host port</code> , <code>mtr host</code> over a few minutes for per-hop loss, <code>tracert</code> , <code>dig ip route get IP</code> , VPC Flow Logs filtered on REJECT
LOGS	Flow logs; load balancer access logs; application client-side timeout logs
ISOLATION	Test from the same subnet, then from another subnet, then from outside. Each hop that works eliminates a layer
FIX	Correct the security group, NACL (remember the ephemeral return range) or route; replace a failing NAT gateway; lower MTU or enable MSS clamping
VALIDATION	Consistent connectivity over a sustained test, not a single successful curl
RCA	Which layer dropped the traffic and what changed
PREVENTION	Flow logs enabled; NAT gateway per AZ; connection and read timeouts set in every client; synthetic checks between tiers

16.12 Incident 11 - DNS Issue

Stage	Detail
SYMPTOM	"Could not resolve host"; intermittent failures; traffic going to an old endpoint
POSSIBLE CAUSES	Record missing or wrong; TTL caching an old value; resolver unreachable; CoreDNS down or throttled in Kubernetes; a stale <code>/etc/hosts</code> ; split-horizon DNS; JVM caching resolution forever
FIRST CHECK	<code>dig name</code> and <code>dig @8.8.8.8 name</code> - if the authoritative answer is right but yours is wrong, it is caching or your resolver
COMMANDS	<code>dig</code> , <code>dig +trace</code> , <code>getent hosts</code> , <code>cat /etc/resolv.conf</code> , <code>nslookup</code> from inside a pod, CoreDNS pod status and logs
LOGS	CoreDNS logs; resolver logs; application DNS error messages
ISOLATION	Fails everywhere = the record. Fails from one network = that resolver. Fails intermittently = one of several records is dead, or CoreDNS is under-provisioned
FIX	Correct the record; flush or wait out the TTL; restore the resolver; scale CoreDNS; remove the stale hosts entry
VALIDATION	Resolution correct from multiple vantage points and consistently over time
RCA	Change management around DNS, and TTL planning
PREVENTION	Lower TTLs before planned changes; monitor resolution latency and CoreDNS errors; avoid hardcoding IPs; set the JVM DNS TTL explicitly

16.13 Incident 12 - SSL/TLS Issue

Stage	Detail		
SYMPTOM	Browser warnings; <code>certificate verify failed</code> ; handshake failures; a client that suddenly cannot connect		

Stage	Detail		
POSSIBLE CAUSES	Expired certificate; missing intermediate chain; hostname/SAN mismatch; untrusted internal CA; protocol or cipher mismatch after hardening; clock skew		
FIRST CHECK	<code>`echo \</code>	<code>openssl s_client -connect host:443 -servername host 2>/dev/null \</code>	<code>openssl x509 -noout -dates -subject -issuer`</code>
COMMANDS	<code>openssl s_client -connect host:443 -servername host -showcerts, curl -vI https://host, check ACM/cert-manager status</code>		
LOGS	Proxy error log; client error text - it usually names the exact failure		
ISOLATION	Fails in curl but works in a browser = missing intermediate. Fails everywhere with "expired" = renewal. Fails for one client = that client's trust store or an old TLS version		
FIX	Renew and deploy the certificate with the full chain; correct the SAN; distribute the internal CA; align the TLS version		
VALIDATION	Handshake succeeds from multiple clients; the expiry date is far in the future		
RCA	Renewal was manual, or the automation failed silently		
PREVENTION	ACM or cert-manager for automatic renewal; <code>probe_ssl_earliest_cert_expiry</code> alerting 21-30 days out; serve the full chain by default		

16.14 Incident 13 - Deployment Failure

Stage	Detail
SYMPTOM	Pipeline red at the deploy stage, or green but the application is broken
POSSIBLE CAUSES	Bad artifact; missing config or secret in the target environment; incompatible database migration; insufficient capacity; health check misconfigured; permissions on the deploy credential
FIRST CHECK	Which stage failed and what was the exit code? Then: are the new instances or pods healthy?
COMMANDS	Pipeline console log; <code>kubectl rollout status</code> , <code>kubectl describe pod</code> , <code>kubectl logs --previous</code> ; target group health; <code>systemctl status</code> on the target
LOGS	Deploy logs, application startup logs, platform events
ISOLATION	Did the artifact change, the config change, or the environment change? Diff the working release against the failing one
FIX	Roll back immediately - previous image tag, <code>rollout undo</code> , or previous release symlink - then reproduce in a lower environment
VALIDATION	Health checks pass, error rate at baseline, and a smoke test of the critical user path
RCA	Why staging did not catch it - usually an environment difference in config, data or scale
PREVENTION	Build once and promote the same artifact; environment parity; smoke tests in the pipeline; automated rollback on a failed health check; progressive delivery so the blast radius is small

16.15 Incident 14 - Rollback Required

Stage	Detail
SYMPTOM	The new release is live and causing errors

Stage	Detail
DECISION	Roll back if: users are affected, the cause is not obvious within a few minutes, or the fix is not trivially safe. Fix forward only if the fix is one line, tested, and rollback is impossible - for example after an irreversible migration
COMMANDS	<code>kubectl rollout undo deployment/app</code> ; redeploy the previous image tag; re-run the last good pipeline; repoint the blue-green load balancer; <code>ln -sfn</code> the previous release directory
CAUTION	Rollback covers code, not data. A migration that dropped a column or rewrote rows cannot be undone by redeploying the old image. Check migrations before rolling back
VALIDATION	Error rate returns to baseline; confirm the running version by image digest, not by the tag
RCA	Then, with service restored, investigate the failing artifact - which you still have because tags are immutable
PREVENTION	Backward-compatible migrations using expand-and-contract; feature flags so a behaviour change can be disabled without redeploying; rehearsed, automated rollback

16.16 Incident 15 - Monitoring Alert Storm

Stage	Detail
SYMPTOM	Dozens of alerts at once, or an alert that fires and clears repeatedly
POSSIBLE CAUSES	One root cause with many symptom alerts (a node or AZ failure); a monitoring outage producing false alarms; thresholds too tight; no <code>for:</code> duration; a maintenance window without a silence
FIRST CHECK	Look at the earliest alert by timestamp - the first one is usually the cause and the rest are symptoms
COMMANDS	Alertmanager UI grouped view; <code>kubectl get nodes</code> ; cloud health dashboard; check whether Prometheus itself is healthy
ISOLATION	Do all the alerts share a label - the same node, AZ, cluster or service? That is your root cause
FIX	Address the single root cause; silence the symptom alerts while you work so the channel stays readable
VALIDATION	Alerts clear on their own, not by being silenced
RCA	Both the technical cause and the alerting design that produced fifty notifications for one event
PREVENTION	Inhibition rules so a cause suppresses symptoms; grouping in Alertmanager; <code>for:</code> durations; symptom-based alerting; a regular review of which alerts fired and what action followed

16.17 Incident 16 - Application Latency

Stage	Detail
SYMPTOM	Requests slow, p95/p99 up, timeouts at the edge, no outright errors
POSSIBLE CAUSES	Slow database query or missing index; connection or thread pool saturation; GC pauses; CPU throttling from a container limit; a slow downstream dependency; disk I/O or exhausted EBS burst balance; DNS resolution latency; traffic increase without capacity
FIRST CHECK	Where is the time spent? Compare edge latency with application latency with database latency - the gap tells you which layer
COMMANDS	<code>curl -w '%{time_namelookup} %{time_connect} %{time_starttransfer} %{time_total}'</code> to split DNS, connect and server time; <code>kubectl top pods</code> ; <code>container_cpu_cfs_throttled_seconds_total</code> ; slow query log; <code>iostat -xz 1</code> ; EBS BurstBalance
LOGS	Application timing logs, slow query log, proxy upstream response time
ISOLATION	All endpoints or one? All instances or one? Latency proportional to load, or constant? Constant added latency usually means DNS, a retry, or a handshake; load-proportional means saturation
FIX	Add the index, raise the pool, remove the CPU limit bottleneck, scale out, cache, or make the slow path asynchronous
VALIDATION	p95 and p99 back to baseline under the same traffic, not just the average
RCA	Which resource saturated first, and why capacity headroom was not monitored
PREVENTION	Percentile-based latency SLOs and alerts; capacity headroom alerting; load testing before releases; query performance review in code review

16.18 Incident 17 - HTTP 500

Stage	Detail
SYMPTOM	The application returns 500 to users
MEANING	The application code raised an unhandled error - so DNS, TCP, TLS and routing all worked
FIRST CHECK	The application log for the stack trace, correlated to the exact request time; then whether it started at a deploy
COMMANDS	<code>kubectl logs -f deploy/app, journalctl -u app</code> , Logs Insights filtered on ERROR and grouped by endpoint, and the ALB HTTPCode_Target_5XX_Count metric
ISOLATION	Every request or some? One endpoint or all? One instance or all? Started with a deploy or with a dependency failure?
FIX	Roll back if it followed a release; otherwise restore the dependency or correct the configuration
VALIDATION	5xx rate back to zero at the load balancer, not just "the error disappeared from the log"
RCA	The specific exception, plus why an unhandled exception reached the user
PREVENTION	Error rate alerting; tests covering that path; graceful degradation so a dependency failure returns a controlled response instead of a 500

16.19 Incident 18 - HTTP 502

Stage	Detail
SYMPTOM	The proxy or load balancer returns 502 Bad Gateway
MEANING	The proxy reached a backend but got an invalid response or the connection was closed unexpectedly
POSSIBLE CAUSES	Backend crashed or is restarting (OOMKilled is classic); proxy pointing at the wrong port; backend speaking HTTPS while the proxy expects HTTP; response headers too large; backend killed mid-response; keep-alive timeout mismatch where the backend closes the connection before the load balancer does
COMMANDS	Proxy error log (<code>/var/log/nginx/error.log</code>); <code>curl -v http://backend:8080/health</code> from the proxy host; <code>kubectl get pods</code> restart counts; ALB HTTPCode_ELB_5XX versus Target_5XX
ISOLATION	Intermittent 502 with healthy pods usually means the keep-alive timeout mismatch; constant 502 means wrong port, wrong protocol, or a dead backend
FIX	Restore the backend; correct the upstream port or protocol; increase header buffers; set the backend's keep-alive timeout above the load balancer's idle timeout
VALIDATION	Sustained clean requests, including at the rate that previously produced errors
RCA	Which of the causes, and whether the backend was crashing silently
PREVENTION	Restart-count alerting; correct memory limits; a documented port and protocol contract between proxy and service; keep-alive timings reviewed

16.20 Incident 19 - HTTP 503

Stage	Detail
SYMPTOM	503 Service Unavailable
MEANING	There is no healthy backend to serve the request
POSSIBLE CAUSES	All targets failing health checks; a Kubernetes Service with zero endpoints because no pod is Ready; a deployment that scaled to zero; deliberate maintenance mode; rate limiting; the upstream queue full
COMMANDS	<code>kubectl get endpoints <svc></code> - empty is the answer; <code>kubectl get pods</code> for readiness; ALB target group health and its failure reason; nginx upstream status
ISOLATION	Zero endpoints = readiness or selector. Unhealthy targets = the health check path, port or timing. Rate limiting = check the proxy's configuration
FIX	Get one backend Ready. Very often the fix is the readiness probe itself - wrong path, wrong port, or a timeout shorter than the application's response time under load
VALIDATION	Healthy host count above zero and stable; 503 rate at zero
RCA	Why every replica became unhealthy simultaneously - a common cause is a liveness or readiness probe that checks a shared dependency

Stage	Detail
PREVENTION	Probes that test the process, not its dependencies; minimum replica counts; PodDisruptionBudgets; alerting on healthy host count with missing data treated as breaching

16.21 Incident 20 - HTTP 504

Stage	Detail
SYMPTOM	504 Gateway Timeout after a long wait
MEANING	The backend was reached but did not respond within the proxy's timeout
POSSIBLE CAUSES	Slow database query; a downstream API hanging; thread or connection pool exhausted; deadlock; proxy timeout shorter than the real processing time; an unbounded operation such as a full-table report
COMMANDS	<code>curl -w '%{time_total}'</code> directly against the backend, bypassing the proxy; slow query log; thread dump for a JVM; <code>kubect1 top</code> for throttling; upstream response time metrics
ISOLATION	If the backend is also slow when called directly, it is the application or its dependencies. If it is fast directly but times out through the proxy, it is the proxy configuration or connection pooling in between
FIX	Fix the slow path - index the query, add a cache, bound the result set. Raising the timeout is only correct when the work genuinely takes that long, and then the right answer is usually to make it asynchronous with a job and a status endpoint
VALIDATION	p99 latency well below the timeout; zero 504s under the same load
RCA	Which operation exceeded the budget, and why there was no timeout inside the application
PREVENTION	Timeouts at every layer, decreasing inward; circuit breakers on downstream calls; query performance gates; latency SLOs alerting before the timeout is reached

16.22 The Incident Communication Template

What to post in the incident channel, every 15-30 minutes

```
[INCIDENT] SEV2 - Payments API elevated errors
Status:  INVESTIGATING | IDENTIFIED | MONITORING | RESOLVED
Started:  2026-09-23 14:02 UTC (detected 14:05 by alert HighErrorRate)
Impact:   ~12% of payment requests returning 500. Checkout affected; browsing unaffected.
Current:  Error rate 12%, stable, not increasing.
Actions:  Rolled back release 1.4.3 to 1.4.2 at 14:18. Monitoring recovery.
Next:     Confirm error rate at baseline by 14:35, then investigate 1.4.3 in staging.
Owner:    Aasim | Next update: 14:35 UTC
```

COMMUNICATION IS HALF OF INCIDENT RESPONSE

Interviewers ask "what do you do during an incident?" partly to see whether you mention communication at all. Say: "I post a status update on a fixed cadence even when there is nothing new, because silence makes people interrupt the responders to ask. The update says impact in user terms, what we have done, what we are doing next, and when the next update will be."

16.23 The RCA Template

A blameless RCA that produces real change

```
TITLE      Payments API 500 errors for 34 minutes on 2026-09-23

SUMMARY    Release 1.4.3 introduced a configuration key that was absent in production,
           causing the payment service to throw on startup for 12% of requests.
           Resolved by rolling back to 1.4.2.

IMPACT     34 minutes. Approximately 12% of payment requests failed.
           No data loss. No customer data exposed.

TIMELINE (UTC)
14:02  Release 1.4.3 deployed to production
14:05  HighErrorRate alert fired (3 min detection)
14:07  On-call acknowledged; incident channel opened
14:12  Error correlated with the deploy; stack trace identified a missing config key
```

14:18 Rollback to 1.4.2 initiated
 14:24 Error rate returned to baseline
 14:36 Monitoring period complete; incident closed

ROOT CAUSE The new config key was added to the staging config map but not to production.
 The application read it at startup without a default and raised an exception.

DETECTION Alert fired 3 minutes after deploy. Adequate, but the deployment pipeline
 did not gate on post-deploy error rate, so the bad release stayed live.

MITIGATION Rollback to the previous immutable image tag.

WHAT WENT WELL

- Immutable tags made rollback a one-command operation.
- The alert fired quickly and named the affected service.

WHAT DID NOT

- Configuration drift between staging and production went undetected.
- No automated rollback on post-deploy error-rate regression.
- The application failed hard on a missing optional key instead of defaulting.

ACTION ITEMS

- | | | |
|---|--------------|----------------|
| 1. Config schema validation in CI, comparing environments | OWNER: Aasim | BY: 2026-10-07 |
| 2. Post-deploy error-rate gate with automatic rollback | OWNER: Aasim | BY: 2026-10-14 |
| 3. Application defaults for non-critical config keys | OWNER: Dev | BY: 2026-10-14 |

PART 16 RECALL POINTS

- Always: what changed, blast radius, when it started, is it worsening.
- Mitigate first, preserve evidence, diagnose after.
- Refused versus timeout splits every network problem in half.
- 500 = app, 502 = bad backend response, 503 = no healthy backend, 504 = too slow.
- `df -i` as well as `df -h; lsof | grep deleted` for space that does not return.
- Look at **available** memory, not free; exit 137 means OOM.
- Roll back on any doubt - but check whether a migration makes rollback unsafe.
- Every RCA ends in action items with an owner and a date.
- Communicate on a fixed cadence, even with nothing new.

PART 17

SCENARIO-BASED INTERVIEW QUESTIONS

50 scenarios, each with what is being tested and how to answer

Scenario questions are where offers are decided. The interviewer is not looking for the right answer - they are looking for a method. This part gives you, for each scenario: what the interviewer is actually testing, a strong answer, the step-by-step approach, the commands, likely follow-ups, and the wrong answers that lose marks.

THE UNIVERSAL SCENARIO-ANSWER STRUCTURE

- 1) **Clarify and scope** - "Is it all users or some? When did it start? Has anything been deployed?" Asking this first is itself a scoring point.
- 2) **State the goal** - "My first priority is restoring service; I will preserve evidence as I go."
- 3) **Work in layers** - narrow from user-visible symptom to the failing component, naming the command at each step.
- 4) **Say what each result would mean** - "If it is refused rather than timing out, I know the network path is fine."
- 5) **Fix, validate, and prevent** - always finish with how you would stop it recurring.

17.1 Scenarios 1-10: Availability

Q. 1. Production application suddenly stopped responding. What do you do?

Testing: whether you panic, whether you have a method, and whether you mitigate before diagnosing. **Answer:** "First I establish scope and timing - is it all users or a subset, one environment or all, and when did it start, because that usually points at a change. In parallel I check whether anything was deployed in the last hour, since most sudden failures follow a change. Then I work from the outside in: is DNS resolving correctly, is the load balancer reporting healthy targets, are the instances or pods running and Ready, is the application listening on its port, and can it reach its dependencies. The load balancer's healthy host count and the split between ELB 5xx and target 5xx tell me immediately whether it is the platform or the application. If it correlates with a deploy, I roll back first and investigate afterwards - restoring users is the priority, and the rollback also confirms the hypothesis. I communicate status on a fixed cadence throughout, and once it is stable I do the RCA and turn it into action items." **Follow-ups:** what if there was no deploy; how do you preserve evidence; what if rollback does not fix it. **Wrong answers:** jumping straight to "restart the server"; diagnosing root cause for twenty minutes while users are down; not mentioning communication.

Q. 2. Users report the site is slow but nothing is erroring. Where do you start?

Testing: whether you can reason about latency rather than availability. **Answer:** "I split the request path and find where the time goes. `curl -w` gives me DNS lookup, connect, TLS and time-to-first-byte separately, which immediately tells me whether it is name resolution, the network, or the server. Then I compare load balancer latency with application latency with database latency - the layer where the jump appears is the culprit. I look at percentiles, not averages, because an average hides the tail that users actually notice. The usual causes are a slow query or missing index, a saturated connection or thread pool, CPU throttling from a container limit, GC pauses, or exhausted EBS burst credits. Then I check whether latency is proportional to load, which means saturation, or constant, which usually means a handshake, a retry or DNS." **Follow-ups:** how would you find the slow query; what is CPU throttling; how do you decide between scaling and optimising.

Q. 3. Only some users are affected. What does that tell you?

Testing: whether you use blast radius as a diagnostic. **Answer:** "It eliminates whole classes of cause. If it were DNS or a certificate, everyone would be affected. A subset usually means one of a few things: one instance or pod in the pool is unhealthy but still receiving traffic, so I check per-target metrics rather than aggregates; or one availability zone is affected; or it is data-dependent - a particular account, region or feature flag. I test each backend directly rather than through the load balancer, because an aggregate graph averages away a single bad host. If it is a canary or partial rollout, that is the obvious first suspect."

Q. 4. Everything looks healthy in monitoring but users say it is broken. What now?

Testing: whether you trust dashboards over users. **Answer:** "The users are right and my monitoring has a gap - that is the starting assumption. I reproduce it from the user's position: from outside the network, through the real DNS name, with a real browser or client. Internal checks bypass DNS, the CDN, the WAF and certificate validation, which is exactly where these

failures live. Synthetic monitoring from outside exists for this reason. Then I close the monitoring gap as an action item, because the incident was really two failures - the outage and the blind spot."

Q. 5. Production is down and you have no idea why. What are your first three commands?

Testing: prioritisation under pressure. **Answer:** "It depends on the platform, but the shape is the same: first something that tells me whether the service is reachable at all - the load balancer's healthy target count, or `curl -v` against the health endpoint. Second, the state of the workloads - `kubectl get pods -o wide` or `systemctl status`, which tells me whether things are running, restarting or gone. Third, `df -h` and `free -h` on the hosts, because a full disk or exhausted memory causes an astonishing proportion of unexplained outages. In parallel I ask what changed in the last hour - that question often answers the whole incident before any command does."

Q. 6. An entire availability zone goes down. What happens to your application?

Testing: architectural understanding of failure domains. **Answer:** "If the design is correct, capacity drops but the service stays up: the load balancer stops sending traffic to targets in that AZ, the Auto Scaling Group launches replacements in the healthy AZs, and an RDS Multi-AZ deployment fails over automatically. What breaks it is anything that exists only once - a single NAT gateway, a single-AZ database, or a stateful workload with a volume bound to that AZ. So my answer is that the honest test is to trace the request path and name every component with only one instance, because that list is the real availability of the system."

Q. 7. A deployment completed successfully but users still see the old version.

Testing: understanding of immutability and caching layers. **Answer:** "I verify what is actually running rather than what the pipeline believes - the image digest, not the tag. The most common cause by far is a mutable tag like `latest` with `imagePullPolicy: IfNotPresent`, so the node reused the cached image. Second, check whether pods actually restarted - if their age predates the deploy, the pod template never changed, which happens when only a `ConfigMap` was updated. Third, whether old targets are still in the load balancer's rotation. Fourth, caching further out - CDN, browser, or a client holding a keep-alive connection to an old instance. The permanent fix is immutable tags per build and deploying by digest."

Q. 8. Your health check passes but the application does not work.

Testing: whether you understand what a health check should assert. **Answer:** "The health check is testing the wrong thing - typically it returns 200 from the web framework without touching anything the application actually needs. A useful readiness check verifies the things whose absence makes the instance unable to serve: can it reach its database, is its cache available, has it finished loading configuration. The balance matters though - a readiness check may test dependencies, but a liveness check must not, otherwise a database blip restarts every replica and turns a small problem into an outage."

Q. 9. The application works in staging but fails in production.

Testing: whether you can reason about environment parity. **Answer:** "Something differs and my job is to find what. In order of likelihood: configuration and secrets that exist in one environment and not the other; data - production has volume, edge cases and legacy rows staging does not; scale - concurrency, connection pool limits and timeouts only bite under real load; network - production has stricter security groups, private subnets and a proxy; and permissions - a production IAM role that is more restricted. I diff the configuration between environments first, because that is the single most common answer, and it is also the one worth automating a check for."

Q. 10. Traffic doubled unexpectedly. What happens and what do you do?

Testing: capacity thinking. **Answer:** "First, is it legitimate traffic or an attack or a retry storm - checking the source distribution and the request pattern answers that. If legitimate, I check where saturation is appearing: application CPU and memory, database connections, and any fixed-size pool. Autoscaling handles the stateless tier if the policy is on a meaningful metric, but the database usually does not scale that way, so the bottleneck moves there. Short term I scale out, raise pool sizes carefully, and shed load if necessary via rate limiting rather than letting everything degrade. Then the RCA is about capacity headroom - whether we would have been alerted before saturation rather than at it."

17.2 Scenarios 11-20: CI/CD and Deployment

Q. 11. The Jenkins pipeline worked yesterday and fails today with no code change.

Testing: systematic reasoning about "nothing changed". **Answer:** "Something always changed, just not in our repository. I diff the console log of the last successful build against the first failing one and find the first line that differs - that usually names the cause directly. Then the candidates: a plugin or Jenkins upgrade on the controller; an agent rebuilt with different tool versions; a mutable dependency or base image tag that moved upstream; an expired credential or certificate; disk filling on an agent; or an external service such as the registry or artifact repository being unavailable. Disk on the agent and a moved upstream dependency are the two most frequent in my experience." **Follow-up:** how do you prevent it - pin everything, ephemeral agents, disk alerts.

Q. 12. A build passes locally but fails in CI.

Testing: understanding of build reproducibility. **Answer:** "The difference is almost always environmental. Locally I have a warm cache that may be hiding a broken step, a different tool version, different environment variables, and often different architecture. CI has a clean workspace, restricted network and different credentials. I reproduce by building with `--no-cache` and in a container matching the CI image. The structural fix is to make the build hermetic - containerised builds with pinned tool versions, so local and CI are the same environment by construction."

Q. 13. Your pipeline deploys to production automatically. Is that a good idea?

Testing: judgement, not dogma. **Answer:** "It depends on what supports it. Continuous deployment needs high automated test confidence, feature flags so unfinished work can ship dark, progressive rollout so a bad release reaches few users, and automated rollback triggered by metrics. With those, it is safer than batching changes into a large monthly release. Without them, it just ships failures faster. In my current environment we do continuous delivery with a controlled production step, which fits a product where QA validates releases. I would build the safety mechanisms first and then remove the manual gate, not the other way round."

Q. 14. A pipeline takes 45 minutes. Reduce it.

Testing: whether you measure before optimising. **Answer:** "First I measure per-stage duration - `timestamps()` in Jenkins or the stage timings - because optimising the wrong stage is wasted work. Then: parallelise independent stages like unit tests, integration tests and static analysis; shallow-clone; cache dependencies between builds; use Docker layer caching with a correctly ordered Dockerfile; only run heavy stages when the relevant paths changed; split a long test suite across parallel agents; and move to ephemeral agents if builds are queuing for capacity. I would also ask which stages actually need to gate the merge - some can run after merge rather than blocking it."

Q. 15. A developer says "the pipeline is blocking me". How do you respond?

Testing: collaboration and product thinking. **Answer:** "I treat that as a real problem, not a complaint. I find out specifically what is blocking - a slow pipeline, a flaky test, an unclear failure message, or a gate that is not adding value. Flaky tests are the most corrosive, because they teach people to re-run rather than read failures. I would quarantine the flake immediately so it stops blocking everyone, then fix it properly. The principle is that a pipeline nobody trusts gets bypassed, and a bypassed pipeline protects nothing."

Q. 16. Terraform wants to destroy production infrastructure. What do you do?

Testing: whether you stop and think under pressure. **Answer:** "I do not apply. I read the plan and find exactly which resources and which attribute is forcing replacement - the plan says 'forces replacement' next to it. Then I work out why: an immutable field changed such as an availability zone or a name; a module version bump changed a default; the state was rolled back, partially removed, or I am pointed at the wrong workspace or state key. I verify which state file and which account I am actually using with `aws sts get-caller-identity`. If the change is genuinely necessary, I plan a safe migration - create before destroy, or a manual cutover - rather than letting one apply do it. And this is exactly why `prevent_destroy` on databases and mandatory plan review exist."

Q. 17. Two engineers ran terraform apply at the same time. What happened?

Testing: understanding of state and locking. **Answer:** "With DynamoDB locking configured, the second one waits or fails cleanly and nothing bad happens. Without locking, both read the same state, both apply, and the second write overwrites the first - so state no longer matches reality and the next plan may propose destroying resources Terraform has lost track of. Recovery is restoring the previous state version from the versioned S3 bucket, or importing the orphaned resources. The prevention is locking, which should never be optional, plus applying only from CI so there is one serialised path."

Q. 18. Someone changed a security group in the console. What is the problem and what do you do?

Testing: drift and process. **Answer:** "The immediate problem is that the code no longer matches reality, so the next Terraform apply will revert it - possibly during an unrelated change and possibly causing an outage. I detect it with a scheduled `terraform plan` that alerts on a non-empty plan. Then it is a decision: if the change was correct, I codify it and reconcile with `-refresh-only`; if not, I re-apply to revert. The real fix is process - remove console write access in production so Terraform is the only path, and make the emergency-change procedure explicit so people are not forced to go around the tooling."

Q. 19. Your deploy succeeded but the database migration failed halfway.

Testing: the hardest part of deployment. **Answer:** "This is the scenario where rollback does not save you, so the answer is mostly about prevention. In the moment: stop further rollout, determine exactly which migration steps applied, and decide whether the application version running can work with the partially migrated schema. If the old version cannot, rolling back the code makes things worse. Usually the safest path is to complete or explicitly reverse the migration rather than roll back blindly. Preventing it: migrations must be backward compatible - expand and contract, so the schema change ships before the code that needs it and the old column is dropped a release later; migrations run as a separate gated step, not on every

replica's startup; and they are tested against a production-sized dataset because a migration that is instant on staging can lock a table for minutes in production."

Q. 20. How would you deploy a change that cannot be rolled back?

Testing: risk management. **Answer:** "I would first try to make it rollback-able, because that is usually possible with more work - feature flags, expand-and-contract schema changes, dual writes. If it genuinely cannot be, then the mitigations are: deploy to a small percentage first and watch real metrics; have a tested forward-fix path and the people who can execute it available; take a verified backup and rehearse the restore; schedule it in a low-traffic window; and communicate it as a higher-risk change. The key point is that 'no rollback' raises the bar on everything else, it does not mean you proceed as normal and hope."

17.3 Scenarios 21-32: Kubernetes and Containers

Q. 21. A Kubernetes pod is in CrashLoopBackOff. What do you check?

Testing: the single most common Kubernetes question. **Answer:** "`kubectl logs <pod> --previous` first - the `--previous` flag is essential because the current container may not have started yet, and the dead one holds the error. Then `kubectl describe pod` for the exit code and Last State. Exit 1 is an application error, usually a missing environment variable, an unreachable dependency at startup or bad config. Exit 137 with OOMKilled means the memory limit is too low or there is a leak. Exit 0 repeatedly means the process finishes and exits - a wrong command, or an app that daemonises instead of running in the foreground. Exit 127 means the command is not in the image. A liveness probe failing during a slow startup also produces this, and the fix there is a `startupProbe`, not disabling liveness. If I cannot exec in because it keeps dying, I run the same image with a sleep command to get a shell and inspect the environment."

Q. 22. A pod is stuck in Pending.

Testing: scheduling. **Answer:** "`kubectl describe pod` and read the scheduler's message in Events - it states the reason explicitly. Insufficient CPU or memory means no node has enough allocatable capacity for the pod's requests, so I either reduce requests, add nodes, or check why the cluster autoscaler is not scaling. A taint message means I need a toleration. A node affinity or selector message means nothing matches. 'Unbound immediate PersistentVolumeClaims' means the storage is the problem - often a zone mismatch. Reading that one message solves this scenario almost every time."

Q. 23. A Service returns nothing but the pods are running.

Testing: service networking. **Answer:** "`kubectl get endpoints <service>` - if it is empty, that is the whole diagnosis. Either the Service's label selector does not match the pods' labels, which I confirm with `kubectl get pods --show-labels`, or the pods are Running but not Ready, so they are excluded from endpoints. If endpoints are populated, I check `targetPort` against the actual container port, and then test the pod directly with `kubectl port-forward` to separate the pod from the Service from the Ingress."

Q. 24. Pods are being OOMKilled. Walk me through it.

Answer: "`kubectl describe pod` confirms it - Last State Terminated, Reason OOMKilled, exit code 137. Then I compare actual usage against the limit. If usage is steady just above the limit, the limit is too low and I raise it based on measured usage. If usage climbs steadily over hours until it hits the limit and repeats, that is a leak and raising the limit only lengthens the interval between crashes. For a JVM or Node application I check whether the runtime is sizing its heap from the host's memory rather than the container limit, because that guarantees an OOM kill - the fix is `-XX:MaxRAMPercentage` or `--max-old-space-size`. Memory limits are a hard kill, unlike CPU which is throttled, so the limit must be above the real peak."

Q. 25. A node goes NotReady. What happens to the pods and what do you do?

Answer: "After the toleration period, the node is tainted and its pods are evicted and rescheduled on healthy nodes - assuming there is capacity and their storage allows it. For the node itself, `kubectl describe node` shows the conditions: DiskPressure is the most common in practice, then memory, then a kubelet or container runtime failure, then network or an expired kubelet certificate. I check the kubelet journal on the node. In a cloud environment, replacing the node is usually faster and more reliable than repairing it - cordon, drain, terminate, let the ASG replace it."

Q. 26. A rollout is stuck and `kubectl rollout status` never completes.

Answer: "The new pods are failing in one of the ways we have already covered, so I look at them directly - Pending, ImagePullBackOff, CrashLoopBackOff, or Running but never Ready. `kubectl describe rs` on the new ReplicaSet shows events about why pods could not even be created, such as a quota being exceeded. A specific trap is `maxUnavailable: 0` with no spare capacity: the new pod cannot be scheduled, the old one cannot be removed, and the rollout stalls at step one forever. A PodDisruptionBudget can block progress the same way. Operationally I roll back first to restore a known state, then diagnose."

Q. 27. A container cannot connect to the database. Where do you look?

Answer: "First, is the application using localhost? Inside a container that means the container itself, and it is the single most common containerisation mistake. Then, is name resolution working - nslookup from inside the pod for the service or endpoint name. Then network reachability - can it open the port. Then credentials from the Secret, and whether the Secret in that namespace holds what I expect. Then the database side - is it accepting connections, is it at max connections, does the security group allow the source. I test each step from inside the pod, because the answer from my laptop is not the same question."

Q. 28. Explain what happens when a pod is deleted.

Testing: graceful shutdown. **Answer:** "The pod is marked Terminating and removed from Service endpoints; a preStop hook runs if defined; the container receives SIGTERM; after the termination grace period anything still running gets SIGKILL. Two things commonly go wrong. First, the application ignores SIGTERM - usually because a shell is PID 1 rather than the app, which is why entrypoints must exec. Second, endpoint removal propagates asynchronously across nodes, so for a brief window traffic still arrives at the terminating pod; a short preStop sleep covers that. Without both, every deployment drops in-flight requests."

Q. 29. You need to debug a production pod but the image has no shell.

Answer: "kubectl debug with an ephemeral container attached to the pod's namespaces using an image that does have tools - that is the supported mechanism and it does not require restarting the pod. Otherwise kubectl logs, kubectl describe, and port-forward to test it directly. The deeper point is that distroless images make this harder deliberately, in exchange for a much smaller attack surface, and the right answer is better instrumentation rather than adding a shell back into production images."

Q. 30. Your cluster is running out of capacity. What do you do?

Answer: "Short term, either add nodes or free capacity. Longer term I look at whether requests reflect reality - the most common cause of a 'full' cluster is over-requested pods where the requests are three times the actual usage, so nodes look full while sitting at 20% real utilisation. I compare requests to actual usage across the top consumers and right-size. Then cluster autoscaler configuration, whether there are pods that should have lower priority, and whether any workload should be on different instance types. It is a capacity planning problem, not just a scaling button."

Q. 31. A container works on your laptop but fails in the cluster.

Answer: "The differences are usually: configuration and secrets, which exist in your local env file and not in the cluster; networking, since localhost and host-mounted services do not exist in a pod; storage, where a bind mount becomes a volume with different ownership; user and permissions, because production runs non-root with a read-only root filesystem; resource limits, which do not exist locally; and architecture if you are on Apple silicon building for amd64 nodes. I check the pod's actual environment with kubectl exec ... env and compare it against what the app expects."

Q. 32. How would you find which pod is consuming the most memory across a cluster?

Answer: "kubectl top pods -A --sort-by=memory for the immediate answer, and in Prometheus topk(10, container_memory_working_set_bytes) for history, which matters more because I usually want to see the trend rather than the instant. The more useful query is usage as a fraction of the limit, because a pod using 4 GB of an 8 GB limit is fine while one using 480 MB of a 512 MB limit is about to be killed."

17.4 Scenarios 33-42: Linux, AWS and Networking

Q. 33. EC2 instance is running but SSH does not work.

Testing: layered network diagnosis. **Answer:** "First I distinguish the two failure modes, because they lead to completely different investigations. 'Connection timed out' is a network problem - security group inbound 22, NACL inbound and outbound because it is stateless, route table, public IP, or the host being unreachable. 'Permission denied (publickey)' means I reached sshd, so it is the key, the username or the server's authorized_keys - and the correct default user differs by AMI. I run ssh -v and read where it stops. If neither applies, I check the instance status checks and get-console-output - a bad /etc/fstab entry blocking boot or a full disk breaking sshd both show up there. If SSH is unrecoverable, SSM Session Manager, or detaching the root volume and mounting it on another instance to fix the file."

Q. 34. Disk reaches 95% on a production server at 2am. What do you do?

Answer: "Mitigate first, safely. df -h and df -i to see whether it is blocks or inodes, then du -xh --max-depth=1 / and descend to find the offender. I compress and archive logs rather than deleting anything live, vacuum the journal, and prune Docker objects with a time filter if it is a container host. If deleting a large file does not return space, a process is holding the descriptor - lsof | grep deleted - and I restart that process. Once stable, I grow the volume if that is the real answer. Then the RCA is why nothing alerted at 80%, and the prevention is logrotate, log driver limits, retention policies and a predictive alert that gives hours of warning."

Q. 35. CPU suddenly hits 100% on all application servers simultaneously.

Answer: "Simultaneous across all servers rules out a single bad host and points at either a traffic change or something we shipped. I check request rate first - if traffic is flat and CPU jumped, it is a release or a cron job; if traffic doubled, it is demand. `top` with `1` tells me whether it is user, system, `iowait` or `steal`, which are four different problems. If it followed a deploy, I roll back. If it is demand, I scale out. If it is a batch job that should not run during peak, I kill it and reschedule. High `steal` time on burstable instances is a different answer entirely - we have outgrown the instance type."

Q. 36. An application cannot connect to MySQL. Walk me through it.

Answer: "I go layer by layer from the application host. Does the hostname resolve, and to the IP I expect? Can I open port 3306 - and is the failure refused or a timeout, because refused means the network is fine and MySQL is not accepting, while timeout means a security group or route. Then I try an actual connection with credentials, since an authentication error is a completely different problem from a connection failure. Then the database side: is it at `max_connections`, is a query holding a lock, is its disk full, did it just fail over so the DNS endpoint points somewhere new. 'Too many connections' usually means the application is leaking connections rather than that the limit is wrong, and restarting the application is a mitigation while the pool configuration is the fix."

Q. 37. S3 uploads suddenly fail with AccessDenied, and nothing changed.

Answer: "I start from the error, which names the principal, action and resource. Then I confirm which identity is actually being used - `aws sts get-caller-identity` - because a stale profile or an environment variable overriding the role is common. Then I look for an explicit Deny, in the bucket policy, a permissions boundary or an SCP, since Deny always wins. Then the ARN form: object actions need `bucket/*` while `ListBucket` needs `bucket` - mixing those up is the classic bug. If the bucket is KMS-encrypted, the principal also needs `kms:GenerateDataKey`, and the error still says S3 AccessDenied, which is misleading. CloudTrail shows the exact denied call with full context."

Q. 38. Your application in a private subnet cannot reach an external API.

Answer: "Private subnets need a NAT gateway for outbound traffic, so I check the subnet's route table for `0.0.0.0/0` pointing at a NAT gateway, and that the NAT gateway itself is in a public subnet with a route to the internet gateway. Then outbound security group rules and NACL rules including the inbound ephemeral range for return traffic. I test from the instance to separate DNS from connectivity - `dig` the hostname, then `curl -v`. If only S3 or DynamoDB are failing, it is a gateway endpoint issue instead. And if this started suddenly, someone deleting a NAT gateway to save cost is a surprisingly common cause."

Q. 39. A load balancer shows all targets unhealthy.

Answer: "I read the health check failure reason, which the target group provides - a 404 means the health check path is wrong, a timeout means the security group or the application is not responding. Then: does the target's security group allow the load balancer's security group on the target port; is the application listening on `0.0.0.0` rather than `loopback`; is the health check port the right one; and are the thresholds and timeouts sane - a timeout shorter than the app's response time under load will take out a healthy fleet. If all targets went unhealthy at once, either something was deployed or the health check endpoint depends on a shared resource that is down."

Q. 40. A cron job stopped running and nobody noticed for a week.

Answer: "Two problems: why it stopped, and why nobody noticed. For the first, I check whether cron is running, whether the crontab still exists, whether the script path or permissions changed, whether the host was replaced, and whether output was going anywhere - cron's minimal `PATH` silently breaks scripts that call `aws` or `kubectl` by name. For the second, which matters more: the job needed a heartbeat. Alerting on failure only catches it when it runs; the dangerous case is when it stops running entirely, which only a staleness check catches - the age of the last success exceeding its interval. I would add that and move the job to a `systemd` timer or a `Kubernetes CronJob` so its output lands somewhere visible."

Q. 41. Certificate expired in production. How did that happen and how do you prevent it?

Answer: "It happened because renewal was manual, or automated renewal failed silently and nobody was watching. Immediate fix: issue and deploy a new certificate with the full chain, and verify from outside with `openssl s_client`. Prevention is layered - automated renewal through ACM or `cert-manager` so no human is in the path, plus monitoring of actual expiry as observed on the live endpoint via `blackbox_exporter`, alerting at 30 days and again at 7. The monitoring matters even with automation, because the failure mode is the automation breaking quietly."

Q. 42. You inherit an environment with no documentation. Where do you start?

Answer: "I map reality before changing anything. What is running - instances, clusters, databases, load balancers - and how traffic reaches the users, traced from DNS inwards. Where the code and the pipelines are. What monitoring and alerting exist and who receives them. What the backup and restore situation is, and whether a restore has ever been tested. Then I write that down, because the documentation gap is itself a risk. My first changes would be the cheap, high-value ones: alerting on the things that cause outages - disk, certificates, healthy host count - and getting infrastructure into code

incrementally, starting with new resources rather than attempting a big-bang import."

17.5 Scenarios 43-50: Judgement, Process and People

Q. 43. You are asked to make a change to production during peak hours.

Testing: risk judgement and communication. **Answer:** "I would ask what makes it urgent, because the answer changes the decision entirely. If it is fixing a live incident, the risk of waiting is higher than the risk of changing, so we proceed carefully with a rollback ready. If it is routine, I would explain the risk and propose a scheduled window, giving a concrete alternative rather than just saying no. If the business decides it must happen now, that is a legitimate decision - I make sure it is made with the risk understood, we have a tested rollback, someone is watching the dashboards, and the change is as small as possible."

Q. 44. You realise you caused the outage. What do you do?

Testing: honesty and psychological safety. **Answer:** "I say so immediately, because knowing the cause is the fastest route to resolution and withholding it costs everyone time. Restore service first, then contribute what I know to the timeline. In the RCA I focus on the system, not myself - if one person's action could take production down, the real finding is the missing guard rail: no review, no staging verification, no confirmation step. Teams where people hide mistakes have longer outages, and I would rather be the person who says it early."

Q. 45. A developer wants production database access to debug. How do you respond?

Answer: "I would find out what they actually need, because 'production access' is usually a proxy for 'I cannot see what is happening'. Often the real need is logs, metrics or a read-only query, and the right answer is better observability rather than credentials. If direct access is genuinely needed, it should be read-only, time-limited, audited, ideally through a break-glass process with approval, and against a replica rather than the primary. The goal is to unblock them without creating a standing risk - saying no without offering a path just gets worked around."

Q. 46. Your manager wants to skip testing to meet a deadline.

Answer: "I would make the trade-off explicit rather than argue. I would say which specific tests would be skipped and what risk that carries - for example, we would not know whether the payment path still works. Then offer options: ship a smaller scope that is fully tested, ship behind a feature flag so it can be disabled instantly, or ship with a reduced test set and an agreed rollback plan and someone watching. That turns it into an informed business decision, which it legitimately is, rather than a disagreement about process."

Q. 47. Two teams disagree about a technical approach and you are in the middle.

Answer: "I would try to get the disagreement onto facts. Usually both sides are optimising for different things - one for delivery speed, one for operability - and neither is wrong. I would write down the requirements and constraints both agree on, then evaluate the options against them, including what each costs to run and to reverse. If it is genuinely close, I would favour the option that is easiest to change later. And if a decision is needed and consensus is not coming, I would say so and escalate rather than let it drift - an unmade decision is usually worse than either option."

Q. 48. How do you prioritise when three incidents happen at once?

Answer: "By user impact and blast radius, not by who is shouting. A complete outage affecting all users beats a degraded feature beats a single internal tool. I would also check whether they are actually one incident with three symptoms, which is common - a shared dependency failing produces alerts everywhere. Then I would get help rather than trying to work three at once: assign owners, keep a single incident channel per issue, and make sure someone is coordinating and communicating rather than everyone debugging in silence."

Q. 49. You are on call and an alert fires at 3am for something that is not actually broken.

Answer: "I handle it, then I fix the alert - because an alert that pages a human for nothing is a defect, and the cost is that people start ignoring alerts, which is how real incidents get missed. Concretely that means checking whether the threshold is right, whether it needs a for: duration, whether it should be a ticket rather than a page, and whether it is a symptom of something already alerted. I would track which alerts fire and what action was taken, and delete or demote any that never lead to action."

Q. 50. What would you do in your first 90 days in this role?

Testing: whether you think beyond tasks. **Answer:** "First month, understand before changing: the architecture and how a request flows end to end, the deployment path from commit to production, what monitoring and alerting exist, where the pain is - I would ask the team what wakes them up and what they dread doing. I would also pick up on-call early, because nothing teaches you a system faster. Second month, fix the cheap high-value things: alerting gaps around disk, certificates and availability; pipeline friction; documentation for the things only one person knows. Third month, take on something structural - whatever the first two months showed was the biggest recurring cost, whether that is environment drift, manual

deployment steps or toil in the on-call rotation. I would rather earn the right to change things by fixing the obvious pain first."

PART 17 RECALL POINTS

- Always open by scoping: who, when, what changed, getting worse?
- Mitigate before diagnosing; roll back on doubt.
- Refused vs timeout; Pending vs CrashLoop vs not-Ready - each splits the problem.
- Say what each check would tell you, not just what you would type.
- Name the prevention at the end of every scenario answer.
- For people scenarios: make trade-offs explicit, offer options, never just say no.

PART 18

PROJECT EXPLANATION

Four versions of your project story, plus 50 cross-questions

Your CV names three projects: CI/CD Deployment Automation, Infrastructure as Code with Terraform, and Containerized Deployment and Observability. This part builds one consistent narrative across all three - because in an interview they are not three projects, they are one platform - and then cross-questions it the way an interviewer will.

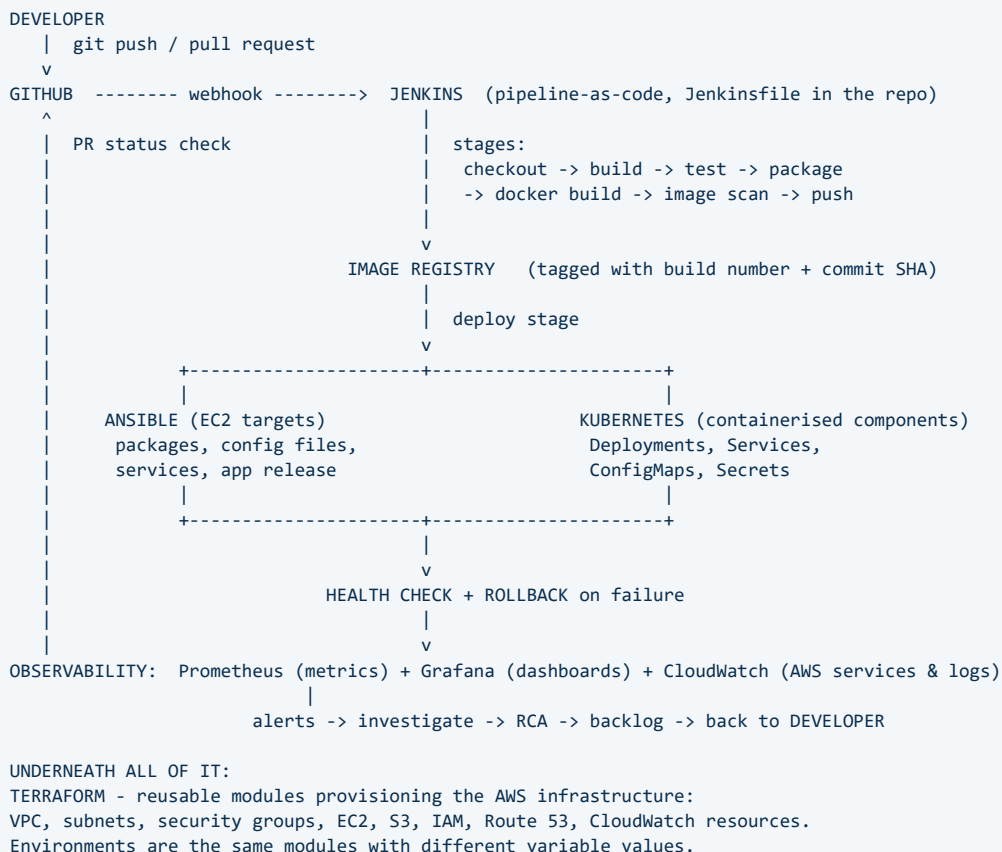
THE EXACT WORDING YOU MUST STAY CONSISTENT WITH

- **CI/CD Deployment Automation:** "Designed Jenkins pipeline-as-code workflows integrating Git, automated builds, testing and deployment stages to standardize application release workflows."
- **Infrastructure as Code with Terraform:** "Built reusable Terraform modules to provision AWS infrastructure, enabling repeatable environment creation and reducing manual configuration."
- **Containerized Deployment and Observability:** "Deployed Dockerized components on Kubernetes (Pods, Deployments, Services, ConfigMaps, Secrets) with Prometheus, Grafana and CloudWatch dashboards and alerting."

THE RULES FOR THIS ENTIRE PART

Do not add numbers your CV does not contain - no traffic volumes, no percentage improvements, no team sizes, no uptime figures, no customer names. Describe mechanism and responsibility. If an interviewer presses for scale, say what you know and be explicit about what you did not own. A confident "I did not own that measurement" costs you nothing; an invented figure that collapses under follow-up costs you the offer.

18.1 The Architecture You Are Describing



One platform, three project names

18.2 The 30-Second Version

30 SECONDS - FOR "TELL ME ABOUT A PROJECT YOU WORKED ON"

"The project I'd point to is our deployment automation platform. I built Jenkins pipelines as code that take a commit from Git through build, test, packaging and deployment, so releases became a triggered job rather than a manual sequence. Underneath it, the AWS infrastructure is provisioned with reusable Terraform modules and configured with Ansible, so environments are repeatable rather than hand-built. Application components are containerised with Docker and deployed to Kubernetes, and we monitor all of it with Prometheus, Grafana and CloudWatch, with dashboards and alerting so we see problems before users report them."

18.3 The One-Minute Version

1 MINUTE - ADDS THE MECHANISM

"The work spans three connected pieces that are really one platform.

The first is CI/CD. I designed Jenkins pipeline-as-code workflows - the Jenkinsfile lives in the repository, so the build definition is versioned and reviewed like any other change. A push or pull request triggers a webhook, and the pipeline checks out that commit, builds, runs tests, packages the application, builds a Docker image tagged with the build number and commit SHA, pushes it to the registry, and deploys. Because the image tag is immutable, the same artifact moves through environments and rollback is redeploying a previous tag.

The second is infrastructure. I built reusable Terraform modules that provision the AWS infrastructure - VPC, subnets, security groups, EC2, IAM, S3 and the monitoring resources - so a new environment is a set of variable values, not a new set of resource blocks. Ansible then handles configuration on the instances: packages, config files rendered from templates, users and services, with handlers so a service only restarts when its configuration actually changed.

The third is containers and observability. Components are containerised with Docker and run on Kubernetes using Deployments, Services, ConfigMaps and Secrets, and Prometheus, Grafana and CloudWatch give us metrics, dashboards and alerting across both the application and the AWS services it depends on."

18.4 The Three-Minute Version

3 MINUTES - ADD THE WHY AND THE DECISIONS

"I'll take it in the order a change actually flows.

The problem. Deployments were manual and the steps varied between environments, which is where release inconsistency comes from - not from anyone being careless, but because a sequence a person performs is never identical twice. Infrastructure was configured by hand, so environments drifted apart and 'works in test' stopped meaning anything.

CI/CD. I built the pipelines as code. The Jenkinsfile sits in the repository, so changing the build is a pull request with review, and each branch can have its own pipeline through a multibranch job. GitHub webhooks trigger it rather than polling, so feedback is immediate. The stages are checkout, build, test with results published so the build goes unstable rather than silently green, package, Docker build, push to the registry, then deploy followed by a health check. Credentials come from the Jenkins credential store and are bound only to the stage that needs them, never hardcoded. Concurrent builds are disabled on anything that deploys, because two simultaneous deploys to one environment is a self-inflicted outage.

The artifact discipline. The image is built once and tagged with the build number and the short commit SHA. That tag is immutable, so what was tested is exactly what is deployed, I can tie anything running back to a commit, and rollback is deploying the previous tag rather than rebuilding.

Infrastructure as code. Terraform provisions AWS - VPC and subnets, security groups, EC2, IAM roles, S3 and the monitoring resources. I built it as reusable modules rather than one flat configuration, so an environment is a module call with different variables. That is what makes environments reproducible and what makes a change to the pattern a single edit rather than three. State is remote and locked so two applies cannot collide, and plans are reviewed before apply.

Configuration. Ansible handles what Terraform deliberately does not - packages, configuration files rendered from Jinja2 templates with per-environment variables, users, and service state. Tasks are idempotent, so re-running is safe, and handlers mean a service restarts only when its configuration actually changed.

Containers and Kubernetes. Components are containerised with Docker, which is what removed the environment inconsistency - the runtime and libraries travel with the application. On Kubernetes we use Deployments for rollout and self-healing, Services for stable addressing, and ConfigMaps and Secrets for configuration. Most of my Kubernetes work is deploying and troubleshooting workloads - pods, deployments, services, configmaps and connectivity - rather than administering the cluster itself.

Observability. Prometheus scrapes application and infrastructure metrics, Grafana provides the dashboards, and CloudWatch covers the AWS services we cannot instrument ourselves - load balancers, RDS - plus log aggregation. Alerts go on symptoms rather than raw resource numbers, so we are paged for things users would notice.

Operations. When something breaks, I work through application logs, service state on both Linux and Windows, database connectivity and network diagnostics - HTTP, DNS, TCP and RTSP in our case, because video components are part of the stack. I do the RCA, coordinate the fix with development and QA, and validate the upgrade or release afterwards."

18.5 The Five-Minute Technical Version

5 MINUTES - WHAT A SENIOR ENGINEER WANTS TO HEAR

"Let me go through it as a technical walkthrough of one change reaching production, then the operational side.

1. Trigger and checkout. A developer works on a short-lived branch and opens a pull request. A GitHub webhook hits Jenkins, and a multibranch pipeline builds that branch - which also means pull requests are validated before merge, not after. The pipeline does `checkout scm`, so it builds the exact commit that triggered it rather than whatever is currently on a named branch, and I capture the short SHA into an environment variable because it becomes part of the artifact identity.

2. Build and test. Build produces the application artifact. Tests run with results published through the JUnit step, which marks the build unstable rather than failed - that distinction matters because a compile failure and a flaky test deserve different responses. Anything that does not need to gate the merge runs in parallel with the tests to keep the pipeline short, because a slow pipeline is one people route around.

3. Package. Docker build produces the image. The Dockerfile copies dependency manifests and installs dependencies before copying source, so the expensive layer stays cached across code commits; the final image runs as a non-root user and has a health check. The image is tagged with the build number and the commit SHA - never just latest, because a mutable tag means you cannot tell what is running and cannot roll back precisely.

4. Publish. The image is pushed to the registry using credentials bound from the Jenkins credential store inside a `withCredentials` block, with the password piped to `docker login --password-stdin` so it never appears on a command line or in the console output.

5. Deploy. The deploy stage targets the environment. For EC2-based components that is Ansible - a playbook that renders configuration from templates, manages services, and uses `serial` so hosts are updated in batches rather than all at once, with a health check before moving to the next batch. For containerised components it is Kubernetes - updating the Deployment's image and then waiting on `kubectl rollout status`, so the pipeline fails if the new pods never become Ready rather than reporting success and walking away.

6. Verify. A health check with retries against the application's health endpoint. This is the gate that makes the difference between 'the deployment command returned zero' and 'the application is actually serving'.

7. Rollback. Because artifacts are immutable and versioned, rollback is deploying the previous tag, or `kubectl rollout undo`, which works because Kubernetes keeps the previous ReplicaSet. The important caveat I would raise is that rollback covers code, not data - schema changes need to be backward compatible, expand-and-contract, or rollback is not actually available to you.

8. The infrastructure underneath. Terraform modules provision it: VPC with public and private subnets across availability zones, security groups that reference each other rather than CIDR ranges so the tiering is explicit and survives instance replacement, EC2 with instance profiles rather than access keys, S3, Route 53 and the CloudWatch resources. Remote state in S3 with DynamoDB locking, one state key per environment so a mistake in one cannot affect another, and providers and modules pinned so a build is reproducible. In a pipeline, plan on the pull request and apply the saved plan file on merge, so what a human reviewed is exactly what runs.

9. Observability. Prometheus scrapes the application and node exporters; Grafana dashboards show request rate, error rate and latency percentiles alongside resource saturation; CloudWatch covers the managed AWS services and log aggregation, because you cannot install an exporter inside an ALB or RDS. Alerting is symptom-based with a `for:` duration so transient spikes do not page anyone.

10. Security posture. No credentials in code or images; secrets from the credential store and from AWS; IAM roles rather than long-lived keys; least privilege on the deployment identity; image scanning in the pipeline; and infrastructure changes going through pull request review like application code.

11. Operations. On the production support side, incidents come in through alerts or from QA and development. I work through application logs, systemd services on Linux and Windows services on the Windows side, database connectivity, and network diagnostics across HTTP/HTTPS, TCP/IP, DNS and RTSP. I do root cause analysis, coordinate the fix, and validate application upgrades and releases afterwards - which is really the feedback loop closing back into the backlog."

HOW TO ADAPT THIS TO THE JOB DESCRIPTION

If the role is Kubernetes-heavy, spend longer on stages 5-7 and the observability section and be explicit that your Kubernetes work is workload-level. If it is cloud-infrastructure-heavy, spend longer on stage 8. If it is SRE or production support, lead with stage 11 - that is your differentiator and most 3-year candidates cannot describe a troubleshooting method at all.

18.6 Fifty Questions an Interviewer May Ask About Your Project

Q. 1. Why pipeline-as-code rather than configuring the job in the Jenkins UI?

Because a UI-configured job exists only inside Jenkins - it is not versioned, not reviewed, and lost if the instance is rebuilt. A Jenkinsfile in the repository means changing the build is a pull request, each branch can have its own pipeline, and a code change and its build change ship as one reviewable unit.

Q. 2. What triggers your pipeline?

A GitHub webhook on push and on pull request, with a multibranch job so each branch gets its own pipeline. Webhooks rather than polling, because polling adds latency up to the poll interval and generates load for nothing. Some jobs also run on a schedule using H in the cron expression so they do not all fire at the same instant.

Q. 3. Walk me through your pipeline stages.

Checkout the triggering commit, build, test with results published, package, Docker build, image scan, push to the registry, deploy to the target environment, health check with retries, notify. Post-actions archive artifacts and clean the workspace regardless of outcome, and a failure in the deploy stage triggers rollback.

Q. 4. How do you handle secrets in the pipeline?

The Jenkins credential store, scoped to the folder rather than globally, bound with withCredentials only around the step that needs them. In the shell I use single quotes so the shell expands the variable rather than Groovy interpolating the value into the command string, and secrets are never echoed. For AWS the better pattern is a role assumed by the pipeline rather than stored keys at all.

Q. 5. How do you version your artifacts?

Build number plus short commit SHA, with a semantic version tag for releases. The tag is immutable, which is what makes "the artifact that was tested is the artifact that is deployed" true, lets me trace anything running back to a commit, and makes rollback a precise operation.

Q. 6. Why not use latest?

Because it is mutable. You cannot tell which build is running, two nodes can have different images under the same tag, imagePullPolicy: IfNotPresent will happily reuse a stale cached image, and there is no specific version to roll back to.

Q. 7. How do you roll back?

Deploy the previous immutable tag - either by re-running the last known-good deployment or through a parameterised rollback job - or kubectl rollout undo for Kubernetes, which reverts to the previous ReplicaSet. The prerequisite is immutable versioned artifacts, and the caveat is that database changes are not rolled back by either.

Q. 8. What happens if the deployment succeeds but the health check fails?

The pipeline fails the stage and triggers rollback, because the health check is the real definition of success - a deployment command returning zero only proves the command ran. I use retries with a delay so a slow-starting application is not failed prematurely, but with a bounded total timeout.

Q. 9. How do you prevent two deployments running at once?

disableConcurrentBuilds() on the job, and a lockable resource when several different jobs target the same environment. Concurrent deploys to one environment produce interleaved state and are a genuine outage cause.

Q. 10. What is in your Terraform modules?

The pattern - networking, security groups, compute and the supporting IAM and monitoring resources - parameterised by name, environment, CIDR, sizing and tags, exporting the IDs that other modules need. No hardcoded values and no provider block inside the module, because providers are configured by the root and inherited.

Q. 11. How do you separate environments in Terraform?

A directory per environment with its own backend key, tfvars and variable values, calling the same modules. I prefer that over workspaces for dev versus production because the separation is explicit - with workspaces, a forgotten workspace select applies to the wrong environment and nothing in the command you type warns you.

Q. 12. Where is your Terraform state and why?

Remote, in S3 with versioning and encryption, with a DynamoDB table for locking, and one key per environment. Remote because the team shares one source of truth; locked because concurrent applies corrupt state; versioned because it lets me recover from a bad apply; encrypted because state contains every resource attribute in plaintext, including secrets.

Q. 13. How do you run Terraform in CI?

`fmt -check, init, validate` and a security scan, then `plan -out=tfplan` on the pull request with the plan posted for review, and `apply tfplan` on merge. Applying the saved plan file matters - a bare `apply` re-evaluates and may do something different from what the reviewer approved.

Q. 14. What does Ansible do that Terraform does not?

Configuration of what exists - packages, config files rendered from templates, users, service state, and the application release itself. Terraform provisions infrastructure and tracks it in state; Ansible configures systems and has no state file, checking reality per task each run. Terraform builds the house, Ansible furnishes it.

Q. 15. How do you deploy with Ansible without downtime?

`serial` to process hosts in batches, removing each host from the load balancer before changing it and returning it after a health check passes, with `max_fail_percentage: 0` so a failure stops the rollout rather than breaking the whole fleet. Handlers ensure services restart only when configuration actually changed.

Q. 16. How do you know your Ansible playbooks are idempotent?

Running them twice - the second run should report zero changed tasks. Anything that reports changed every time is usually a `command` or `shell` task with no guard, and `I add creates, removes` or a `when` condition, or replace it with a proper module.

Q. 17. What is in your Dockerfile and why in that order?

Base image pinned to a specific tag, then dependency manifests copied and dependencies installed, then the application source, then a non-root user, then the health check and entrypoint. The ordering is deliberate: source changes on every commit and dependencies rarely, so installing dependencies before copying source keeps that layer cached and turns a multi-minute build into seconds.

Q. 18. How do you keep images small?

Multi-stage builds so the compiler and dev dependencies never reach the final image, a slim base, cleaning package caches within the same RUN layer rather than a later one, and a `.dockerignore` so the build context does not carry `.git` and `node_modules`. `docker history` tells me which layer is responsible when an image is unexpectedly large.

Q. 19. Why do your containers run as non-root?

Because a container escape or an exploited application otherwise has root in the host namespace, and a root process writing to a mounted volume creates root-owned files on the host. Adding a user and a `USER` line costs nothing and removes an entire escalation path.

Q. 20. Which Kubernetes objects do you use and why?

Deployments for rollout and self-healing, Services for stable addressing by label selector, ConfigMaps for configuration and Secrets for credentials, all within namespaces. That is the set my CV claims and the set I work with - my Kubernetes work is deploying and troubleshooting workloads rather than administering the cluster.

Q. 21. How do you handle configuration differences between environments?

ConfigMaps and Secrets per namespace for Kubernetes, and per-environment variable files for Terraform and Ansible. The manifests or templates are the same; only the values differ. That is what keeps environments comparable - if the code differs per environment, you cannot reason about what you tested.

Q. 22. How do you update a ConfigMap and make the application pick it up?

Environment variables from a ConfigMap are injected at pod start, so changing it does not affect running pods - a rolling restart is required. The cleaner pattern is annotating the pod template with a checksum of the config, so any config change produces a new pod template and therefore a normal rolling update with readiness gating.

Q. 23. Are your Kubernetes Secrets actually secure?

Not by default - they are base64-encoded in etcd, which is encoding, not encryption, so anyone who can read Secrets in that namespace can decode them. Making them secure needs encryption at rest for etcd, tight RBAC, and ideally an external secret store. I would rather state that clearly than pretend otherwise.

Q. 24. What do you monitor with Prometheus?

Application signals first - request rate, error rate and latency percentiles - then saturation: CPU and memory against limits, restart counts, and queue or pool usage. Node-level metrics from `node_exporter`, and Kubernetes object state from `kube-state-metrics`.

Q. 25. What do you use CloudWatch for if you have Prometheus?

The AWS services I cannot instrument - load balancer metrics, RDS, NAT gateways - plus log aggregation. Those metrics only exist in CloudWatch, and they are exactly the ones that tell me whether a problem is the platform or my application. I point Grafana at both so one dashboard shows them together.

Q. 26. What do you alert on?

Symptoms users would notice - error rate, latency, healthy target count, pod restart rate - rather than raw resource numbers, with a `for` duration so a transient spike does not page anyone. Plus predictive alerts where possible, like disk filling within four hours, and absence alerts, because a target that stops reporting entirely is the most dangerous state.`

Q. 27. Give an example of an alert you would not create.

A pure "CPU above 80%" page. High CPU is frequently normal and is not by itself something a human needs to act on at 3am. If high CPU matters, it matters because latency or errors rose - so I alert on those, and keep CPU on the dashboard for diagnosis.

Q. 28. How do you know your monitoring is actually working?

Because it has caught real problems before users reported them, and because the `up` metric and staleness alerts cover the monitoring itself. The uncomfortable test is an incident where the dashboards looked fine - that is a monitoring gap and it becomes an action item in the RCA.`

Q. 29. How do you troubleshoot a production incident?

I scope it first - all users or some, which environment, when it started, and what changed, because most incidents follow a change. Then I work through the layers: DNS and the load balancer, the instance or pod state, whether the process is listening, then application logs, then dependencies - database connectivity and network diagnostics. I mitigate before diagnosing, preserve evidence, and communicate status on a fixed cadence. Afterwards, RCA with action items.

Q. 30. Tell me about a production issue you investigated.

Keep it factual and within what your CV supports - for example: "A component became unreachable while the process was still running. I confirmed the service state, checked whether it was listening and on which interface, then tested connectivity from the client side to separate a network problem from an application problem. The logs showed it was failing to reach a dependency, so I verified that path specifically - name resolution, port reachability and credentials - and found the dependency was the actual failure. I coordinated the fix and then validated the application end to end before closing it." Describe method, not invented drama.

Q. 31. What is the hardest problem you have solved?

Choose something where the difficulty was diagnosis rather than heroics - an intermittent failure, a problem that only appeared under load, or a difference between environments. Explain how you narrowed it down and what made it hard to see. Interviewers value the method, and an honest medium-sized problem explained well beats an exaggerated one.

Q. 32. What would you improve about your current setup?

Have a real answer - it shows engineering judgement. Good candidates: automated rollback triggered by post-deploy error rate rather than a manual decision; drift detection running on a schedule; better environment parity checks so a config key present in one environment and missing in another is caught in CI; and more of the Kubernetes work moved to GitOps so the cluster state is reconciled from Git rather than pushed by a pipeline.

Q. 33. What was the biggest mistake you made on this project?

Pick something real, small and instructive, and focus on the system fix - for example not setting resource limits initially, and learning that a container without limits can starve its neighbours and that the scheduler cannot place work sensibly without requests. The answer an interviewer wants is that you changed the system, not just your behaviour.

Q. 34. How long did this take and who else was involved?

Answer honestly about your scope. It is fine - and better - to say "I built and maintained the pipelines and the Terraform modules; the application teams owned the application code, and QA owned validation" than to imply you built everything alone. Interviewers are experienced enough to know that a solo claim on a whole platform is usually inflated.

Q. 35. How do you test infrastructure changes?

Static checks in CI - format, validate and a security scan - then the plan itself as the review artifact, applied first in a lower environment. For modules, integration tests that provision into a sandbox, assert and destroy. And a scheduled plan against production to catch drift.

Q. 36. How do you handle database changes in your pipeline?

As a separate, gated step rather than something that happens implicitly on application startup, and written to be backward compatible - expand and contract - so the old and new application versions can both run during a rolling update. That is what keeps rollback available, which is the whole point.

Q. 37. What does your pull request process look like?

Branch protection on main requiring a pull request, at least one approval, passing status checks and the branch being up to date, with force-push disabled and the rules applying to administrators. The pipeline result is a required check, so a red

build cannot merge.

Q. 38. How do you handle a hotfix?

Branch from the released commit or tag rather than from main, make the minimal change, run the full pipeline, deploy, and then merge back so the fix is not lost in the next release. Cherry-picking onto the release branch is the same idea when the release line is separate.

Q. 39. What monitoring gaps did you find?

A genuinely good answer here is memory and disk on EC2, because CloudWatch does not provide them without the agent, so teams often discover during an incident that they have no memory history. Another is certificate expiry, which is entirely predictable and still causes outages.

Q. 40. How do you decide what to automate?

Anything done more than twice, anything error-prone by hand, anything that must run out of hours, and anything more than one person needs to be able to do. I would not automate something whose process is still changing weekly, because you end up maintaining automation for a moving target.

Q. 41. How do you onboard a new application onto your platform?

A Jenkinsfile from the shared library so the pipeline is consistent, a Dockerfile following the same pattern, Terraform module calls for whatever infrastructure it needs, manifests or an Ansible role for deployment, and dashboards and alerts from a template. The goal is that onboarding is configuration rather than invention.

Q. 42. What is a shared library and do you use one?

A versioned repository of reusable pipeline code so the same logic is not copied into every Jenkinsfile. Pinned to a tag, not to main, so a change to the library cannot silently alter every pipeline in the organisation at once. If you have not used one, say so and explain why it would help - that is a stronger answer than pretending.

Q. 43. How do you handle Windows servers in this setup?

They are part of the estate, so pipelines target them with Windows agents and `bat` or `powershell` steps rather than `sh`, Ansible connects over WinRM using the `win_*` modules, and troubleshooting uses Windows services and event logs rather than `systemd` and `journald`. Having both in one pipeline is a real constraint and it shapes how stages are labelled and routed.

Q. 44. What is RTSP and why does it matter in your environment?

It is the control protocol for streaming media, typically on port 554 with the media itself over RTP. It matters operationally because it is not HTTP - an application load balancer cannot route it, firewalls need the media ports open as well as the control port, and diagnosing it means testing the actual stream rather than curling an HTTP endpoint.

Q. 45. How do you validate a release after deployment?

The automated health check in the pipeline first, then the dashboards - error rate and latency compared against the pre-deploy baseline rather than against zero - and then functional validation with QA on the critical paths. Deployment is not complete when the pipeline goes green; it is complete when the signals confirm it.

Q. 46. What would you do differently if you started again?

Immutable tags and a health-check gate from day one rather than added later; resource requests and limits from the start; alerting built alongside the first deployment rather than after the first incident; and Terraform state split by component from the beginning, because splitting a large state later is significantly more work than starting that way.

Q. 47. How does your team handle on-call and escalation?

Describe what is actually true for you. If there is no formal rotation, say so and describe how incidents actually reach you and how you handle them - that is a legitimate answer. Then say what you would want: a documented rotation, runbooks linked from alerts, and clear severity definitions so escalation is not a judgement call at 3am.

Q. 48. What is the most valuable thing this project achieved?

Consistency. The mechanism is that the same artifact, built once, moves through every environment, and the environments themselves are created from the same code - so a release behaves the same way each time and a failure has a known rollback path. Everything else - speed, fewer manual steps - follows from that rather than being the goal.

Q. 49. If I gave you more headcount and a quarter, what would you build?

Progressive delivery - canary with automated analysis and rollback based on real metrics - because that reduces the blast radius of every future release rather than fixing one problem. Alongside it, GitOps for the Kubernetes workloads so cluster state is reconciled from Git, and environment parity checks in CI.

Q. 50. Why should we believe you did this?

Because I can describe the mechanism at any level of detail you want - the pipeline stage by stage, why the Dockerfile is ordered the way it is, what happens when a rollout gets stuck, how state locking prevents two applies colliding, and what I would check first if the deployment went green and the application stayed broken. And I am equally clear about where my depth ends: I deploy and troubleshoot Kubernetes workloads, I have not administered a cluster in production. Ask me anything inside that line and I can go as deep as you want.

PART 18 RECALL POINTS

- One platform, three project names - tell it as one story.
- Stay inside your CV's verbs: built, provisioned, containerized, support, implemented.
- Mechanism over magnitude; never invent metrics.
- The artifact discipline - build once, immutable tag, deploy everywhere - is your strongest technical point.
- Terraform provisions, Ansible configures, Docker packages, Kubernetes runs, Prometheus/Grafana/CloudWatch observe.
- Always be explicit that Kubernetes is workload-level for you.
- Have a real "what I would improve" answer ready.

PART 19

RESUME CROSS-QUESTIONING

Every bullet on your CV, turned into an interrogation

This is the most important part of the handbook. An interviewer is entitled to pick any line from your CV and question you on it for ten minutes. For each bullet you get: what they may ask, the depth expected, a strong answer, follow-ups, the traps, and a practical example. Work through it line by line and mark anything you cannot answer fluently - that is your study list.

HOW TO USE THIS PART

Read the bullet. Cover the answer. Say your version out loud and time it - a good answer is 45 to 90 seconds. Then compare. Anything where you hesitated, pause and go back to that part of the handbook before moving on. Do this twice before any interview.

19.1 Summary Line: "3 years of experience automating CI/CD pipelines, cloud infrastructure and production deployments"

Field	Content
What they ask	"Three years - walk me through where those years were spent and what changed between them."
Depth expected	A clear timeline with increasing scope, no gaps, and a concrete difference between the two roles
Strong answer	"I moved into DevOps in January 2024 at Naresh I Technology, where I built Jenkins pipelines, packaged applications with Docker, and automated AWS provisioning with Terraform and Ansible. In May 2025 I moved to I2V Systems, where the scope widened - I now own the DevOps side of our application environments end to end, including Kubernetes deployments, monitoring implementation and production incident work across both Linux and Windows. The first role taught me the tooling; the second added ownership and production responsibility."
Follow-ups	What was the biggest difference? What could you not do at the first company that you can now? Why did you leave?
Traps	Vague timelines; claiming the same depth in both roles; not being able to explain the progression
Prepare	Know your dates precisely and be able to state what each role added

19.2 "Hands-on with AWS, Kubernetes, Docker, Terraform, Ansible, Jenkins, GitHub Actions, Prometheus, Grafana, Python and Bash"

Field	Content
What they ask	"Rank those by your own confidence, honestly." Or: "Which one would you not want me to ask about?"
Depth expected	Self-awareness. They already suspect an 11-item list is not uniformly deep
Strong answer	"Strongest are Jenkins and CI/CD pipelines, Docker, and production troubleshooting on Linux and Windows - those are daily. Then Terraform and Ansible, which I use regularly for provisioning and configuration. Then AWS within the services I actually operate - EC2, S3, IAM, VPC, CloudWatch, Route 53 and security groups. Kubernetes I would put next: I deploy and troubleshoot workloads, but I have not administered a cluster, and I am actively strengthening that. Prometheus and Grafana I use for monitoring and dashboards rather than having built the platform from scratch."
Follow-ups	Any of them, in depth. GitHub Actions is often picked because it is last in the list
Traps	Claiming equal depth in all eleven; being unable to name a weakness; sounding rehearsed
Prepare	Have one concrete thing you have done with each - especially GitHub Actions and PowerShell, which are the easiest to be caught on

19.3 "Skilled in Infrastructure as Code, containerized deployments, monitoring and production incident resolution"

Field	Content
What they ask	"Define infrastructure as code and tell me what problem it solved for you specifically."
Depth expected	Declarative vs imperative, state, plan before apply, reproducibility and review
Strong answer	"IaC means the infrastructure is defined in code that a tool applies, rather than built by hand. The problem it solved for us was environment reproducibility - hand-built environments drift apart, so 'it worked in test' stops meaning anything. With Terraform modules, creating an environment is supplying variable values, the change goes through a pull request like application code, and <code>plan</code> shows exactly what will be created, changed or destroyed before anything happens."
Follow-ups	What is state and why does it matter? How do you handle drift? What happens if two people apply at once?
Traps	Defining it as "writing scripts to build servers" - that is imperative automation, not IaC

19.4 Skills Line: "AWS (EC2, S3, IAM, VPC, CloudWatch, Route 53, Security Groups)"

Field	Content
What they ask	"Draw me the VPC you work with." Or: "What makes a subnet public?"
Depth expected	Public/private subnet design, route tables, IGW vs NAT, SG vs NACL, IAM roles vs users
Strong answer	"A subnet is public because its route table sends 0.0.0.0/0 to an Internet Gateway - there is no setting called public. The standard layout is public subnets across two AZs holding only the load balancer and NAT gateways, private subnets for the application tier whose route to the internet goes through NAT, and private data subnets with no internet route at all. Security groups are stateful and can reference each other, so the database's group allows the app tier's group rather than a CIDR - which keeps working as instances are replaced."
Follow-ups	How many usable IPs in a /24? SG vs NACL? What does the NAT gateway cost you? What is 169.254.169.254?
Traps	Saying "I used EC2" with no architectural understanding; confusing IGW and NAT gateway
Prepare	Be able to draw the diagram from Part 8.3 on paper from memory

19.5 "Docker, Kubernetes (Pods, Deployments, Services, ConfigMaps, Secrets)"

Field	Content
What they ask	"Your CV lists specific Kubernetes objects. Why those five?"
Depth expected	Honest scope. That list is workload-level, and a good interviewer will notice
Strong answer	"Those are the objects I work with - I deploy and troubleshoot workloads rather than administering clusters. Deployments for rollout and self-healing, Services for stable addressing by label selector, ConfigMaps and Secrets for configuration. My day-to-day is debugging why a pod is not starting, why a Service has no endpoints, or why a rollout is stuck. I have not run a control plane in production, and I am currently strengthening cluster administration."
Follow-ups	What is CrashLoopBackOff and how do you debug it? What are endpoints? Difference between liveness and readiness?
Traps	Being drawn into claiming cluster administration. Hold the line - it is a credible and respected position

19.6 "Terraform, Terraform Modules, Infrastructure Provisioning"

Field	Content
What they ask	"Show me a module you wrote. What were its inputs and outputs?"
Depth expected	Module structure, variables, outputs, versioning, and why modules rather than one configuration
Strong answer	"A module takes the pattern - networking, security groups, compute - and parameterises it: name, environment, CIDR, sizing, tags. It exports the IDs that other modules consume, so the application module takes the network module's subnet IDs rather than hardcoding them. No provider block inside the module, because providers are configured at the root and inherited. Modules are pinned to a version tag, because an unpinned module means someone else's commit can change every environment the next time you run <code>init</code> ."
Follow-ups	count vs for_each? What forces resource replacement? How do you test a module? Where does state live?
Traps	"I used Terraform" without being able to discuss state, locking or plan review

19.7 "Ansible, YAML" and Configuration Management

Field	Content
What they ask	"Why both Terraform and Ansible? Could you not do it all in one?"
Depth expected	The provisioning/configuration boundary, and idempotency
Strong answer	"You could, but each would be doing the other's job badly. Terraform is declarative with a state file, so it knows what it owns and can show a plan - that is right for infrastructure. Ansible is task-based with no state, checking reality per task each run - that is right for configuring systems, where you want packages, files and services converged. Using Terraform provisioners for configuration is explicitly discouraged, and using Ansible to build a VPC gives you no state and no plan."
Follow-ups	What is idempotency and where does it break? What is a handler? How do you do a rolling update with Ansible?
Traps	Not being able to define idempotency precisely, or not knowing that shell tasks break it

19.8 "Jenkins, Jenkins Pipelines (pipeline-as-code), GitHub Actions"

Field	Content
What they ask	"You list both Jenkins and GitHub Actions. When would you use each?"
Depth expected	A real distinction, not a preference
Strong answer	"GitHub Actions is tightly integrated with the repository, needs no infrastructure to operate, and is excellent for build, test and publish where the code already lives on GitHub. Jenkins is better when you need to orchestrate across systems, run inside a private network, support complex approval workflows, or you have existing investment in pipelines and shared libraries. In practice I use both - Actions for repository-scoped CI and Jenkins for deployment orchestration."
Follow-ups	Show me a Jenkinsfile. What is a multibranch pipeline? What is H in cron? How do you handle credentials? Write a GitHub Actions workflow
Traps	Being strong on Jenkins and unable to write a GitHub Actions workflow at all - it is on your CV, so prepare at least one

A GitHub Actions workflow you should be able to write from memory

```

name: build-and-push
on:
  push:
    branches: [main]
  pull_request:

jobs:
  build:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      id-token: write          # OIDC - no stored AWS keys
    steps:
      - uses: actions/checkout@v4

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3

      - name: Run tests
        run: make test

      - name: Configure AWS credentials
        if: github.ref == 'refs/heads/main'
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789012:role/github-ci
          aws-region: eu-west-1

      - name: Login to ECR
        if: github.ref == 'refs/heads/main'
        uses: aws-actions/amazon-ecr-login@v2

```

```
- name: Build and push
  if: github.ref == 'refs/heads/main'
  uses: docker/build-push-action@v5
  with:
    push: true
    tags: |
      ${ steps.login-ecr.outputs.registry }/payments-api:${ github.sha }
    cache-from: type=gha
    cache-to: type=gha,mode=max
```

19.9 "Prometheus, Grafana, AWS CloudWatch, Application Monitoring, Log Analysis, Alerting"

Field	Content
What they ask	"Write me a PromQL query for error rate." Or: "What do you alert on and why?"
Depth expected	Counters vs gauges, rate(), symptom-based alerting
Strong answer	"Error rate is $100 * \text{sum}(\text{rate}(\text{http_requests_total}\{\text{status}=\sim'5.\.'\}[5m])) / \text{sum}(\text{rate}(\text{http_requests_total}[5m]))$. I use rate because a counter's absolute value is meaningless. For alerting I go for symptoms users feel - error rate, latency percentiles, healthy target count - with a for: duration so a transient spike does not page anyone, and every alert links a runbook. I would not page on raw CPU, because high CPU is often normal; it belongs on the dashboard for diagnosis."
Follow-ups	Counter vs gauge? What is cardinality? What does Alertmanager add? Why CloudWatch as well?
Traps	Not being able to write any PromQL at all - practise the five queries in Part 12.4

19.10 "Python, Bash, PowerShell, Automation Scripts"

Field	Content
What they ask	"Write a script that checks disk usage and alerts above 80%." Live, on a whiteboard
Depth expected	Working code with error handling and a meaningful exit code
Strong answer	Write it, talking through decisions: "I'll use <code>shutil.disk_usage</code> rather than parsing <code>df</code> , because parsing command output is fragile across distributions. I'll exit non-zero on breach so cron or the pipeline can act on it, and log rather than print so it is visible when scheduled."
Follow-ups	How would you make it check multiple mounts? How would you send the alert? How would you test it?
Traps	PowerShell is on your CV - be ready for at least <code>Get-Service</code> , <code>Get-EventLog/Get-WinEvent</code> , <code>Test-NetConnection</code> and a basic loop

The PowerShell you should know, given your CV

```
Get-Service -Name "MyAppService"
Get-Service | Where-Object { $_.Status -eq "Stopped" -and $_.StartType -eq "Automatic" }
Restart-Service -Name "MyAppService" -Force
Get-WinEvent -LogName System -MaxEvents 50 | Where-Object { $_.LevelDisplayName -eq "Error" }
Get-EventLog -LogName Application -EntryType Error -Newest 20
Test-NetConnection -ComputerName db01 -Port 1433
Get-PSDrive -PSProvider FileSystem | Select-Object Name, @{n='UsedGB';e={[math]::Round(($_Used/1GB),1)}},
  @{n='FreeGB';e={[math]::Round(($_Free/1GB),1)}}
Get-Process | Sort-Object -Property WS -Descending | Select-Object -First 10 Name, Id, WS
Invoke-WebRequest -Uri "http://localhost:8080/health" -UseBasicParsing | Select-Object StatusCode

# A practical check loop
$services = @("MyAppService", "W3SVC")
foreach ($svc in $services) {
    $s = Get-Service -Name $svc -ErrorAction SilentlyContinue
    if ($null -eq $s) { Write-Warning "$svc not installed" }
    elseif ($s.Status -ne "Running") {
        Write-Warning "$svc is $($s.Status) - restarting"
        Restart-Service -Name $svc -Force
    } else { Write-Output "$svc OK" }
}
```

19.11 "Git, GitHub, Branching, Pull Requests"

Field	Content
What they ask	"Explain merge versus rebase. When do you use each?"
Depth expected	Correct, and with the golden rule stated
Strong answer	"Merge integrates two branches and creates a merge commit, preserving the real topology. Rebase replays my commits onto a new base, producing new hashes and linear history. I rebase my own feature branch to stay current with main, and I merge to integrate into a shared branch. The rule is never to rebase anything others have pulled, because rewriting shared history forces everyone into a painful recovery."
Follow-ups	reset vs revert? How do you resolve a conflict? How do you recover a lost commit? What is branch protection?
Traps	Confusing reset and revert - that is the single most common Git mistake in interviews

19.12 "Linux, Ubuntu, Windows Server, TCP/IP, HTTP/HTTPS, DNS, RTSP, Ports, Network Troubleshooting"

Field	Content
What they ask	"An application is unreachable. Walk me through your troubleshooting."
Depth expected	A layered method with specific commands and what each result means
Strong answer	Use the Part 4.10 method: scope, DNS, reachability, port, is it listening and on which interface, firewall path including NACLs, TLS, then application layer and logs, then dependencies, then capacity. Say what each result would tell you - "refused means the path is fine and the service is not listening; timeout means something is dropping packets"
Follow-ups	What is RTSP and how is it different? Why can an ALB not route it? What does <code>ss -tulpn</code> tell you that <code>netstat</code> does not?
Traps	Jumping straight to "I'd restart it". Also: RTSP is explicitly on your CV, so know what it is

19.13 "Production Support, Incident Troubleshooting, RCA, Release Management"

Field	Content
What they ask	"Walk me through an RCA you have written."
Depth expected	Structure, blamelessness, and action items with owners
Strong answer	"The structure is summary, impact in factual terms, a timestamped timeline including when detection happened, the actual root cause, how we mitigated, and action items with owners and dates. I keep it blameless - if one person's action could take production down, the finding is the missing guard rail, not the person. And I always include the detection gap, because how long it took to notice is usually as important as the cause."
Follow-ups	Give me an example. What is the 5 Whys? What is the difference between mitigation and fix? How do you communicate during an incident?
Traps	"Root cause: human error" - that is a symptom, not a cause

19.14 Current Role Bullet 1: "Manage DevOps activities across Linux and Windows"

Field	Content
What they ask	"What is different about supporting Windows versus Linux?"
Depth expected	Genuine operational differences, not just "different commands"
Strong answer	"Practically: services are Windows Services managed with <code>Get-Service</code> and <code>Restart-Service</code> rather than <code>systemd</code> ; logs are Event Viewer rather than <code>journald</code> , which changes how you search them; Ansible connects over WinRM with the <code>win_*</code> modules rather than SSH; pipelines need Windows agents and <code>powershell</code> or <code>bat</code> steps rather than <code>sh</code> ; and patching and reboots are a bigger operational event. Having both in one estate means pipelines have to be label-aware and runbooks have to exist in two forms."

Field	Content
Follow-ups	How do you check a Windows service from a pipeline? How do you read Windows logs? How does Ansible reach Windows?
Traps	Having only Linux answers when your CV explicitly claims both

19.15 Current Role Bullet 2: "Built and maintain Jenkins CI/CD pipelines... reducing manual deployment effort and improving release consistency"

Field	Content
What they ask	"Reducing manual effort - by how much?"
Depth expected	Mechanism, honestly, without invented figures
Strong answer	"I did not own that measurement so I will not quote a number I cannot defend. What I can describe is the mechanism: the manual deployment was a sequence - copy the build, stop the service, replace files, start it, verify - performed by a person, and it differed slightly between environments. The pipeline made it one triggered job running the same steps every time, which removed the manual variance and the need for someone to be available at release time. The consistency improvement is the same mechanism: the same artifact, built once, deployed the same way everywhere."
Follow-ups	Walk me through the pipeline. How do you handle credentials? How do you roll back? What happens if the health check fails?
Traps	Inventing a percentage and then contradicting yourself on how it was measured

19.16 Current Role Bullet 3: "Provisioned AWS infrastructure with Terraform... automated configuration with Ansible"

Field	Content
What they ask	"What exactly did you provision, and what happens when you run apply?"
Depth expected	Specific resources, plus the apply lifecycle
Strong answer	Name what you actually built - VPC, subnets, security groups, EC2, IAM roles, S3, Route 53 records, CloudWatch resources - and then describe apply: lock state, read state, refresh from the API, build the dependency graph, diff, execute in dependency order, write state, unlock. Mention that a -/+ in the plan means destroy and recreate and that you always read why
Follow-ups	Where is your state? What is locking? How do you handle drift? What would you do if the plan wanted to destroy production?
Traps	Being unable to explain what state actually is

19.17 Current Role Bullet 4: "Containerized application components with Docker and support Kubernetes deployments"

Field	Content
What they ask	"Show me a Dockerfile you would write." Live
Depth expected	Correct layer ordering, multi-stage, non-root, healthcheck, exec-form entrypoint
Strong answer	Write it while explaining: "Dependencies before source so the cache survives code changes. Multi-stage so the compiler never reaches the final image. A non-root user. A health check. ENTRYPOINT in exec form so the process is PID 1 and gets SIGTERM - otherwise docker stop takes the full grace period and drops connections on every deploy."
Follow-ups	CMD vs ENTRYPOINT? COPY vs ADD? Why is your image large? How do you debug a container that will not start?
Traps	Writing a Dockerfile that copies source before installing dependencies - the interviewer will notice immediately

19.18 Current Role Bullet 5: "Implemented monitoring with Prometheus, Grafana and AWS CloudWatch; maintain GitHub Actions workflows"

Field	Content
What they ask	"What does 'implemented' mean here - did you deploy Prometheus, or configure it?"
Depth expected	Honest scope again
Strong answer	Be precise about what you did: configured scrape targets and exporters, wrote alert rules, built dashboards, set up CloudWatch agent configuration and alarms - whichever is true. "I set up the monitoring for our applications and infrastructure: exporters and scrape configuration, alert rules, and Grafana dashboards, plus CloudWatch for the AWS services we cannot instrument and for log aggregation."
Follow-ups	Write a PromQL query. What is the difference between Prometheus and Grafana? Why CloudWatch too? What alerts did you create?
Traps	Implying you architected an observability platform if you configured one

19.19 Current Role Bullet 6: "Developed Python and Bash scripts for deployment support, system checks, log analysis and network troubleshooting"

Field	Content
What they ask	"Describe the most useful script you wrote and why."
Depth expected	A real script with a real purpose and design decisions
Strong answer	Use the log parser from Part 14.7 or the health check: "The most useful was a log analysis script. The insight was normalising variable parts out of error messages - numbers, hex IDs - before counting, which turns thousands of near-identical lines into a short list of actual distinct problems, plus a per-hour histogram showing exactly when it started. Before that, finding the real error in a large log was manual."
Follow-ups	Write it. How do you handle a 5 GB log file? How do you make it safe to schedule? Why Python rather than Bash here?
Traps	"I wrote some scripts" with no specifics

19.20 Current Role Bullet 7: "Investigate production incidents... perform RCA, coordinate fixes with development and QA, validate application upgrades"

Field	Content
What they ask	"Tell me about a production incident you handled end to end."
Depth expected	Method, communication, RCA, prevention - and no invented drama
Strong answer	Follow the structure: how it was detected, how you scoped it, how you narrowed it down layer by layer, what you did to mitigate, how you validated recovery, the root cause, and the action item that came out of it. Emphasise that you mitigated before diagnosing and that you coordinated with development and QA, because your CV says that explicitly
Follow-ups	How did you know it was fixed? What was the detection gap? What would have prevented it? What did you tell stakeholders?
Traps	A story with no validation step, or one where you never say how you confirmed recovery

19.21 Previous Role Bullets

Bullet	Likely question	Key point to make	
"Created and maintained Jenkins pipelines... integrated with Git"	"How is that different from what you do now?"	Same tooling, smaller scope - pipelines rather than the whole environment	
"Packaged applications with Docker to standardize environments"	"What inconsistency did that actually remove?"	Runtime and library versions travelling with the app rather than being installed per server	
"Operated AWS services (EC2, S3, IAM, VPC, CloudWatch, Route 53)"	"What does 'operated' mean?"	Be precise - created, configured and maintained these, within an architecture you did not design alone	
"Wrote Python and Bash scripts for operational tasks, environment checks and service management"	"Give me one."	Have a specific script ready	

Bullet	Likely question	Key point to make	
"Monitored application and infrastructure health... and investigated alerts"	"What did you do when an alert fired?"	A method: verify it is real, assess impact, find the layer, mitigate, then fix the alert if it was noise	
"Performed Linux and Windows troubleshooting (service failures, disk utilization, network connectivity, application logs)"	"Disk utilization - walk me through a disk-full incident."	The full runbook from Part 16.5, including <code>df -i</code> and <code>lsdf \</code>	grep d eleted
"Documented recurring issues and procedures"	"Why does that matter?"	Runbooks reduce MTTR and remove single points of knowledge - and a recurring issue that needs a runbook is really a backlog item	

19.22 Education and Certifications

Line	Likely question	Strong answer
B.Tech Mechanical Engineering, 2018-2022	"Why the switch?"	The three-sentence answer from Part 1.5.1 - systems thinking, deliberate transition, formal training
"AWS, DevOps, Terraform, Ansible" certifications	"Which certification exactly? Can you give me the number?"	Be precise about what is a completed certification versus completed training. "I completed AWS, DevOps, Terraform and Ansible training as part of my DevOps program; I am preparing for the AWS Solutions Architect Associate exam." Never claim a credential you cannot produce
"Currently strengthening Kubernetes administration, AWS architecture, GitHub Actions and cloud-native observability"	"How are you strengthening them? What have you learned recently?"	Have a real answer - a specific course, a home lab, a cluster you run locally, something you built. This line invites the question, so it must be true

THE BACKGROUND-CHECK TRAP

Certification claims are verified. If your CV line "AWS | DevOps | Terraform | Ansible" refers to training rather than certification exams, say so clearly the first time you are asked. Being upfront costs you nothing; being caught costs you the offer at the final stage, after everyone has invested time.

19.23 The Self-Audit Checklist

MARK EACH ONE HONESTLY BEFORE YOUR INTERVIEW

For each line below, can you speak for 60 seconds without hesitating?

1. My three-year timeline and what changed between the two roles.
2. Why Mechanical Engineering, without apologising.
3. My full pipeline, stage by stage, and what each stage protects against.
4. A Jenkinsfile, written from memory, with credentials handled correctly.
5. A GitHub Actions workflow, written from memory.
6. A production-quality Dockerfile, and why the layers are in that order.
7. CMD vs ENTRYPOINT, exec form vs shell form.
8. What Terraform state is, where mine lives, and why it is locked.
9. count vs for_each, and what a -/+ in the plan means.
10. Terraform vs Ansible, in two sentences.
11. Idempotency, and where Ansible breaks it.
12. CrashLoopBackOff debugging, including --previous.
13. Why a Service has no endpoints.
14. Liveness vs readiness, and why liveness must not check dependencies.
15. A PromQL error-rate query.
16. Why CloudWatch as well as Prometheus.
17. Why there is no EC2 memory metric by default.
18. What makes a subnet public.
19. Security group vs NACL.
20. Refused vs timeout.
21. 500 vs 502 vs 503 vs 504.
22. The disk-full runbook including inodes and deleted-but-open files.
23. A disk-check script in Python, written live.
24. A PowerShell service check.
25. Merge vs rebase, reset vs revert.
26. My RCA structure, and a real incident told end to end.
27. What I would improve about my current setup.
28. Where my Kubernetes knowledge ends - stated confidently.

PART 20

HR AND MANAGERIAL ROUND

Believable answers, in your own situation

The HR and managerial round is not a formality - candidates are rejected here for sounding rehearsed, for badmouthing an employer, or for giving an answer that contradicts something they said in the technical round. Every answer below is written to fit your actual CV. Adapt the wording so it sounds like you; keep the structure.

THREE RULES FOR THIS ROUND

- 1) **Never criticise a current or previous employer.** Frame every move as movement towards something, not away from something.
- 2) **Every behavioural answer needs a concrete example**, using Situation, Task, Action, Result - and the Result must be something you can defend.
- 3) **Consistency matters more than polish.** If you told the technical interviewer you support Kubernetes workloads, do not tell the manager you run the cluster.

20.1 Tell Me About Yourself

Use the full answer from Part 1.7. Structure: Present (40%), Past (30%), Future (20%), Hook (10%). The hook is the last line that steers the next question onto ground you have prepared.

SHORT REMINDER VERSION

"I'm a DevOps Engineer with three years of experience, currently at I2V Systems in Gurgaon. My work breaks into four areas: CI/CD with Jenkins and GitHub Actions; infrastructure automation on AWS with Terraform and Ansible; containers and monitoring with Docker, Kubernetes, Prometheus, Grafana and CloudWatch; and production support, where I investigate incidents, do root cause analysis and coordinate fixes with development and QA. Before this I was at Naresh I Technology from January 2024 doing the same core work at a smaller scope. My degree is a B.Tech in Mechanical Engineering, and I moved into DevOps deliberately through formal training. What I'm looking for now is more depth in Kubernetes and cloud architecture - which is why this role interested me. Happy to go deeper on the pipeline or on a production incident, whichever is more useful."

20.2 Why DevOps?

WHY DEVOPS

"Two reasons, one from my background and one from the work itself. From my background - my degree is Mechanical Engineering, and the part I enjoyed most was systems: how a process behaves end to end, where it fails, and how you make it repeatable. That is exactly what DevOps rewards.

From the work: I like that DevOps has an immediate, visible effect. When a deployment goes from a manual sequence someone performs to a pipeline that runs identically every time, you can see the difference in how releases go. And I genuinely enjoy the troubleshooting side - working out why something is failing when the logs do not say so directly is the part of the job I find most satisfying. Those two things - building automation and diagnosing failures - are the whole job, and I did not want to do either of them in isolation."

20.3 Why Are You Switching / Why Are You Leaving?

WHY YOU ARE LOOKING

"I'm not leaving because of a problem - my current role has given me good breadth, from pipelines and infrastructure automation through to production support across Linux and Windows. The reason I'm looking is depth. I want Kubernetes and cloud architecture to be a bigger part of my day-to-day rather than one part of a broad role, and I want to work somewhere with more engineers doing the same thing, because that is how you learn fastest. When I looked at this role, that is what stood out."

ANSWERS THAT END INTERVIEWS

"My manager doesn't appreciate me." / "The processes there are a mess." / "There's no growth." / "I want more money." Even when true, these tell the interviewer you will describe them the same way in a year. If the real reason is compensation, phrase it as wanting a role with more scope and responsibility - which is also true, since scope is what compensation follows.

20.4 Why Should We Hire You?

WHY YOU

"Three things. First, I cover the full path from commit to running system - I build the pipelines, provision the infrastructure as code, containerise and deploy the application, and monitor it afterwards. That means I am not handing work over at each boundary. Second, production troubleshooting is genuinely my strongest area: I work incidents through logs, service state on both Linux and Windows, database connectivity and network diagnostics, and I do the root cause analysis afterwards - a lot of engineers at my level can build pipelines but cannot diagnose a live failure. Third, I know where my knowledge ends and I say so. I am strong on Kubernetes workloads and I am still building cluster administration depth, and I would rather tell you that now than have you find out in month two."

20.5 Why This Company?

YOU MUST CUSTOMISE THIS ONE

A generic answer here is worse than a short one. Before the interview, find three specific things: what the product actually does, something in the job description that matches your experience, and something in it that stretches you. Then say all three.

STRUCTURE TO FILL IN

"Three reasons. First, the role itself - the job description mentions [specific technology or responsibility], and that is exactly what I do now with [your matching experience], so I could contribute early. Second, it stretches me where I want to grow - [the part that is new: deeper Kubernetes, multi-region architecture, SRE practices]. Third, [something specific about the company: the product, the scale, the engineering culture, something you read or were told]. I would rather join somewhere that needs what I already do well and will push me on the rest."

20.6 Strengths and Weaknesses

"WHAT ARE YOUR STRENGTHS?"

"Troubleshooting is the one I would name first. When something breaks I work it methodically rather than guessing - scope it, narrow it layer by layer, mitigate before diagnosing, and then actually close the loop with a root cause and a preventive action. It is also the thing I am asked to do most."

The second is that I work across boundaries. Because I cover the pipeline, the infrastructure, the containers and the monitoring, I can usually tell quickly whether a problem is the application, the platform or the network - and I work directly with development and QA rather than passing tickets."

"WHAT IS YOUR WEAKNESS?"

"Kubernetes cluster administration. I am strong at the workload level - deploying, debugging pods and services, understanding rollouts and probes - but I have not run a control plane in production. I am addressing it directly: I run clusters locally to practise, I am working through cluster administration material, and it is one of the reasons I am looking for a role where Kubernetes is more central. I would rather name that specifically than give you a non-answer."

WEAKNESS ANSWERS THAT FAIL

"I'm a perfectionist" / "I work too hard" - these are transparent and interviewers are tired of them. So is naming a weakness that is critical to the job with no plan attached. The formula that works: a real gap, stated specifically, plus what you are actively doing about it, plus why it does not block you doing this job.

20.7 Tell Me About a Difficult Incident

STAR STRUCTURE - ADAPT TO SOMETHING YOU ACTUALLY DID

Situation: A component became unreachable in production while the process itself was still running, so the usual 'is it up' checks said everything was fine.

Task: Restore service, and work out why the monitoring had not caught it.

Action: I scoped it first - which users, which environment, when it started, and whether anything had been deployed. Then I worked outward: the service was running, and it was listening, but connections from the client side were failing, which told me the process was alive but not serving correctly. The application logs showed it was failing on a dependency, so I tested that path specifically - name resolution, port reachability and credentials - and confirmed the dependency was the actual failure. I mitigated by restoring that path, preserved the logs first so we could still investigate, and validated recovery using the same signal that had shown the failure rather than just a local check.

Result: Service was restored, and the RCA produced two action items - an alert on the dependency itself rather than only on the application, and a health check that actually tests the dependency so the instance is taken out of rotation instead of failing silently. The second one was the more valuable outcome, because it changed the failure mode from silent to visible."

CHOOSING YOUR STORY

Pick something true, medium-sized and diagnostically interesting. Do not pick something that makes you the lone hero - interviewers are suspicious of those. The best stories are ones where the fix was ordinary and the diagnosis was the hard part, and where the prevention changed the system rather than just the behaviour.

20.8 Tell Me About a Production Issue You Caused

HONEST, WITH THE SYSTEM FIX

"I have made mistakes that caused problems - most engineers who touch production have. The one I would describe is a change where the configuration in one environment did not match another, so it worked where I tested it and failed where it mattered. I noticed it quickly because the error rate moved, said so immediately in the channel rather than trying to quietly fix it, and rolled back.

What I took from it was not 'be more careful' - that never works. It was that the system allowed an environment-specific config to reach production without anything comparing the two, so the fix was a check in the pipeline. The other thing I took from it was the value of saying it out loud straight away: knowing the cause is the fastest route to resolution, and a team where people hide mistakes has longer outages."

20.9 Tell Me About a Disagreement

DISAGREEMENT

"We disagreed about whether a release should go out without a particular test stage passing, because the deadline was tight. I did not argue about process - I made the trade-off explicit: here is specifically what we would not know if we skipped it, and here is the risk that carries. Then I offered options: ship a smaller scope that is fully tested, ship it with a rollback plan agreed and someone watching the dashboards, or delay.

We went with shipping and watching, with the rollback ready. It was fine. The point for me was that it became an informed decision rather than a disagreement - my job is to make the risk visible, not to be the person who says no. And if it had gone wrong, we would have had the rollback rehearsed rather than improvised."

20.10 How Do You Handle Pressure?

PRESSURE

"By being systematic, because that is what pressure erodes first. In an incident, the temptation is to start changing things - restart it, redeploy it, try something. I make myself do the first two steps properly: scope it, and ask what changed. Those two questions usually eliminate most of the possibilities in under a minute, which is faster than guessing.

The other thing is communication. I post status updates on a fixed cadence even when there is nothing new, because silence makes people interrupt the person who is actually working the problem. And I ask for help early rather than at the point where I am stuck and out of time."

20.11 How Do You Prioritise Incidents?

PRIORITISATION

"By user impact and blast radius, not by volume of noise. A complete outage affecting everyone comes before a degraded feature, which comes before an internal tool. I also check early whether multiple alerts are actually one incident with several symptoms, which is common - a shared dependency failing lights up alerts everywhere, and the earliest alert by timestamp is usually the cause.

If there genuinely are several at once, I would rather assign owners and coordinate than try to work them all myself. And for anything outside an incident, I prioritise by whether it reduces the chance or the cost of the next incident - alerting gaps and missing runbooks usually beat feature work on that measure."

20.12 Are You Comfortable With On-Call?

ON-CALL

"Yes. I already handle production issues in my current role, so being the person who responds is not new to me. I would want to understand the practicalities - the rotation size, what the escalation path looks like, and whether alerts have runbooks attached - because those are what make on-call sustainable rather than exhausting.

And I would want to improve it where I could. An alert that pages a human for something that is not actionable is a defect, and it is how real incidents end up ignored. I would track what fires and what action was actually taken, and remove or demote the ones that never lead to anything."

20.13 Where Do You See Yourself?

FIVE YEARS

"Technically deeper rather than further from the technology. In the next couple of years I want to be genuinely strong on Kubernetes and cloud architecture - designing systems rather than only operating them - and I would like to move towards the SRE side, where reliability is measured and engineered rather than just responded to.

Longer term I would like to be the person a team relies on for the hard problems, and to be mentoring engineers earlier in that path. I am not aiming at management in the near term - I want more technical depth first, and I would rather be honest about that than tell you what I think you want to hear."

20.14 Expected Salary

HOW TO HANDLE IT

Never name the first number if you can avoid it, and never name a number you have not researched. Three usable positions, in order of preference.

DEFLECT FIRST

"I'd rather understand the role and the scope properly first - the responsibilities here look broader than my current position. Do you have a band for this role? I am confident we can align if the role is the right fit."

IF PRESSED FOR A NUMBER

"Based on my three years of experience across CI/CD, cloud infrastructure and production support, and the market for DevOps engineers in the NCR region, I am looking in the range of [X to Y]. I am flexible for the right role, and I am considering the whole package rather than just the base."

IF ASKED FOR YOUR CURRENT SALARY

"My current compensation is [X]. I would rather we talk about the value of this role, since the scope is different - but I am happy to be transparent." (In some jurisdictions you are not obliged to answer; in India you usually will be asked and expected to answer, and a refusal can stall the process.)

SALARY MISTAKES

- Naming a number before you understand the role.
- Naming a number you have not researched, then discovering it was well below the band.
- Saying "whatever you think is fair" - it signals you have not valued yourself.
- Inflating your current salary. It is verified from your payslips and an inflated figure ends offers.

20.15 Notice Period

NOTICE PERIOD

"My notice period is [X days/months]. I would serve it properly - I would not want to leave my current team without a handover, and you would not want someone who does. If there is flexibility on the start date I am happy to discuss it, and if buyout is an option that is a conversation I am open to."

20.16 Other Questions You Will Be Asked

Q. How do you keep your skills current?

"A mix of deliberate study and building. I am currently working through Kubernetes administration and AWS architecture material, and I run things locally to practise rather than only reading - a local cluster, Terraform against a sandbox account. I also read the change logs of the tools I depend on, because most 'nothing changed' incidents turn out to be something upstream that moved. And honestly, incidents are a large part of it - every real failure teaches you something the documentation does not."

Q. How do you work with developers who do not follow process?

"I start by finding out why, because usually there is a real reason - the pipeline is slow, a test is flaky, or the process does not fit how they actually work. A process people route around is not protecting anything, so the first question is whether it deserves to exist in that form. If it does, I make the right path the easy path - templates, shared libraries, good error messages - rather than relying on enforcement. Enforcement is the last layer, not the first."

Q. What do you do if you do not know the answer?

"I say so, and then say how I would find out. In an interview that is exactly what I would do here. In the job, it is the same - I would rather say 'I have not done that, here is my understanding and here is what I would verify first' than guess and send someone down the wrong path during an incident."

Q. How do you handle repetitive work?

"I automate it. My rule is that anything I do twice manually gets scripted, and anything that touches production gets a dry-run mode and logging before it gets a schedule. That said, I would not automate something whose process is still changing weekly - you end up maintaining automation against a moving target, which costs more than the toil did."

Q. Describe a time you improved something without being asked.

Use a real example within your CV's scope - adding an alert for a gap you noticed, writing a runbook for a recurring issue, or adding a check to a pipeline after an incident. The shape is: I noticed a recurring cost or risk, I made a small change, and here is the specific thing that got better.

Q. How do you handle a task you have never done before?

"I find out what good looks like before I start - documentation, how it is done elsewhere in the organisation, and whoever has done it before. Then I do it somewhere safe first: a lower environment, a dry run, a plan I can read before applying. And I say early that it is new to me, because the cost of asking is a few minutes and the cost of quietly getting it wrong in production is much higher."

Q. What kind of team do you work best in?

"One where people share ownership rather than throwing work over a boundary, and where it is safe to say 'I broke it' or 'I don't know'. I work well with developers and QA directly - my current role involves coordinating fixes with both - and I would rather be in a team that reviews each other's work honestly than one that is polite about it."

Q. Do you prefer working alone or in a team?

"Both, for different parts of the job. Deep work - writing a module, debugging something subtle - I do better with uninterrupted time. But incidents, design decisions and anything that will outlive me need other people, because the failure mode of working alone is that knowledge stays in one head. I try to make sure nothing I build is only understandable by me."

Q. What are you looking for in your next role?

"Depth in Kubernetes and cloud architecture, a team I can learn from, and ownership of something that matters. I would also like reliability to be engineered rather than only responded to - SLOs, error budgets, proper post-incident follow-through - because that is the direction I want my career to go."

Q. Do you have any questions for us?

Always yes - see Part 25.6 for the list. Never "no, I think you covered everything."

20.17 The Consistency Check

DO NOT CONTRADICT YOURSELF BETWEEN ROUNDS

The technical interviewer and the manager compare notes. Before the interview, fix these answers so they are the same in every room:

- Your exact dates at each company.
- The scope of your Kubernetes experience - workloads, not cluster administration.
- Whether your certifications are completed exams or completed training.
- Why you are leaving - one reason, stated the same way.
- Your notice period.
- Your current compensation.
- The project you describe, and what you personally did within it.

PART 20 RECALL POINTS

- Never criticise an employer; frame every move as towards something.
- Weakness = real gap + what you are doing about it + why it does not block this job.
- Every behavioural answer is STAR, with a defensible result.
- Deflect salary once, then give a researched range, never a bare number.
- "I don't know, but here is how I would find out" is a strong answer, not a weak one.
- Always have questions for them.
- Consistency across rounds matters more than polish.

PART 21

RAPID REVISION

Everything on a few pages - the night-before section

Read this section the evening before an interview and again on the morning. It contains no new material - it is the highest-value 5% of the handbook, compressed. If something here is unfamiliar, go back to its part immediately.

21.1 Linux Command Sheet

Task	Command				
Disk usage / inodes	df -h / df -i				
Biggest directories	du -xh --max-depth=1 / \	sort -h			
Biggest files	find / -xdev -type f -size +500M -exec ls -lh {} \; 2>/dev/null				
Deleted-but-open files	lsof +L1 or lsof \	grep deleted			
Memory	free -h (read AVAILABLE) / vmstat 1 5				
OOM evidence	dmesg -T \	grep -i "killed process"			
Top CPU	ps -eo pid,%cpu,etime,cmd --sort=-%cpu \	head			
Top memory	ps -eo pid,rss,%mem,cmd --sort=-rss \	head			
Live view	top (press 1 for per-core, M memory, P cpu)				
Process tree	ps -ef --forest / pstree -p				
Kill gracefully then hard	kill PID then kill -9 PID				
Reload config	kill -HUP PID / systemctl reload svc				
Service state	systemctl status svc -l				
Failed units	systemctl list-units --failed				
After editing a unit	systemctl daemon-reload				
Service logs	journalctl -u svc -n 200 --no-pager				
Previous boot	journalctl -b -1				
Live tail (rotation-safe)	tail -F /var/log/app.log				
Search with context	grep -C 5 "Exception" app.log				
Count errors	grep -c ERROR app.log				
Top IPs in access log	awk '{print \$1}' access.log \	sort \	uniq -c \	sort -rn \	head
Status code counts	awk '{print \$9}' access.log \	sort \	uniq -c \	sort -rn	
Edit in place with backup	sed -i.bak 's/old/new/g' file				
Listening sockets	ss -tulpn (check the BIND ADDRESS)				

Task	Command			
Test a port	nc -zv host 443 or timeout 3 bash -c "</dev/tcp/host/443"			
HTTP timing breakdown	curl -w '%{time_namelookup} %{time_connect} %{time_total}\n' -o /dev/null -s URL			
DNS	dig +short name / dig @8.8.8.8 name / getent hosts name			
Route for an IP	ip route get 8.8.8.8			
Append as root	echo "line" \	sudo tee -a /etc/file		
Find recently changed	find /etc -mmin -60 -type f			
Certificate expiry	echo \	openssl s_client -connect h:443 -servername h 2>/dev/null \	openssl x509 -noout -dates	

21.2 Git Command Sheet

Task	Command
Status / short	git status / git status -s
Stage a hunk	git add -p
Undo working change	git restore file
Unstage	git restore --staged file
Fix last commit	git commit --amend
Undo last commit, keep work	git reset --soft HEAD~1
Undo a PUSHED commit	git revert SHA
Recover lost commit	git reflog then git reset --hard SHA
Park work	git stash -u / git stash pop
Take one commit	git cherry-pick SHA
Update my branch onto main	git switch feat && git rebase main
Safe force push	git push --force-with-lease
History picture	git log --oneline --graph --decorate --all
What is on my branch only	git log main..feature
Who changed this line	git blame file
Find the breaking commit	git bisect start / bad / good v1.2
Tag a release	git tag -a v1.4.2 -m "..." && git push origin v1.4.2
Clean workspace	git clean -fdx (preview with -n)

21.3 Docker Command Sheet

Task	Command
Build	docker build -t app:1.0 .
Run detached with ports	docker run -d -p 8080:80 --name web app:1.0
Shell inside	docker exec -it web sh
Logs	docker logs -f --tail 100 web
Exit code / OOM	docker inspect -f '{{.State.ExitCode}} {{.State.OOMKilled}}' web
What it wrote	docker diff web
Live resource use	docker stats
Network with DNS	docker network create appnet
Disk usage	docker system df

Task	Command
Safe cleanup	<code>docker system prune -af --filter "until=72h"</code>
Debug a crashing image	<code>docker run --entrypoint sh -it app:1.0</code>

Exit code	Meaning
0 (repeatedly)	Main process finished - wrong command, or app daemonises
1	Application error - read the logs
126	Not executable / architecture mismatch
127	Command not found in the image
137	SIGKILL - almost always OOMKilled
143	SIGTERM - normal docker stop

21.4 Kubernetes Command Sheet

Task	Command
Overview	<code>kubectl get pods -o wide</code>
WHY something is wrong	<code>kubectl describe pod NAME</code>
Crashed container's logs	<code>kubectl logs NAME --previous</code>
Service debugging	<code>kubectl get endpoints SVC</code>
What happened, in order	<code>kubectl get events --sort-by=.lastTimestamp</code>
Resource usage	<code>kubectl top pods --sort-by=memory</code>
Test a pod directly	<code>kubectl port-forward pod/NAME 8080:8080</code>
Shell in	<code>kubectl exec -it NAME -- sh</code>
Debug a shell-less image	<code>kubectl debug -it NAME --image=busybox --target=app</code>
Update image	<code>kubectl set image deploy/app app=img:1.4.3</code>
Wait for rollout (use in CI)	<code>kubectl rollout status deploy/app --timeout=300s</code>
Roll back	<code>kubectl rollout undo deploy/app</code>
Restart to pick up config	<code>kubectl rollout restart deploy/app</code>
Preview a change	<code>kubectl diff -f manifest.yaml</code>
Node maintenance	<code>kubectl drain node --ignore-daemonsets && kubectl uncordon node</code>
Permission test	<code>kubectl auth can-i create deploy -n prod</code>

Symptom	First command	Usual cause
CrashLoopBackOff	<code>kubectl logs --previous</code>	App error, OOM, wrong command
ImagePullBackOff	<code>kubectl describe pod</code>	Tag, credentials, registry route
Pending	<code>kubectl describe pod</code>	Resources, taints, affinity, PVC
Running but unreachable	<code>kubectl get endpoints</code>	Readiness failing, selector mismatch
Rollout stuck	<code>kubectl get pods</code>	New pods failing one of the above
Old version serving	<code>check image DIGEST</code>	Mutable tag + IfNotPresent

21.5 Terraform and Ansible Sheets

Terraform	Command
Initialise	<code>terraform init</code>
Format check (CI)	<code>terraform fmt -check -recursive</code>
Validate	<code>terraform validate</code>
Plan to a file	<code>terraform plan -out=tfplan</code>

Terraform	Command
Apply the reviewed plan	terraform apply tfplan
Show drift only	terraform apply -refresh-only
List state	terraform state list
Rename without destroying	terraform state mv old new
Stop managing (keeps resource)	terraform state rm addr
Adopt an existing resource	terraform import addr ID
Force recreate one resource	terraform apply -replace="addr"
Clear a stuck lock	terraform force-unlock ID

Ansible	Command
Connectivity	ansible all -m ping
Dry run with diffs	ansible-playbook site.yml --check --diff
One host	ansible-playbook site.yml --limit web01
Part of a playbook	ansible-playbook site.yml --tags deploy
Override a variable	ansible-playbook site.yml -e "app_version=1.4.2"
Inventory tree	ansible-inventory -i inv --graph
Encrypt a value	ansible-vault encrypt_string 'secret' --name 'db_password'

21.6 PromQL Sheet

Question	Query
Is the target up?	up == 0
Requests per second	sum(rate(http_requests_total[5m]))
Error rate %	100 * sum(rate(http_requests_total{status=~"5.."}[5m])) / sum(rate(http_requests_total[5m]))
p95 latency	histogram_quantile(0.95, sum by (le) (rate(http_request_duration_seconds_bucket[5m])))
CPU used %	100 - (avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)
Memory used %	100 * (1 - node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes)
Disk full prediction	predict_linear(node_filesystem_avail_bytes{mountpoint="/" }[6h], 4*3600) < 0
Pod restarts in an hour	increase(kube_pod_container_status_restarts_total[1h]) > 0
CPU throttling	rate(container_cpu_cfs_throttled_seconds_total[5m]) > 0

21.7 AWS Quick Facts

Fact	Detail
What makes a subnet public	Route table has 0.0.0.0/0 -> Internet Gateway
Reserved IPs per subnet	5 (network, router, DNS, future, broadcast) - a /24 gives 251
Security group	Stateful, instance level, allow only, can reference other SGs
NACL	Stateless, subnet level, allow AND deny, numbered order - allow ephemeral returns
IGW vs NAT GW	IGW = two-way for public IPs; NAT = outbound only for private
IAM evaluation	Explicit Deny wins; default deny; SCP is a ceiling
Metadata endpoint	169.254.169.254 - IMDSv2 requires a token (mitigates SSRF)
EBS vs instance store	EBS persists; instance store is lost on stop/terminate
No default metrics	Memory and disk-used need the CloudWatch agent
ALB vs NLB	ALB = L7, host/path routing; NLB = L4, static IP, preserves source IP
ELB 5XX vs Target 5XX	ELB = no healthy targets; Target = the application errored

Fact	Detail
ASG health check type	Set it to ELB, not EC2
CNAME at apex	Not allowed - use a Route 53 ALIAS
S3 versioning	Costs grow forever without a noncurrent-version lifecycle rule
Log group retention	Never expires unless you set it

21.8 The Difference Tables

Docker	Virtual Machine
Shares the host kernel	Its own guest kernel
Megabytes, starts in under a second	Gigabytes, starts in minutes
Namespaces + cgroups	Hypervisor
Weaker isolation	Stronger isolation

Terraform	Ansible
Provisions infrastructure	Configures existing systems
Declarative with a state file	Task-based, no state file
Plan before apply	--check for a dry run
Knows what it owns	Checks reality per task

git merge	git rebase
Preserves topology, adds a merge commit	Linear history, new commit hashes
Safe on shared branches	NEVER on shared branches
Integrating a feature into main	Updating your own branch onto main

git reset	git revert
Moves the pointer, rewrites history	Adds a new commit that undoes
Local commits only	The correct way to undo a pushed commit

CI	CD (Delivery)	CD (Deployment)
Every commit built and tested	Every green build is releasable	Every green build ships automatically
Ends at a tested artifact	Ends at an approved artifact	Ends in production
No human gate	Human decides when	No human gate

Kubernetes Deployment	Service
Manages pods and rollouts	Stable virtual IP and DNS for pods
Creates ReplicaSets	Selects pods by LABEL
Handles self-healing and scaling	Load-balances to READY endpoints

ConfigMap	Secret
Non-sensitive config	Passwords, tokens, keys
Plaintext in etcd	base64 in etcd (encoding, NOT encryption)
Same 1 MiB limit	Same 1 MiB limit

Liveness probe	Readiness probe
Fails -> container RESTARTED	Fails -> pod removed from endpoints
Detects a wedged process	Controls traffic
Must NOT check dependencies	May check dependencies

requests	limits
What the scheduler reserves	The enforced ceiling
Determines placement	CPU throttled, memory OOMKilled
Needed for HPA to work	Prevents noisy neighbours

ALB	NLB
Layer 7, HTTP/HTTPS	Layer 4, TCP/UDP
Routes on host, path, headers	IP and port only
DNS name, X-Forwarded-For	Static IP, preserves source IP
Web apps, microservices	RTSP, databases, extreme throughput

Security Group	NACL
Stateful	Stateless
Instance / ENI	Subnet
Allow only	Allow and Deny
Can reference other SGs	CIDR only

Public subnet	Private subnet
Route 0.0.0.0/0 -> IGW	Route 0.0.0.0/0 -> NAT Gateway
Load balancer, NAT GW, bastion	Application and data tiers
Reachable from the internet	Outbound only

Image	Container
Immutable layered template	Running instance with a writable layer
Stored in a registry	Lives on a host
One image, many containers	Writable layer dies with it

Pod	Deployment
Smallest unit, one or more containers	Manages ReplicaSets of pods
No self-healing on its own	Rolling updates, rollback, self-healing
Rarely created directly	What you actually deploy

Prometheus	Grafana
Collects, stores, alerts	Visualises
Own TSDB, PromQL	Queries data sources, stores nothing
Pull-based scraping	Dashboards and variables

Horizontal scaling	Vertical scaling
More instances	Bigger instance
Near-unbounded, no downtime	Bounded, needs a restart
Requires stateless design	Works for databases

CMD	ENTRYPOINT
Default, fully replaced by run args	Always runs; args are appended
docker run img sh -> sh	docker run img --flag -> entrypoint --flag
Use for default arguments	Use for the binary

Named volume	Bind mount
Docker-managed, portable	Arbitrary host path

Named volume	Bind mount
Handles ownership	Inherits host UID/GID - permission errors
Production state	Development source, config files

21.9 HTTP Status Decision Tree

```

4xx = the request      5xx = the server
|                      |
401 -> token missing/expired    500 -> the APPLICATION threw
403 -> IAM / bucket policy / file    -> application logs, stack trace
      permission / WAF rule          -> did we just deploy? roll back
404 -> wrong path, ingress/ALB rule
      not matching, missing file    502 -> proxy got a bad/closed response
413 -> body size limit              -> backend crashed/OOM, wrong port,
429 -> rate limited                  http vs https mismatch

                                   503 -> NO healthy backend
                                   -> kubectl get endpoints (empty?)
                                   -> target group health
                                   -> readiness probe

                                   504 -> backend too slow
                                   -> slow query, pool exhausted,
                                       downstream hanging, timeout too short

```

From the code to the first place you look

21.10 Troubleshooting Flowcharts

```

Does the name resolve? --NO--> DNS: dig, resolv.conf, record, TTL, CoreDNS
| YES
Can I reach the port?
|-- REFUSED --> reachable; service not listening
|               -> ss -tulpn (check the BIND ADDRESS: 127.0.0.1 vs 0.0.0.0)
|               -> systemctl status / docker ps / kubectl get pods
|-- TIMEOUT --> packets dropped
|               -> security group -> NACL (stateless! ephemeral returns)
|               -> route table -> host firewall -> is the host alive
| OPEN
Does TLS complete? --NO--> openssl s_client: expiry, chain, SAN, protocol
| YES
What does HTTP return? --> apply the 4xx/5xx tree above

```

"I cannot connect"

```

Where is the time? curl -w namelookup / connect / starttransfer / total
|
DNS slow -----> resolver, ndots search path, CoreDNS capacity
Connect slow ----> network path, load balancer, TLS handshake
Server slow -----> compare LB latency vs app latency vs DB latency
|
+--> app saturated? CPU throttling (cfs_throttled), memory, thread/conn pool
+--> DB slow?       slow query log, missing index, locks, connections at max
+--> disk?          iostat await, EBS BurstBalance, high wa in vmstat
+--> proportional to load? saturation. constant? handshake/retry/DNS.

```

"It is slow"

```

Did the pipeline actually succeed? --NO--> read the FIRST differing log line
| YES
Are new pods/instances healthy? --NO--> Pending / ImagePull / CrashLoop / not Ready
| YES                                -> describe + logs --previous
Is the new version actually serving?
|-- NO --> check the image DIGEST, pod AGE, endpoints, CDN/browser cache
|         usual cause: mutable tag + imagePullPolicy IfNotPresent
|-- YES but broken --> roll back FIRST, then diff config between environments

```

"The deploy did not work"

21.11 Definitions You Must Be Able to State in One Sentence

Term	One-sentence definition
DevOps	Removing the manual handoff between writing code and running it, through automation, shared ownership and measurement
CI	Every change is merged and automatically verified continuously
CD	Every verified build is always releasable (delivery) or automatically released (deployment)
IaC	Infrastructure defined in versioned code that a tool applies, giving reproducibility and review
Idempotency	Running it again produces the same result and no further changes
Immutable infrastructure	Servers are replaced rather than modified in place
Container	An isolated process on the host kernel, using namespaces and cgroups
Orchestration	Scheduling, healing, scaling and networking containers across many machines
Terraform state	The map from configuration addresses to real resource IDs
Drift	Reality no longer matching the code
Blue-green	Two environments, one live; cutover is a router change
Canary	A small slice of real traffic on the new version, watched before proceeding
Rolling update	Replacing instances in waves, gated on readiness
Observability	Being able to ask new questions about a system without shipping new code
SLI / SLO / SLA	The measurement / the internal target / the contractual promise
Error budget	100% minus the SLO - the failure you are allowed
Toil	Manual, repetitive, automatable operational work
RCA	The systemic cause, plus action items with owners - never "human error"
MTTR	Mean time to restore service
Blast radius	How much is affected when this fails

21.12 The Twelve Sentences That Win Interviews

SAY THESE, UNPROMPTED, WHEN THE TOPIC ARISES

1. "Build once, deploy many - the artifact tested in staging is the exact artifact deployed to production."
2. "Refused means the path works and nothing is listening; timeout means packets are being dropped. Different investigations."
3. "Terraform provisions, Ansible configures."
4. "Readiness controls traffic, liveness controls restarts - and liveness must never check a dependency."
5. "Mitigate first, diagnose second - and preserve the evidence before restarting anything."
6. "Requests schedule, limits enforce. Memory over limit is an OOM kill; CPU over limit is throttling."
7. "State contains every attribute in plaintext, so it is encrypted, locked and never in Git."
8. "Kubernetes Secrets are base64-encoded, which is encoding, not encryption."
9. "latest is mutable, so you cannot tell what is running and you cannot roll back precisely."
10. "Rollback covers code, not schema - migrations have to be expand-and-contract."
11. "An alert that is not actionable is a dashboard, not an alert."
12. "I support and troubleshoot Kubernetes workloads; I have not administered a cluster in production."

PART 22

FLASHCARDS

310 cards for active recall

Cover the answer, say yours out loud, then check. Do 30 a day and cycle. The memory trick line is there to give you a hook - a phrase you can recall under pressure when the full answer will not come.

22.1 DevOps Fundamentals (1-30)

Q. What is DevOps? A. Removing the manual handoff between writing code and running it, through automation, shared ownership and measurement. <i>Trick: No wall to throw things over.</i>	Q. The four pillars of DevOps? A. Culture, Automation, Measurement, Sharing. <i>Trick: CAMS.</i>
Q. What are the four DORA metrics? A. Deployment frequency, lead time for changes, change failure rate, MTTR. <i>Trick: How often, how fast, how often broken, how fast fixed.</i>	Q. CI vs CD? A. CI verifies every change; Continuous Delivery keeps every build releasable; Continuous Deployment ships automatically. <i>Trick: Verify - ready - shipped.</i>
Q. What is "build once, deploy many"? A. The same artifact moves through every environment; you never rebuild per environment. <i>Trick: Test what you ship, ship what you tested.</i>	Q. What is Infrastructure as Code? A. Infrastructure defined in versioned code that a tool applies. <i>Trick: Servers in Git.</i>
Q. Declarative vs imperative? A. Declarative describes the end state; imperative describes the steps. <i>Trick: What vs how.</i>	Q. What is idempotency? A. Running it again produces the same result and no further changes. <i>Trick: Safe to re-run.</i>
Q. What is configuration management? A. Getting existing systems into a desired state - packages, files, services, users. <i>Trick: Furnishing the house.</i>	Q. What is containerization? A. Packaging an app with its dependencies so it runs identically anywhere. <i>Trick: Ship the environment with the code.</i>
Q. What is orchestration? A. Scheduling, healing, scaling and networking containers across many machines. <i>Trick: One container is easy; a thousand needs a scheduler.</i>	Q. Monitoring vs observability? A. Monitoring answers questions you knew to ask; observability lets you ask new ones. <i>Trick: Known unknowns vs unknown unknowns.</i>
Q. The three pillars of observability? A. Metrics, logs, traces. <i>Trick: How much, what happened, where in the path.</i>	Q. The four golden signals? A. Latency, traffic, errors, saturation. <i>Trick: LTES.</i>
Q. The USE method? A. Utilisation, Saturation, Errors - for each resource. <i>Trick: USE for resources, golden signals for services.</i>	Q. What makes a good alert? A. Symptom-based, actionable, with a for: duration and a runbook link. <i>Trick: If nobody acts, it is a dashboard.</i>
Q. What is an RCA? A. The systemic cause of an incident plus action items with owners and dates. <i>Trick: Never "human error".</i>	Q. What is the 5 Whys? A. Asking why repeatedly until you reach a systemic cause, not a symptom. <i>Trick: Stop when you find something you can fix.</i>
Q. Rolling deployment? A. Replace instances in waves, gated on readiness. <i>Trick: Kubernetes default.</i>	Q. Blue-green deployment? A. Two full environments, one live; cutover is a router change. <i>Trick: Flip the switch, flip it back.</i>
Q. Canary deployment? A. A small slice of real traffic on the new version, watched before proceeding. <i>Trick: Smallest blast radius.</i>	Q. What is immutable infrastructure? A. Servers are replaced, never modified in place. <i>Trick: Cattle, not pets.</i>

Q. Why is rollback hard for databases? A. Schema and data changes are often irreversible. <i>Trick:</i> Expand and contract.	Q. What is expand-and-contract? A. Add the new schema, ship code that works with both, drop the old one a release later. <i>Trick:</i> Two versions must coexist.
Q. What is SLI? A. The measurement of service quality, e.g. proportion of requests under 300ms. <i>Trick:</i> I = Indicator = the number.	Q. What is SLO? A. The internal target for an SLI, e.g. 99.9% over 30 days. <i>Trick:</i> O = Objective = the goal.
Q. What is SLA? A. The contractual promise to a customer, with penalties. <i>Trick:</i> A = Agreement = the contract.	Q. What is an error budget? A. 100% minus the SLO - the amount of failure you are allowed. <i>Trick:</i> 99.9% = about 43 minutes a month.
Q. What is toil? A. Manual, repetitive, automatable operational work with no lasting value. <i>Trick:</i> If a machine could do it, it is toil.	Q. High availability vs fault tolerance? A. HA keeps serving despite failure; fault tolerance keeps working correctly during failure. <i>Trick:</i> Redundancy vs graceful handling.

22.2 Linux (31-70)

Q. Which permission lets you delete a file? A. Write on the DIRECTORY, not on the file. <i>Trick:</i> The directory owns the name.	Q. chmod 755 means? A. Owner rwx, group r-x, others r-x. <i>Trick:</i> 4+2+1 / 4+1 / 4+1.
Q. What is the sticky bit? A. On a directory, only the file's owner can delete files inside. /tmp is 1777. <i>Trick:</i> Sticky stops you deleting mine.	Q. What is SUID? A. The executable runs as the file's OWNER, not the caller. chmod 4755. <i>Trick:</i> passwd needs root to edit /etc/shadow.
Q. What is SGID on a directory? A. New files inherit the directory's group. chmod 2775. <i>Trick:</i> Shared team folders.	Q. usermod -G vs -aG? A. -G REPLACES all secondary groups; -aG appends. <i>Trick:</i> Always 'a' for add.
Q. SIGTERM vs SIGKILL? A. TERM (15) can be caught and cleaned up; KILL (9) cannot be caught. <i>Trick:</i> Ask politely first.	Q. What does SIGHUP do to most daemons? A. Reload configuration without restarting. <i>Trick:</i> HUP = re-read.
Q. Process state D means? A. Uninterruptible sleep, usually blocked on I/O - cannot be killed, even with -9. <i>Trick:</i> D = Disk.	Q. What is a zombie process? A. Exited but the parent has not reaped its exit status; fix the parent. <i>Trick:</i> Dead but still in the table.
Q. df shows free space but writes fail. Why? A. Inode exhaustion - check df -i. <i>Trick:</i> Millions of tiny files.	Q. Deleted a big file and space did not return. Why? A. A process holds the file descriptor - lsof grep deleted, then restart it. <i>Trick:</i> The kernel keeps blocks until the handle closes.
Q. free vs available memory? A. Free is unused; available includes reclaimable cache - available is the real number. <i>Trick:</i> Linux uses spare RAM as cache on purpose.	Q. What is the OOM killer? A. The kernel kills the highest-scoring process when memory cannot be reclaimed. <i>Trick:</i> dmesg grep -i oom.
Q. Which command shows listening ports and the owning process? A. sudo ss -tulpn <i>Trick:</i> Read the BIND ADDRESS, not just the port.	Q. 127.0.0.1:8080 vs 0.0.0.0:8080? A. Loopback-only vs all interfaces - loopback is unreachable remotely. <i>Trick:</i> The number one "works locally" bug.
Q. tail -f vs tail -F? A. -f follows the descriptor and dies on rotation; -F follows by name. <i>Trick:</i> Capital F for production.	Q. How do you append to a root-owned file as a normal user? A. echo "x" sudo tee -a /etc/file <i>Trick:</i> sudo does not apply to your shell's redirect.
Q. What does 2>&1 do? A. Sends stderr wherever stdout currently points. Order matters. <i>Trick:</i> Redirect stdout first, then merge.	Q. systemctl reload vs restart? A. Reload re-reads config without dropping connections; restart stops and starts. <i>Trick:</i> Reload for config-only changes.

Q. You edited a systemd unit. What must you run? A. systemctl daemon-reload <i>Trick:</i> systemd caches unit files.	Q. Which command shows failed services? A. systemctl list-units --failed <i>Trick:</i> First command after an unexpected reboot.
Q. How do you see logs from the previous boot? A. journalctl -b -1 <i>Trick:</i> -1 = one boot back.	Q. What does grep -C 5 do? A. 5 lines of Context either side of each match. <i>Trick:</i> -A after, -B before, -C context.
Q. Top 10 IPs in an access log? A. awk '{print \$1}' log sort uniq -c sort -rn head -10 <i>Trick:</i> cut, sort, count, sort again.	Q. Why must you sort before uniq? A. uniq only removes ADJACENT duplicates. <i>Trick:</i> uniq is blind beyond one line.
Q. sed -i without a backup is risky. Safer form? A. sed -i.bak 's/a/b/g' file <i>Trick:</i> Always leave yourself an undo.	Q. How do you handle filenames with spaces in find? A. -print0 with xargs -0, or read -r -d ''. <i>Trick:</i> Null is the only safe separator.
Q. What does set -euo pipefail do? A. Exit on error, error on unset variable, fail on any pipeline stage. <i>Trick:</i> The safety header.	Q. Why do cron jobs fail when the script works by hand? A. Minimal PATH and no shell profile - use absolute paths and redirect output. <i>Trick:</i> Cron has amnesia.
Q. What is /proc? A. A virtual filesystem exposing kernel and process state as files. <i>Trick:</i> Nothing there is on disk.	Q. How do you find files changed in the last hour? A. find /etc -mmin -60 -type f <i>Trick:</i> "What changed?" in one command.
Q. Which command shows a process's system calls? A. strace -p PID <i>Trick:</i> For "permission denied" with no useful log.	Q. What is a hard link vs a symlink? A. Hard link = another name for the same inode; symlink = a file containing a path. <i>Trick:</i> Symlinks break, hard links do not.
Q. What is the deployment symlink pattern? A. Release to a dated directory, point current at it - rollback is repointing. <i>Trick:</i> Atomic swap.	Q. su vs sudo? A. su becomes another user with their password; sudo runs one command with yours, and is logged. <i>Trick:</i> sudo = least privilege + audit.
Q. apt update vs apt upgrade? A. update refreshes the package index; upgrade installs newer versions. <i>Trick:</i> Refresh then install.	Q. Which package owns a file? A. dpkg -S /path on Debian, rpm -qf /path on RHEL. <i>Trick:</i> S for search, f for file.
Q. What does ulimit -n control? A. The maximum number of open file descriptors. <i>Trick:</i> "Too many open files".	Q. What is load average measuring on Linux? A. Runnable PLUS uninterruptible (I/O-blocked) processes - compare to core count. <i>Trick:</i> High load with low CPU means disk.

22.3 Networking (71-105)

Q. Connection refused means? A. Host reachable, nothing listening on that port. <i>Trick:</i> Someone answered the door and said no.	Q. Connection timed out means? A. Packets dropped - firewall, security group, NACL, route, or host down. <i>Trick:</i> Nobody came to the door.
Q. The TCP three-way handshake? A. SYN, SYN-ACK, ACK. <i>Trick:</i> Hello, hello back, confirmed.	Q. TCP vs UDP? A. TCP is connection-oriented, reliable, ordered; UDP is fire-and-forget. <i>Trick:</i> Registered post vs a postcard.
Q. Many sockets in CLOSE_WAIT means? A. The application is not calling close() - an app bug leaking descriptors. <i>Trick:</i> Your side forgot to hang up.	Q. What does a /24 give you in AWS? A. 256 addresses, 251 usable (AWS reserves 5). <i>Trick:</i> Always subtract five.
Q. The RFC1918 private ranges? A. 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16. <i>Trick:</i> Docker's bridge sits in 172.17.	Q. What is 169.254.169.254? A. The instance metadata endpoint - identity and temporary IAM credentials. <i>Trick:</i> Link-local, why IMDSv2 exists.

Q. DNS resolution order? A. /etc/hosts and cache, stub resolver, recursive resolver, root, TLD, authoritative. <i>Trick:</i> Local, then walk down the tree.	Q. A record vs CNAME? A. A maps a name to an IP; CNAME maps a name to another NAME. <i>Trick:</i> CNAME cannot live at the apex.
Q. Why does Route 53 have ALIAS records? A. To point the zone apex at AWS resources, which a CNAME cannot do. <i>Trick:</i> Apex needs ALIAS.	Q. What is DNS TTL and when do you change it? A. Cache lifetime - lower it 24-48 hours BEFORE a planned cutover. <i>Trick:</i> You cannot un-cache the past.
Q. HTTP 401 vs 403? A. 401 = not authenticated; 403 = authenticated but not allowed. <i>Trick:</i> Who are you vs you may not.	Q. HTTP 500 means? A. The application threw an unhandled error - go to application logs. <i>Trick:</i> The app broke.
Q. HTTP 502 means? A. The proxy got an invalid or closed response from the backend. <i>Trick:</i> Bad answer from behind.	Q. HTTP 503 means? A. No healthy backend available. <i>Trick:</i> Nobody is home.
Q. HTTP 504 means? A. Backend reached but too slow - the proxy timed out. <i>Trick:</i> Too slow to answer.	Q. Forward proxy vs reverse proxy? A. Forward sits in front of clients; reverse sits in front of servers. <i>Trick:</i> Egress vs ingress.
Q. ALB vs NLB? A. ALB is layer 7 with host/path routing; NLB is layer 4 with static IPs and source IP preservation. <i>Trick:</i> NLB for non-HTTP like RTSP.	Q. Security group vs NACL? A. SG is stateful, instance-level, allow-only; NACL is stateless, subnet-level, allow and deny. <i>Trick:</i> Stateless needs the return rule.
Q. What is a NAT gateway for? A. Outbound internet for private subnets, with no inbound access. <i>Trick:</i> One-way door.	Q. What makes a subnet public? A. Its route table sends 0.0.0.0/0 to an Internet Gateway. <i>Trick:</i> It is the route, not a checkbox.
Q. What is X-Forwarded-For? A. The original client IP, added by the proxy, since the backend only sees the proxy. <i>Trick:</i> Only trust it from a known proxy.	Q. What is SNI? A. The hostname sent in plaintext in the ClientHello so the server picks the right certificate. <i>Trick:</i> openssl needs -servername.
Q. Works in a browser, fails in curl - TLS. Why? A. Missing intermediate certificate; browsers often fetch or cache it. <i>Trick:</i> Serve the full chain.	Q. How do you check certificate expiry from the shell? A. echo openssl s_client -connect h:443 -servername h openssl x509 -noout -dates <i>Trick:</i> Pipe s_client into x509.
Q. Ping fails but the service works. Why? A. ICMP is often blocked while the TCP port is open. <i>Trick:</i> Ping proves nothing.	Q. What does traceroute actually do? A. Sends packets with increasing TTL; each router returns time-exceeded. <i>Trick:</i> Stars are often just silent routers.
Q. What is MTU and when does it matter? A. Largest frame without fragmentation, usually 1500 - matters on VPNs and tunnels. <i>Trick:</i> Small requests work, large ones hang.	Q. What is the ephemeral port range? A. Source ports for outbound connections, typically 32768-60999 on Linux. <i>Trick:</i> NACLs must allow the returns.
Q. Testing a port with no tools installed? A. timeout 3 bash -c "</dev/tcp/host/443" <i>Trick:</i> Bash has a built-in socket.	Q. Why is VPC peering not transitive? A. Routes are point-to-point - A-B and B-C does not give A-C. <i>Trick:</i> Use Transit Gateway for a hub.
Q. What is a bastion host, and what replaces it? A. A hardened SSH entry point; SSM Session Manager replaces it with no open port. <i>Trick:</i> No port, no key, full audit.	Q. HTTP/1.1 vs HTTP/2? A. HTTP/2 multiplexes many streams over one connection and compresses headers. <i>Trick:</i> Fewer connections per client.
Q. What is a CDN for? A. Caching content at the edge - latency, spike absorption, origin offload, DDoS buffer. <i>Trick:</i> The hard part is invalidation.	

22.4 Git (106-130)

Q. The four areas of Git? A. Working tree, staging area (index), local repository, remote repository. <i>Trick:</i> Edit, stage, commit, push.	Q. fetch vs pull? A. Fetch downloads only; pull is fetch plus merge and changes your working tree. <i>Trick:</i> Fetch is always safe.
Q. merge vs rebase? A. Merge preserves topology and adds a merge commit; rebase replays commits linearly with new hashes. <i>Trick:</i> Merge in, rebase yours.	Q. The golden rule of rebase? A. Never rebase a branch other people have pulled. <i>Trick:</i> Do not rewrite shared history.
Q. reset vs revert? A. Reset rewrites history (local only); revert adds a new undo commit (safe on shared branches). <i>Trick:</i> Revert what is pushed.	Q. The three reset modes? A. --soft keeps changes staged, --mixed unstages them, --hard destroys them. <i>Trick:</i> Soft, mixed, hard = increasing destruction.
Q. How do you recover a commit lost by reset --hard? A. git reflog, find the SHA, reset or branch to it. <i>Trick:</i> Reflog remembers for ~90 days.	Q. What is git cherry-pick for? A. Applying one commit from another branch - typically a hotfix onto a release branch. <i>Trick:</i> Pick one cherry.
Q. What is git stash? A. Parking uncommitted work so you can switch context; pop to restore. <i>Trick:</i> -u includes untracked.	Q. What is git bisect? A. Binary search through history to find the commit that introduced a bug. <i>Trick:</i> Eight steps for 200 commits.
Q. --force vs --force-with-lease? A. With-lease refuses if the remote has commits you have not fetched. <i>Trick:</i> Never bare force.	Q. Lightweight vs annotated tag? A. Annotated has an author, date and message; use it for releases. <i>Trick:</i> git describe needs annotated.
Q. What is a detached HEAD? A. HEAD points at a commit, not a branch - commits there have no name. <i>Trick:</i> git switch -c to rescue them.	Q. How do you resolve a merge conflict? A. Edit the file, remove all three markers, git add to mark resolved, commit, then run the tests. <i>Trick:</i> HEAD is yours, below ===== is theirs.
Q. What does .gitignore NOT do? A. It does not untrack files already tracked - use git rm --cached. <i>Trick:</i> Ignore is for new files.	Q. A secret was pushed. What is the first action? A. Rotate it - history rewriting is secondary, everyone with a clone already has it. <i>Trick:</i> Assume compromised.
Q. What is a shallow clone and why use it in CI? A. --depth 1 fetches only the latest commit - much faster checkouts. <i>Trick:</i> No history means no bisect or describe.	Q. What is branch protection? A. PR required, approvals, passing checks, up-to-date branch, no force-push, applied to admins. <i>Trick:</i> Nothing reaches main unreviewed.
Q. What is a git hook example for DevOps? A. pre-commit for lint and secret scanning; commit-msg for a ticket convention. <i>Trick:</i> Hooks are not versioned - use the pre-commit framework.	Q. git diff vs --staged? A. diff = tree vs index (unstaged); --staged = index vs last commit (about to commit). <i>Trick:</i> What you have not staged vs what you will commit.
Q. How does Git store data? A. As snapshots of trees, content-addressed by SHA - not as diffs. <i>Trick:</i> Unchanged files reuse the same blob.	Q. What is a fast-forward merge? A. Moving the pointer forward when there are no divergent commits - no merge commit. <i>Trick:</i> --no-ff forces one anyway.
Q. How do you see who changed a line? A. git blame file, then git show SHA. <i>Trick:</i> git log -S finds when a string appeared.	Q. Your push is rejected as non-fast-forward. What now? A. Someone pushed since your fetch - pull --rebase, resolve, test, push. Never force. <i>Trick:</i> Behind, not broken.
Q. What is a monorepo's CI challenge? A. You must build only what changed - path-aware pipelines. <i>Trick:</i> Otherwise every commit rebuilds everything.	

22.5 Jenkins and CI/CD (131-160)

Q. Jenkins controller vs agent? A. The controller schedules and stores config and credentials; agents execute builds. <i>Trick:</i> Zero executors on the controller.	Q. Why should builds not run on the controller? A. Build scripts would have access to credentials and could take down the instance. <i>Trick:</i> Never build where the secrets live.
Q. What is an executor? A. One concurrent build slot on an agent. <i>Trick:</i> Roughly one per core.	Q. What is \$JENKINS_HOME and what must you back up? A. All Jenkins state - config.xml, jobs/, credentials.xml AND secrets/, plugins/. <i>Trick:</i> credentials.xml is useless without secrets/.
Q. Freestyle vs pipeline? A. Freestyle config lives only in the UI; a Jenkinsfile is versioned and reviewed. <i>Trick:</i> Build definition as code.	Q. Declarative vs scripted pipeline? A. Declarative is structured and validated up front; scripted is full Groovy. <i>Trick:</i> Declarative by default, script{} when you must.
Q. What does 'H' mean in a Jenkins cron? A. Hashed - spreads jobs across the window instead of all firing at once. <i>Trick:</i> H prevents the thundering herd.	Q. Webhook vs poll SCM? A. Webhook is pushed instantly; polling asks repeatedly and adds latency. <i>Trick:</i> Push beats pull.
Q. What is a multibranch pipeline? A. A job per branch with a Jenkinsfile, including PR builds of the merge result. <i>Trick:</i> Test what main will become.	Q. How do you handle credentials in Jenkins? A. Credential store, folder-scoped, bound with withCredentials, single-quoted in sh. <i>Trick:</i> Single quotes keep secrets out of Groovy strings.
Q. FAILURE vs UNSTABLE? A. FAILURE means a step errored; UNSTABLE usually means tests failed via junit. <i>Trick:</i> Broken vs disappointing.	Q. What does disableConcurrentBuilds() protect against? A. Two simultaneous deployments to the same environment. <i>Trick:</i> Concurrency plus deploys equals outage.
Q. Why wrap input in a timeout? A. It occupies an executor while waiting. <i>Trick:</i> An overnight approval blocks a slot.	Q. What is a Jenkins shared library? A. A versioned repo of reusable pipeline code, pinned to a tag. <i>Trick:</i> Pin it, or one commit changes every pipeline.
Q. How do you version Docker images from CI? A. Build number plus short commit SHA - immutable, traceable. <i>Trick:</i> Never deploy latest.	Q. How do you roll back from a pipeline? A. Deploy the previous immutable tag, or kubectl rollout undo. <i>Trick:</i> Rollback needs immutable artifacts.
Q. buildDiscarder does what? A. Limits retained builds so \$JENKINS_HOME does not fill. <i>Trick:</i> Disk is the top Jenkins failure.	Q. Why does cleanWs() matter? A. Prevents stale workspace files affecting the next build and reclaims disk. <i>Trick:</i> Fresh workspace, honest build.
Q. Agent shows offline. First thing to check? A. Disk space on the agent - Jenkins takes nodes offline below a threshold. <i>Trick:</i> Then Java version, then network.	Q. Pipeline worked yesterday, fails today, no code change. Why? A. Something external moved - plugin, agent, base image tag, credential, disk. <i>Trick:</i> Diff the console logs.
Q. What is a quality gate? A. A stage that fails the build on a measured threshold - coverage, vulnerabilities. <i>Trick:</i> Automated, not a report nobody reads.	Q. How do you speed up a slow pipeline? A. Measure first, then parallelise, cache dependencies, shallow clone, path filters. <i>Trick:</i> Optimise the slowest stage, not all of them.
Q. Why apply a saved Terraform plan in CI? A. So what runs is exactly what a human reviewed. <i>Trick:</i> plan -out, apply tfplan.	Q. What is Jenkins Configuration as Code? A. Defining the controller's configuration in YAML so the instance is reproducible. <i>Trick:</i> Terraform for Jenkins.
Q. Jenkins vs GitHub Actions? A. Actions is repo-integrated and needs no infrastructure; Jenkins orchestrates across systems and private networks. <i>Trick:</i> Actions for CI, Jenkins for orchestration.	Q. What does checkout scm do? A. Checks out the exact commit and repo that triggered this pipeline. <i>Trick:</i> Never hardcode the branch.
Q. How do you capture command output in a Jenkinsfile? A. sh(script: '...', returnStdout: true).trim() <i>Trick:</i> returnStatus for the exit code.	Q. What should a post{} block always do? A. Publish results, archive artifacts, notify, clean the workspace - even on failure. <i>Trick:</i> always{} runs regardless.

Q. What is a Docker agent in Jenkins? A. Each build runs in a declared container image - no toolchain drift. <i>Trick:</i> Environment as code.	Q. How do you test a Jenkinsfile change safely? A. On a feature branch with a multibranch job, plus Replay to iterate. <i>Trick:</i> Branch first, never edit main's pipeline live.
---	---

22.6 Docker (161-190)

Q. Image vs container? A. Image is the immutable template; container is a running instance with a writable layer. <i>Trick:</i> Class vs object.	Q. Container vs VM? A. Container shares the host kernel; a VM runs its own. <i>Trick:</i> Seconds vs minutes, MB vs GB.
Q. What creates a Docker layer? A. Each filesystem-changing instruction in the Dockerfile. <i>Trick:</i> Cached by content.	Q. Why copy dependency manifests before source? A. So the dependency install layer stays cached across code changes. <i>Trick:</i> The number one Dockerfile question.
Q. CMD vs ENTRYPOINT? A. CMD is a default that run args replace; ENTRYPOINT always runs and args are appended. <i>Trick:</i> ENTRYPOINT the binary, CMD the args.	Q. Exec form vs shell form? A. Exec makes the app PID 1 and it receives SIGTERM; shell form wraps it in sh which does not forward signals. <i>Trick:</i> Shell form = 10-second stops.
Q. COPY vs ADD? A. ADD also auto-extracts tars and fetches URLs - prefer COPY. <i>Trick:</i> Magic causes surprises.	Q. What does EXPOSE do? A. Documents the port. It does NOT publish it. <i>Trick:</i> -p publishes.
Q. What is a multi-stage build? A. Build in one stage, copy only artifacts into a clean final stage. <i>Trick:</i> Compilers never reach production.	Q. What is localhost inside a container? A. The container itself - not the host, not another container. <i>Trick:</i> The top containerisation mistake.
Q. Default bridge vs user-defined bridge? A. User-defined gives DNS resolution by container name; the default does not. <i>Trick:</i> Always docker network create.	Q. Named volume vs bind mount? A. Volume is Docker-managed and portable; bind mount maps a host path and inherits its UID/GID. <i>Trick:</i> Bind mounts cause permission errors.
Q. Exit code 137 means? A. SIGKILL - almost always OOMKilled. <i>Trick:</i> 128 + 9.	Q. Exit code 127 means? A. Command not found in the image. <i>Trick:</i> Missing in a slim base.
Q. Container exits immediately with code 0. Why? A. The main process finished - often a daemonising app that needs foreground mode. <i>Trick:</i> nginx -g "daemon off;".	Q. Container runs but is unreachable. Three causes? A. App bound to 127.0.0.1; no -p mapping; wrong port in the mapping. <i>Trick:</i> docker exec ... ss -tulpn.
Q. How do you debug an image that will not start? A. docker run --entrypoint sh -it image <i>Trick:</i> Get a shell without running the app.	Q. What does docker diff show? A. Files added, changed or deleted in the writable layer since the image. <i>Trick:</i> "What has it written?"
Q. Why not deploy :latest? A. It is mutable - you cannot tell what is running or roll back precisely. <i>Trick:</i> Immutable tags per build.	Q. Docker host disk full - first command? A. docker system df <i>Trick:</i> Then prune with a time filter.
Q. What does docker system prune -a --volumes delete? A. All unused images AND unused volumes - potential data loss. <i>Trick:</i> Know before you run it.	Q. Why run containers as non-root? A. A container escape or bind-mount write otherwise has root on the host. <i>Trick:</i> USER costs nothing.
Q. What is a distroless image? A. Only the app and runtime - no shell or package manager. <i>Trick:</i> Smaller attack surface, harder debugging.	Q. What is HEALTHCHECK for? A. Docker marks the container healthy/unhealthy; Compose can gate on it. <i>Trick:</i> Kubernetes ignores it - it has probes.
Q. Why does depends_on not guarantee readiness? A. It only waits for the container to START, not for the app to be ready. <i>Trick:</i> condition: service_healthy.	Q. What is the build context? A. The directory sent to the daemon - nothing outside it can be COPYed. <i>Trick:</i> .dockerignore keeps it small.

Q. Where do container logs go? A. Whatever PID 1 writes to stdout/stderr, captured by the log driver. <i>Trick:</i> Set max-size or the disk fills.	Q. How do you limit container resources? A. --memory and --cpus, or limits in Compose/Kubernetes. <i>Trick:</i> No limit = noisy neighbour.
Q. A JVM keeps getting OOMKilled in a container. Why? A. The heap is sized from host RAM, not the container limit. <i>Trick:</i> -XX:MaxRAMPercentage=75.	Q. Docker Compose's main limitation? A. Single host - no cross-machine scheduling or self-healing. <i>Trick:</i> That is where Kubernetes starts.

22.7 Kubernetes (191-235)

Q. What does the API server do? A. Front door - REST API, auth, admission - and the only component that writes to etcd. <i>Trick:</i> Everything goes through it.	Q. What is etcd? A. The consistent key-value store holding ALL cluster state. <i>Trick:</i> Back this up or lose the cluster.
Q. What does the scheduler do? A. Assigns pending pods to nodes based on requests, taints and affinity. <i>Trick:</i> Placement only.	Q. What does the controller manager do? A. Runs reconciliation loops closing the gap between desired and actual state. <i>Trick:</i> Self-healing lives here.
Q. What does the kubelet do? A. Node agent - starts containers, runs probes, reports status. <i>Trick:</i> One per node.	Q. What does kube-proxy do? A. Programmes iptables/IPVS so Service virtual IPs route to pod IPs. <i>Trick:</i> Service networking.
Q. What is a pod? A. One or more containers sharing a network namespace and volumes, scheduled together. <i>Trick:</i> Sidecars share localhost.	Q. Deployment, ReplicaSet, Pod relationship? A. Deployment manages ReplicaSets, which manage Pods. <i>Trick:</i> Old ReplicaSets enable instant rollback.
Q. Deployment vs StatefulSet? A. Deployment pods are interchangeable; StatefulSet gives stable identity, DNS and per-pod storage. <i>Trick:</i> Databases need identity.	Q. What is a DaemonSet for? A. One pod per node - log shippers, node_exporter, CNI. <i>Trick:</i> Node agents.
Q. Job vs CronJob? A. Job runs to completion; CronJob creates Jobs on a schedule. <i>Trick:</i> concurrencyPolicy: Forbid for backups.	Q. How does a Service find its pods? A. By LABEL SELECTOR - ready matching pods become its Endpoints. <i>Trick:</i> Labels, not names.
Q. Service returns nothing. First command? A. kubectl get endpoints SVC <i>Trick:</i> Empty = selector mismatch or not Ready.	Q. The Service types? A. ClusterIP, NodePort, LoadBalancer, ExternalName, and headless. <i>Trick:</i> ClusterIP is the default.
Q. Service vs Ingress? A. Service is L4 with a virtual IP; Ingress is L7 HTTP routing by host and path. <i>Trick:</i> One LB fronting many services.	Q. In-cluster DNS name format? A. service.namespace.svc.cluster.local <i>Trick:</i> Short names work in the same namespace.
Q. requests vs limits? A. Requests are reserved for scheduling; limits are the enforced ceiling. <i>Trick:</i> Requests schedule, limits enforce.	Q. What happens over a CPU limit vs a memory limit? A. CPU is throttled; memory is an OOM kill. <i>Trick:</i> You cannot throttle memory.
Q. What are QoS classes? A. Guaranteed, Burstable, BestEffort - eviction order is the reverse. <i>Trick:</i> BestEffort dies first.	Q. Liveness vs readiness probe? A. Liveness restarts the container; readiness removes it from endpoints. <i>Trick:</i> Restarts vs traffic.
Q. Why must liveness not check dependencies? A. A database blip would restart every replica and turn a small problem into an outage. <i>Trick:</i> Readiness may, liveness must not.	Q. What is a startupProbe for? A. Delaying liveness until a slow-starting app has come up. <i>Trick:</i> Instead of a huge initialDelaySeconds.

Q. Debugging CrashLoopBackOff - first command? A. kubectl logs POD --previous <i>Trick:</i> The dead container has the error.	Q. What causes ImagePullBackOff? A. Wrong tag, missing imagePullSecrets, rate limit, no registry route, arch mismatch. <i>Trick:</i> describe names the exact error.
Q. Pod is Pending. What do you do? A. kubectl describe pod and read the scheduler's message. <i>Trick:</i> It literally tells you why.	Q. OOMKilled - how do you tell limit vs leak? A. Steady usage just over the limit = sizing; steadily climbing = leak. <i>Trick:</i> Look at the shape of the graph.
Q. maxSurge and maxUnavailable control what? A. How many extra and how many missing pods are allowed during a rolling update. <i>Trick:</i> 0 unavailable needs spare capacity.	Q. How do you roll back a Deployment? A. kubectl rollout undo deploy/NAME <i>Trick:</i> The old ReplicaSet is still there.
Q. Changing a ConfigMap does not update pods. Why? A. Env vars are injected at pod start - you need a rollout restart. <i>Trick:</i> Or annotate with a config checksum.	Q. Are Kubernetes Secrets encrypted? A. No - base64-encoded. Encoding is not encryption. <i>Trick:</i> Needs etcd encryption plus RBAC.
Q. What is a namespace, and what does it NOT isolate? A. A scope for names, RBAC and quotas - it does NOT isolate networking by default. <i>Trick:</i> Need NetworkPolicy for that.	Q. What is a NetworkPolicy? A. A pod-level firewall; selecting a pod makes that direction default-deny. <i>Trick:</i> Needs a CNI that enforces it.
Q. What is a ServiceAccount? A. The identity a pod uses against the API - and on EKS, mapped to an IAM role. <i>Trick:</i> IRSA = no stored keys.	Q. What does HPA need to work? A. metrics-server AND resource requests set. <i>Trick:</i> No request, no percentage.
Q. HPA vs VPA vs Cluster Autoscaler? A. Number of pods / size of pods / number of nodes. <i>Trick:</i> Three different layers.	Q. What does RWO access mode actually mean? A. One NODE at a time - not one pod. <i>Trick:</i> Multi-replica on one EBS volume fails.
Q. PVC stuck Pending. Common causes? A. No StorageClass, broken provisioner, zone mismatch, unsatisfiable size or access mode. <i>Trick:</i> WaitForFirstConsumer fixes zones.	Q. What is a reclaim policy? A. What happens to the volume when the PVC is deleted - Delete or Retain. <i>Trick:</i> Retain anything that matters.
Q. What is a PodDisruptionBudget? A. Minimum availability during VOLUNTARY disruption like a node drain. <i>Trick:</i> Does not protect against crashes.	Q. What are taints and tolerations? A. Taints repel pods from a node; tolerations allow scheduling there. <i>Trick:</i> Affinity attracts, taints repel.
Q. What is an init container? A. A container that runs to completion before the app containers start. <i>Trick:</i> Wait for a dependency, run a migration.	Q. What happens when a pod terminates? A. Removed from endpoints, preStop hook, SIGTERM, then SIGKILL after the grace period. <i>Trick:</i> preStop sleep covers endpoint propagation lag.
Q. Deployed but the old version is still serving. Usual cause? A. Mutable tag plus imagePullPolicy IfNotPresent, so the cached image was reused. <i>Trick:</i> Check the DIGEST, not the tag.	Q. What is CPU throttling and why is it sneaky? A. Exceeding the CPU limit slows the app with no error anywhere. <i>Trick:</i> container_cpu_cfs_throttled_seconds_total.
Q. What is GitOps? A. A controller in the cluster reconciles it to the desired state stored in Git. <i>Trick:</i> Deploy by merge, drift auto-corrected.	

22.8 Terraform and Ansible (236-265)

Q. What is Terraform state? A. The map from configuration addresses to real resource IDs, plus last-seen attributes. <i>Trick:</i> Without it, Terraform cannot tell create from update.	Q. Why must state be remote? A. Shared truth, encryption, access control and - critically - locking. <i>Trick:</i> Local state means collisions.
Q. What provides Terraform state locking on AWS? A. A DynamoDB table alongside the S3 backend. <i>Trick:</i> Two applies would corrupt state.	Q. Can state go in Git? A. No - it contains every attribute in plaintext, including secrets. <i>Trick:</i> sensitive = true hides output, not state.

Q. What is drift? A. Reality no longer matching state, usually from a console change. <i>Trick: Detect with a scheduled plan.</i>	Q. How do you reconcile drift without changing infrastructure? A. terraform apply -refresh-only <i>Trick: Accept reality into state.</i>
Q. count vs for_each? A. count is indexed and re-indexes on removal; for_each is keyed and stable. <i>Trick: for_each for anything with identity.</i>	Q. What does -/+ mean in a plan? A. Destroy and recreate - an immutable attribute changed. <i>Trick: The plan says "forces replacement".</i>
Q. Why apply a saved plan file? A. A bare apply re-evaluates and may differ from what was reviewed. <i>Trick: plan -out, apply tfplan.</i>	Q. What does terraform import do? A. Brings an existing resource under management - it does not write the config for you. <i>Trick: Import, plan, adjust until empty.</i>
Q. terraform state rm does what? A. Stops managing a resource; the real resource is untouched. <i>Trick: Orphan, not delete.</i>	Q. How do you rename a resource without destroying it? A. terraform state mv, or a moved block in code. <i>Trick: moved blocks are reviewable.</i>
Q. What is create_before_destroy for? A. Avoiding downtime and "resource in use" errors when replacing. <i>Trick: Use name_prefix to avoid collisions.</i>	Q. What does prevent_destroy do? A. Makes Terraform error rather than destroy - put it on databases. <i>Trick: A guard rail against a bad plan.</i>
Q. Why pin module and provider versions? A. An unpinned reference can change every environment on the next init. <i>Trick: .terraform.lock.hcl is committed.</i>	Q. Why split state by environment and component? A. Blast radius, speed and lock contention. <i>Trick: One giant state is a liability.</i>
Q. What is a data source? A. A read-only lookup of something Terraform does not manage. <i>Trick: Reference without owning.</i>	Q. Why avoid provisioners? A. Imperative, run only at create time, and a failure taints the resource. <i>Trick: Use user data or Ansible.</i>
Q. Terraform vs Ansible in one line? A. Terraform provisions infrastructure; Ansible configures systems. <i>Trick: Builds the house, furnishes it.</i>	Q. Is Ansible agent-based? A. No - it connects over SSH or WinRM, copies the module, runs it, removes it. <i>Trick: Nothing to install on targets.</i>
Q. What breaks Ansible idempotency? A. command and shell tasks with no creates/removes/when guard. <i>Trick: Always prefer a real module.</i>	Q. What is an Ansible handler? A. A task that runs once at the end, only if notified by a changed task. <i>Trick: Restart only when config actually changed.</i>
Q. What is the Ansible dry run? A. ansible-playbook --check --diff <i>Trick: The Ansible equivalent of plan.</i>	Q. How do you do a rolling update in Ansible? A. serial: N plus health checks, with max_fail_percentage: 0. <i>Trick: Batches, not the whole fleet.</i>
Q. What is delegate_to for? A. Running a task on a different host, usually localhost for a load balancer API call. <i>Trick: run_once for one-time actions.</i>	Q. Which Ansible variable source wins? A. --extra-vars, always. <i>Trick: defaults lowest, extra-vars highest.</i>
Q. What is Ansible Vault? A. Encryption of secrets at rest in the repository. <i>Trick: A secrets manager is better in production.</i>	Q. How do you reach Windows with Ansible? A. WinRM connection plus the win_* modules. <i>Trick: Control node is always Linux.</i>
Q. What does block/rescue/always give you? A. Structured error handling - deploy, roll back, log. <i>Trick: try/catch/finally.</i>	Q. How do you prove a playbook is idempotent? A. Run it twice - the second run must report zero changed. <i>Trick: Molecule automates this test.</i>

22.9 AWS and Monitoring (266-310)

Q. Region vs availability zone? A. A region contains multiple AZs - independent data centres with separate power and networking. <i>Trick: AZs for HA, regions for DR.</i>	Q. How many IPs does AWS reserve per subnet? A. Five. <i>Trick: A /24 gives 251 usable.</i>
--	---

Q. IGW vs NAT gateway? A. IGW is two-way for public IPs; NAT is outbound only for private subnets. <i>Trick:</i> Public vs one-way.	Q. Why use an IAM role instead of access keys? A. Temporary, auto-rotated credentials with nothing to leak or forget to rotate. <i>Trick:</i> Roles for anything running on AWS.
Q. IAM evaluation order? A. Explicit Deny wins; otherwise an explicit Allow is required; default is deny. <i>Trick:</i> Deny always wins.	Q. What is an SCP? A. An Organizations-level permissions ceiling - it can only restrict, never grant. <i>Trick:</i> Blocks even the root user.
Q. What is IMDSv2 for? A. Requiring a token before metadata access, defeating SSRF credential theft. <i>Trick:</i> 169.254.169.254 is a target.	Q. EBS vs instance store? A. EBS persists independently; instance store is lost on stop or terminate. <i>Trick:</i> Ephemeral means ephemeral.
Q. What is user data and its big caveat? A. A first-boot script - and it is readable from metadata, so no secrets. <i>Trick:</i> /var/log/cloud-init-output.log.	Q. ASG health check type - which and why? A. ELB, not EC2 - an instance can be running with a dead application. <i>Trick:</i> EC2 checks are hardware-level.
Q. What is a launch template vs a launch configuration? A. Templates support versioning and newer features; configurations are legacy. <i>Trick:</i> Versioning makes rollback easy.	Q. ALB vs NLB in one line? A. ALB routes HTTP by host and path; NLB is L4 with static IPs and source IP preservation. <i>Trick:</i> RTSP needs NLB.
Q. ELB 5XX vs Target 5XX? A. ELB = no healthy targets or connection failure; Target = the application errored. <i>Trick:</i> Different investigations entirely.	Q. What is deregistration delay? A. Draining time so in-flight requests complete before a target is removed. <i>Trick:</i> Too short cuts live connections.
Q. S3 durability vs availability? A. Durability is not losing data (11 nines); availability is being able to reach it. <i>Trick:</i> People conflate them.	Q. What is the S3 policy ARN trap? A. bucket vs bucket/* - ListBucket needs the first, object actions need the second. <i>Trick:</i> The most common S3 policy bug.
Q. What grows an S3 bill silently? A. Versioning with no noncurrent-version lifecycle rule, and incomplete multipart uploads. <i>Trick:</i> Add AbortIncompleteMultipartUpload.	Q. What is a pre-signed URL? A. A time-limited URL giving temporary access to a private object without credentials. <i>Trick:</i> Keeps large transfers off your servers.
Q. Why is there no EC2 memory metric by default? A. CloudWatch sees the hypervisor, which cannot see inside the guest OS. <i>Trick:</i> Install the CloudWatch agent.	Q. What does treat-missing-data breaching protect against? A. An alarm never firing because the source stopped publishing entirely. <i>Trick:</i> Silence is the worst state.
Q. What is a CloudWatch metric filter? A. Turns a log pattern into a metric you can alarm on. <i>Trick:</i> Alert on ERROR lines.	Q. What is the CloudWatch Logs cost trap? A. Log groups never expire unless you set retention. <i>Trick:</i> Set it in IaC.
Q. CloudWatch vs CloudTrail? A. CloudWatch is operational telemetry; CloudTrail is an audit of API calls. <i>Trick:</i> What is broken vs who changed it.	Q. Why use a VPC gateway endpoint for S3? A. Keeps traffic off the internet and off the NAT gateway - free and more secure. <i>Trick:</i> Removes NAT data charges.
Q. What is RDS Multi-AZ vs a read replica? A. Multi-AZ is a synchronous standby for availability; a replica is asynchronous for read scale. <i>Trick:</i> You cannot read from the standby.	Q. What is the shared responsibility model? A. AWS secures the cloud; you secure what you put in it. <i>Trick:</i> The line moves per service.
Q. Prometheus - pull or push? A. Pull - it scrapes /metrics on a schedule. <i>Trick:</i> The up metric is free health checking.	Q. Counter vs gauge? A. Counter only increases (use rate); gauge goes up and down (read directly). <i>Trick:</i> Raw counter graphs are useless.
Q. Why does cardinality matter? A. Each label combination is a separate stored series - IDs as labels kill the server. <i>Trick:</i> Never label with user_id.	Q. How do you get p95 from a histogram? A. histogram_quantile(0.95, sum by (le) (rate(..._bucket[5m]))) <i>Trick:</i> le = less-or-equal bucket.

Q. Error rate query? A. $100 * \text{sum}(\text{rate}(\text{reqs}\{\text{status}=\sim"5.."}[5m])) / \text{sum}(\text{rate}(\text{reqs}[5m]))$ <i>Trick:</i> Aggregate both sides the same way.	Q. What does for : do in an alert rule? A. Requires the condition to hold continuously before firing. <i>Trick:</i> Stops transient spikes paging you.
Q. What does Alertmanager add? A. Grouping, deduplication, silencing, inhibition and routing. <i>Trick:</i> Prometheus decides when, Alertmanager decides who.	Q. What is inhibition? A. A cause alert suppressing its symptom alerts. <i>Trick:</i> Node down should not also page about its pods.
Q. How do you alert on a job that stopped running? A. On the age of the last success, or absent(). <i>Trick:</i> Failure alerts miss "never ran".	Q. Why also use CloudWatch if you have Prometheus? A. You cannot instrument ALB, RDS or NAT - those metrics only exist in CloudWatch. <i>Trick:</i> Platform vs application.
Q. Prometheus vs Grafana? A. Prometheus collects, stores and alerts; Grafana visualises and stores nothing. <i>Trick:</i> Complementary, not alternatives.	Q. rate vs irate? A. rate averages over the window; irate uses the last two samples. <i>Trick:</i> rate for alerts, irate for fast graphs.
Q. What is predict_linear used for? A. Extrapolating a trend - alerting that a disk will fill in four hours. <i>Trick:</i> Hours of warning, not an outage.	Q. What is a recording rule? A. A precomputed query stored as a new series for fast dashboards and simple alerts. <i>Trick:</i> Cache the expensive query.
Q. What is Container Insights / kube-state-metrics for? A. Kubernetes OBJECT state - replicas, phases, restart counts. <i>Trick:</i> cAdvisor gives container resources.	Q. What is blackbox_exporter good for? A. Probing endpoints from outside - including certificate expiry. <i>Trick:</i> probe_ssl_earliest_cert_expiry.
Q. What is a Synthetics canary? A. A scripted external check of a real user flow. <i>Trick:</i> Everything internal can look fine while users cannot log in.	Q. Which CloudWatch metric is the best availability signal for an ALB? A. HealthyHostCount, with missing data treated as breaching. <i>Trick:</i> Zero healthy hosts equals 503.
Q. What is the one alert every service should have? A. up == 0 - the target is not responding at all. <i>Trick:</i> Free, and it catches total failure.	

HOW TO USE THESE CARDS

Three passes. First pass: read both sides, build familiarity. Second pass: cover the answer and say it out loud - speaking is what builds interview fluency, reading is not. Third pass: only the ones you missed. Then cycle the missed pile daily until it is empty.

PART 23

FIVE COMPLETE MOCK INTERVIEWS

With expected depth, follow-ups and scoring

Do these out loud, timed, recorded on your phone. Reading them silently builds recognition; speaking them builds fluency, and fluency is what the interviewer is actually assessing. Score yourself honestly against the criteria - a 3 is a pass, a 2 means go back to that part.

HOW TO RUN A MOCK

Set a timer. Answer each question out loud before reading the model answer. Record it. Then listen back and check three things: did you ramble past 90 seconds, did you say "um" while thinking instead of pausing, and did you answer the question that was asked. Most candidates fail on the third.

23.1 Mock Interview 1 - Junior / Intermediate DevOps (45 minutes)

Q. [Opening] Tell me about yourself.

Expected depth: 60-90 seconds, structured, ending with a hook. **Strong answer:** the 60-second introduction from Part 1.6.2. **Score 4:** structured, specific tools tied to responsibilities, no rambling, ends by inviting the next question. **Score 2:** a chronological list of jobs with no shape. **Score 1:** over three minutes, or life story.

Q. What is DevOps, in your own words?

Testing: whether you understand the problem, not the buzzword. **Strong:** "Removing the manual handoff between writing code and running it - pipelines instead of tickets, infrastructure as code instead of hand-built servers, and monitoring so both sides see the same truth." **Score 4:** problem statement first, then tools. **Score 1:** "DevOps is Jenkins, Docker and Kubernetes."

Q. Explain CI versus CD.

Strong: CI verifies every change continuously; Continuous Delivery keeps every verified build releasable with a human deciding when; Continuous Deployment removes that human. **Follow-up:** which do you do? **Expected:** CI plus continuous delivery, with an honest reason.

Q. What happens when you run `docker run nginx`?

Strong: CLI sends the request to the daemon; the daemon checks locally and pulls if needed; creates namespaces, cgroups and a writable layer; sets up bridge networking; starts the entrypoint as PID 1 and streams its output. **Score 4:** mentions namespaces and cgroups. **Score 2:** "it starts a container."

Q. What is the difference between an image and a container?

Strong: image = immutable layered template; container = running instance with a writable layer that dies with it, which is why data goes in volumes. **Score 4:** connects it to persistence.

Q. Write a Dockerfile for a Python web application.

Testing: layer ordering above all. **Score 4:** pinned base, requirements copied and installed BEFORE source, non-root USER, EXPOSE, HEALTHCHECK, exec-form ENTRYPOINT. **Score 2:** works but copies source first. **Score 1:** runs as root with `CMD python app.py` in shell form.

Q. A container exits immediately. How do you debug it?

Strong: `docker ps -a` for the exit code, `docker logs` for the error, then map the code - 0 means the process finished, 1 is an app error, 127 command not found, 137 OOM. Then `docker run --entrypoint sh -it image` to inspect without running the app.

Q. What is a Jenkinsfile and why use one?

Strong: the pipeline as code in the repository, so the build definition is versioned, reviewed and branch-specific, and survives rebuilding Jenkins.

Q. Walk me through your pipeline stages.

Score 4: checkout, build, test with published results, package, image build and scan, push, deploy, health check, notify - and explains what each protects against. **Score 2:** lists stages with no reasoning.

Q. What is the difference between `git merge` and `git rebase`?

Strong: merge preserves topology with a merge commit; rebase replays commits linearly with new hashes. **Must include:** never rebase shared history. **Score 1:** gets them the wrong way round.

Q. How do you check disk usage and find what is filling a disk?

Strong: `df -h` and `df -i`, then `du -xh --max-depth=1 | sort -h` and descend. Bonus for mentioning `lsdf | grep deleted`.

Q. What does `chmod 755` mean?

Strong: owner `rwx`, group and others `r-x`. Bonus for noting it is the normal mode for executables and directories.

Q. [Closing] Do you have any questions for me?

Score 4: two or three specific questions about the team, the stack or how incidents are handled. **Score 1:** "No, I think you covered everything."

Mock 1 scoring	What it means
4 - Strong	Correct, concise, with a reason and often a caveat
3 - Pass	Correct but thin; no trade-off mentioned
2 - Weak	Partially correct or needs prompting
1 - Fail	Incorrect, or a confident wrong answer
Target	Average 3+, with no 1s on Docker, Git or Linux basics

23.2 Mock Interview 2 - Two to Three Year DevOps (60 minutes)

Q. Your CV says three years. Walk me through what changed between your two roles.

Expected: a clear progression - tooling depth in the first role, ownership and production responsibility in the second. **Score 1:** cannot articulate any difference.

Q. You list eleven technologies. Rank them by your real confidence.

Testing: self-awareness. **Score 4:** honest ranking with Kubernetes placed accurately and a named area you are improving. **Score 1:** claims equal depth in all.

Q. Explain your Terraform state setup.

Strong: S3 with versioning and encryption, DynamoDB locking, one key per environment, never in Git because it holds plaintext secrets. **Follow-up:** what happens without locking? **Expected:** overlapping applies corrupt state and the next plan proposes destroying resources.

Q. What is the difference between `count` and `for_each`?

Strong: `count` is indexed and re-indexes on removal, destroying unrelated resources; `for_each` is keyed and stable. **Score 4:** explains the re-index trap with an example.

Q. Terraform plan shows `-/+` on your database. What do you do?

Strong: do not apply; find which attribute forces replacement; verify the state key and account; plan a safe migration; note that `prevent_destroy` exists for this. **Score 1:** "apply it, Terraform knows best."

Q. Why both Terraform and Ansible?

Strong: Terraform provisions with state and a plan; Ansible configures with idempotent tasks and no state. Each doing the other's job is possible and worse.

Q. Define idempotency and tell me where Ansible breaks it.

Strong: re-running produces no further changes; `command` and `shell` tasks break it unless guarded with `creates/removes/when`.

Q. A pod is in `CrashLoopBackOff`. Walk me through it.

Score 4: `logs --previous` first, then describe for exit code, then maps codes to causes, mentions that you cannot exec into a crashing container. **Score 2:** describes but forgets `--previous`.

Q. A Service returns nothing but the pods are Running. Why?

Score 4: `kubectl get endpoints` immediately, then selector mismatch versus not-Ready. **Score 2:** starts randomly checking things.

Q. Liveness versus readiness - and what is the classic mistake?

Strong: liveness restarts, readiness controls traffic; the mistake is checking a dependency in liveness, which turns a database blip into a fleet-wide restart loop.

Q. How do you handle secrets across your stack?

Strong: Jenkins credential store bound per-stage; AWS via IAM roles not keys; Kubernetes Secrets acknowledged as base64 not encrypted; Terraform state encrypted; nothing in Git; rotate anything exposed.

Q. Write me a PromQL query for the error rate.

Strong: writes it correctly and explains why rate is needed for a counter.

Q. What do you alert on, and what would you refuse to alert on?

Score 4: symptoms with a for: duration and runbooks; refuses to page on raw CPU with a reason.

Q. Describe a production incident end to end.

Score 4: detection, scoping, layered narrowing, mitigation before diagnosis, validation, RCA, action item. **Score 1:** a story with no validation and no prevention.

Q. What would you improve about your current setup?

Score 4: a specific, credible improvement - automated rollback on post-deploy error rate, drift detection, config parity checks. **Score 1:** "nothing, it works well."

23.3 Mock Interview 3 - AWS and Cloud Infrastructure (60 minutes)

Q. Draw the VPC architecture you work with.

Score 4: two AZs, public subnets with ALB and NAT, private app subnets, private data subnets with no internet route, and explains WHY each layer exists - removing a single point of failure or an exposure. **Score 2:** draws boxes without the reasoning. **Score 1:** cannot distinguish public from private.

Q. What makes a subnet public?

Strong: the route table sends 0.0.0.0/0 to an Internet Gateway. **Score 1:** "there's a public setting."

Q. How many usable IPs in a /26 in AWS?

Strong: 64 total minus 5 reserved = 59.

Q. Security group versus NACL, and why does it matter operationally?

Strong: stateful versus stateless, instance versus subnet, allow-only versus allow and deny, SG can reference other SGs. **The operational point:** stateless NACLs need the ephemeral return range, which is behind most "I opened the port but it still times out" tickets.

Q. An EC2 instance is running but SSH does not work.

Score 4: immediately separates timeout from permission-denied and gives different paths for each; mentions status checks and console output; mentions SSM as an alternative.

Q. What is 169.254.169.254 and why does IMDSv2 exist?

Strong: instance metadata including temporary role credentials; IMDSv2 requires a token, defeating SSRF-based credential theft.

Q. You get AccessDenied on an S3 upload. Walk through it.

Score 4: reads the error, confirms the identity with `sts get-caller-identity`, looks for explicit Deny including SCPs, checks the bucket vs bucket/* ARN distinction, mentions KMS permissions. **Score 2:** "add s3:* to the policy."

Q. Why is there no memory metric for EC2 in CloudWatch?

Strong: CloudWatch sees the hypervisor, not the guest OS; you install the agent.

Q. How do you do a blue-green deployment on AWS?

Strong: two target groups behind one ALB, deploy green, smoke test via a test listener, switch the production listener, rollback by switching back; weighted target groups make it a canary.

Q. ALB or NLB for a video streaming component using RTSP?

Strong: NLB - an ALB only understands HTTP and cannot route RTSP; NLB also preserves the client source IP and offers static IPs.

Q. How would you reduce AWS cost without hurting reliability?

Score 4: right-size from real metrics, Savings Plans for baseline and Spot for CI, S3 lifecycle and noncurrent version expiry, delete unattached volumes and old snapshots, S3 gateway endpoints to cut NAT charges, budgets and anomaly detection,

tagging discipline.

Q. Your application in a private subnet cannot reach an external API.

Strong: route to NAT, NAT in a public subnet, SG egress, NACL both directions, then test DNS and connectivity separately from the instance.

Q. How would you design for disaster recovery?

Strong: start from RTO and RPO, then pick the pattern - backup/restore, pilot light, warm standby, active-active - and emphasise testing the failover, because an untested DR plan does not work.

Q. Terraform or CloudFormation, and why?

Strong: Terraform for provider-agnosticism and module reuse, with the trade-off that you own state and locking; CloudFormation is AWS-managed state with built-in rollback but AWS-only.

23.4 Mock Interview 4 - Kubernetes and CI/CD (60 minutes)

Q. Explain the Kubernetes control plane.

Score 4: API server as the only writer to etcd, etcd holding all state, scheduler for placement, controller manager running reconciliation loops - and frames the whole thing as desired-versus-actual reconciliation. **Score 2:** lists components without the model.

Q. What is your actual Kubernetes experience?

Score 4: states clearly that it is workload-level - deploying and troubleshooting pods, deployments, services, configmaps and connectivity - and that cluster administration is something being strengthened. **Score 1:** overclaims and then fails the follow-up on etcd or node draining.

Q. Deployment, ReplicaSet and Pod - explain the relationship.

Strong: Deployment manages ReplicaSets, ReplicaSets manage Pods; a new pod template makes a new ReplicaSet, and old ones are retained at zero replicas, which is what makes rollback instant.

Q. Walk me through a rolling update and how you would make it zero-downtime.

Strong: maxSurge and maxUnavailable, readiness gating each step, maxUnavailable: 0 with maxSurge: 1 for strict zero downtime, minReadySeconds, and progressDeadlineSeconds so a stuck rollout fails rather than hanging.

Q. A rollout is stuck. Debug it.

Strong: look at the NEW pods - they are Pending, ImagePullBackOff, CrashLooping or never Ready; describe rs for creation failures like quota; the maxUnavailable-zero-with-no-capacity trap; roll back first, diagnose after.

Q. You deployed and the old version is still serving.

Score 4: check the image DIGEST not the tag; mutable tag plus IfNotPresent; pod AGE showing they never restarted; endpoints; CDN and keep-alive caching.

Q. How do you handle configuration and secrets in Kubernetes?

Strong: ConfigMaps and Secrets per namespace; env vars fixed at pod start so config changes need a rollout restart or a checksum annotation; Secrets are base64 not encrypted, so etcd encryption, RBAC and ideally an external store.

Q. Design the pipeline for a new microservice.

Score 4: PR pipeline for fast feedback without deployment; main pipeline builds once with an immutable tag, scans, pushes, deploys to staging, smoke tests, gated production deploy, rollout status wait, health check, notify, automated rollback on failure; credentials from the store; Jenkinsfile in the repo; shared library pinned to a tag.

Q. How do you ensure the artifact you tested is the artifact you deploy?

Strong: build once, tag immutably with build number and SHA, promote the same image through environments, never rebuild per environment, and deploy by digest where possible.

Q. How would you implement a canary in Kubernetes?

Strong: two Deployments behind one Service for a crude split, or ingress canary annotations / a service mesh for real percentage control - and stresses that it only works with per-version metrics to decide on.

Q. How do you handle database migrations with rolling deployments?

Score 4: expand-and-contract so both versions coexist; migration as a separate gated step, not on every replica's startup; tested against production-sized data; acknowledges that an irreversible migration removes rollback as an option.

Q. What is CPU throttling and how would you spot it?

Strong: exceeding the CPU limit causes cgroup throttling - the app gets slower with no errors; `container_cpu_cfs_throttled_seconds_total` reveals it. A great answer because most candidates never mention it.

Q. An HPA is not scaling. Why?

Strong: metrics-server missing, or no resource requests set so utilisation cannot be computed - it shows `<unknown>`.

Q. How would you introduce GitOps, and why?

Strong: Argo CD or Flux reconciling the cluster from Git - deployment becomes a merge, drift is auto-corrected, the audit trail is the Git history, and CI no longer needs cluster credentials.

23.5 Mock Interview 5 - Production Troubleshooting (45 minutes)

THIS IS YOUR INTERVIEW TO WIN

Production troubleshooting is the strongest, most differentiated claim on your CV. Most candidates at three years cannot describe a method. Go in expecting to score highest here and let it carry the round.

Q. It is 3am, you are paged, the site is down. First five minutes.

Score 4: scope (all users or some, which environment), timing, what changed; then a layered check from the edge inwards; mitigate first; communicate on a cadence. **Score 1:** starts restarting things.

Q. How do you decide between rolling back and fixing forward?

Strong: roll back if users are affected and the cause is not obvious within a few minutes; fix forward only when the fix is trivial and tested, or when a migration makes rollback unsafe. **Score 4:** mentions the migration caveat unprompted.

Q. Disk is at 97% on a production database server.

Score 4: `df -h` and `df -i`, find the offender with `du`, compress rather than delete live logs, check for deleted-but-open files, then grow the volume; RCA on why nothing alerted at 80%; prevention with logrotate, retention and a predictive alert.

Q. The application is returning 502 intermittently.

Score 4: proxy error log, backend health from the proxy, restart counts for OOM, wrong port or protocol - and specifically mentions the keep-alive timeout mismatch as the classic cause of INTERMITTENT 502s. That detail is a standout answer.

Q. Latency went from 100ms to 3 seconds with no deploy.

Strong: split the time with `curl -w`; compare edge, app and database latency; check throttling, pool saturation, GC, disk burst balance; ask whether it is proportional to load; check whether a dependency or a cron job changed.

Q. Users cannot log in but the dashboards are all green.

Strong: the users are right and monitoring has a gap; reproduce from outside through real DNS with a real client; suspect DNS, certificate, CDN or WAF, which internal checks bypass; close the monitoring gap as an action item.

Q. Pods are restarting every few minutes.

Strong: describe for the reason - OOMKilled versus liveness probe failure; if OOM, limit versus leak; if liveness, an over-aggressive probe or one checking a dependency; `logs --previous` for the application's own error.

Q. The database is at max connections.

Strong: restart the application as mitigation to release leaked connections; check whether the pool is leaking or genuinely undersized; add a pooler; kill blocking queries; alert on connection count approaching the max rather than at it.

Q. How do you write an RCA?

Score 4: summary, factual impact, timestamped timeline including the detection gap, root cause, mitigation, what went well and badly, action items with owners and dates; blameless, naming systems not people.

Q. You caused the outage. What do you do?

Score 4: say so immediately, restore first, then focus the RCA on the missing guard rail rather than on yourself. **Score 1:** hedges or describes covering it up.

Q. An alert fires at 3am for something that is not broken.

Strong: handle it, then fix the alert - it is a defect; adjust the threshold, add a `for:`, demote it to a ticket, or delete it; track which alerts fire and what action followed.

Q. What is the hardest production problem you have solved?

Score 4: a diagnostically interesting problem explained with a clear method and a prevention outcome. **Score 1:** an exaggerated heroic story with invented scale.

Q. What would you check first in ANY incident, regardless of symptom?

Strong: what changed in the last hour, and what the blast radius is. Those two questions eliminate most possibilities faster than any command.

23.6 Self-Scoring Sheet

Area	Score 1-4	If below 3, revisit
Introduction and narrative		Part 1
DevOps fundamentals		Part 2
Linux		Part 3
Networking		Part 4
Git		Part 5
Jenkins / CI/CD		Part 6
Docker		Part 7
AWS		Part 8
Terraform		Part 9
Ansible		Part 10
Kubernetes		Part 11
Monitoring		Parts 12-13
Scripting		Parts 14-15
Troubleshooting method		Parts 16-17
Project explanation		Part 18
Resume defence		Part 19
HR answers		Part 20

THE THREE HABITS THAT RAISE EVERY SCORE

- 1. Answer the question asked, then stop.** Most weak answers are correct information delivered past the point of usefulness. Aim for 45-90 seconds.
- 2. Give a reason, not just a fact.** "Use for_each" is a 2. "Use for_each because count re-indexes and destroys unrelated resources when you remove an item" is a 4.
- 3. Name the limit of your knowledge before they find it.** "I have not administered a cluster in production" said by you is a strength; discovered by them, it is a problem.

PART 24

30-DAY STUDY PLAN

Realistic for someone working full time

Roughly 2 hours on weekdays and 3-4 hours on weekends. Each day has theory, hands-on, questions and revision. Do not skip the hands-on - reading about a CrashLoopBackOff and debugging one are different skills, and interviewers can tell which one you have.

BEFORE DAY 1 - SET UP YOUR LAB

You need three things: a Linux VM (an AWS free-tier EC2 instance or a local VM), Docker installed, and a local Kubernetes (minikube, kind or Docker Desktop). Also create a scratch GitHub repository and an AWS account with a billing alarm set at a low threshold. Total setup time: about an hour. Everything below assumes you have it.

24.1 Week 1 - Foundations and Linux

Day	Topic	Theory (45m)	Hands-on (45m)	Questions (20m)	Outcome
1	DevOps fundamentals	Part 2.1-2.8	Draw the DevOps lifecycle from memory; map each phase to a tool you use	Part 2 questions	Can define DevOps, CI/CD and IaC without hesitating
2	Deployment strategies and SRE	Part 2.15-2.21	Draw rolling, blue-green and canary from memory; write down when you would use each	Part 2 recall box	Can compare strategies with trade-offs
3	Linux architecture, files, permissions	Part 3.1-3.4	Create a shared SGID directory; set a sticky bit; find all SUID binaries	Linux Q1-10	Permissions are automatic, including the directory-delete rule
4	Linux text processing	Part 3.5	Download any access log; produce top 10 IPs, status code counts, and all 5xx in the last hour using only grep/awk/sort/uniq	Linux Q11-20	The incident one-liners are muscle memory
5	Processes, services, systemd	Part 3.6-3.7	Write a systemd unit for a Python HTTP server; break it deliberately; diagnose with status and journalctl only	Linux Q21-30	Can debug a service that will not start
6	Disk, memory, logs	Part 3.8-3.10	Fill a disk with falloccate; delete the file while tail holds it; prove with lsof that space did not return; write a logrotate config and test with -d	Linux Q31-44	The disk-full runbook is second nature
7	Week 1 review	Rapid revision 21.1	Re-do anything from days 3-6 that felt slow	Flashcards 31-70	No hesitation on Linux fundamentals

24.2 Week 2 - Networking, Git, Jenkins

Day	Topic	Theory (45m)	Hands-on (45m)	Questions (20m)	Outcome
8	Networking models and TCP	Part 4.1-4.4	Use ss, nc and curl -v against a service; deliberately bind to 127.0.0.1 and observe the failure from another host	Networking Q1-10	Refused vs timeout is instant
9	DNS, HTTP, TLS	Part 4.5-4.6	dig a domain with +trace; inspect a certificate with openssl s_client; break SNI deliberately	Networking Q11-25	Can diagnose a TLS problem from the error text

Day	Topic	Theory (45m)	Hands-on (45m)	Questions (20m)	Outcome
10	Load balancing, firewalls, the full request path	Part 4.7-4.9	Explain "what happens when you type a URL" out loud, recorded, in under 3 minutes	Networking Q26-40	The layered answer is fluent
11	Network troubleshooting scenarios	Part 4.10-4.11	Break connectivity three ways (bind address, firewall rule, wrong port) and diagnose each without looking at what you changed	Part 4 recall	A repeatable method, not guessing
12	Git fundamentals and branching	Part 5.1-5.5	Create a conflict and resolve it; do an interactive rebase squashing 4 commits into 1	Git Q1-15	Merge vs rebase is confident
13	Git recovery and workflow	Part 5.6-5.8	Reset --hard, then recover with reflog; cherry-pick a commit; run git bisect on a scripted repo	Git Q16-40	Can recover from any local Git mistake
14	Week 2 review	Rapid revision 21.2, 21.9	Re-draw the request path and the "I cannot connect" flowchart from memory	Flashcards 71-130	Networking and Git are solid

24.3 Week 3 - Containers, CI/CD, Cloud

Day	Topic	Theory (45m)	Hands-on (45m)	Questions (20m)	Outcome
15	Docker architecture and Dockerfile	Part 7.1-7.3	Write a multi-stage Dockerfile; measure the size difference against a single-stage version; run as non-root	Docker Q1-15	Layer ordering and CMD vs ENTRYPOINT are automatic
16	Docker networking, volumes, Compose	Part 7.4-7.7	Run an app and a database on a user-defined network; break it by using the default bridge and observe the DNS failure	Docker Q16-30	localhost-inside-a-container will never catch you
17	Docker troubleshooting	Part 7.8-7.9	Reproduce four failures: exit 0, exit 127, OOM with a memory limit, unreachable port	Docker Q31-40	Exit codes map instantly to causes
18	Jenkins architecture and pipelines	Part 6.1-6.4	Write the full Jenkinsfile from Part 6.4 from memory; explain every block out loud	Jenkins Q1-20	Can write a Jenkinsfile on a whiteboard
19	Jenkins credentials, libraries, agents	Part 6.5-6.9	Write a GitHub Actions workflow (Part 19.8) that builds and pushes an image	Jenkins Q21-35	Comfortable with both CI tools
20	Jenkins troubleshooting and the full flow	Part 6.10-6.11	Narrate the complete Git-to-production flow, recorded, in under 3 minutes	Jenkins Q36-50	The pipeline story is your strongest set piece
21	Week 3 review	Rapid revision 21.3, 21.4	Re-write the Dockerfile and Jenkinsfile from memory, no notes	Flashcards 131-190	CI/CD and containers are interview-ready

24.4 Week 4 - Kubernetes, IaC, Monitoring, Practice

Day	Topic	Theory (45m)	Hands-on (45m)	Questions (20m)	Outcome
22	Kubernetes architecture and pods	Part 11.1-11.2	Deploy an app to minikube; write the pod spec with probes, requests and limits by hand	K8s Q1-20	Architecture and the reconciliation model are clear
23	Deployments, Services, config	Part 11.3-11.5	Do a rolling update; break the Service selector and fix it using get endpoints	K8s Q21-40	Service debugging is a reflex

Day	Topic	Theory (45m)	Hands-on (45m)	Questions (20m)	Outcome
24	Kubernetes troubleshooting	Part 11.8	Deliberately create CrashLoopBackOff, ImagePullBackOff, Pending and OOMKilled; debug each	K8s Q41-70	All 15 scenarios understood, 6 done by hand
25	Terraform	Part 9 (all)	Write a module with variables and outputs; run plan, apply, change something in the console, run plan again to see drift	Terraform Q1-40	State, modules and drift are confident
26	Ansible	Part 10 (all)	Write a playbook installing and configuring nginx with a template and a handler; run it twice and prove zero changes	Ansible Q1-30	Idempotency is demonstrable, not theoretical
27	AWS	Part 8 (all)	Draw the three-tier VPC from memory; build a VPC with subnets and an EC2 instance in Terraform	AWS Q1-60	Can draw and justify the architecture
28	Monitoring and scripting	Parts 12, 13, 14, 15	Write the disk-check and health-check scripts from memory; write five PromQL queries without looking	Monitoring Q1-25, Python Q1-30	Can write a script and a query live
29	Project and resume defence	Parts 18, 19	Deliver the 3-minute project explanation, recorded; then work through the Part 19.23 self-audit and mark every gap	Part 18 - all 50	Every CV line is defensible
30	Mock interviews	Part 23	Run all five mocks out loud, timed, recorded. Score yourself	Part 25 checklist	Ready

24.5 Daily Routine

Slot	Duration	What
Morning commute / start	15 min	30 flashcards from Part 22 - recall only, no reading
Core session	45 min	The day's theory - read actively, take notes in your own words
Hands-on	45 min	The day's practical. Break something deliberately, then fix it
Questions	20 min	Answer the day's questions OUT LOUD before checking
Evening / wind down	10 min	Re-read the recall box of that part

THE TWO RULES THAT MAKE THIS PLAN WORK

- Out loud, always.** Interviews are spoken. An answer you can write but cannot say fluently will fail. Record yourself at least twice a week and listen back - you will hear the rambling immediately.
- Break it, then fix it.** Reading about CrashLoopBackOff teaches you the words. Creating one and debugging it teaches you the reflex. When the interviewer asks, the difference is obvious within ten seconds.

24.6 If You Only Have One Week

Day	Priority	Why
1	Part 1 (your narrative) + Part 18 (project)	You will be asked these in every single interview
2	Part 19 (resume cross-questioning), all of it	Anything on your CV is fair game and this is where candidates get exposed
3	Part 6 (Jenkins) + Part 7 (Docker)	Your strongest technical claim and the most commonly tested
4	Part 11 (Kubernetes) troubleshooting + Part 3 (Linux) command sheet	The two highest-frequency technical areas
5	Part 8 (AWS) + Part 9 (Terraform)	Draw the VPC, explain state
6	Parts 16-17 (troubleshooting and scenarios)	Your differentiator - make it fluent

Day	Priority	Why
7	Part 21 (rapid revision) + Part 23 (mocks) + Part 20 (HR)	Consolidate and rehearse

24.7 If You Only Have One Day

THE FOUR-HOUR EMERGENCY PLAN

Hour 1: Part 1.6-1.8 (introductions and project honesty) and Part 18.3-18.4 (the 1 and 3-minute project versions). Say each one out loud three times.

Hour 2: Part 21 in full - the rapid revision section. Every table, every flowchart.

Hour 3: Part 21.12 (the twelve sentences), plus flashcards for your two weakest areas.

Hour 4: Mock Interview 2 and Mock Interview 5 from Part 23, out loud, timed.

Then sleep. Cramming past this point reduces performance rather than improving it.

24.8 Progress Tracker

Week	Target	Done?
1	Linux fundamentals fluent; can run the disk-full runbook from memory	
2	Networking layered method fluent; Git merge/rebase/reset/revert confident	
3	Can write a Dockerfile and a Jenkinsfile from memory; pipeline story recorded	
4	Six Kubernetes failures debugged by hand; VPC drawn from memory; five mocks completed	
Final	Part 19.23 self-audit: all 28 items answerable in 60 seconds	

PART 25

FINAL INTERVIEW CHECKLIST

The last thing you read before you walk in

25.1 The Week Before

Item	Done
Re-read your CV as if you had never seen it, and list every question it invites	
Complete the Part 19.23 self-audit - 28 items, 60 seconds each	
Record your 30-second, 60-second and 2-minute introductions and listen back	
Record the 3-minute project explanation and listen back	
Run all five mock interviews from Part 23	
Research the company: what the product does, who the customers are, recent news	
Read the job description line by line and map each requirement to your experience	
Prepare three specific reasons you want THIS role	
Prepare your salary range, researched, and your notice period	
Decide in advance how you will describe your certifications versus training	
Prepare five questions to ask them	

25.2 The Day Before

Item	Done
Read Part 21 in full - the rapid revision section	
Read Part 21.12 - the twelve sentences - twice	
Review flashcards for your two weakest areas only	
Confirm the time, timezone, format and joining link or address	
Test your camera, microphone and internet if it is remote; have a phone backup	
Lay out a notepad and pen - you will want to sketch diagrams	
Have your CV printed or open in a separate window	
Stop studying by 9pm. Sleep matters more than one more topic	

25.3 Thirty Minutes Before

Item	Done
Re-read Part 21.12 and your 60-second introduction	
Re-read the job description	
Water within reach; phone silenced; notifications off	
Close every other application if screen-sharing is possible	
Two minutes of slow breathing - lowers the shake in your voice for the first answer	

25.4 During the Introduction

Do	Do not
Smile and greet by name	Launch straight into technical detail

Do	Do not
Deliver the prepared 60-second introduction	Improvise it for the first time
Structure: who, what I do now, stack, proof, why here	List every job chronologically from 2022
End with a hook that invites the next question	Trail off with "...yeah, that's about it"
Speak slightly slower than feels natural	Fill silence with "um" - pause instead

25.5 During the Technical Round

THE SEVEN RULES

1. **Clarify before answering.** "Do you mean in Kubernetes or on EC2?" is a scoring point, not a weakness.
2. **Answer the question asked, then stop.** 45-90 seconds. Most weak answers are correct information delivered past the point of usefulness.
3. **Give the reason, not just the fact.** "Use for_each" scores a 2; "use for_each because count re-indexes and destroys unrelated resources" scores a 4.
4. **Think out loud on scenarios.** They are assessing your method, not your recall. Narrate what each check would tell you.
5. **Name your limits before they find them.** "I have not administered a cluster in production" is a strength when you say it.
6. **Use the whiteboard or paper.** Draw the VPC, draw the pipeline. Candidates who draw are remembered.
7. **If you do not know: say so, then say how you would find out.** That answer scores far higher than a confident guess.

WHEN YOU ARE COMPLETELY STUCK

"I have not come across that. My instinct would be to start from [the nearest thing you do know] - for example, I would check whether it behaves like [an analogous case], and I would verify it against the documentation before acting on it. Is that the right direction?" That answer shows reasoning and coachability, which is most of what they are testing anyway.

25.6 Questions to Ask Them

ASK THREE OR FOUR - AND ACTUALLY LISTEN TO THE ANSWERS

About the work:

- What does the deployment path look like today, from commit to production?
- What is the biggest source of toil for the team right now?
- What would you want the person in this role to have improved in six months?

About reliability:

- How does on-call work, and how noisy is it?
- What was the last significant incident, and what changed afterwards?
- How do you decide between shipping features and paying down reliability work?

About the team:

- How is the team structured, and how does it work with development and QA?
- How are technical decisions made - who decides on tooling?
- What does a good first 90 days look like in this role?

About growth:

- Where does the team want its skills to be a year from now?
- Is there budget and time for certification or conferences?

Never ask in a first round: how much leave do I get, can I work from home every day, how soon can I be promoted, what does the company do (you should have found that out).

25.7 Things Not to Say

Never say	Because	Say instead
"I managed a large Kubernetes cluster" (if you did not)	One follow-up on etcd or node draining ends the round	"I deploy and troubleshoot workloads; cluster administration is what I am currently strengthening"
"We reduced deploy time by 80%" (unmeasured)	You will contradict yourself on methodology	"I did not own that measurement, but the mechanism was..."
"My previous manager was difficult"	You will describe them the same way in a year	"I'm looking for more depth in Kubernetes and cloud architecture"
"I'm a perfectionist" as a weakness	Transparent and tired	A real gap plus what you are doing about it
"I don't know" and then silence	Wastes the opportunity	"I don't know, but here is how I would find out"
"We had zero downtime for a year"	Unprovable, invites "what was your SLO?"	"We had a rollback path for every release and used it when we needed to"
"That's not my job"	Ends the DevOps conversation immediately	"That sits outside what I own, but I would work with whoever does"
"I just followed the runbook"	Sounds like you do not understand the system	"The runbook covered it, and here is why each step mattered"
Naming a certification you do not hold	Verified at background check, after everyone has invested time	"I completed the training; the exam is something I am preparing for"
Criticising a specific former colleague	Reads as a red flag regardless of who was right	Describe the situation and the outcome, not the person

25.8 Handling Questions You Cannot Answer

Situation	Response
Completely unfamiliar topic	"I have not worked with that. Can you tell me a little about how you use it? I would rather understand it than guess."
Familiar but you have forgotten the detail	"I know the concept - it is [what you do remember]. The exact syntax I would check, but the reasoning is [explain the reasoning]."

Situation	Response
A trick question where you suspect a trap	"My instinct is [answer], but I suspect there is a catch - is this about [name the likely trap]?"
Asked to do live coding and you are blanking	"Let me talk through the structure first, then fill in the syntax." Narrating the approach earns most of the marks.
You realise mid-answer that you are wrong	"Actually, let me correct that - [right answer]. I was confusing it with [the other thing]." Self-correction is respected.
You answered confidently and they look sceptical	"Would you like me to go deeper on that, or have I misunderstood the question?"

25.9 During the Managerial and HR Round

Do	Do not
Use STAR for every behavioural question	Give a hypothetical when asked for an example
Keep every employer reference neutral or positive	Explain why your last company was badly run
Be consistent with what you told the technical interviewer	Upgrade your Kubernetes experience between rounds
Have a researched salary range ready	Say "whatever you think is fair"
Say your real notice period	Promise a start date you cannot meet
Ask about on-call, incidents and how the team works	Ask only about leave and perks

25.10 After the Interview

Item	Done
Write down every question you were asked, within the hour, while you remember	
Mark the ones you handled badly - that is your study list for the next round	
Send a short thank-you note if you have the interviewer's email: two or three sentences, one specific thing from the conversation	
If you promised to follow up on something, do it within 24 hours	
If rejected, ask for specific feedback once, politely, and accept whatever you get	

25.11 The Final Reminders

READ THIS LAST

- **Honesty outperforms bluffing.** Every experienced interviewer has been bluffed at, and they all test for it. Your CV is strong and honest - defend it exactly as written.
- **Your differentiator is troubleshooting.** Most three-year candidates can describe a pipeline. Far fewer can describe a method for diagnosing a live failure. Lead with it when you get the chance.
- **Method beats memory.** They are not checking whether you memorised a flag. They are checking whether you would be useful at 3am.
- **Draw things.** A candidate who sketches the VPC or the pipeline is remembered after the interview; one who only talks is not.
- **Say what each check would tell you.** "I'd run `kubectl get endpoints` - if it is empty, it is either a selector mismatch or the pods are not Ready" is worth three times "I'd check the endpoints".
- **You will not know everything.** Nobody does. The people who get hired are the ones who are clear about what they know, honest about what they do not, and obviously capable of finding out.
- **Then go and do the job.** This handbook gets you through the door; the work itself is what makes the next one easier.

ONE LAST THING

Before you close this book, go back to Part 1.4 and re-read your gaps, and Part 1.3 and re-read your strengths. Walk in knowing both. Confidence built on an accurate picture of yourself holds up under questioning; confidence built on hoping they do not ask does not.